Spring Boot 入門

JAC0177G-T1-04

~サーブレット&JSP からのステップアップ~

商標

- アマゾン ウェブ サービス、AWS および Amazon Web Services ロゴは、米国その他の諸国における、Amazon.com, Inc.またはその関連会社の商標です。
- Cisco、Cisco Systems、Cisco Systems ロゴは、Cisco Systems, Inc.および関連会社の米国ならびに他の国における商標もしくは登録商標です。
- IIBA®、BABOK® は、IIBA® (International Institute of Business Analysis™) の登録商標です。
- Intel、インテルは、米国およびその他の国における Intel Corporation の商標です。
- ITIL®は、PeopleCert group の登録商標であり、PeopleCert Group の許可のもとに使用されています。すべての権利は留保されています。
- Microsoft、Windows は、米国 Microsoft Corporation の、米国およびその他の国における登録商標または商標です。
- OracleとJava は、Oracle Corporation およびその子会社、関連会社の米国およびその他の国における登録商標です。
- PMI、PMP、PMBOK は、プロジェクトマネジメント協会(Project Management Institute, Inc.)の登録商標です。
- VMware、VMware ロゴは、VMware, Inc.の米国および各国での商標または登録商標です。
- BOOT CAMP、NEW TRAIN、MIX TRAIN、TRAINOCAMP は、トレノケート株式会社の登録商標です。

その他の会社名、製品名およびサービス名は、それぞれ各社の商標または登録商標です。

目　次

はじめに 7

第 1 章　Spring Boot と開発環境の概要 15

1.1. フレームワークとは 18
1.2. Spring Framework 19
1.3. Spring Boot とは 21
1.4. Maven でライブラリ管理 24
章のまとめ 29

第 2 章　Spring Web アプリケーション 31

2.1. Web アプリケーションの概要 34
2.1.1.【 参考】サーブレットについて 36
2.1.2.【 参考】MVC パターンと Spring アプリケーションの流れ 38
2.2. Controller の基本 39
2.3. データ送信とパラメータの取得 47
2.4. Form クラスでパラメータ取得 53
2.5. Controller から View へのデータ処理 57
章のまとめ 62

第 3 章　Spring Data JPA 63

3.1. Spring Data JPA の利用 66
3.2. エンティティの定義 72
3.3. リポジトリの定義 77
3.3.1. 依存性の注入 79
3.3.2. コントローラーの基本 URL 設定 81
【 参考】既存の全件検索方法 82
3.4. 全件検索 85
3.5. 主キーによる条件検索 90
3.5.1. 主キーではない列による条件検索 95
3.5.2. 複数の列による条件検索 99

3.5.3. その他の条件検索 103
3.6. 登録処理 105
3.7. 更新処理 114
3.8. 削除処理 122
3.9.【 参考】Service クラスでビジネスロジックを分離 125
章のまとめ 132

第 4 章　セッションと入力チェック 133

4.1. 基礎的なログイン機能の実装 136
4.2. セッションによるユーザ情報の保持 142
4.3. ログイン時の入力チェック 148
4.3.1.【 参考】その他の Validation アノテーション 157
4.3.2.【 参考】メッセージのカスタマイズ 159

第 5 章　Thymeleaf と Web ページデザイン 164

5.1. Thymeleaf の基本構文 167
5.1.1. 動的にイメージとリンク生成(th:attr) 170
5.1.2. 条件式による表示制御(th:if) 175
5.2. CSS と JavaScript の利用 178
5.3. レイアウト機能 181

第 6 章　外部参照と複雑な SQL 操作 192

6.1. 外部参照を持つエンティティの定義 195
6.2. 外部参照を用いた検索 203
6.2.1.【 参考】検索結果のエラーハンドリング 208
6.3. 外部参照付きのデータ登録 213
6.4. 複雑な SQL 文の操作方法 222
6.4.1. 複雑な検索をメソッド名で定義する際のデメリット 223
6.4.2. JPQL の基本構文 225
章のまとめ 236

第 7 章　Bootstrap 238

7.1. Bootstrap とは 241
7.2. Bootstrap の導入方法 245
7.3. Bootstrap を使ったスタイルの適用 250

7.3.1. テキスト....251
7.3.2. 色....252
7.3.3. ボタン....254
7.3.4. ボーダー(Border)....256
7.3.5. 余白....257
7.3.6. ブロック要素の幅の調整(高さと横幅)....260
7.4. テーブル....262
7.5. グリッドシステムを使ったレスポンシブデザイン....265
7.6. フォーム....272
章のまとめ....280

付録 A 環境構築手順....282

Import 手順....283

新規プロジェクト作成方法....286
セキュリティ問題で、Eclipse 上で新規プロジェクト作成ができない場合....288

付録 B よく起きるエラーまとめ....290

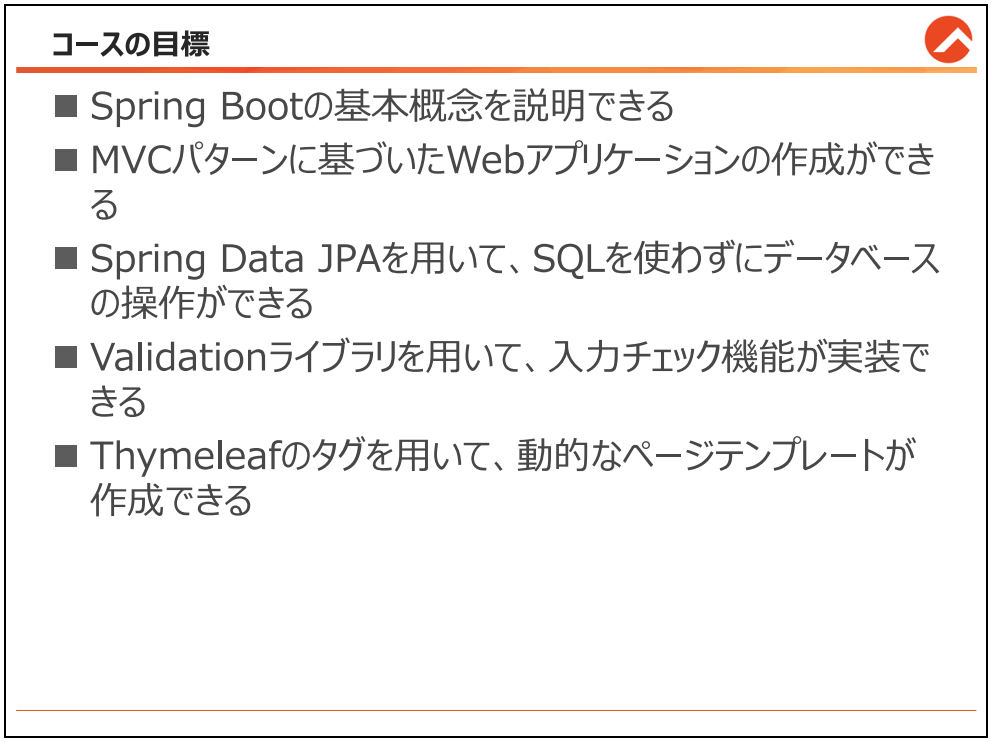
404 エラーが出る場合:....291

500 Thymeleaf エラー....291

8080 ポート衝突エラー....292
Eclipse 上で赤線が発生したり、呼び出したいメソッドが候補から出ない....293
設定ファイル・イメージ・CSS・JavaScript などを修正しても反映されない....294
インポート時にエラーが発生する時....294

はじめに

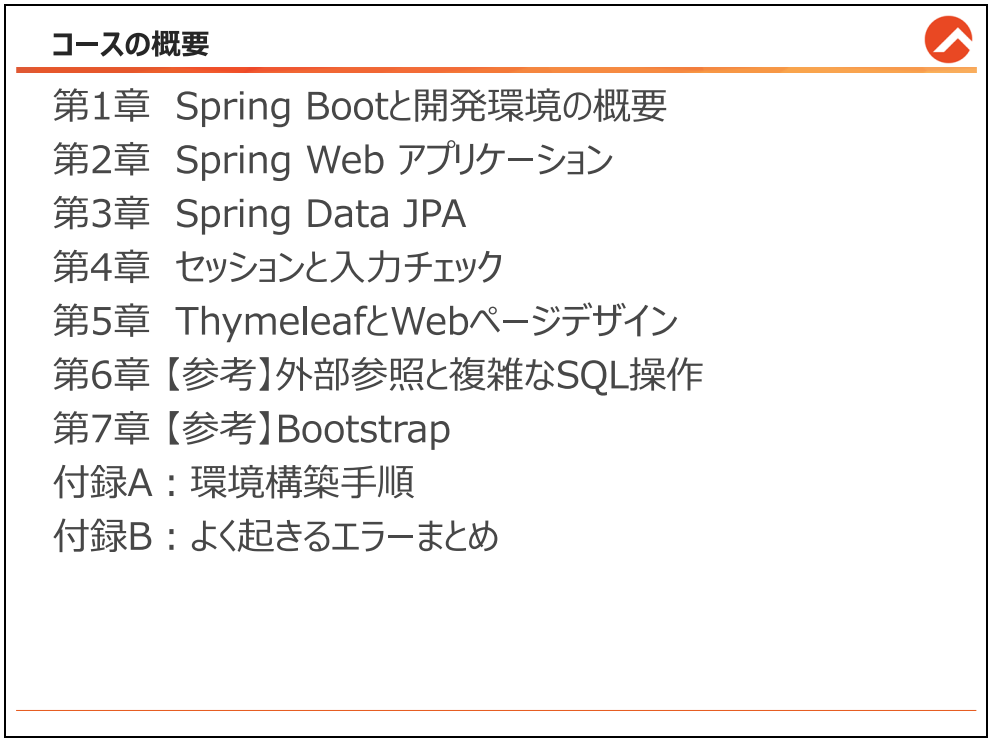
コースの目標



本コースを修了することで、以下のことができるようになります。

- Spring Boot の基本概念を説明できる
- MVC パターンに基づいた Web アプリケーションの作成ができる
- Spring Data JPA を用いて、効率的なデータベースの操作ができる
- Validation ライブラリを用いて、入力チェック機能が実装できる
- Thymeleaf のタグを用いて、動的なページテンプレートが作成できる

コースの概要



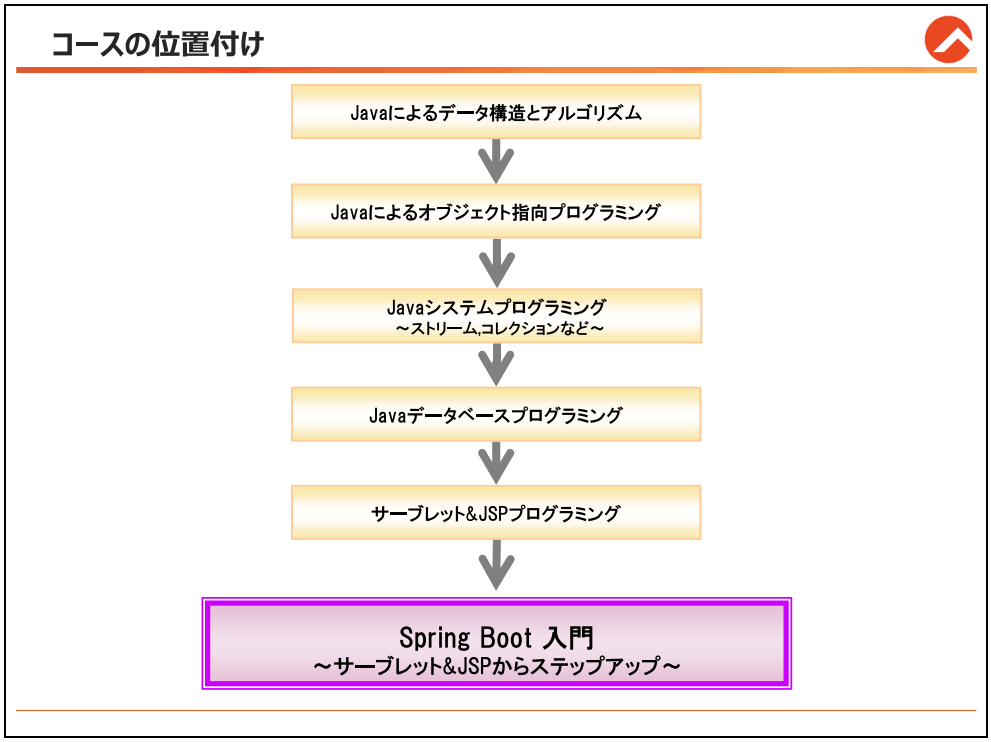
目標を達成するため、以下の章で構成されています。

1. Spring Boot と開発環境の概要
2. Spring Web アプリケーション
3. Spring Data JPA
4. セッションと入力チェック
5. Thymeleaf と Web ページデザイン

なお、参考として、以下の章の内容も学習すると良いです。

6. 外部参照と複雑な SQL 操作
7. Bootstrap

コースの位置付け



本コースでは、サーブレット&JSPを用いた、クラシックなWebアプリケーション開発からステップアップして、より効率よくWeb開発ができる「Spring Boot」の習得を目指します。

このコースでは、以下の条件を満たしていることを前提としています。

- Windows の基本操作(ファイルの作成、コピー、移動、削除など)ができる。
- 「Java によるオブジェクト指向プログラミング」を受講しているか、
  基本的な Java プログラミングが記述できる(基本文法・クラス・インスタンス・継承)
- JavaAPI(例:ArrayList)の使い方を知っている
- 「データベース入門」を受講しているか、
  SQL(SELECT/INSERT/UPDATE/DELETE)の基本構文を知っている
- 「基礎から学べる HTML」を受講しているか、
  HTML を使って簡単な Web ページを作成できる
- Eclipse の基本的な操作ができる

テキストの表記

■ 画面に表示される文字とキーボードキー

メニュー・コマンド・ダイアログボックス等画面に表示される文字とキーボードキーは、[ ]で囲んで表記しています。

表記例

画面に表示される文字	[プロパティ]を選択します
キーボードキー	[Ctrl]・[Tab]・[Caps Lock]・[↑]・[→]

■ コンソール上の英字やコード・コマンド・パラメータ

コンソール上の英字やコード・コマンド・パラメータは、Courier New のフォントで表記しています。

表記例

```
          Vlan Name                    Status             Ports
----------------------------------
1         default    Enabled  3-12,AUI,A
10        test1                Enabled  1
```

■ ユーザが任意に指定する値(コード・コマンド・パラメータ)

ユーザが任意に指定する値は、Courier New Bold italic のフォントで表記しています。

表記例

```
C:¥AppSource> csc /debug+ /out: MyApp.exe MyApp.cs
```

```
If command1 returns success, command2 runs
$ ls -l sample.txt
```

■ UNIX と Linux 関連のテキストでの表記

ユーザが任意に指定する値

ユーザが任意に指定する値は Times New Roman Italic のフォントで表示しています。

表記例

```
more [file1[file2…]]
ls -l file
```

■ 重要な語句や強調

本文中の重要な語句や強調は太字で表示しています。

表記例

この認証プロセスのことをログオンといいます。

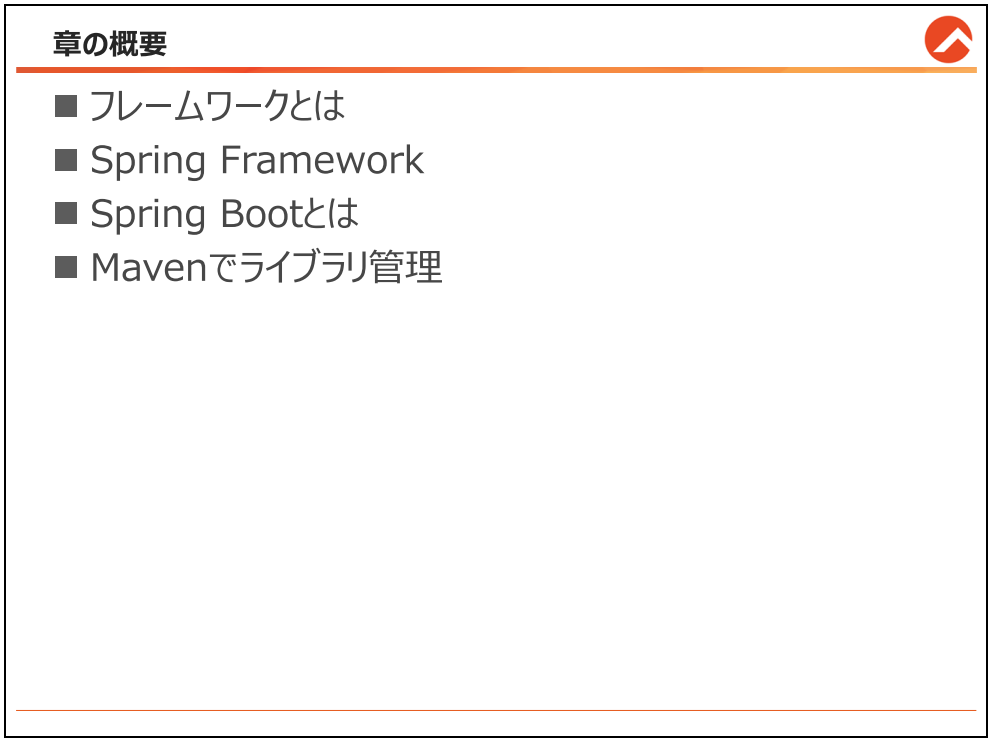
1

第1章 Spring Boot と開発環境の概要

章の目標

- フレームワークと Spring Framework について理解する
- Spring Boot の基本概念と特徴を把握する
- Maven を用いてライブラリのインストールができる

章の概要

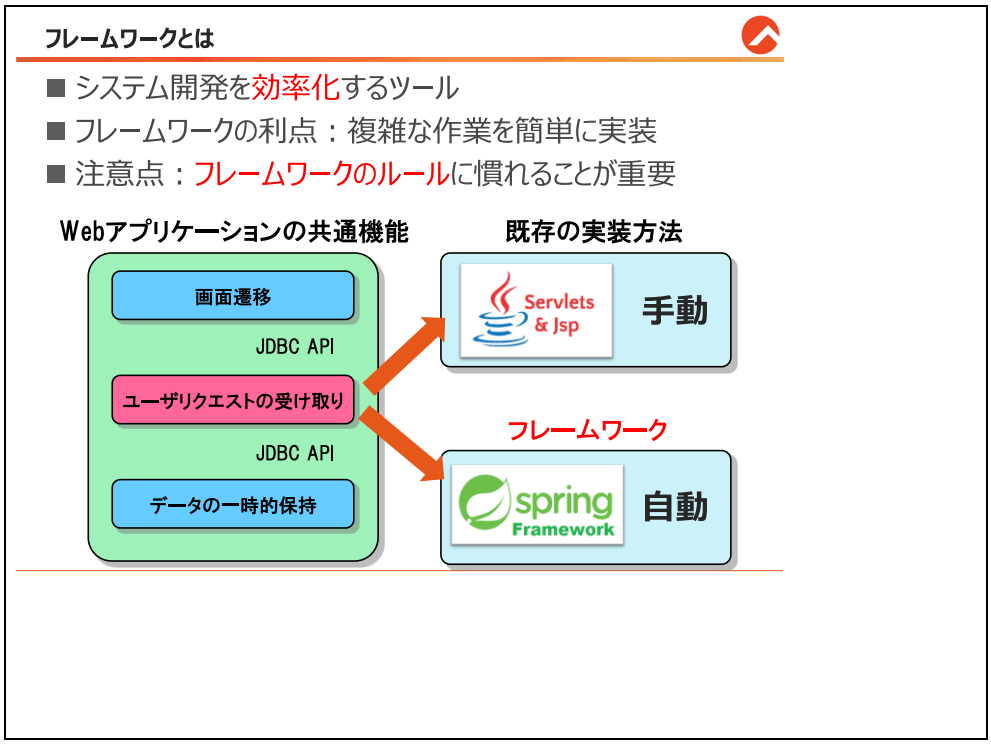


この章では、Java での Web 開発における、フレームワークの概念とそのフレームワークの中でもよく活用される Spring Boot の基本概念について学習します。

続いて、Spring Boot のプロジェクトの作成方法について学習します。Spring Boot は、プロジェクトを作成する時、しっかり設定をすることで、元々はややこしい環境設定を簡単にできるようになるので、しっかり習得する必要があります。

そして、フレームワークの環境設定で欠かせないビルドツールである、Maven についてもご紹介します。Maven を学ぶことにより、ライブラリの導入と、ビルドを効率化することができます。

1.1. フレームワークとは



フレームワークとは

フレームワークは、システム開発の世界でよく使われるツールの一種です。このフレームワークを使うと、**特定の種類のシステムをより速く、効率的に作ることができます。**

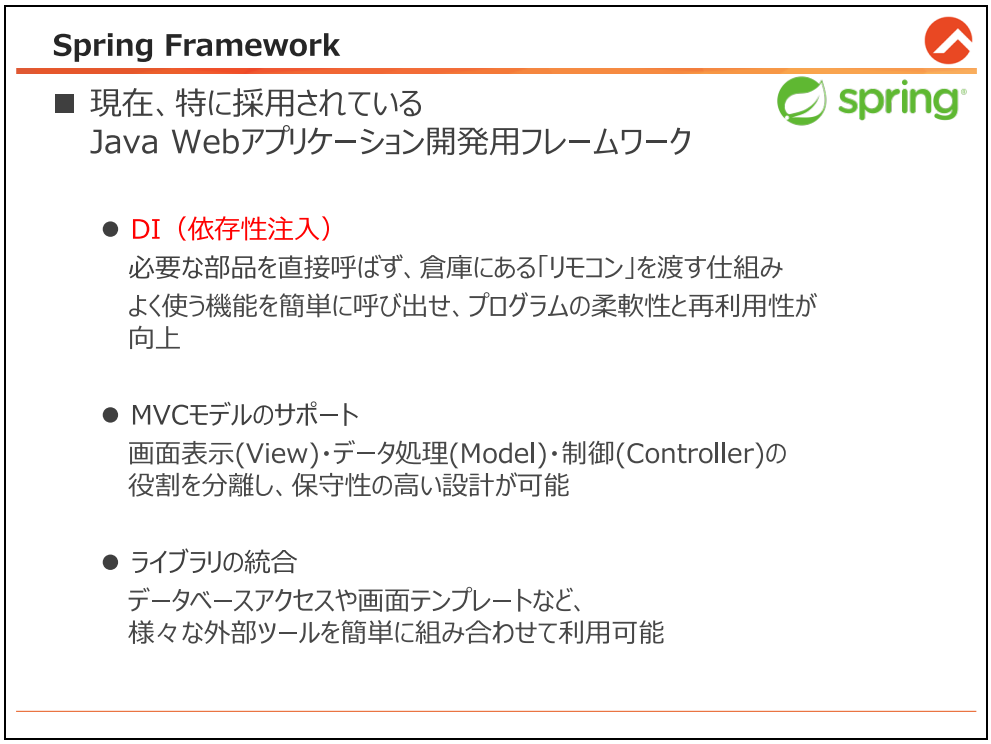
例えば、Web アプリケーションを作る時、画面を切り替えたり、ユーザのリクエストを受け取ったり、データを一時的に保持したりすることがよくあります。これらは、どんな Web アプリケーションにも必要な基本的な機能です。

以前は、サーブレットという技術を使ってこれらの機能を実装していましたが、ちょっと複雑で時間がかかる作業でした。ですが、フレームワークを使うと、これらの作業を簡単なコードやキーワードで実装できるようになります。つまり、フレームワークは開発者の作業効率を高めてくれます。

フレームワークの注意点

フレームワークは便利ですが、フレームワークによって**定められたルールや方法に従って作業する必要**があります。最初は、これらのルールに慣れることに集中しましょう。また、エラーが発生した時の対処方法も、フレームワークを使いこなす上で大切です。エラーメッセージを見て、何が問題なのかを理解し、解決する力を身につけましょう。もし、問題が発生した場合は、本書付録の「よく起きるエラーまとめ」を参考にしてください。

1.2. Spring Framework



Java Web アプリケーションの歴史と Spring Framework

「Spring Framework」、通称「Spring」は、Java 言語を使用してエンタープライズアプリケーション全体を開発するためのフレームワークです。Web アプリケーションの開発もその中の一部でした。

1995 年に Java が公開され、その後の 1990 年代後半には、サーブレットと JSP を活用した Web アプリケーション開発の時代が始まりました。サーブレット&JSP は Java Web アプリケーションの開発の基盤となりましたが、これらの技術だけでは複雑なアプリケーションの開発が難しく、冗長な作業が増えることが課題となりました。

2000 年代に入ると、この課題を解決するためのフレームワークがいくつか登場しました。Struts は Apache Software Foundation から公開され、初期の頃の主要な MVC フレームワークとして広く採用されました。一方、2002 年には Spring Framework が Rod Johnson 氏によって初めて公開されました。Seasar2 などの他のフレームワークもこの時期に登場しました。

2010 年代に入ると、多くの Java フレームワークの中でも Spring はその柔軟性、拡張性、そして豊富なサブプロジェクト（例: Spring Boot, Spring Data）のおかげで非常に人気があり、多くの企業やプロジェクトで採用されるようになりました。その結果、多くの Java のフレームワークの中で、**Spring は特に注目される存在**となりました。

Spring の特徴

1. DI(依存性注入)

Spring では、「 DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスから B クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンを A クラスに渡す。というイメージに近いです。
A クラスは渡されたリモコンの「 スイッチ(メソッド) 」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring は MVC(Model-View-Controller) モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示(View) 、データの処理(Model) 、そしてユーザの入力と画面表示の制御(Controller) といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章から MVC パターンを活用した実装の仕方を学習します。

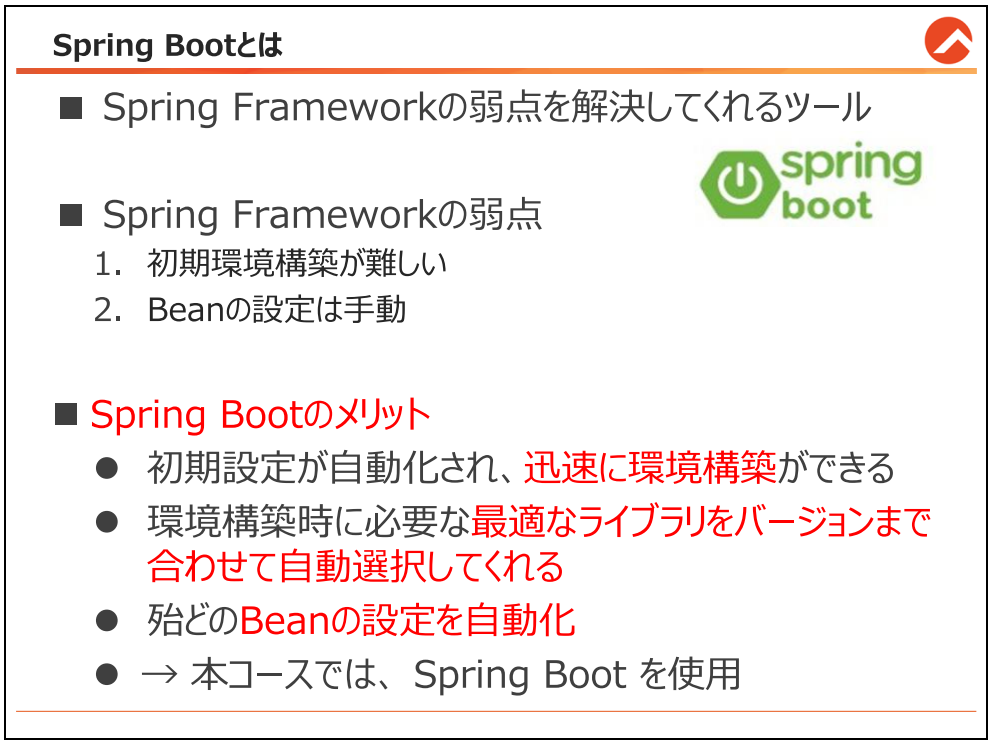
3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「 Thymeleaf」、データベースアクセスに「 Hibernate」、ログ出力に「 Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring は Java の Web アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年 11 月リリース)を利用します。

1.3. Spring Boot とは



Spring Boot とは

Spring Framework は機能が豊富なフレームワークですが、アプリケーションの開発を始める際には、**環境構築の手順が複雑で難しい**というデメリットがありました。

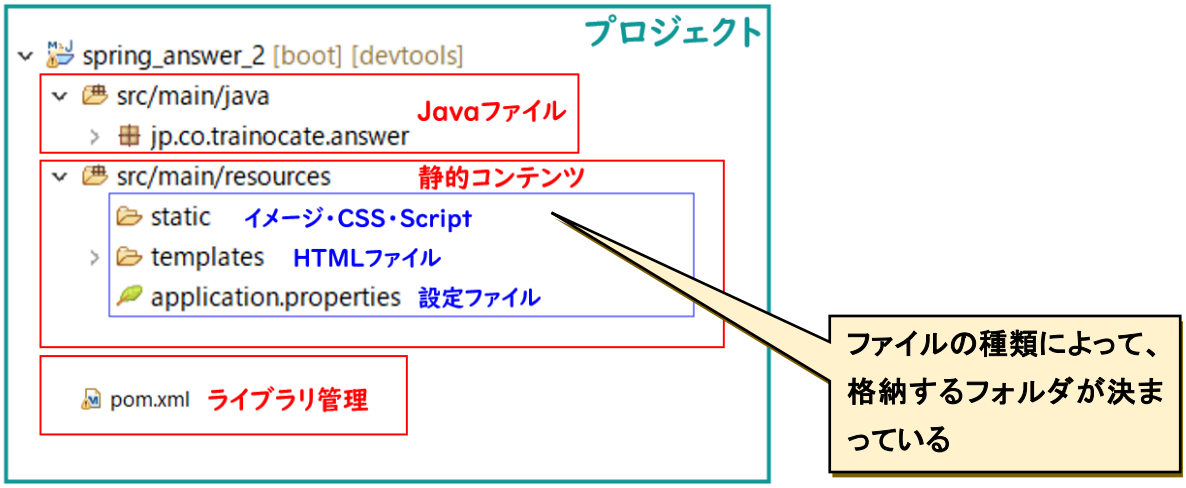
この課題を解決するために、Spring Boot が開発されました。当コースでは、Spring Boot を使用し、研修を進めます。Spring Boot は、Spring Framework をより簡単に使えるようにするツールのようなものです。

Spring Boot は、開発者が簡単に Web アプリケーションを作成できるように設計されています。以下にその主なメリットを紹介します。

1. セットアップが簡単

Spring Boot は、多くの設定を自動で行ってくれるため、開発を始めるための準備が簡単です。そのおかげで、プログラミングの初心者でも迅速にプロジェクトを立ち上げることができます。Spring Boot によって作成されたプロジェクトは、自動で以下のようなフォルダ構成になります。

＜Spring Boot プロジェクトの構成＞



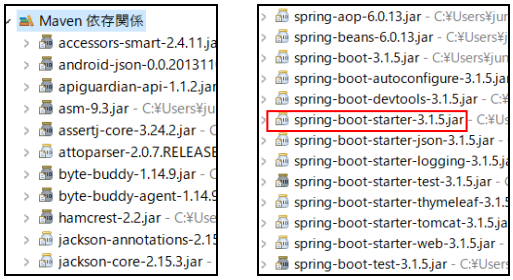
Spring Boot では、ファイルの種類によって、決まったフォルダに格納しないと、正常動作しません。spring_answer_2 プロジェクトを展開して、ファイルの構成を確認してください。

2. 依存ライブラリの簡単な管理

Spring Boot の次のメリットは、ライブラリのバージョン管理を自動で行ってくれることです。

通常、複数のライブラリを組み合わせて開発する場合、「どのライブラリの、どのバージョン同士なら競合せずに動くか」を調整するのは非常に手間がかかります。しかし Spring Boot では、**初期環境構築時にベースとなる「Spring Boot のバージョン」を決めるだけで、それに最適な関連ライブラリのバージョンを自動で選択してくれます。**

以下の画像は、Maven により必要なライブラリが自動導入されている様子です。開発者はバージョンの相性に悩まされることなく、スムーズに開発を始めることができます



3. Bean の自動設定

Spring Framework では、アプリケーションの多くの機能が**「Bean」**と呼ばれる特別なオブジェクトとして管理されます。Bean は、先ほど DI（依存性の注入）で例えた「予め倉庫（DI コンテナ）に準備されている、リモコン本体」のことです。

通常、この Bean を倉庫（DI コンテナ）に登録・管理するためには、以下のように手動で一つ一つ設定を書く必要があります。

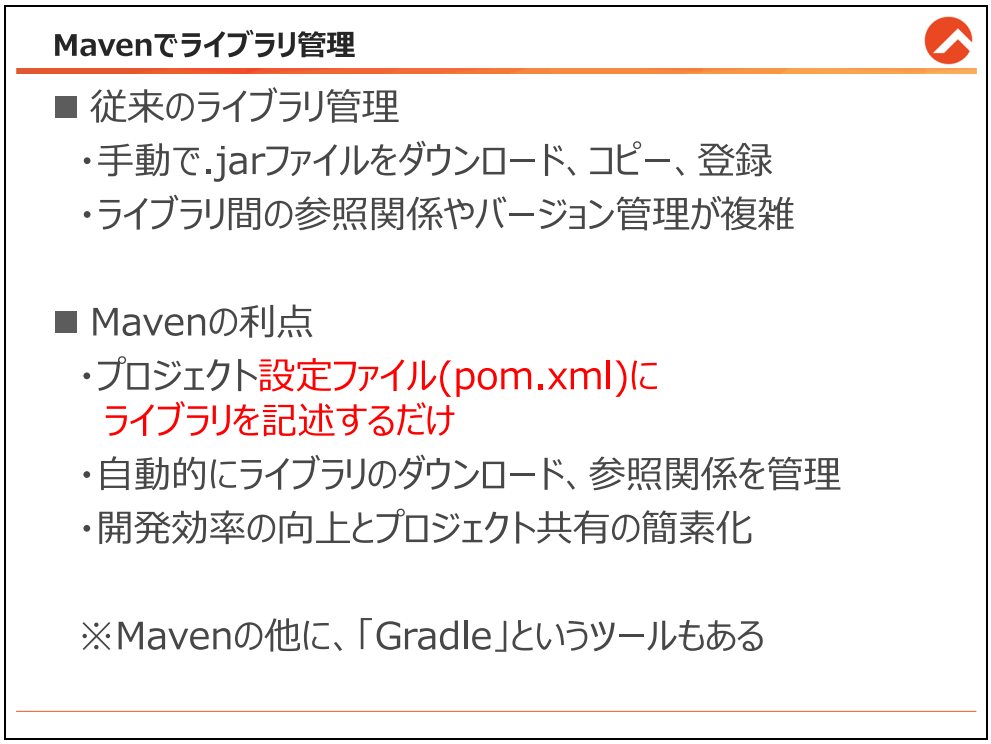
```
@Configuration
public class AppConfig {
  //myService を Bean に登録すると便利だが、登録に手間がかかる
        @Bean
        public MyServiceImpl myService() {
                return new MyServiceImpl();
        }
}
```

ただし、**Bean の数が多くなると、それぞれの設定が複雑になり、管理が難しくなります。**
Spring Boot は、多くの一般的な Bean を、最初から自動で DI コンテナに準備・設定してくれます。

開発者が手動で Bean を設定する手間が大幅に減るため、アプリケーションの本来の目的（ビジネスロジック）の実装により集中できるようになります。

このように、Spring Boot は Spring Framework の機能を拡張し、開発プロセスを簡略化して、より効率的にするための大変便利なツールとなっています。

1.4. Maven でライブラリ管理



Maven でライブラリ管理

Spring Boot では、Maven などのビルドツールを使ってライブラリの管理をするのが一般的です。**Maven を使うと、従来は複雑なライブラリの管理を簡単に行うことができる**ようになります。

Java Web プロジェクトを開発する際、データベースの接続やその他の多くの機能を実装するために、外部ライブラリの利用はほぼ必須です。しかし、従来の方法では以下のような手順が必要でした。

【 参考】従来のライブラリ導入方法

1. インターネットから適切な.jar ファイル(例：mysql-connector-j.jar)を探し出し、ダウンロードする。
2. ダウンロードした.jar ファイルをプロジェクトの特定のフォルダにコピーする。
3. プロジェクトのビルドパスにこれらの.jar ファイルを登録する。

手動でのライブラリ管理は、時間がかかるだけでなく、ライブラリ間の参照関係やバージョンの整合性を手動で管理する必要があり、非常に面倒です。さらに、プロジェクトを他の開発者と共有する際も、全てのライブラリファイルを含める必要があるため、非効率です。

「Maven」や「Gradle」といったビルドツールを利用することで、問題を解決できます。これらのツールを使うと、**必要なライブラリをプロジェクトの設定ファイルに記述するだけで、ツールが自動的にライブラリをダウンロードし、参照関係を管理**してくれます。このおかげで、プロジェクトの設定と共有が簡単になり、開発効率が大幅に向上します。

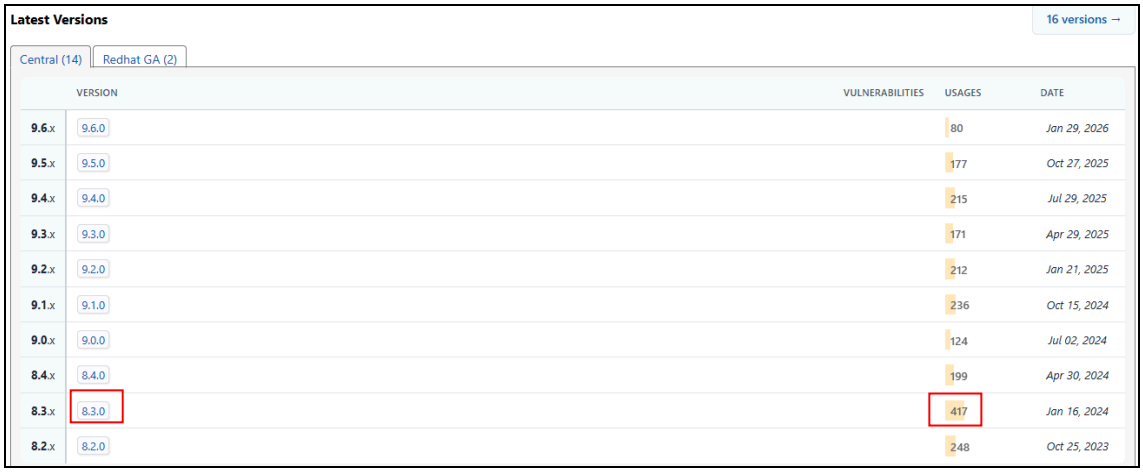
Maven を利用し、ライブラリを導入する手順

1. インターネットから、導入したいライブラリ名を検索し、[Maven Repository]に接続する。

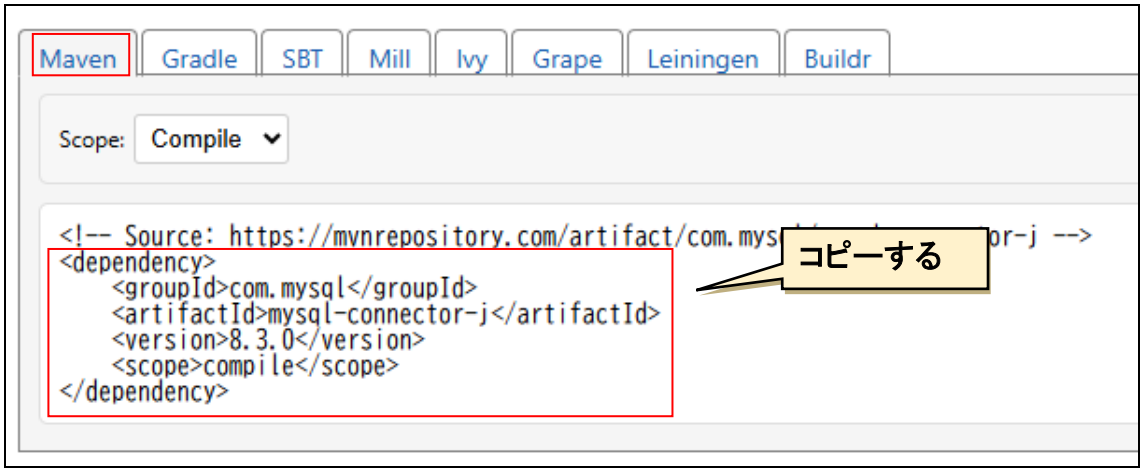
   (今回の例では、MySQL Connector J と検索しました。)

   (https://mvnrepository.com/artifact/com.mysql/mysql-connector-j)

2. 導入したいバージョン名をクリックする。
   (よくわからない場合は、Usages が一番多い方を選ぶ)

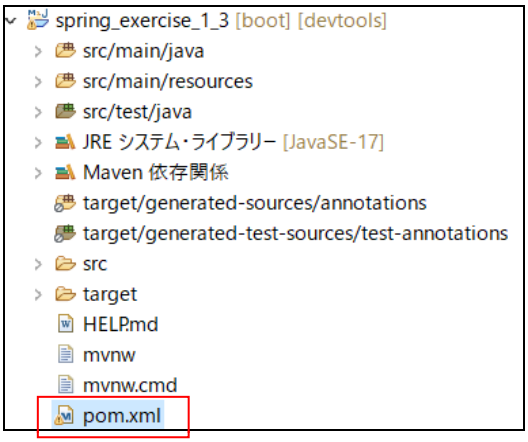


3. 利用するビルドツールに合わせて、タブを切り替えする。表示されたテキストをコピーしておく。




```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
    <scope>compile</scope>
</dependency>
```

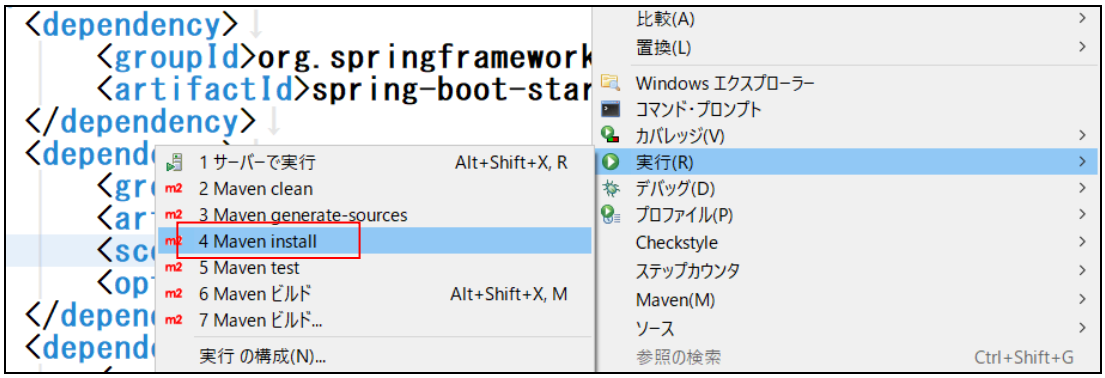
4. Eclipse 上の[spring_exercise_1_3]プロジェクトをダブルクリックし、[pom.xml]ファイルをダブルクリックする。



5. **<dependencies>と、</dependencies>の間の任意の場所**に、手順 3 でコピーしたテキストを貼り付けて、保存する。

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- この場所に、コピーしたテキストを貼り付けしてください -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.3.0</version>
            <scope>compile</scope>
        </dependency>
    </dependencies>
```

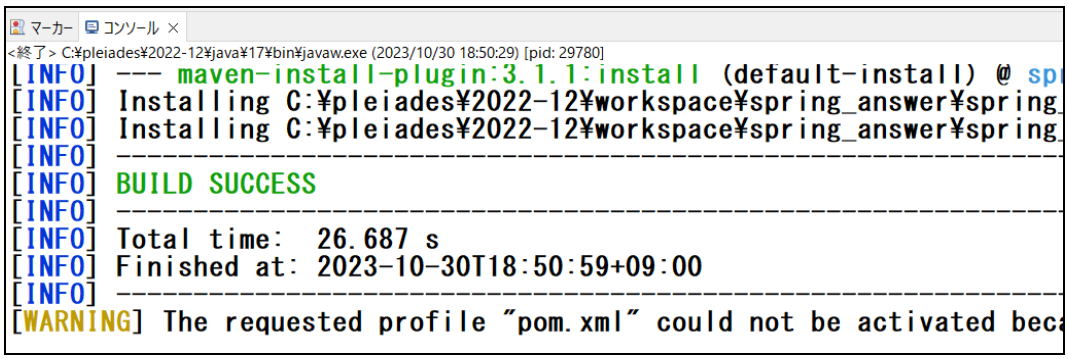
6. pom.xml の任意の場所を右クリック→[実行]→[Maven install]をクリックすると、ライブラリの導入が完了する。



7. コンソール上で、ダウンロードとインストールのログが流れる。

   以下のように、[BUILD SUCCESS]が表示されると、完了。

   ※[FAILED]になった際は、誤字を確認して修正後、再度試す。



Maven によるライブラリ管理の利点

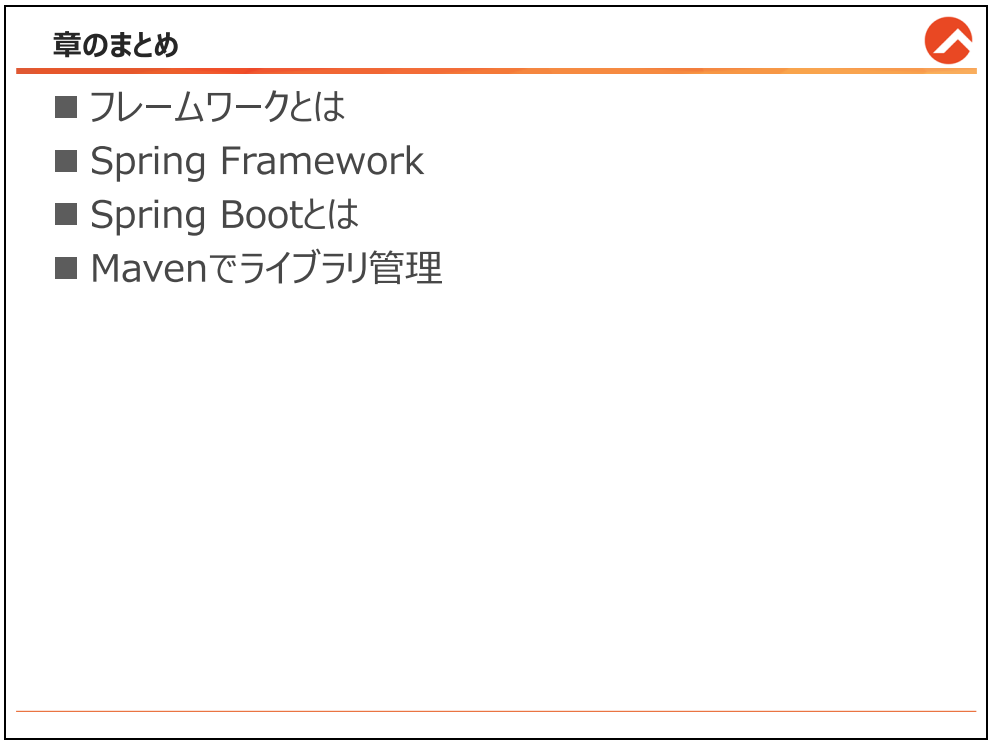
以上が、Maven を利用したライブラリの適用方法です。Maven を使用する最大のメリットは、「プロジェクトを他の人と共有するのが非常に簡単になること」です。

通常、作成したプロジェクトを第三者に渡す際、使用している重いライブラリ(部品)のファイルを一つ一つセットにして渡す必要があります。しかし Maven を使えば、ライブラリの「お買い物リスト」である pom.xml ファイルだけを渡せば済みます。

プロジェクトを受け取った人が maven install を実行すると、リストに書かれた「全く同じバージョンのライブラリ」がインターネットから自動的にダウンロードされ、インストールされます。

これにより、「A さんのパソコンでは動くのに、B さんのパソコンでは動かない」といった環境のズレを防ぎ、チーム全員が同じ環境で効率よく開発を進めることができます。

章のまとめ



この章では、フレームワークの概念とその必要性を理解しました。特に、Java の開発で広く利用されている Spring Framework の特徴と強みについて学びました。また、Spring Boot の導入によって、Spring Framework の開発をより手軽に始められる方法について説明しました。最後に、Maven を使った効率的なライブラリ管理の方法を学び、プロジェクトの構築と管理がいかに簡単になるかを理解できました。

これで Spring Boot での Web アプリケーション開発への第一歩を踏み出す準備が整いましたので、次の章から本格的に Spring Boot を使った Web アプリケーションの開発に挑戦しましょう。

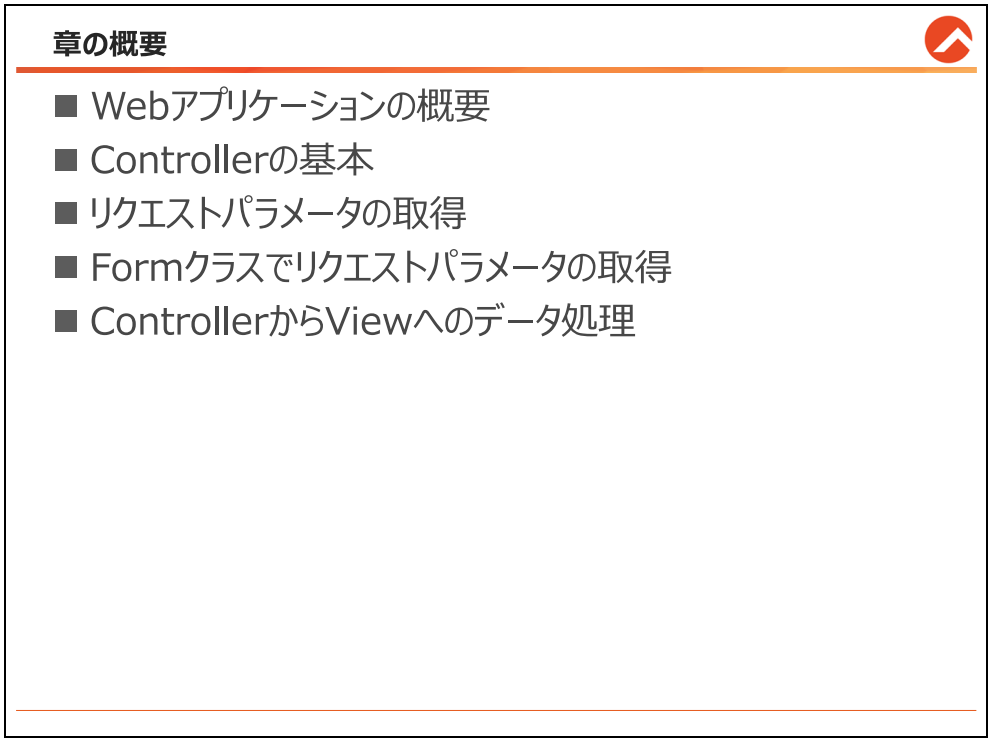
2

第2章 Spring Web アプリケーション

章の目標

- Web アプリケーションの基本的な流れを理解できる
- Controller を作成し、利用できる
- View から Controller へのリクエストパラメータを取得できる
- Form クラスを使用してリクエストパラメータを取得できる
- Model を通じて Controller から View にデータを表示できる

章の概要

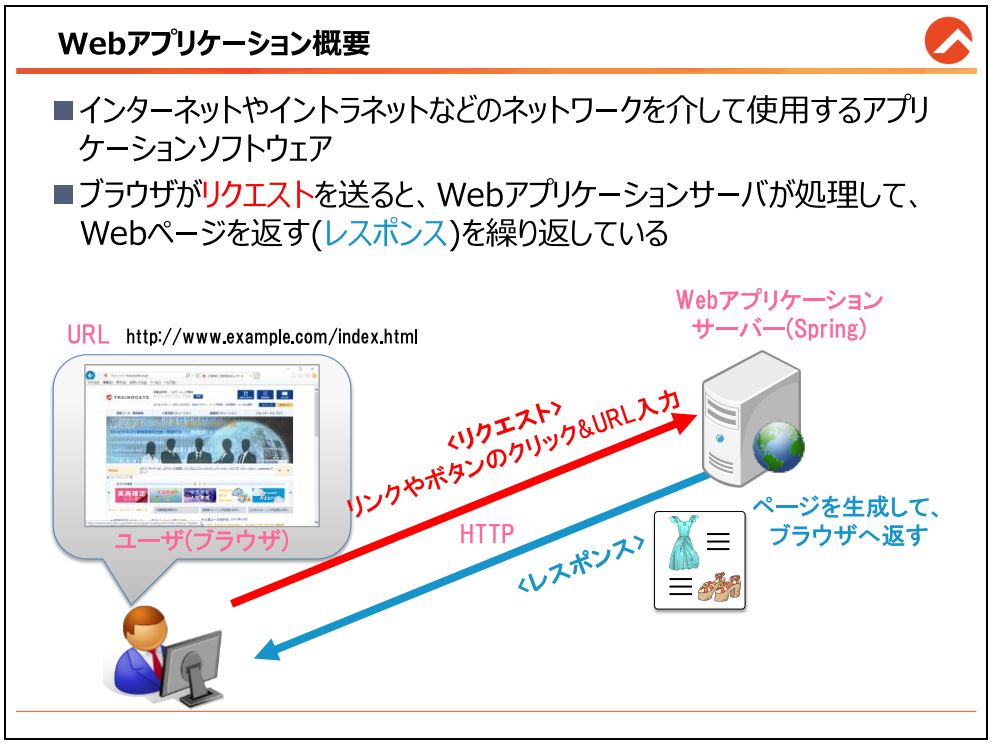


この章では、インターネットを介したアプリケーションの基本的な概念である Web アプリケーションについて学びます。

まず、Web アプリケーションの基本的な構造と機能について理解を深め、次に、Spring Framework を使用した Web アプリケーション開発における Controller の基本について学習します。その後、Web アプリケーションにおけるリクエストパラメータの受け渡しについて学びます。最後に、「Thymeleaf」というテンプレートエンジンを使って、サーバー側で処理されたデータを Web ページにどのように表示するかについても学びます。

この章を通して、Web アプリケーションの基礎的な概念と Spring Framework を使った Web アプリケーション開発の基本を理解し、実際に Web アプリケーションを構築するための基本的なスキルを身につけてみましょう。

2.1. Web アプリケーションの概要



Web アプリケーションとは？

Spring で Web アプリケーションの作成を学ぶ前に、まずは Web アプリケーションについてご説明します。

Web アプリケーションは、インターネットやイントラネットを介して使用されるソフトウェアです。日常的に使っているウェブサイト(例えば、ニュースサイトやオンラインショッピング、SNS など) は、実はすべて Web アプリケーションの一部です。

Web アプリケーションは、ブログや記事のような Web ページと異なり、ユーザの入力に応じて動的にコンテンツを変更したり、データベースと連携して情報を更新したりすることができます。

リクエストとレスポンスのプロセス

Web アプリケーションの基本的な動作は、**「リクエスト」**と**「レスポンス」**のやり取りによって成り立っています。ここでの「リクエスト」とは、皆様がブラウザを通じてサーバーに対して行う要求のことで、例えば**特定のページを表示**したい、あるいはフォームに**入力したデータを送信**したいといったことが含まれます。

一方で、「レスポンス」とはサーバーがリクエストに応じてブラウザに返す応答のことを指します。これは通常、HTML 形式の Web ページとして表示され、ユーザに情報が送られます。

例えば、オンラインの服のショッピングサイトを訪れて、特定のブランドの服や靴を探しているとします。検索ボックスに「夏のファッション」と入力して検索ボタンをクリックします。これが「リクエスト」です。

このリクエストは、Web アプリケーションサーバーに送信され、Web アプリケーションはユーザが入力したブランド名に基づいて商品のデータベースを検索します。検索結果として、そのブランドの商品一覧が見つかります。この商品一覧が Web ページとして整形され、ユーザのブラウザに送り返されます。これが「レスポンス」です。

その結果、のブラウザ上には求めていたブランドの商品一覧が表示されます。この一連のプロセスが、リクエストとレスポンスによって成り立っているのです。

Spring での Web アプリケーション開発

Spring Framework を利用すると、このリクエストとレスポンスのプロセスを簡単かつ効率的に扱うことが可能になります。Spring は記載した Java コードを基にして、ブラウザからのリクエストに適切に応答し、必要な処理を行い、適切なレスポンスを生成します。

この章では、Spring を使った基本的な Web アプリケーションの構築方法について学びます。それでは、Spring を使った Web アプリケーション開発の魅力的な世界へ一緒に踏み込んでいきましょう。

2.1.1.【 参考】サーブレットについて

【参考】サーブレットについて

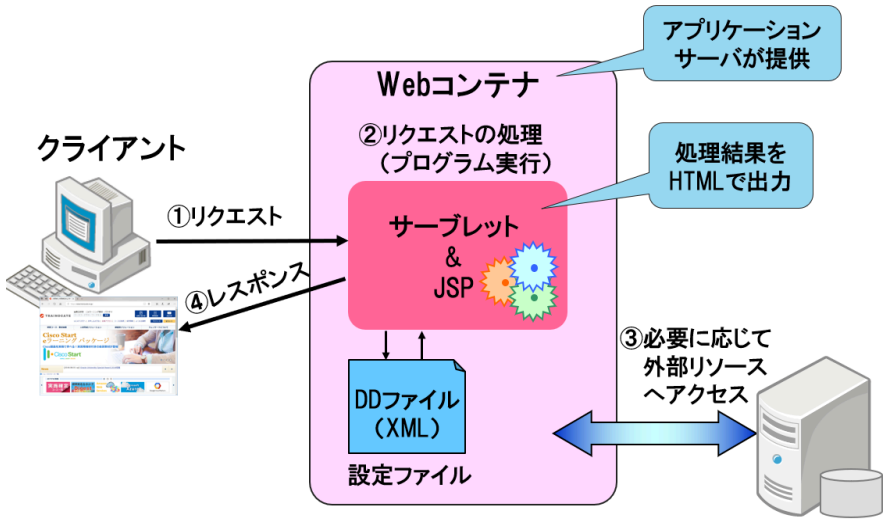
■ サーブレット
JavaのWeb開発でリクエスト受け取りとレスポンス送信を担当

■ 形
普通のJavaクラスに、Web関連のクラスを多数継承した形

■ 課題
Web関連機能を実装するためのコードの量が多く、冗長

■ Springでは？
サーブレットに代わりControllerクラスを採用。効率的な実装が可能

【 参考】サーブレットについて



サーブレットは、Java における Web アプリケーション開発の基礎を提供する技術の一つです。サーブレットは Web サーバー上で動作し、クライアントからのリクエストを受け取り、適切なレスポンスを生成して返す役割を持つクラスです。

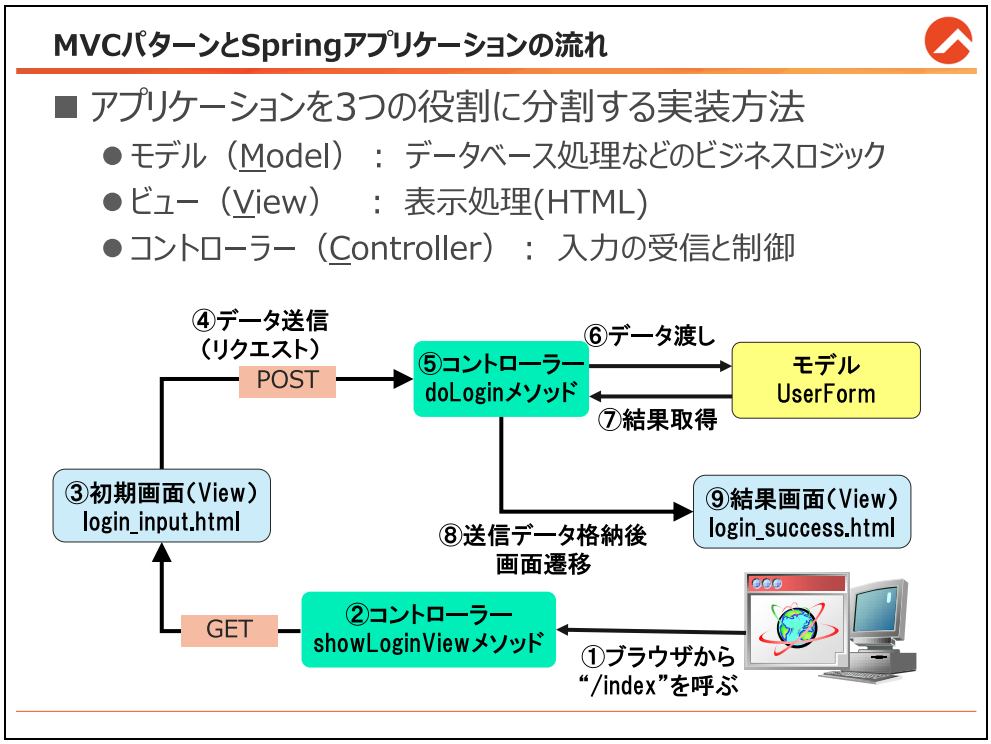
【参考】サーブレットの基本

```
 1:import java.io.IOException;
 2:
 3:import javax.servlet.RequestDispatcher;
 4:import javax.servlet.ServletException;
 5:import javax.servlet.annotation.WebServlet;
 6:import javax.servlet.http.HttpServlet;
 7:import javax.servlet.http.HttpServletRequest;
 8:import javax.servlet.http.HttpServletResponse;
 9:
10:@WebServlet(urlPatterns = "/index") //URL マッピング
11:public class Sample1 extends HttpServlet { //HttpServlet の継承必要
12:
13:     //メソッド単位の URL マッピング不可
14:     protected void doGet(HttpServletRequest request,
15:             HttpServletResponse response)
16:                     throws ServletException, IOException {
17:
18:             //index.jsp へ遷移のための長いコード
19:             RequestDispatcher dispatcher =
20:
        request.getRequestDispatcher("/index.jsp");
21:             dispatcher.forward(request, response);
22:     }
23:}
```

サーブレットの開発では、リクエストの処理やレスポンスの生成のために、比較的冗長なコードを書く必要がありました。サーブレットは Web アプリケーションの中でコントローラーとしての役割を果たし、クライアントとサーバー間のデータのやり取りを管理します。

一方、Spring Framework では、**サーブレットクラスの代わりに**、よりシンプルで効率的な **Controller クラス**を提供しています。Spring の Controller クラスは、サーブレットの機能を簡潔に実装するためのアノテーションやツールを提供し、開発者が Web アプリケーションのロジックにより集中できるように設計されています。

2.1.2.【参考】MVC パターンと Spring アプリケーションの流れ



MVC パターンについて

この章では、複数の Java メソッドと、HTML ファイルを作成します。そして、「MVC パターン」と呼ばれる開発手法で実装を進めていきます。

MVC パターンでは、Model・View・Controller で役割を分割します。

このパターンは、明確に役割が分かれていて、保守性が高いプログラムを作ることができます。サーブレット/JSP の開発の際も多く使われていて、Spring では基本的に MVC パターンに基づいた実装する前提で設計されました。

Spring アプリケーションの MVC パターン

上の図には、Spring アプリケーションの典型的な流れが表現されています。

2 章で扱うファイル名も載ってありますので、**2 章の途中で必要であれば、随時このページに戻って、現在どこを説明しているかをご確認ください。**

それでは、次のページでコントローラーについてご説明します。

2.2. Controller の基本

Controllerの基本

```
@Controller
public class IndexController {
    @GetMapping("/index")
    public String index() {
        return "index";
```

■ Controllerクラス
・サーブレットのように、ユーザーからのリクエストを
受け取り、適切なレスポンスを生成するクラス

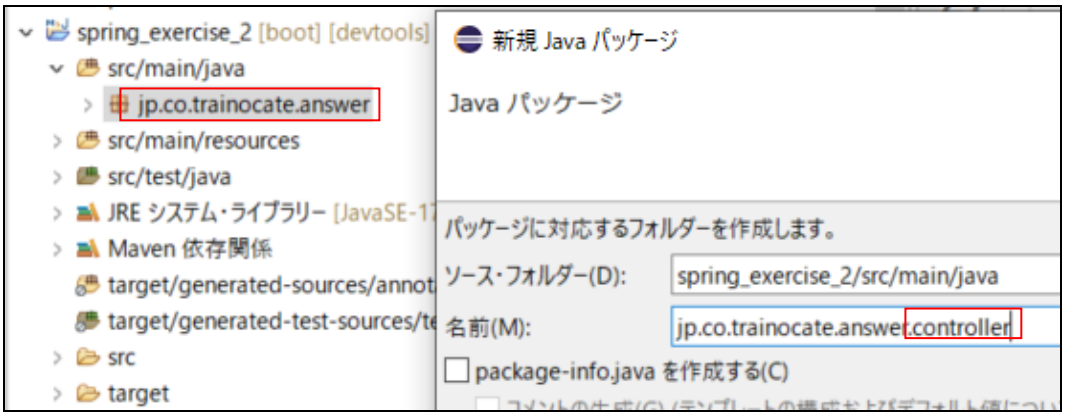
■ Controllerクラスの基本構文
・クラス名に@Controller アノテーションをつける
・メソッド名に@GetMappingをつけて、
URLとクラスのメソッドを関連付ける。
・returnには、表示したいhtmlファイル名を記載

■ ビューの作成
・Thymeleafというテンプレートエンジンを利用する

【 参考】controller パッケージの作成

Spring でクラスを作成する際は、クラスに役割によって、異なるパッケージの中に作成をします。今からは、コントローラーの役割をする、IndexController クラスを作成する予定ですので、[controller]というパッケージを作成します。

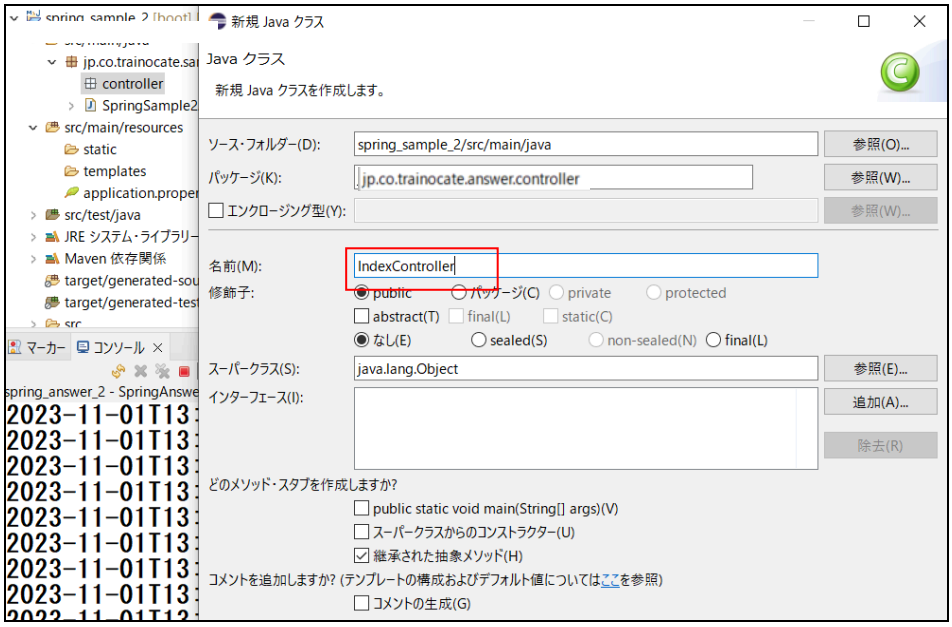
パッケージの作成方法は、プロジェクトを展開し、src/main/java のパッケージを右クリック→[新規]→[パッケージ]→[名前]フォームに、**既存のパスを消さずに、[.controller]を追加すると作成されます。**



【参考】Controller クラス作成

Controller クラスを作成する際は、controller パッケージに、main メソッドを持たないクラスを作成します。

作成された[controller]パッケージを右クリック→[新規]→[クラス]→[名前]に「IndexController」と入力し、[完了]をクリック



【 作業手順①】Controller クラスの作成

…/controller/IndexController.java

```
 1:package jp.co.trainocate.answer.controller;
 2:
 3:import org.springframework.stereotype.Controller;
 4:import org.springframework.web.bind.annotation.GetMapping;
 5:
 6:@Controller
 7:public class IndexController {
 8:     @GetMapping("/index")
 9:     public String index() {
10:             return "index";
11:     }
12:}
```

上記の IndexController が、一番基本となる Controller の形です。前の節で確認した、サーブレットと比べて、とても簡略で短いコードになっています。

@Controller アノテーション

サーブレットで、Controller の役割をするためには、予め Web 関連のクラスを継承するなど、冗長なコードを書かないといけませんでしたが、Spring では、**クラス名の上に、@Controller をつけるだけ**で、Controller としての役割を付与することができます。

```
6 @Controller
7 public class IndexController {
```

サーブレットのように、HttpServlet を継承して、HttpServletRequest と HttpServletResponse を設定する必要はありません。

リクエストマッピング

Controllerクラスには、プログラムの開始点となるmainメソッドはありません。では、Controllerのメソッドはいつどのように実行されるのでしょうか？

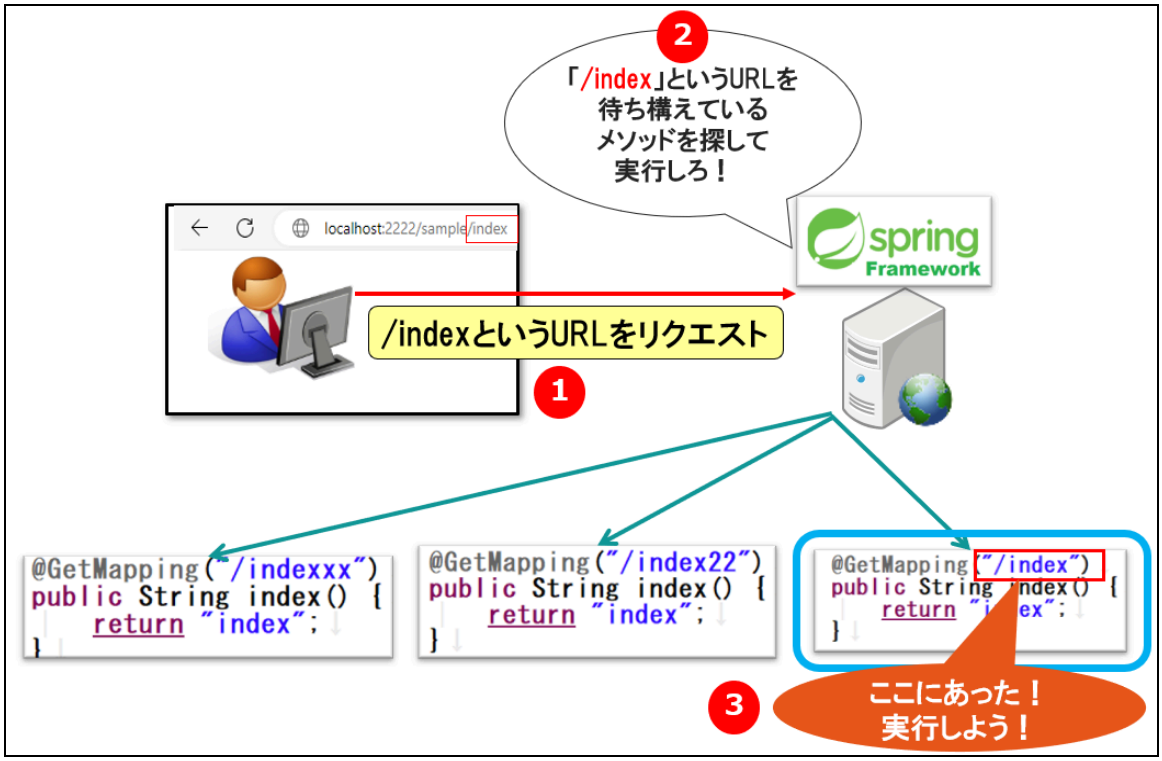
実は、Controller のメソッドは「**リクエストが来るまで待機**」しています。リクエストは、ユーザが Web ページ上でリンクやボタンをクリックしたときに URL として送信されます。

このリクエストは Spring 環境のアプリケーションサーバーに届きます。サーバーは、登録されている全ての Controller を調べ、リクエストされた URL に対応するメソッドを探します。

URL を対応させる方法は、後述する「@GetMapping」アノテーションを、メソッド名の上につけるだけです。

```
8    @GetMapping("/index")
9    public String index() {
10       return "index";
11   }
```

対応する URL を見つけると、その URL にマッピングされた Controller のメソッドが実行されます。今回の例では、ユーザーがブラウザで **http://ドメイン/index** にアクセスすると、このメソッドが呼び出されます。



【参考】サーブレットで URL を対応させるコード

```
@WebServlet(urlPatterns = "/index")
public class Sample1 extends HttpServlet{
```

サーブレットの@WebServlet(urlPatterns)アノテーションと似たような役割をすると覚えると良いでしょう。ただ、サーブレットの URL 対応と決定的な異なる部分は、**サーブレットではクラス単位**でしたが、**Spring の@GetMapping では、メソッド単位で URL をマッピング**することができるということです。サーブレットでは、機能ごと 1 つずつのクラスを作らないといけませんでしたが、Spring では 1 つのクラス内に、複数の機能を実装できるので非常に便利です。

index メソッドの解説

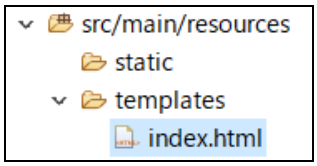
```
8    @GetMapping("/index")
9    public String index() {
10       return "index";
11   }
```

・9 行 public String index()

public は、このメソッドが外部からアクセス可能であることを意味します。String は、メソッドが文字列を返すことを示しています。index() はメソッド名で、このメソッドが何をするのかを表します。**Controller の中のメソッドは、通常は戻り値を String にします。**

・10 行 return “index”;

return には文字列を指定します。通常、この文字列は表示させたい html ファイル名を指定します。



Spring Boot では、ビュー(html)の名前を単純に return 文で返すだけで、適切な HTML 画面が表示されます。つまり、この場合は「index.html」という名前のページがユーザーに表示されます。**.html は省略しないといけません。**

【参考】サーブレットで index.jsp を表示するためには

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/index.jsp");
dispatcher.forward(request, response);
```

今回の return は、サーブレットの request.getRequestDispatcher と似た役割をするコードだと覚えると良いでしょう。

Spring の return は、正確には forward をしているのではなく、「指定した HTML ページを呼び出す」に近いですが、MVC パターンでの画面出力という観点では似ています。

Spring Boot の方がより簡潔で直感的なコードを書くことが可能になり、開発者の負担が軽減されます。

【作業手順②】ビューの作成と構成

Web アプリケーションにおいて、ユーザが見る画面を形作るファイルを「ビュー(View)」と称します。View は、HTML や JSP ファイルで構成されていることが一般的です。本書では、Java の Web アプリケーション開発で広く用いられる「Thymeleaf(タイムリーフ)」というテンプレートエンジンを利用します。

テンプレートエンジンとは、HTML ファイルに数値や文字列などを動的に埋め込むことができる特別なライブラリのことです。このテンプレートエンジンを使用することにより、ユーザの名前をページに表示したり、商品一覧をリストアップしたりするような動的な処理が可能になります。

では、Controller の index メソッドから遷移するビューをどのように作成するかをご紹介します。

…/resources/templates/index.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org">
 3:<head>
 4:     <meta charset="UTF-8" />
 5:     <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
 6:     <title>2 章 サンプル 1</title>
 7:</head>
 8:
 9:<body>
10:     <div>トップページ</div>
11:</body>
12:</html>
```

このビューの特徴として、2 行目に注目してください。ここでは、Spring Boot と相性が良いテンプレートエンジン「Thymeleaf」を使用するという宣言をしています。
xmlns:th を記載することで、HTML ファイル内で Thymeleaf の機能が利用可能になります。

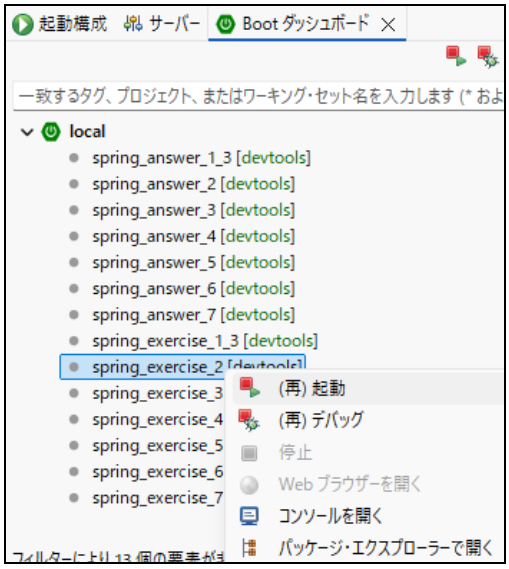
[トップページ]と表示される部分(10 行目)からは、ユーザに見える画面の内容が入ります。今後、さまざまな情報やデザイン要素を追加していくことになります。

【 作業手順③】Spring Boot アプリケーションの実行

IndexController の index メソッドをブラウザからリクエストすることで、実行してみましょう。以下の手順に従ってください。

1. IndexController に誤字がないか確認し、保存します。
2. Eclipse の上段メニューから、緑の「 ⏻ 」ボタンをクリックします。

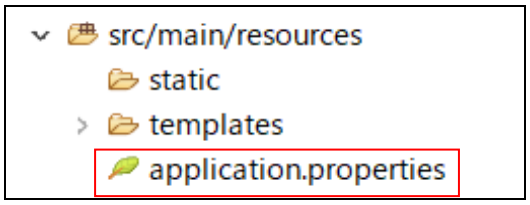
3. local の部分をダブルクリックし、展開します。
4. [spring_exercise_2]プロジェクトを右クリックし、**[(再)起動]を選択**します。



※ポップアップが出る際は、全て「 許可」します。

5. 任意のブラウザ(Chrome、Edge など)のアドレスバーに以下の URL を入力します
   http://localhost:8080/index
6. 以下のように、「 トップページ」と表示されたら、実行成功です。

【参考】application.properties ファイルについて




```
application.properties
1 server.servlet.context-path=/
2 server.port=8080
3
4 #サーバーを再起動しなくても、HTMLの修正が反映(ホットデプロイ)
5 spring.thymeleaf.cache=false
6
7 #その他データベース関連のコードは、3章で追記します
```

Spring Boot アプリケーションでは、application.properties ファイルが非常に重要です。このファイルは、アプリケーションの設定情報を保持し、アプリケーションの振る舞いをカスタマイズするために使用されます。application.properties ファイルは、プロジェクトの **src/main/resources ディレクトリ内に配置**されます。

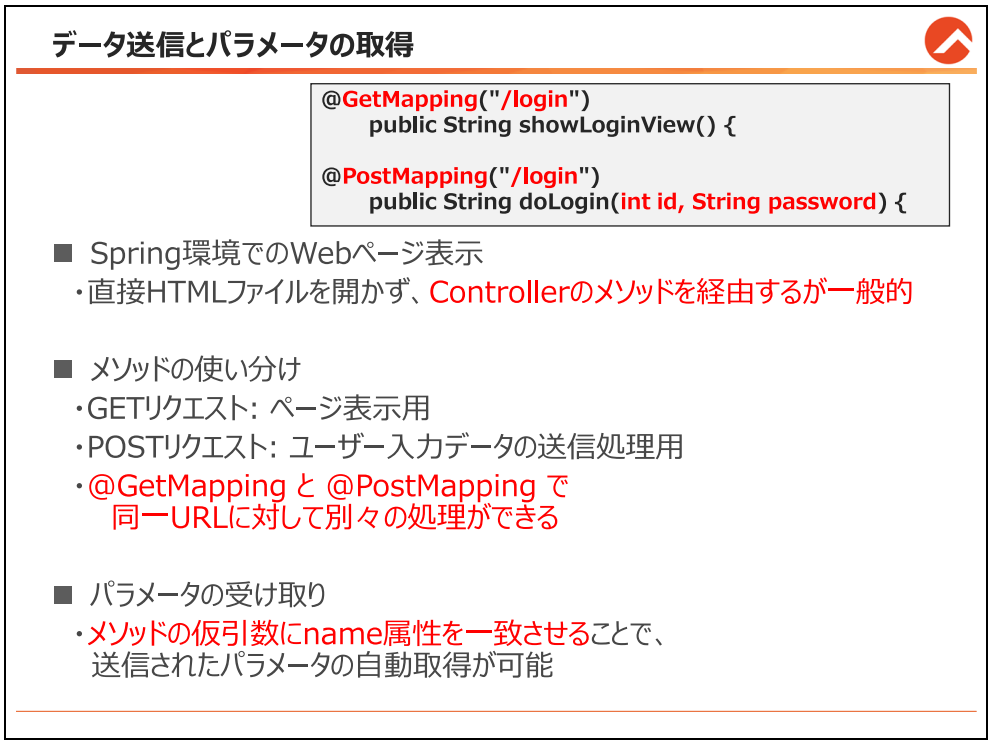
このファイルを修正すると、データベースの接続情報、サーバーのポート番号、コンテキストパスなど、さまざまな設定を行うことができます。特に開発時には、アプリケーションの動作環境に合わせて、これらの設定を調整することが一般的です。

以下は、application.properties ファイルに記載される可能性のある設定の例です。

- server.servlet.context-path=/sample
  この設定により、アプリケーションは /sample というコンテキストパスでアクセスされます。つまり、アプリケーションの URL の一部として /sample が含まれるようになります。
- server.port=2222
  この設定により、アプリケーションはポート番号 2222 でリッスンします。デフォルトのポート番号 8080 ではなく、別の番号を使用することで、他のアプリケーションとの衝突を避けることができます。
- spring.thymeleaf.cache=false
  この設定により、Thymeleaf テンプレートのキャッシュが無効になります。開発中には、テンプレートの変更が即座に反映されるようにするために便利です。

このように、application.properties ファイルを適切に設定することで、アプリケーションの振る舞いを柔軟に制御し、開発プロセスをスムーズに進めることができます。

2.3. データ送信とパラメータの取得



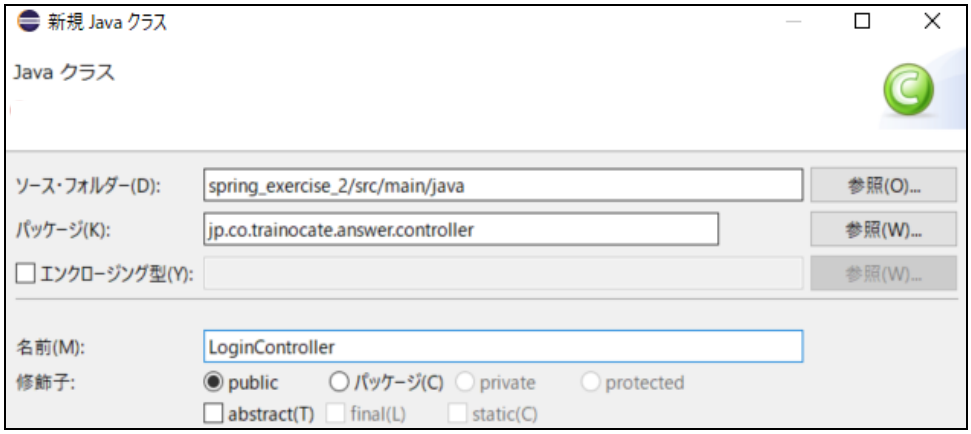
Web ページにおけるデータ送信の概要

Web ページの大きな利点の一つは、ユーザがフォームにデータを入力して送信できる機能を持っている点です。以下のログインフォームの例のように、ユーザは情報を入力し、[ログイン]ボタンをクリックすることでそのデータをアプリケーションサーバーに送信することが可能です。

サーブレットでは、直接このページを開くことができましたが、**Spring では、直接 HTML ページを開くのではなく、Controller のメソッドを経由させます。**よって、実装時は HTML ファイルより先に、画面遷移の処理のみを持つメソッドを作成する必要があります。

【 作業手順①】ログイン画面を表示するメソッドの作成

まずは、ログイン処理用の Controller クラスを作成します。Spring では、画面ごとに Controller クラスを作成するのが一般的です。



従って、controller パッケージに、LoginController クラスを作成します。

…/controller/LoginController.java

```
 1:package jp.co.trainocate.answer.controller;
 2:
 3:import org.springframework.stereotype.Controller;
 4:import org.springframework.web.bind.annotation.GetMapping;
 5:
 6:@Controller
 7:public class LoginController {
 8:
 9:    //Spring 環境では、html ページを直接より、
10:    //遷移用のメソッドを経由する方法が一般的です
11:
12:    @GetMapping("/login")
13:    public String showLoginView() {
14:            return "login_input";
15:    }
```

[showLoginView()]は、ログインフォームのビュー(HTML ページ)を表示するためのメソッドです。このメソッドは処理を何もせず、単に View 名を return しています。

/login という URL がリクエストされたら、login_input.html をユーザへ表示してくれます。

一般的に、Spring では、コントローラーのメソッドを経由して HTML ページを呼び出します。

この手法により、URL とビューとの間に明確な対応関係を持たせ、アプリケーションの構造を整

理しやすくなります。また、必要に応じて追加のデータ処理やビジネスロジックをメソッド内に組み込むことが可能です。

【 作業手順②】入力フォームがある HTML の作成

…/resources/templates/login_input.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org">
 3:<head>
 4:<meta charset="UTF-8"/>
 5:<meta name="viewport" content="width=device-width,
initial-scale=1.0"/>
 6:<title>Insert title here</title>
 7:</head>
 8:<body>
 9:<h1>ログイン画面</h1>

    <!--送信先を「 /login」 という URL に設定-->
10: <form th:action = "@{/login}" method="POST">
11:<p>社員番号   : <input type = "text" name = "id"/></p>
12:<p>パスワード : <input type = "password" name = "password"/></p>
13:<input type = "submit" value = "ログイン">
14:</form>
15:</body>
16:</html>
```

・10 行目 <form th:action = “@{/login}” method = “POST”>

<form>要素を用いてデータ送信のためのフォームを作成します。この要素の th:action 属性は、フォームデータが送信されるサーバー上の URL を指定できます。Thymeleaf では、ただの action ではなく、**th:action** を使うことで、**実行環境に変化があっても URL を正しく解決**できます。**「 @」は、アプリケーションのコンテキストパス(基本 URL)を、自動生成**してくれます。URL が異なるサーバーで実行しても、影響がなくなる効果があります。

ここでは/login がその URL として指定されています。通常、**th:action 属性の URL 先は、@Controller のメソッドを指定**します。そして、method という属性がありますが、この属性は一般的な GET 方式の送信ではなく、POST 方式の送信をするという意味です。POST 方式にすると、パスワードが URL に表示されないなどのメリットがあり、隠したいデータを送信する時によく使われます。

・11 行・12 行目 <input type = “text” name = “パラメータ名”>

<input>要素はユーザがデータを入力するためのフィールドを提供します。type=″text″を指定することでテキスト入力フィールドが、type=″password″を指定することでパスワード入力フィールドがそれぞれ生成されます。ここで、このフォームのデータは、社員番号という日本語で認識されるのではなく、**name 属性の値で認識されます。今回は、「id」と「password」という name でデータが送られるということを覚えてください。**

・13 行目 <input type = ″submit″ value = ″ログイン″>

<input type=″submit″>は送信ボタンを表します。このボタンをクリックすると、フォーム内のデータが th:action 属性で指定された URL に送信されます。value 属性はボタン上に表示されるテキストを定義します。

では、送信したデータを、アプリケーションサーバー側では、どう受け取っているのでしょうか？ **アプリケーションサーバー側では、このフォームからの送信データを受け取る準備をしておく必要があります。**

【 作業手順③】ログインデータの受信と処理

ユーザーがログインフォームに入力した情報をサーバー側で受け取るには、HTML フォームから送信されるデータを扱うためのコントローラーのメソッドを定義する必要があります。ここでは、ログインフォームのデータを処理する Spring のコントローラーメソッドの実装例を見ていきます。

…/controller/LoginController.java

```
 1://【 省略】
 2:
 3:@GetMapping("/login")
 4:public String showLoginView() {
 5:     return "login_input";
 6:}
 7:
 8://同じ「 login」 という URL でも、
 9://Get か Post かによって、異なるメソッドが実行されます
10:@PostMapping("/login")
11:public String doLogin(int id, String password) {
12:     //仮引数を記載するだけで、自動的にデータが格納されます
13:     System.out.println(id);
14:     System.out.println(password);
15:     return "index";
```

LoginController クラスの doLogin メソッドは、ログインフォームから送信されたデータを受け取るために使用されます。

・10 行目 @PostMapping("/login")

@PostMapping は、POST リクエストが送られた際に、このメソッドが実行されることを示しています。3 行でも同じ/login という URL を待ち受けていますが、普通の Get 方式での呼び出しがあった場合は、showLoginView()メソッドが実行され、Post 方式の呼び出しがあった場合は、こちらの doLogin メソッドが実行されます。

・11 行目 仮引数 int id, String password

login_input.html から送信したデータの name 属性に対応する値を受け取るために使用されています。**Spring では、Controller のメソッドの仮引数を html の name 属性の値と合わせるだけで、パラメータを受け取ることができます。**login_input.html では、「 id」と「 password」のデータが送られるので、引数名も、int id, String password に合わせます。

【 参考】サーブレットでパラメータを受け取る方法

```
// 「id」という名前で送られてくるデータを取得し、整数に変換
int id = Integer.parseInt(request.getParameter("id"));

// 「password」という名前で送られてくるデータを取得
String password = request.getParameter("password");
```

サーブレットでは、上記のような冗長なコードが必要でしたが、Spring ではメソッドの引数に変数を入れておくだけで取得できるので、とても便利だというのがわかります。

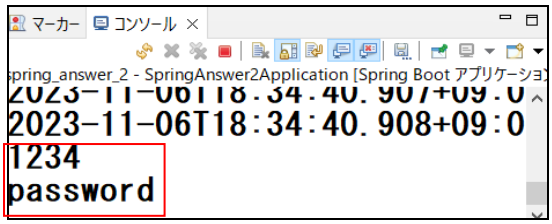
・13 行、14 行目の出力

本当に id と password のデータを取得できたのか、コンソールに出力することで、検証をしています。結果は、以下の画像のように、入力した値を問題なく取得できているのがわかります。

動作確認

http://localhost:8080/login

社員番号:1234
パスワード:password



このように、Spring ではメソッドの仮引数を入力値の型に合わせて記載するだけで、自動的にリクエストパラメータからのデータの取得をしてくれます。

そして、同じ URL に対して異なる HTTP メソッド(GET や POST)を用いることで、異なる処理を行うメソッドを分けて定義することができます。これにより、フォームの表示とフォームからのデータ受信を別々のメソッドで扱うことができます。

Spring を使った開発は、このように「決まったルール通り」命名をすれば、後ろで自動的にやってくれるので、便利です。しかし、その反面、**1 文字でもルール通り命名しないと、思い通りに動かないこともありますので、変数名やメソッド名を命名する時は、注意が必要**です。

2.4. Form クラスでパラメータ取得

Formクラスでパラメータ取得

```
public class UserForm {
    private int id;
    private String password;

    //～～Getter&Setterメソッド～～
```

■ Formクラスの定義

・複数のデータを扱う際は、Formクラスを定義する
・Viewのname属性値と、フィールド名を一致させる
・Getter&Setterメソッドを生成する

```
@PostMapping("/login")
    public String doLogin(UserForm userForm) {
        System.out.println(userForm.getId());
        System.out.println(userForm.getPassword());
```

■ リクエストパラメータを、Formオブジェクトで受け取れる
各フィールドに自動的にデータがセットされる

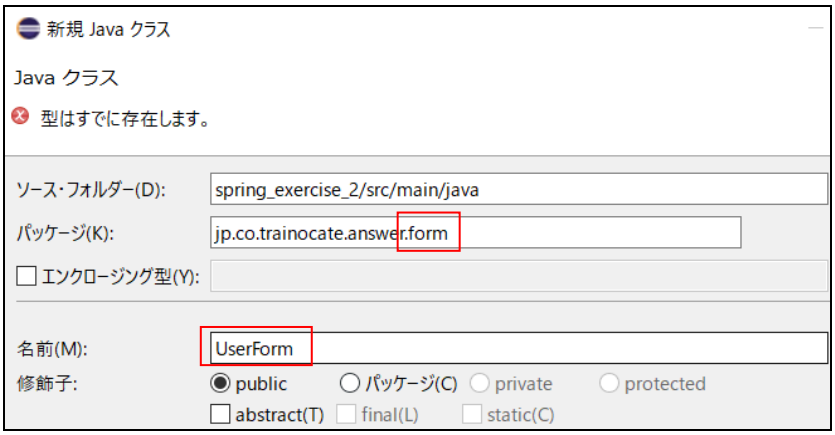
Form クラスを使用する目的

Web アプリケーションではユーザがフォームに複数の情報を入力する場面が頻繁にあります。例えば、登録フォームに名前、住所、電話番号など、多くのフィールドを記入する場合、これらの個別の情報を Controller のメソッドで一つずつ引数として取り扱うのは煩雑であり、コードの複雑さを増加させます。Controller のメソッドの引数をまとめて管理するために、Form クラスを利用できます。

【 作業手順①】 UserForm クラスの作成

まず、spring_exercise_2 を右クリックし、[form]というパッケージを作成します。

その後、[form]パッケージに、[UserForm]というクラスを作成します。



UserForm クラスには、**View から送られるデータをフィールドして記載し、Getter&Setter メソッドを定義**します。今回の例では、2 つのフィールドしか記載していませんが、複数のデータを扱うある場合は、全て記載します。

ここで注意点は、**フィールド名は、必ず View からの name 属性値と一致させる**ことです。

…/form/UserForm.java

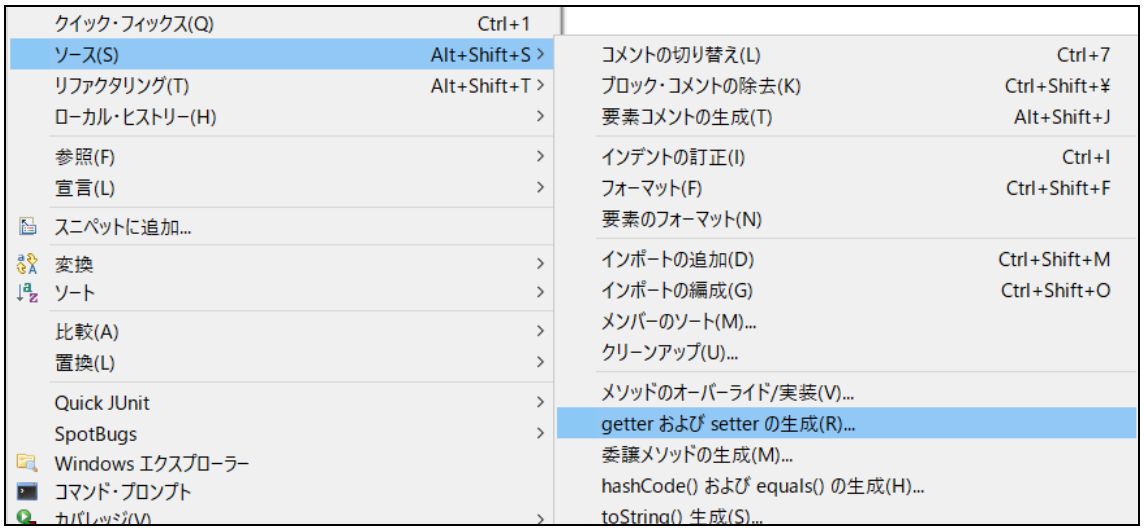
```
 1:package jp.co.trainocate.answer.form;
 2:
 3:public class UserForm {
 4:     private int id;
 5:     private String password;
 6:
 7:     public int getId() {
 8:             return id;
 9:     }
10:     public void setId(int id) {
11:             this.id = id;
12:     }
13:     public String getPassword() {
14:             return password;
15:     }
16:     public void setPassword(String password) {
17:             this.password = password;
18:     }
19:}
```

【参考】Getter&Setter メソッドを Eclipse で簡単に生成する方法

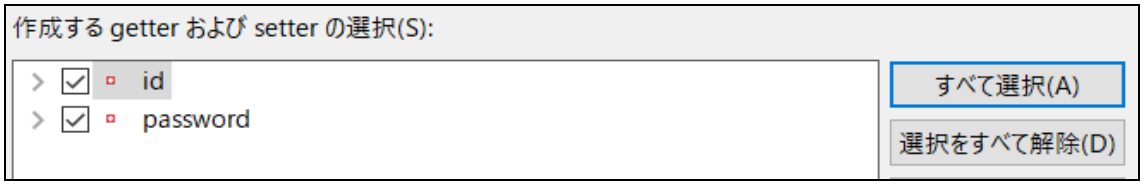
Getter&Setter メソッドは、定型文ですので、Eclipse で簡単に生成する機能があります。

Form クラス内のフィールドの定義が終わったら、以下の手順に従ってください。

[右クリック]→[ソース]→[Getter および Setter の生成]



フィールドにチェックを入れて、[生成]をクリックすると、メソッドが生成されます。



【 作業手順②】Form クラスのオブジェクトでパラメータを取得

それでは、この Form クラスを利用して、複数のリクエストパラメータを受け取るように、既存の LoginController を修正します。(サンプルは、LoginController2)

…/controller/LoginController2.java

```
1://【 省略】
2:
3:@PostMapping("/login2")
4:      public String doLogin(UserForm userForm) {
5:              //仮引数に Form クラスを記載するだけで、
6:              //自動的にデータがフィールドにセットされます
7:              System.out.println(userForm.getId());
8:              System.out.println(userForm.getPassword());
9:              return "index";
10:     }
```

・4 行目 doLogin(UserForm userForm)

doLogin メソッドの仮引数に、UserForm のオブジェクトを入れています。Spring では、**仮引数に Form クラスのオブジェクトを入れるだけで、リクエストパラメータが UserForm オブジェクトの対応するフィールド(id, password)に自動的にセットされます。**
よって、各リクエストパラメータを個別に取得するためのコードを書く必要がなくなります。

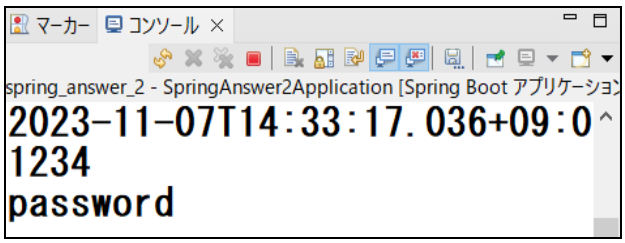
・7-8 行目

UserForm クラスに予め作成しておいた、getId()と getPassword()メソッドを呼び出すことで、userForm オブジェクトにバインドされた ID とパスワードを取得し、コンソールに出力します。

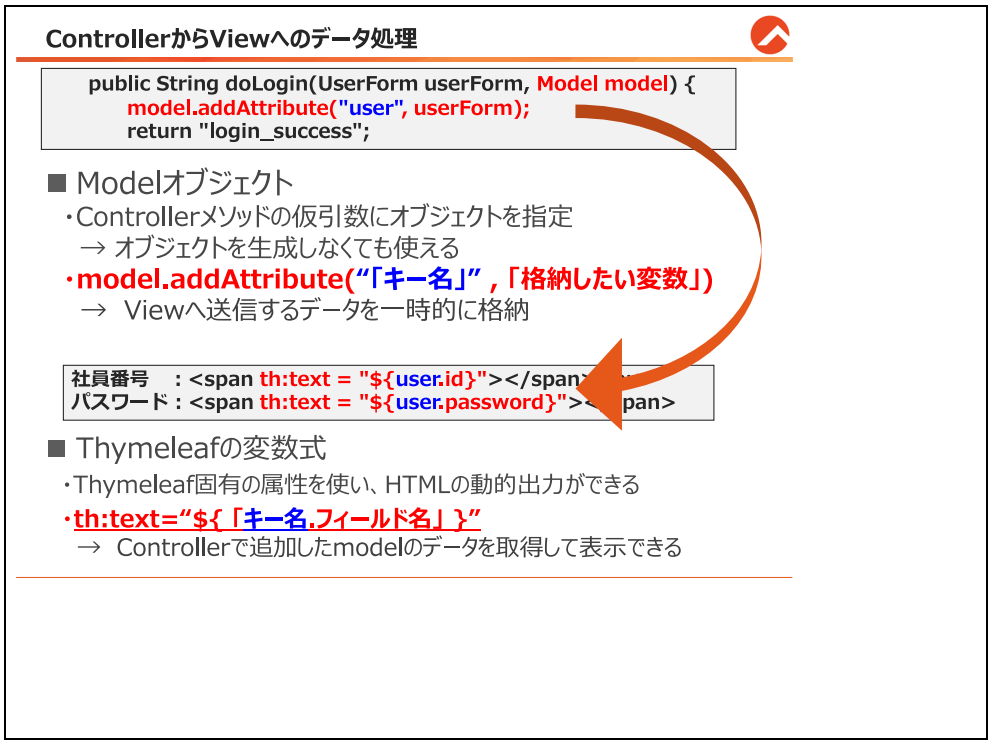
動作確認

以下の URL から、動作確認をすると、同じように View からのリクエストパラメータを取得してコンソール上に出力できているのがわかります。

http://localhost:8080/login2



2.5. Controller から View へのデータ処理



前の節では、View からのリクエストパラメータを、Controller で受け取る方法についてご説明しました。

この節では、逆に Controller から View へデータを送り、画面上に表示する方法についてご説明します。

【 作業手順③】model オブジェクトにデータを格納

Spring Controller から、View へデータを送るためには、サーブレットと同じくリクエストスコープを利用します。リクエストスコープは、別の場所にデータを送るために、**一時的にデータを入れておく箱**だと思ってください。

…/controller/LoginController3.java

```
 1://省略
 2:import org.springframework.ui.Model;
 3:
 4:@Controller
 5:public class LoginController3 {
 6:
 7:     @GetMapping("/login3")
 8:     public String showLoginView() {
 9:             return "login_input3";
10:     }
11:
12:     @PostMapping("/login3")
13:     public String doLogin(UserForm userForm, Model model) {
14:             //リクエストスコープに、userForm のデータを格納
15:             model.addAttribute("user", userForm);
16:
17:             return "login_success";
18:     }
19:}
```

Spring フレームワークでは、Model オブジェクトを使用して、Controller から View にデータを渡します。**Model オブジェクトの addAttribute メソッドを使うと、Model という箱に、データを一時保管できます**。サーブレットの request.setAttribute()に相当しますが、より洗練された方法です。

・13 行目 doLogin(UserForm userForm, Model model)

UserForm オブジェクトは、フォームから送信されたデータを受け取り、
リクエストスコープのオブジェクト(データ一時保管用の箱)を生み出すため、Model オブジェクトをメソッドの仮引数に入れます。

UserForm の時と同様に、**Spring ではメソッドの仮引数にオブジェクト名を書くと、オブジェクトを生成しなくても、使うことができます。**

・14-15 行目 model.addAttribute("user", userForm);

この行は、UserForm オブジェクトを"user"というキーでモデルに追加しています。
第 1 引数:キー名
第 2 引数:格納したい変数

これにより、return で呼び出す login_success.html でも、userForm オブジェクトのデータが使用可能になります。

【 参考】サーブレットでリクエストデータを格納するコード

```
request.setAttribute("id", id);
request.setAttribute("password", password);
```

サーブレットの request.setAttribute と役割も、使い方は同じですので、参考にしてください。

・17 行 return "login_success";

モデルに格納されたデータと共に、login_success.html を呼び出します。

このように Model オブジェクトを使うと、Controller から View へデータを「一時的に入れておく箱」としてリクエストスコープを利用することができ、Spring の MVC パターンに沿った方法でデータを渡すことができます。

【 作業手順④】Thymeleaf を利用したデータ出力

Thymeleaf は、Spring Framework と組み合わせて使用されるモダンなサーバーサイド Java テンプレートエンジンです。**Thymeleaf を利用することで、Controller から送信されたデータを HTML 内で動的に表示することができます。**

…/resources/templates/login_success.html

```
1:<!DOCTYPE html>
2:<html xmlns:th="http://www.thymeleaf.org">
3:<head>
4:<meta charset="UTF-8"/>
5:<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
6:<title>2 章 ログイン成功サンプル 1</title>
7:</head>
8:<body>
```

```
 9:     <h1>ログイン成功</h1>
10:     社員番号   : <span th:text = "${user.id}"></span><br>
11:     パスワード : <span th:text = "${user.password}"></span>
12:</body>
13:</html>
```

Thymeleaf の **th:text 属性は、指定された変数の値を HTML タグのテキスト内容として埋め込む**ために使用されます。

変数式は${...}という形で記述され、スコープ内にある変数の値を参照するために使われます。変数名は大文字・小文字を区別するため、Model に追加されたキー名と正確に一致する必要があります。

・2 行目 <html xmlns:th="http://www.thymeleaf.org">

10 行と、11 行には、th:text という thymeleaf 固有の属性が登場していますが、そのためには、予め宣言をする必要があります。

**・10 行目 **

この部分は Thymeleaf 独自のスクレプト文法の一つの、th:text 属性を使用しています。

Controller から Model に格納した「user」というキーの後ろに、「.フィールド名」を記載することで、id の値を出力できます。
例えば、user.id が 1234 であれば、出力は1234となります。

【参考】JSP でデータを表示する出力するコード

```
<%
        int id = (Integer) request.getAttribute("id");
        String password = (String) request.getAttribute("password");
 %>
    <p>社員番号   : <%= id %></p>
    <p>パスワード: <%= password %></p>
```

JSP では、上記のように request.getAttribute の記述と、<% %>という複雑な書き方をする必要がありましたが、th:text を使うことで、それがかなり簡略化されました。

・11 行目

ここでも th:text 属性を使い、user オブジェクトの password フィールドの値を表示しています。変数式を使用することで、動的に内容を Web ページに埋め込むことができます。

動作確認

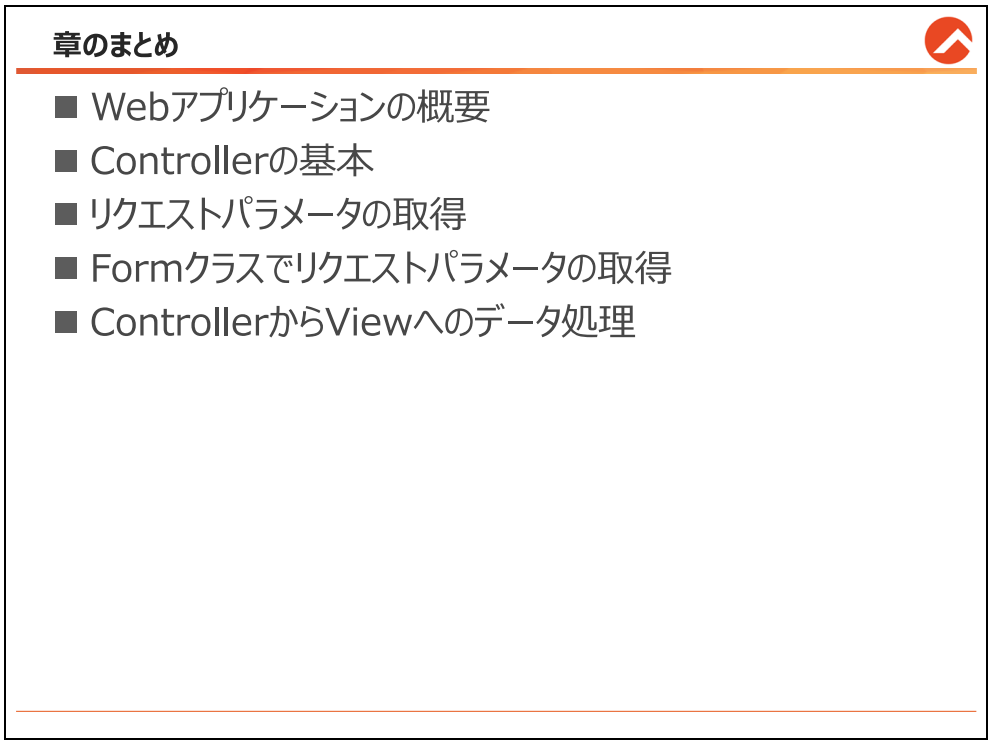
http://localhost:8080/login3
社員番号と、パスワードを入力してください。

このように Thymeleaf テンプレートを使用すると、モデルに格納された属性を HTML ページ内で簡単に参照し、出力することができます。

別の章で紹介しますが、th:text の他にも Thymeleaf は条件分岐、ループ処理、属性の変更など、多くの便利な機能を提供しています。

これにより、サーバーサイド(Controller)のロジックと View(HTML)の表示を分離しながら、動的な Web ページを作成することが可能になります。

章のまとめ



この章では、Controller と View の作成方法と、データの受け渡しについてご説明しました。

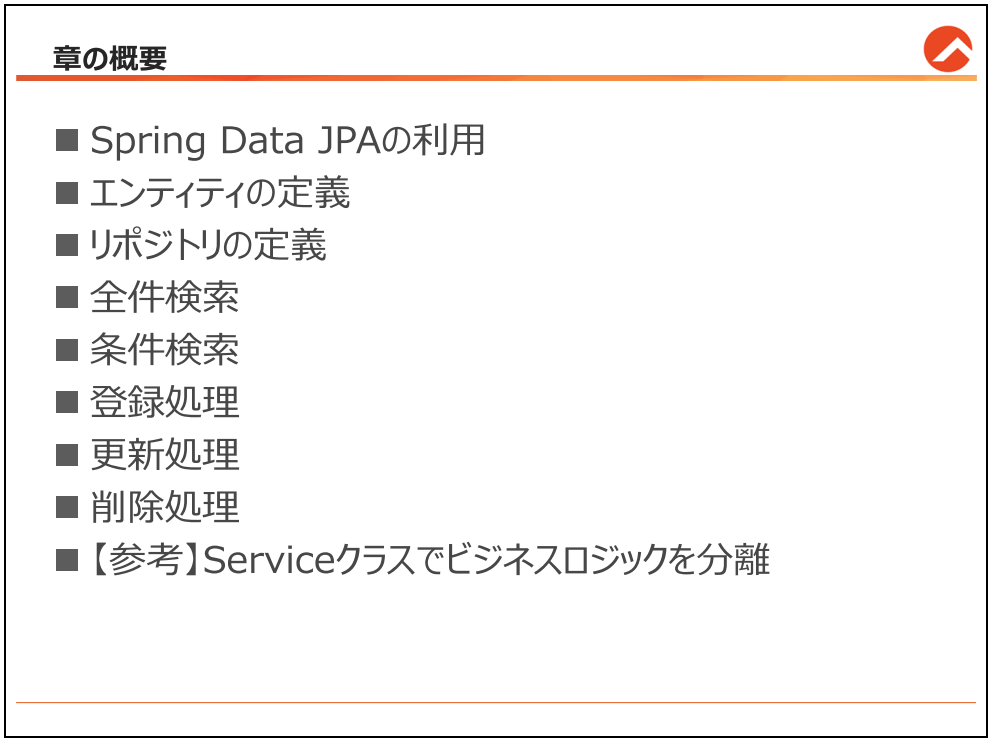
3

第3章 Spring Data JPA

章の目標

- Spring Data JPA の基本概念を理解できる
- エンティティとリポジトリを実装できる
- 全件検索・条件検索ができる
- 登録・更新・削除ができる

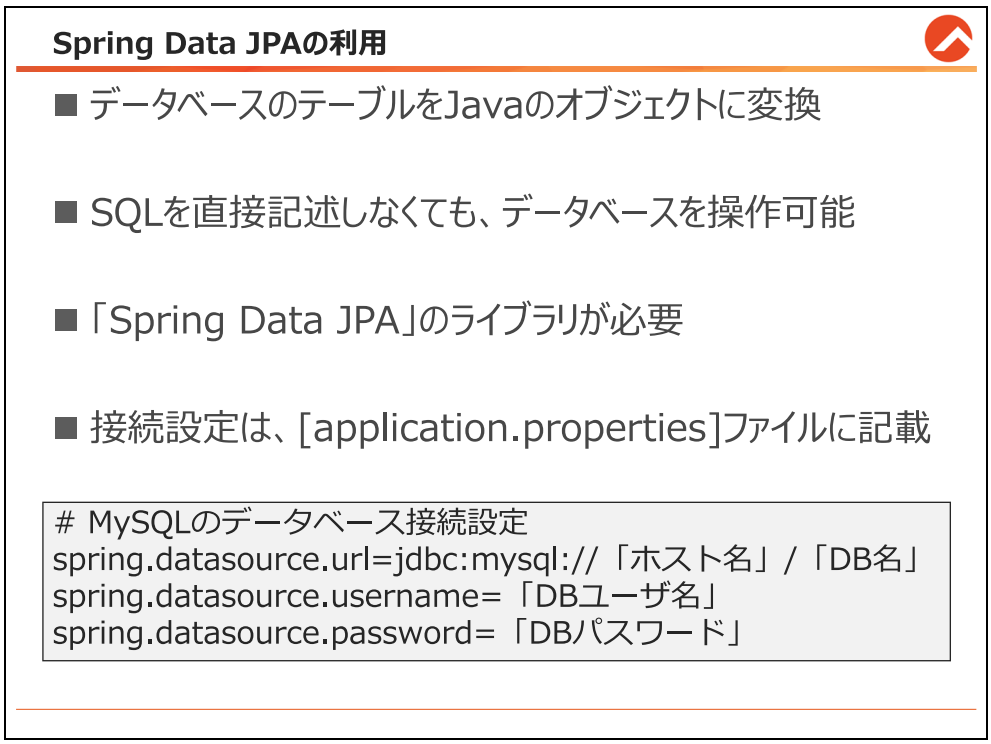
章の概要



第 3 章では、Spring Data JPA の活用方法に焦点を当てて、Spring とデータベースを連携する Web アプリケーションを構築する方法を学びます。

初めに、Spring Data JPA とは何か、そしてその利用のメリットについて理解を深めます。続いて、データを表現するためのエンティティの定義方法について説明し、データへのアクセスを抽象化するリポジトリの作成技術を学習します。さらに、データベースから情報を全件検索する方法、さまざまな条件を用いた検索方法についてもご説明します。その後、データの登録、更新、削除といった CRUD 操作の基本的なプロセスについて学ぶことで、Spring Data JPA を用いた効率的なデータ管理手法をマスターしていきましょう。

3.1. Spring Data JPA の利用



Spring Data JPA とは

Web アプリケーションの開発では、データの保存や取得のためにデータベースとの連携が不可欠です。一般的に、データベースと連携するプログラミングは、以下のようにある程度パターン化されています。

「 データベースへの接続→データの検索→結果を取得→データベースからの切断」

このようなパターンは一見シンプルに思えますが、Spring Framework などのツールを使わないと、膨大な量のコードを書く必要があります。

Spring Framework は、このような繰り返し行われる作業を効率化することに強みがあり、さまざまな支援ツールを提供しています。本書では、特にデータベース操作を簡単にできる「 Spring Data JPA(Jakarta Persistence API)」に注目します。

通常、データベースに命令を出すためには、SQL という特別な言語を使う必要がありますが、**Spring Data JPA を使うと、予めデータベース情報を Java のオブジェクトに変換することで、テーブルの 1 行のデータを Java の 1 つのインスタンスとして扱うことができます。**

そのおかげで、**SQL 文を書かなくても、データベースの操作をすることが可能になります。**この節では、Spring Data JPA を使って、データを効果的に管理する方法について学びます。

【 参考】 サンプルデータベースの定義

本書では、以下のデータベースの情報を基にします。

RDBMS 名	MySQL8.0
データベース名	spring_sample
ユーザ名	spring_sample_user
パスワード	trainocate

3 章では、[Product]テーブルのみ使用します。

<Product テーブル>

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	255	–	–
3	価格	price	INT	–	–	–

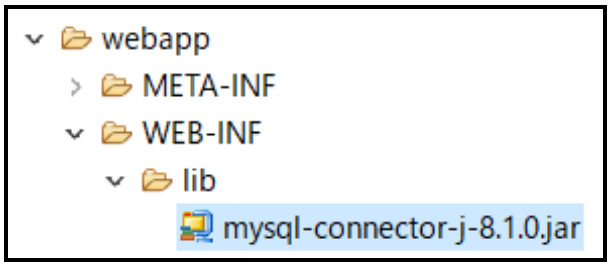
[product]テーブルには、9 行のデータが格納されています。

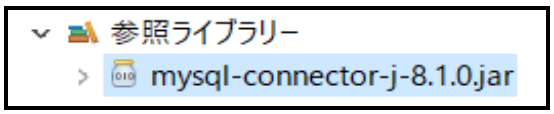
商品番号	商品名	価格
1	スマートテレビ	30000
2	ゲーミングモニター	20000
3	ノートパソコン	50000
4	2 ドア冷蔵庫	50000
5	コンパクト冷蔵庫	15000
6	ワインクーラー	30000
7	全自動洗濯機	25000
8	乾燥機能付き洗濯機	40000
9	洗濯乾燥機	60000

【参考】既存の JDBC 接続(Java+MySQL)方法と問題

既存の Java データベースプログラミングにおいての、MySQL との接続方法を説明します。

1. 事前に mysql-connector-j.jar ファイルを入手し、プロジェクトのビルドパスに追加する





2. クラスに接続情報を記載する

```
String jdbcUrl =
      "jdbc:mysql://localhost/spring_sample?serverTimezone=Asia/Tokyo";
String userId = "spring_sample_user";
String password = "trainocate";
```

3. DriverManager.getConnction(「DB URL」,「ユーザ名」、「パスワード」)メソッドを実行

4. SQL を実行するための処理を記載(例外処理必須)

```
try (Connection con =
            DriverManager.getConnection(jdbcUrl, userId, password)) {
              //データベース処理
      } catch (SQLException e) {
              e.printStackTrace();
      }
```

既存の JDBC 接続方法にはいくつかの欠点があります。そして、Spring Data JPA はその欠点を大きく改善しています。以下は、その比較表です。

要素	JDBC	Spring Data JPA
外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。
ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。
冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。
例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時に SQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。
リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。

Spring Data JPA は、よりシンプルで安全、そして効率的なデータアクセスを提供することで、データベースプログラミングを容易にします。

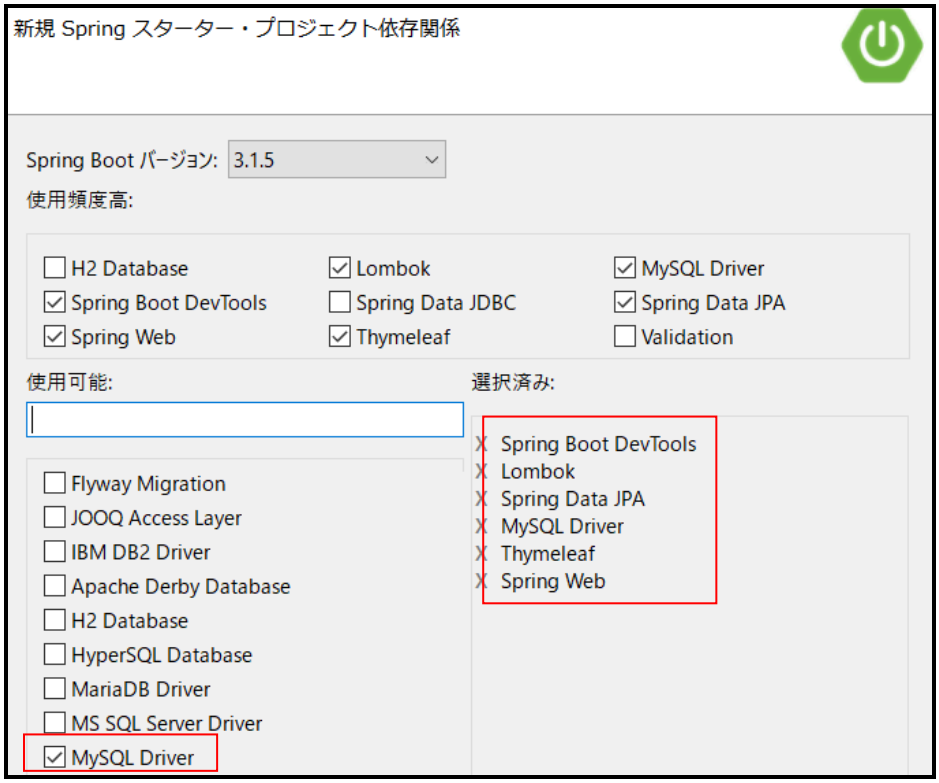
【 作業手順①】Spring Data JPA と、MySQL ドライバーの導入

第 1 章でご説明した通り、Maven を利用すると簡単にライブラリを追加できます。

第 3 章で必要なライブラリは、**「 Spring Data JPA」と、「 MySQL Driver」、「 Lombok」**です。

または、Spring プロジェクトを作成する時、同時にインストールしたいライブラリを選択する方法もあります。

新規でプロジェクトを作成する際は、必要なライブラリを必ず導入してから進めてください。



【 作業手順②】接続情報の設定

Spring Data JPA を使う場合、**データベースとの接続情報は、クラスに記載する必要はなく、「 application.properties」ファイルに記載をします。**そうすると、全クラスがこの設定ファイルを参照することになりますので、管理がしやすくなります。

application.properties

```
spring.thymeleaf.cache=false

# MySQL のデータベース接続設定
spring.datasource.url=
   jdbc:mysql://localhost/spring_sample?serverTimezone=Asia/Tokyo
spring.datasource.username=spring_sample_user
spring.datasource.password=trainocate

# JPA/Hibernate の追加設定( 開発環境ではお勧め)
spring.jpa.show-sql=true
```

jdbc:mysql://[サーバーURL]/[データベース名]?[その他設定]

追記した各設定は以下の情報を表しています。

spring.datasource.url→データベースの URL(サーバー名/データベース名が含まれます)

spring.datasource.username→データベースに接続するユーザ名

spring.datasource.password→データベースに接続するためのパスワード

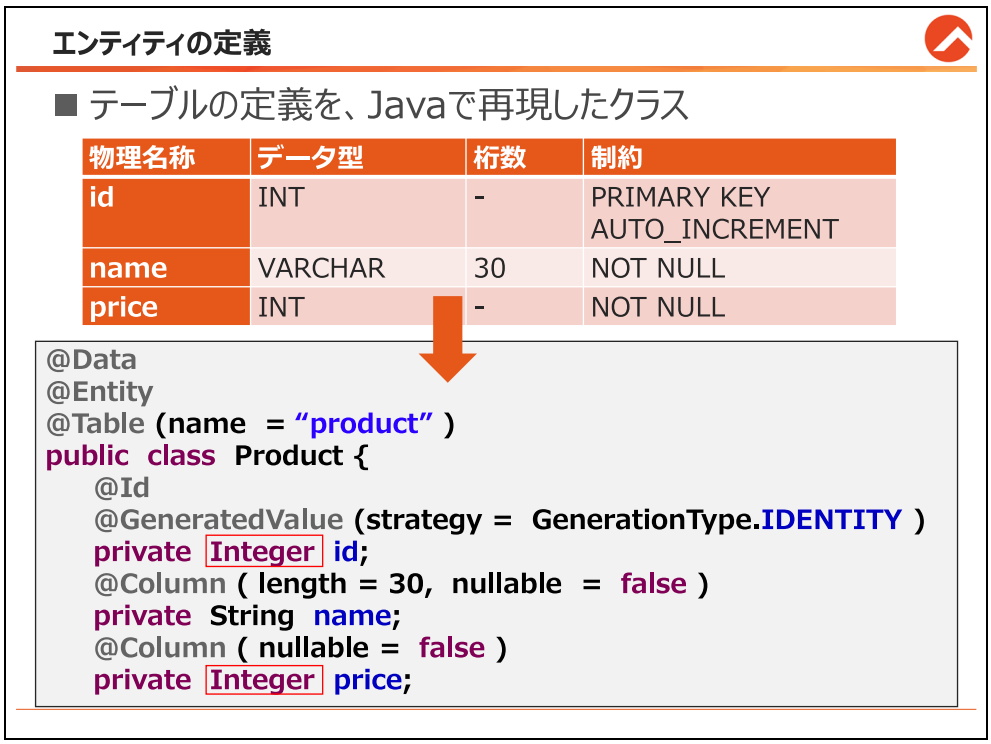
spring.jpa.show-sql=true →生成した SQL 文をログで確認可能

最後の 1 行は、本番環境では使うべきではありませんが、開発時は役に立つ設定です。

この設定は、あくまで MySQL 専用の書き方であり、Oracle や PostgreSQL などの RDBMS と接続する場合は、別途調査が必要です。

設定完了後は、必ず[実行]→[maven install]をしないと反映されませんので、ご注意ください。

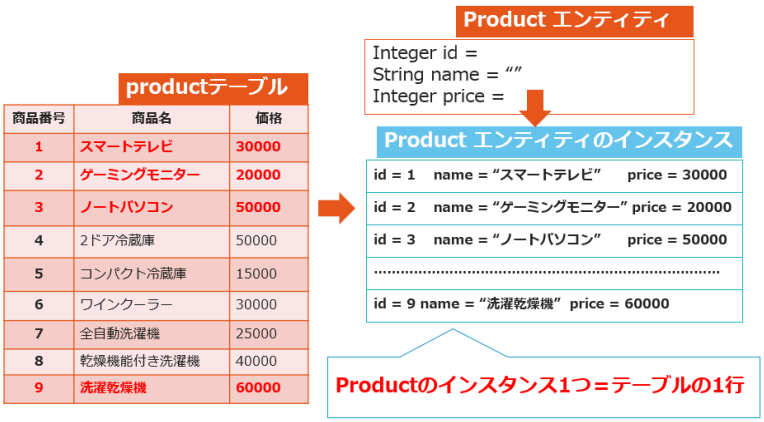
3.2. エンティティの定義



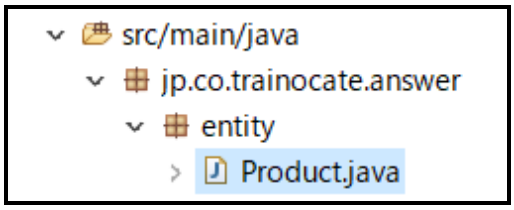
エンティティ(Entity)とは？

Spring Data JPA を使用する際には、**データベース内の各テーブルを Java のクラスで表現**することが重要です。このクラスを「 エンティティ」と呼びます。エンティティは実質的にデータベースのテーブルの各行をオブジェクトとして扱うためのものです。つまり、**テーブルの 1 行がエンティティの 1 インスタンス(オブジェクト)に対応します。**

エンティティの作成を学ぶことは、データベースをプログラムで扱うための最初のステップです。例として、もし「 product」という名前のテーブルがある場合、それに対応するエンティティを「 Product」というクラスとして作成します。このクラスには、テーブルの列に対応するフィールドが含まれ、データベースの行の情報を保持できるようになります。



Entity クラスの作成



Entity クラスは、通常テーブルの名前と同じようなクラス名を付けます。

Controller クラスを、controller パッケージに作成したのと同じく、Entity クラスは、entity パッケージの中に作成します。

【 参考】<Product テーブル>

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	-	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	30	NOT NULL	-
3	価格	price	INT	-	NOT NULL	-

それでは、Product テーブルを、Java のクラスで再現してみます。理解のため、全体のソースコードをお見せする前に、作業順でご説明します。

【 作業手順①】テーブルの列名と型を Java のフィールドで表現する

基本的に、JavaBean クラスを作る感覚と似ていますので、product テーブルで使われるデータを、一つずつ定義していきましょう。

…/entity/Product.java

```
@Entity
@Table(name = "product")
public class Product {
    private Integer id;
    private String name;
    private Integer price;
}
```

このコードにおいて、@Entity アノテーションはこのクラスがエンティティクラスであることを示し、@Table アノテーションは実際のテーブル名を指定します。その後、テーブルの各列を型に合わせて定義します。この例では、id、name、price が該当します。

フィールドの型についてですが、int ではなく、Integer の型が推奨されます。その理由は、データベースの int 列には、null を格納できても、Java の int 型には null を入れることができないためです。

そして、JavaBean と同様にデータのやり取りを行うクラスなので、当然 Getter&Setter メソッドや、初期化のためのコンストラクタ、より完成度を上げるため toString メソッド、equals メソッド、hashcode メソッドを定義するのが理想です。

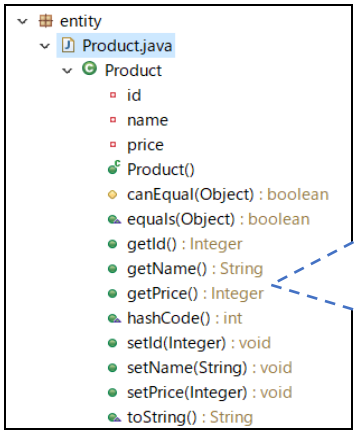
しかし、それを全て書くと、開発効率が落ちるため、lombok ライブラリの@Data アノテーションを付けると便利です。

【 作業手順②】Lombok の@Data アノテーションを付与

…/entity/Product.java

```
import lombok.Data;

@Entity
@Table(name = "product")
@Data
public class Product {
    private Integer id;
    private String name;
    private Integer price;
}
```

@Data アノテーションを付与するだけで、Getter&Setter、コンストラクタ、toString、hashcode、equals メソッドが自動生成されますので、非常に便利になります。



@Data を付けた後、Eclipse 上で確認してみると、いつの間にかメソッドが生成されていることを確認できます。

【 作業手順③】 制約をアノテーションで表現

次は、**制約についても、定義**する必要があります。

…/entity/Product.java

```
@Id  //Primary Key 制約
@GeneratedValue(strategy = GenerationType.IDENTITY)//auto_increment 制約
private Integer id;

@Column(length = 30, nullable = false)  //桁数 30, NOT NULL 制約
private String name;

@Column(nullable = false)  //NOT NULL 制約
private Integer price;
```

Primary Key 制約は、「 @Id」アノテーションで表現します。普通の列については、「 @Column」を付けます。

新規登録をするたびに、自動的に番号が割り当てられる、自動採番の AUTO_INCREMENT 制約は、「 @GeneratedValue(strategy = GenerationType.IDENTITY)」で表現します。

入力値の長さや、NOT NULL 制約の有無なども、表現できます。

完成された Product クラスは、下記となります。

…/entity/Product.java

```
 1:package jp.co.trainocate.answer.entity;
 2:
 3:import jakarta.persistence.Column;
 4:import jakarta.persistence.Entity;
 5:import jakarta.persistence.GeneratedValue;
 6:import jakarta.persistence.GenerationType;
 7:import jakarta.persistence.Id;
 8:import jakarta.persistence.Table;
 9:import lombok.Data;
10:
11:@Data
12:@Entity
13:@Table(name = "Product")
```

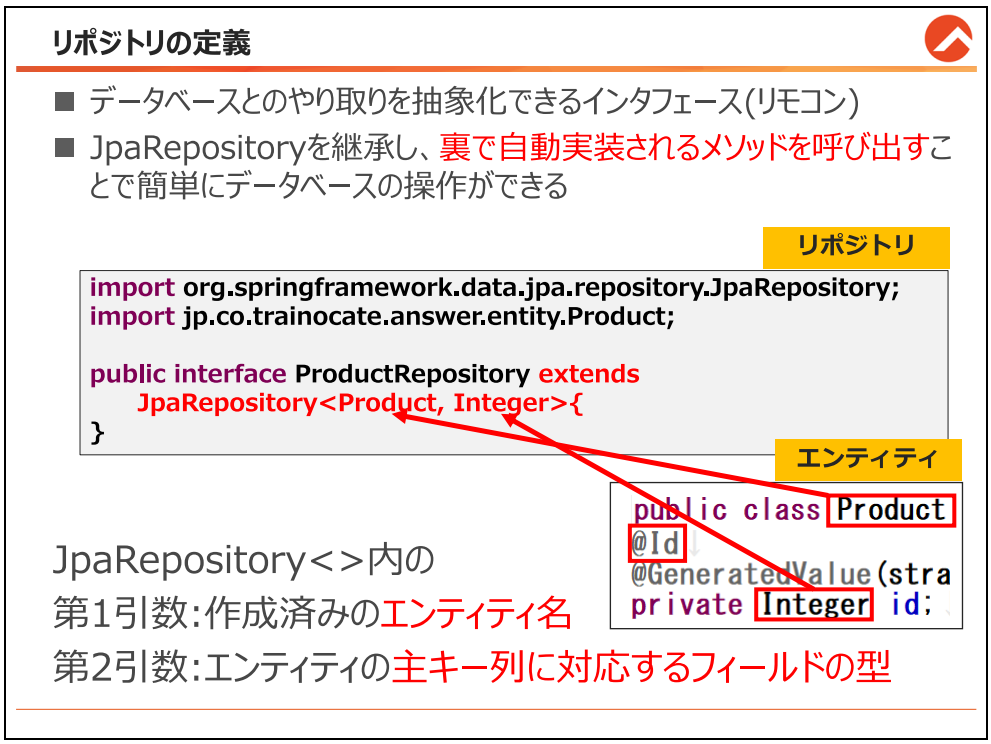
```
14:public class Product {
15:
16:    @Id
17:    @GeneratedValue(strategy = GenerationType.IDENTITY)
18:    private Integer id;
19:
20:    @Column(length = 30, nullable = false)
21:    private String name;
22:
23:    @Column(nullable = false)
24:    private Integer price;
25:
26:    // lombok がコンストラクタと以下のメソッドを自動生成
27:    //Getter()&Setter()
28:    //equals()
29:    //hashCode()
30:    //toString()
31:}
```

以上が、エンティティクラスの作成方法でした。エンティティは、全く言語が異なる、データベースを Java の世界に再現したものです。

データベースの各行はエンティティの Java オブジェクトとして扱われ、SQL を直接書くことなくデータベースとのやり取りが可能になります。

今回は一般的なエンティティクラスの書き方をご説明しましたが、Oracle などの異なる RDBMS の独自の機能を再現する際は、書き方が変わる場合がありますので、それぞれ調査が必要です。

3.3. リポジトリの定義



リポジトリ(Repository)とは？

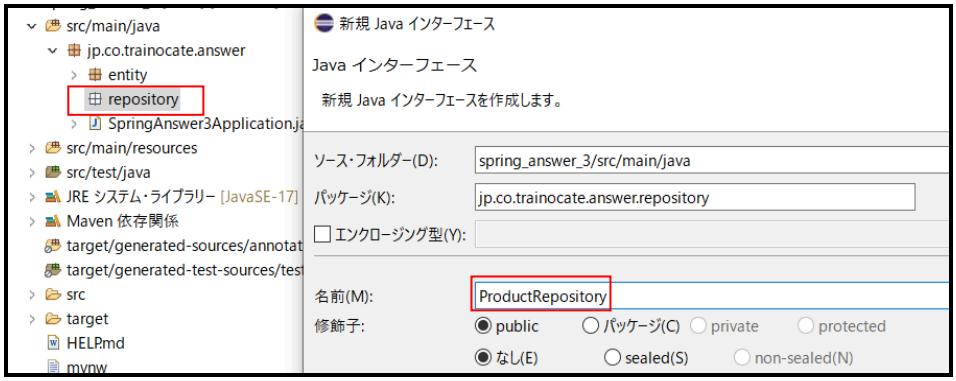
リポジトリは Spring Data JPA において、データベースとのやり取りを抽象化することができるインターフェースです。簡単に言うと、**データベースを操作できるリモコンのようなもの**です。

エンティティとリポジトリを組み合わせることで、**複雑な SQL クエリを書くことなく、Java を通じてデータベース操作を行うことができます**。リポジトリインターフェースを定義するだけで、Spring Data JPA は実行時にその実装を裏で自動的に生成し、開発者がデータベース操作に集中できるようにします。

まるで、**リモコンに最初からついているボタンを押せば、テレビを簡単に操作できるのと似ています。**この節では、リポジトリの作成と基本的な使用方法について詳しく説明していきます。

【 作業手順④】リポジトリの作成

エンティティと同様に、別途パッケージを作成します。
クラスではなく、インターフェースですので、インターフェースとして作成します。
名前は通常、エンティティ名の後ろに、[Repository]を結合することが多いです。



…/repository/ProductRepository.java

```java
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Integer>{  }
```

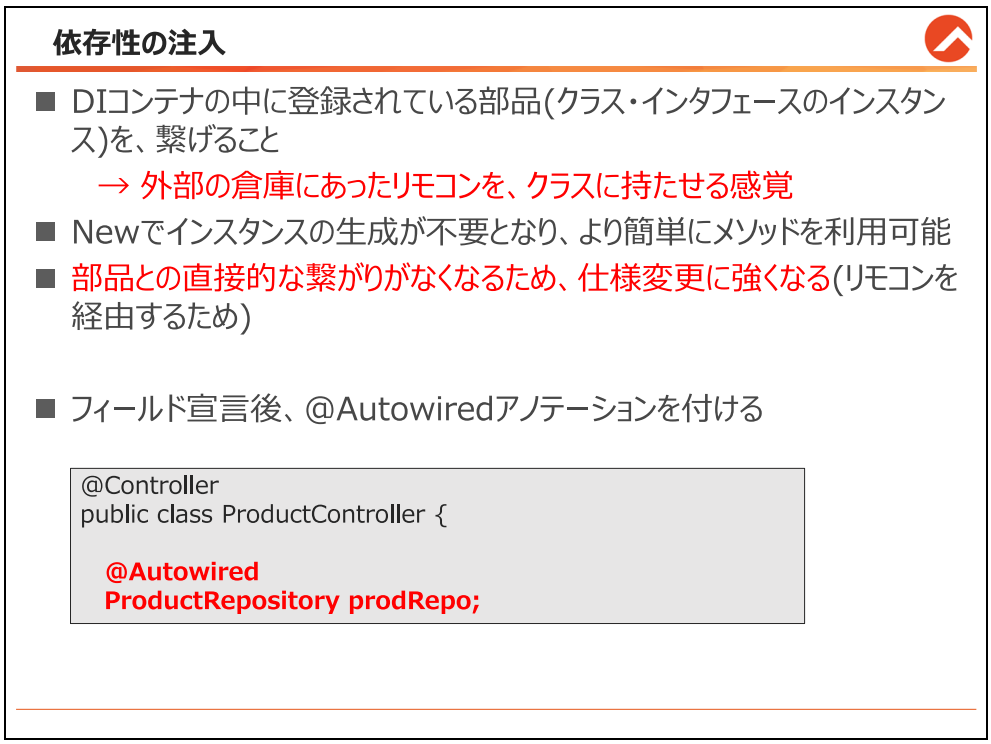
リポジトリを定義する際には、**JpaRepository という既存のインターフェースを extends で拡張**(継承)させます。

JpaRepository には、データベースのデータを検索したり保存したりするためのメソッドが最初から用意されています。それを**継承するだけで、開発者は SQL を記述しなくても、便利にデータベースの操作ができるようになります。**これはまるで、「リモコンによく使う電源 On/Off やチャンネル・音量調整ボタンが最初から付いているため、テレビの中の複雑な処理を知らなくても簡単に操作できる」のと同じイメージです。

extends JpaRepository の後ろには、ジェネリクス(< >)を記載します。 <Product, Integer>のように、**第 1 引数にはエンティティ名を、第 2 引数にはそのエンティティの@Id アノテーションを付けた主キーの型名**を記載します。

今までの内容をまとめると、Product というエンティティクラスがデータベースのテーブルを表し、そのクラスに対応するリポジトリを作ることで、テーブルのデータを簡単に追加、取得、更新、削除できるようになるわけです。
作成したリポジトリを実際の処理で使うためには、「依存性の注入」が必要になります。

3.3.1. 依存性の注入



依存性の注入とは

コントローラーからリポジトリを使用して DB 操作を行うためには、リポジトリを使えるようにする**「依存性の注入」**という準備が必要です。依存性の注入とは、**Spring の DI コンテナ(待機室) の中で待機しているクラスやインターフェースのインスタンスを、使いたいクラスに繋げること**です。

通常、Java プログラミングで A クラスのインスタンスを B クラスで使うためには、
B クラス内で「 A a = new A();」
というオブジェクト生成の式を書く必要があります。しかし、**依存性の注入を行うことで、この New 生成式を書かなくてもインスタンスを繋げて利用できる**ようになります。

第 1 章で、Spring Boot の DI コンテナの中には「 Bean(部品) 」が自動で登録されていると説明しました。リポジトリのインスタンスも Bean として自動登録されるため、必要な時にいつでもクラスに繋げることができます。

メリットはそれだけではなく、クラスのインスタンスを直接呼ばず、外部からのリモコンを使うだけなので、直接的な繋がりがなくなり、**部品の仕様変更にも強く柔軟なプログラムを実現できるようになります。**

では、具体的な注入方法を説明します。

```
@Autowired
ProductRepository prodRepo;
```

Spring では、いくつかの依存性の注入方法があります。一番簡単な方法は、上記のように、フィールドへ直接注入する方法があります。

インターフェースのフィールドを記載し、その上に[@Autowired]アノテーションを付けるだけで依存性の注入が完了し、そのインターフェースに紐づいているメソッドが使えるようになります。

イメージとしては、**クラスに「ProductRepository 型」の「prodRepo」という名前のリモコンを持たせてあげた状態**です。

【参考】その他の依存性の注入方法

フィールドに@Autowired をつける方法は、とても便利ですが、単体テストの際に不便だというデメリットがあります。

ですので、下記の方法で書くこともあります。

依存性の注入方法②

```
public class ProductController {
    private final ProductRepository prodRepo;

    @Autowired
    public ProductController(ProductRepository prodRepo) {
        this.prodRepo = prodRepo;
    }
```

このように、フィールドを定数に変えた後に、コンストラクタを作成し、コンストラクタに @Autowired アノテーションを付ける依存性の注入の書き方もあります。こちらは単体テスト時も大丈夫ですので、よく使われます。

しかし、フィールドに指定する方法より、可読性が悪くなるため、
下記のように、**lombok ライブラリを使って、簡略化させることも可能**です。

依存性の注入方法③

```
import lombok.RequiredArgsConstructor;

@Controller
@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository prodRepo;
```

@RequiredArgsConstructor アノテーションを、クラス名に付けるだけで**依存性の注入のためのコンストラクタのコードを省略できます。**

3.3.2. コントローラーの基本 URL 設定

今までは、メソッド単位でマッピングする URL を指定していましたが、通常、Spring プロジェクトには、複数の異なる機能を持つ、コントローラーが存在します。

例えば、**商品に関する機能を集めたコントローラーと、ユーザに関連する機能を持つコントローラーは、URL も区別しておかないと、URL の管理が難しくなります。**

そこで、コントローラーに基本 URL を指定しておくことで、URL の管理を便利にすることができます。

ProductController の作成

まずは、controller パッケージに[ProductController]クラスを作成します。

…/controller/ProductController.java

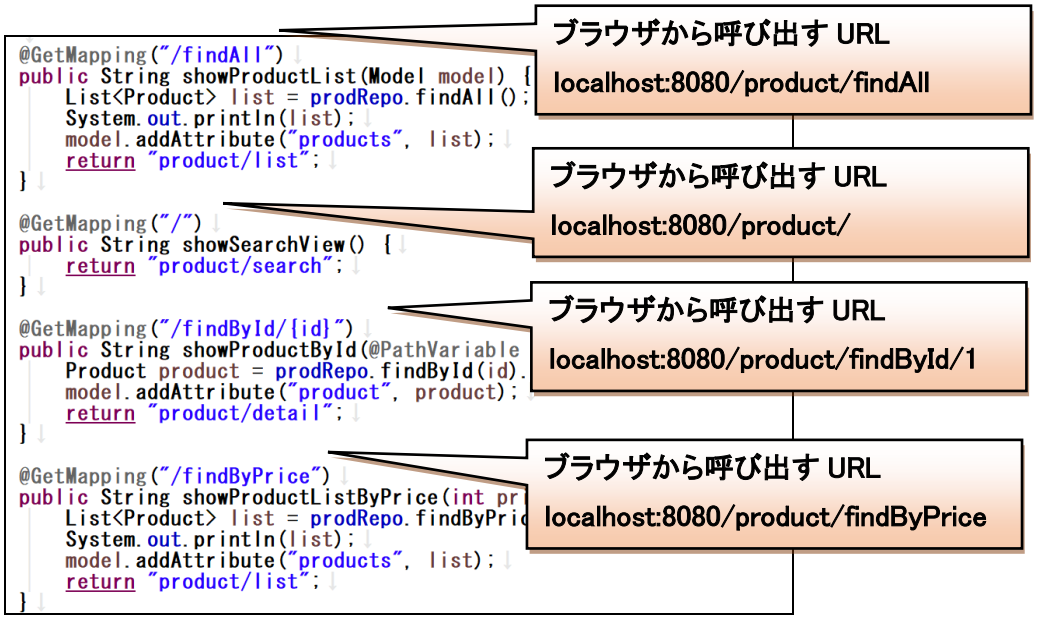
```
@Controller
@RequestMapping("/product")
public class ProductController {
```

class ProductController の上に、@RequestMapping(“URL”)というコードが書かれています。

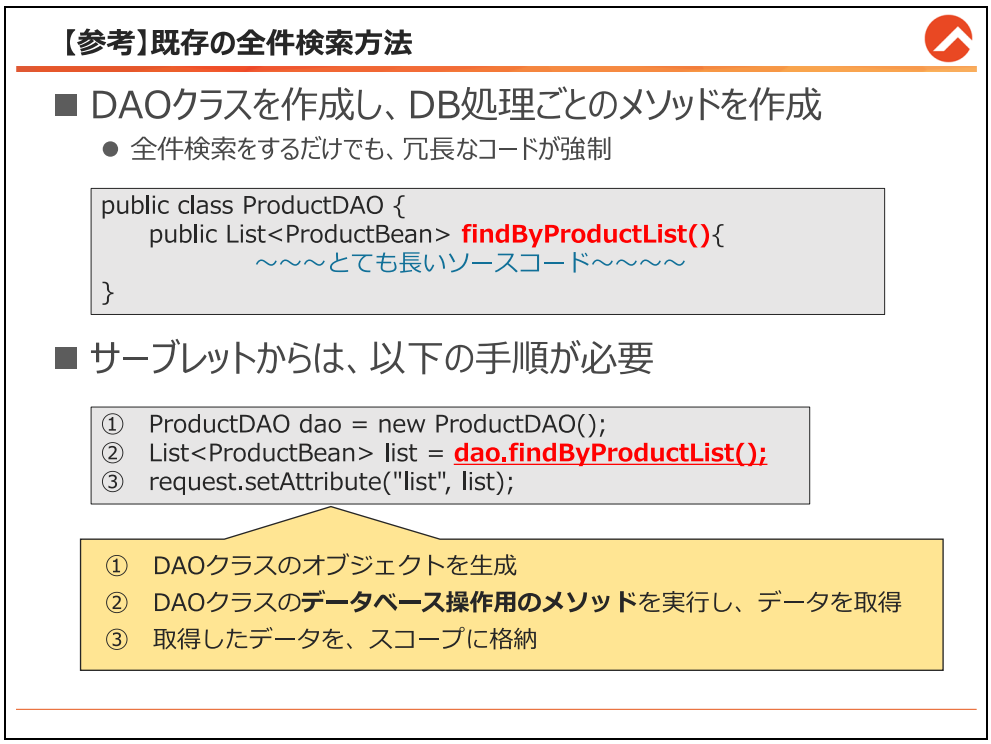
これは、ProductController クラス全体に基本 URL「 /product」を設定しているコードです。

RequestMapping で URL を指定すると、この**コントローラー内のすべてのメソッドは、基本 URL に基づいてアクセスされます**。

たとえば、ProductController クラスに作成されるメソッドの URL は、以下のように呼び出されることになります。



【参考】既存の全件検索方法



データベースのアクセス情報を記載し、エンティティとリポジトリの定義まで終わったら、いよいよデータベース操作を開始できます。

では、本格的な説明の前に、まずは既存の Java データベースプログラミングとサーブレットでの書き方を確認しておきましょう。

【参考】サーブレットでデータベースの操作(全件検索)

サーブレットで、データベースから検索をするためには、別途 DAO クラスを作成し、その中にデータベース操作をするためのメソッドを作成する必要がありました。

```
public class ProductDAO {

        public List<ProductBean> findByProductList(){
                String jdbcUrl = "jdbc:mysql://localhost/spring_sample?
                   serverTimezone=Asia/Tokyo";
                String userId = "spring_sample_user";
                String password = "trainocate";
```

```
                PreparedStatement pst = null;
                ResultSet rs = null;
                ArrayList list = new ArrayList();

                try (Connection con =
                DriverManager.getConnection(jdbcUrl, userId, password)) {
                        String sql = "SELECT * FROM product";
                        pst = con.prepareStatement(sql);
                        rs = pst.executeQuery();
                        while(rs.next()) {
                                ProductBean product = new ProductBean();
                                product.setId(rs.getInt("id"));
                                product.setName(rs.getString("name"));
                                product.setPrice(rs.getInt("price"));
                                list.add(product);
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
                return list;
        }
```

一番簡単な SELECT 文なのに、これほどのソースコードが必要でした。

次は、サーブレット側のソースコードです。

```
@WebServlet("/Sample5")
public class Sample5 extends HttpServlet {
        protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

             //Product の DB 操作用クラスの ProductDAO のオブジェクト生成
                ProductDAO dao = new ProductDAO();

                //複数行を取得するので、リスト型でデータを取得
                List<ProductBean> list = dao.findByProductList();
```

```
            //取得したデータを JSP に送るため、リクエストスコープに格納
              request.setAttribute("list", list);

              //list.jsp へ遷移
              RequestDispatcher dispatcher =
                    request.getRequestDispatcher("/index.jsp");
              dispatcher.forward(request, response);
        }
```

サーブレットでは、以下の手順が必須でした。

① DAO クラスのオブジェクトを生成する。

② DAO クラスに定義されている**データベース操作用のメソッド**を実行し、データを取得する

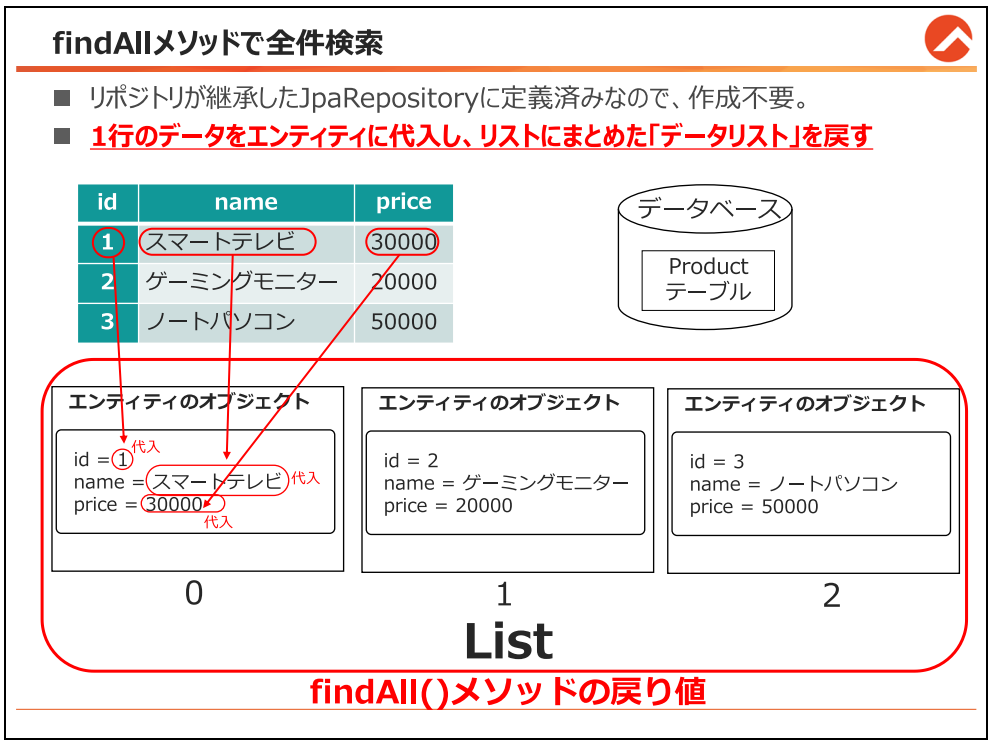
③ 取得したデータを、スコープに格納

④ JSP へ遷移

サーブレットの処理コード自体は、多くはないのですが、全件検索をするための、DAO クラスのメソッドのソースコードがかなりの量ですので、時間がかかっていました。

では、Spring Data JPA を使うと、どれほど簡単になるのか、今からご説明いたします。

3.4. 全件検索



【 作業手順①】 findAll メソッドで全件データ取得

…/controller/ProductController.java

```
 1:
 2:@Controller
 3:@RequestMapping("/product")
 4:public class ProductController {
 5:     @Autowired
 6:     private ProductRepository prodRepo;
 7:
 8:     @GetMapping("/findAll")
 9:     public String showProductList(Model model) {
10:             List<Product> list = prodRepo.findAll();
11:             System.out.println(list);
12:             model.addAttribute("products", list);
13:             return "product/list";
14:     }
15:}
```

3.3.2 で作成した ProductController に、全件データの検索処理用のメソッドとして、[showProductList]メソッドを作成します。

5-6 行目

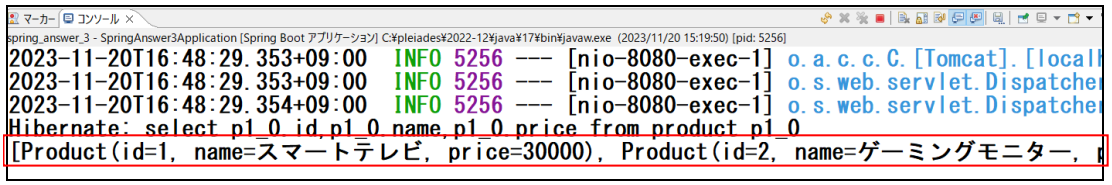
```
 5:    @Autowired
 6:     private ProductRepository prodRepo;
```

全件検索をするためには、ProductRepository から利用できる「findAll」メソッドを呼び出さないといけません。**ProductRepository のメソッドを簡単に利用できるように、ProductRepository のフィールドを宣言して、@Autowired で依存性の注入を**しておきましょう。

この 2 行のコードを書いておけば、[prodRepo]を使って、findAll メソッドを呼び出せます。

10 行目 List<Product> list = prodRepo.findAll();

findAll メソッドを実行すると、Product テーブル内の全データを、List<Product>の形で取得できます。**List の Import 先は「java.util」ですので、ご注意ください。**



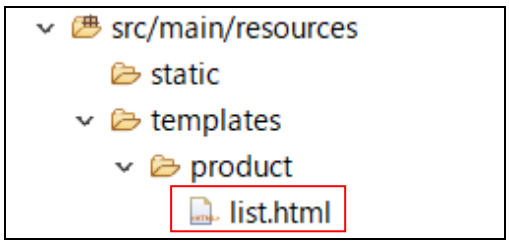
SELECT の SQL 文は、どこにも書いていませんが、データベースの product テーブルから商品データをしっかり取得しているのが確認できます。これが Spring Data JPA の凄いところです。エンティティにテーブルを忠実に再現した結果、**findAll というメソッドを実行するだけで、自動でデータの格納まで完了したリストを取得できます。**

12-13 行目

```
12:     model.addAttribute("products", list);
13:     return "product/list";
```

最後は、データの塊である、list を View へ出力させるために、model に格納します。
このリストは、product が複数入っているので、キーは[products]にしました。
遷移先は、product フォルダの、list.html を指しています。

【 作業手順②】 Thymeleaf でリストの出力



showProductList メソッドの return が[“product/list”]ですので、合わせてフォルダとファイルを作成します。

…/resources/templates/product/list.html

```
1:<h1>商品一覧</h1>
2:    <table>
3:        <tr>
4:            <th>商品番号</th>
5:            <th>商品名</th>
6:            <th>価格</th>
7:        </tr>
8:        <tr th:each="product: ${products}">
9:            <td th:text="${product.id}"></td>
10:           <td th:text="${product.name}"></td>
11:           <td th:text="${product.price}"></td>
12:       </tr>
13:   </table>
```

Spring Controller で model に格納した **キー名を右に。** **左は任意の変数名**

8 行目 <tr th:each="product: ${products}">

th:each は、**Java の拡張 for 文と似た使い方**をします。指定されたコレクション(ここでは products リスト)の各要素に対して、<tr> タグ(テーブルの行)を繰り返し生成します。

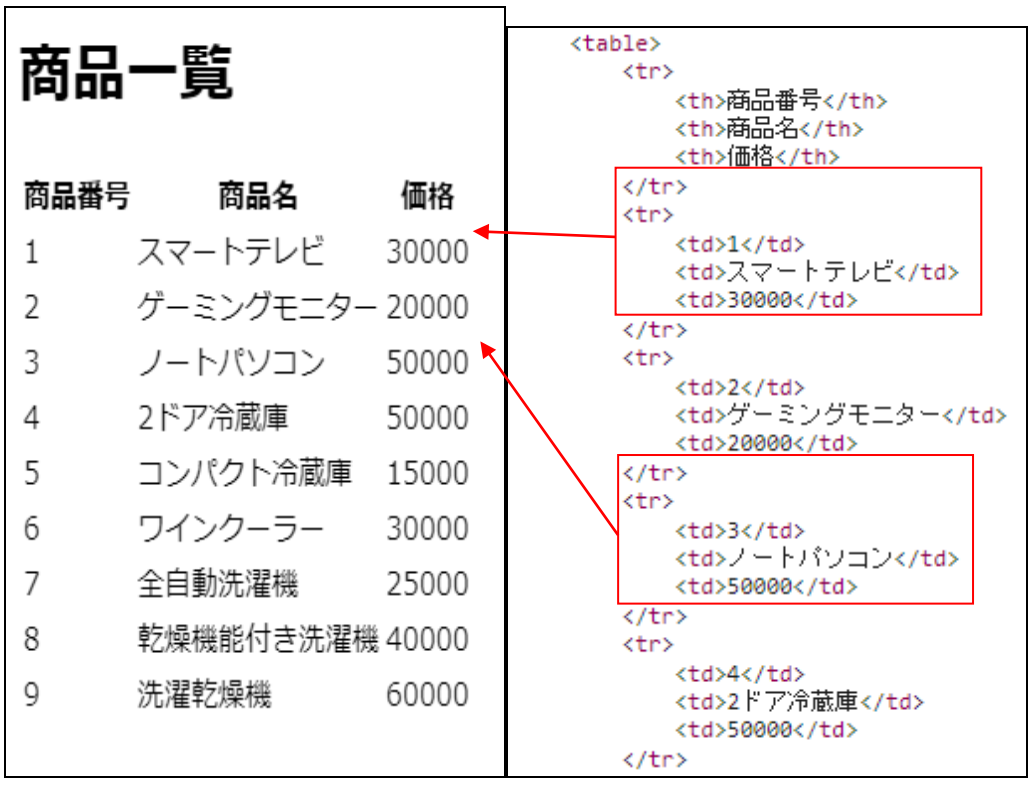
今回は、全部で 9 個のデータが products リストの中に入っているので、products リストの要素が product 変数に 9 回代入されます。**:を中心にして、右は Controller の model で格納したキー名、左はリストの中に格納されているものを意味する変数名を使います。**
今回は、リストの中に、複数の product インスタンスが格納されているため、「 product」としました。

9-11 行目 <td th:text="${product.id}">、<td th:text="${product.name}">～～

product 変数に 1 行～9 行までのデータが順番に格納されていますので、「product.フィールド名」でデータを出力することができます。つまり、各 product オブジェクトの id、name、price を<td>の中に 9 回出力するソースコードが生成されます。

```
<tr th:each="product: ${products}">
      <td th:text="${product.id}"></td>
      <td th:text="${product.name}"></td>
      <td th:text="${product.price}"></td>
</tr>
```

<th:each>と<th:text>によって実際に生成される HTML は、以下のようになります。



このテンプレートは、Spring Boot アプリケーションの ProductController の showProductList メソッドによって返された products リストの内容を、ブラウザ上で見やすい表形式で表示するために使用されます。Thymeleaf の th:each を使用することで、リストの各要素に対してテーブルの行が自動的に生成され、リスト内のデータが簡単かつ効率的に表示されます。

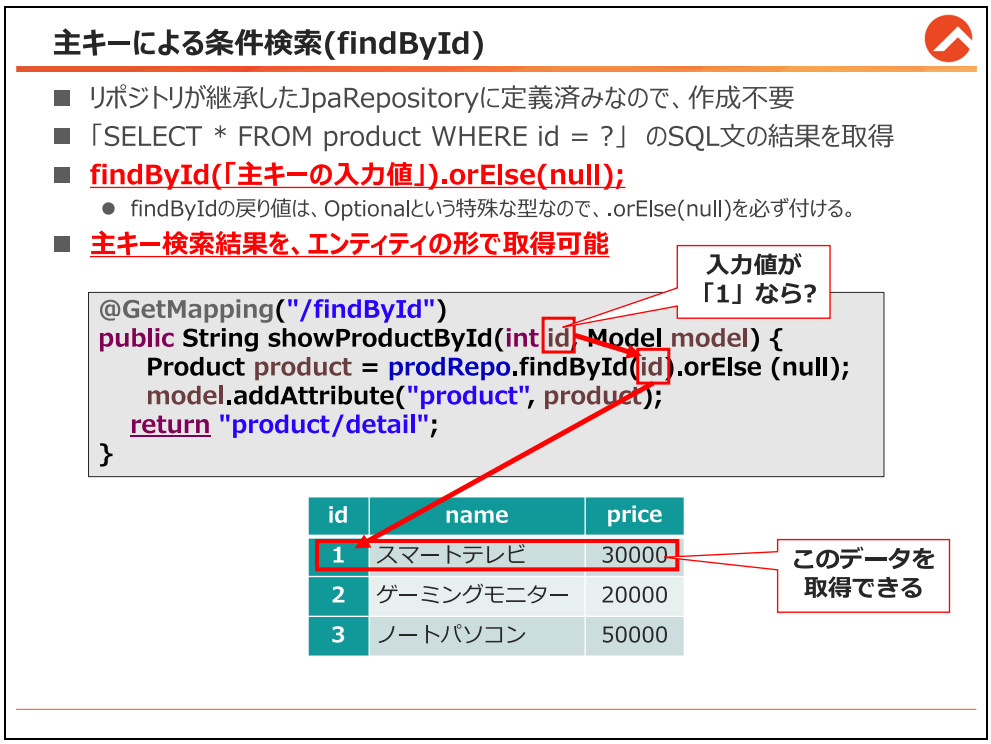
動作確認

http://localhost:8080/product/findAll

Controller の URL マッピングが、[/product]
メソッドの URL マッピングが[/findAll] になっているため、このような URL を指定します。

th:each により、<tr>全体が繰り返しされるため、findAll メソッドで取得して View に送られてきたリストを 1 行ずつ出力することができます。

3.5. 主キーによる条件検索



前の節では、findAll()メソッドを使って、全件検索ができるという内容をご説明しました。

この節では「SELECT * FROM WHERE id = ?」の SQL 文のような、主キーを使った条件検索の仕方について説明します。

検索用ページ作成

まずは、検索する ID を入力してもらうためのページを作成します。

そのためには、ページへ遷移させるためのメソッドを作成する手順が先です。

【 作業手順①】 遷移用のメソッド作成

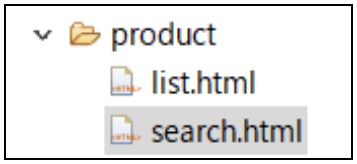
…/controller/ProductController.java

```
//showProductList メソッドの下に追記

@GetMapping("/")
        public String showSearchView() {
                return "product/search";
        }
```

/product を URL で入力すると、呼び出されるページ遷移用のメソッドを作成しましょう。

遷移先は、/product/search.html ですので、次は product フォルダ内に、search.html を作成します。

【 作業手順②】 検索入力 View の作成



…/resources/templates/product/search.html

```
<!-- 省略 -->
<body>
        <h1>検索</h1>
        <h2>ID 検索</h2>
        <form th:action="@{/product/findById}">
                商品番号：<input type="text" name="id" />
                <input type="submit" value="検索">
        </form>
</body>
```

submit ボタンをクリックすると、[/product/findById]のメソッドを呼び出すように記載します。

送られる商品番号の name は、[id]です。

【 作業手順③】 主キー検索メソッドを作成

…/controller/ProductController.java

```
1://ProductController クラスの showSearchView メソッドの下に追記
2:
3:@GetMapping("/findById")
4:      public String showProductById(int id, Model model) {
5:              Product product = prodRepo.findById(id).orElse(null);
6:              System.out.println(product);
7:              model.addAttribute("product", product);
8:              return "product/detail";
9:      }
```

4 行目 showProductById(int id, Model model)

View から送信される主キーの name 属性が「id」ですので、引数名を一致させることで、リクエストパラメータを取得します。

5 行目　Product product = prodRepo.findById(id).orElse(null);

findById メソッドは、findAll と同じく、継承した JpaRepository の親に定義されているメソッドです。**引数には主キーを入れて、後ろに.orElse(null)を付けることで、主キー検索による、1 行の結果を取得**することができます。

.findById メソッドの戻り値は「Optional」という特殊な戻り値ですので、.orElse(null)をつけないとデータを取得できません。ご注意ください。

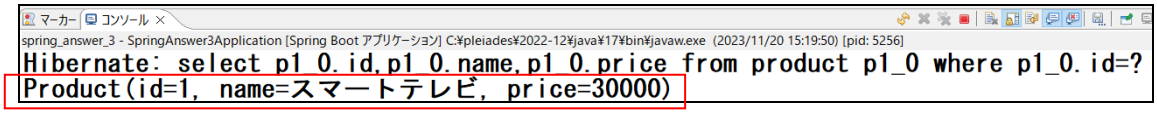
※.orElse(null)の代わりに、.get() というメソッドもありますが、検索結果がない場合は null ではなく、例外を出してアプリケーションを強制終了されてしまうので、お勧めしません。

findAll は、全件検索だったため、複数行が取得されることの前提でした。よって List<Product>型で受け取りました。

主キー検索は、必ず 1 行のみ取得できるので、List での取得よりは、エンティティ単位での[Product product]で受け取るのが一般的です。

6 行目　System.out.println(product);

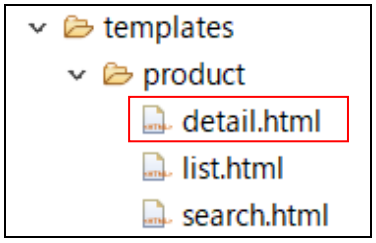
検索する商品番号に「1」を入れて実行させてみると、以下のように 1 行のデータを取得できているのがわかります。



7 行目　model.addAttribute("product", product);

取得した 1 行のデータは、Product エンティティの中に格納されているので、View に出力させるため、model に格納します。

【 作業手順④】 商品詳細ページの作成



product フォルダに、[detail.html]を作成します。

…/resources/templates/product/detail.html

```
<h1>商品詳細</h1>
        <table>
                <tr>
                        <th>商品番号</th>
                        <th>商品名</th>
                        <th>価格</th>
                </tr>
                <tr>
                        <td th:text="${product.id}"></td>
                        <td th:text="${product.name}"></td>
                        <td th:text="${product.price}"></td>
                </tr>
        </table>
```

showProductById メソッドの model に追加したキー名は、「product」ですので、th:text で各項目を表示させます。

Model に格納されているのは、複数のデータではなく、商品 1 個分のデータですので、th:each を使う必要なく、そのまま model のキー名で出力できます。

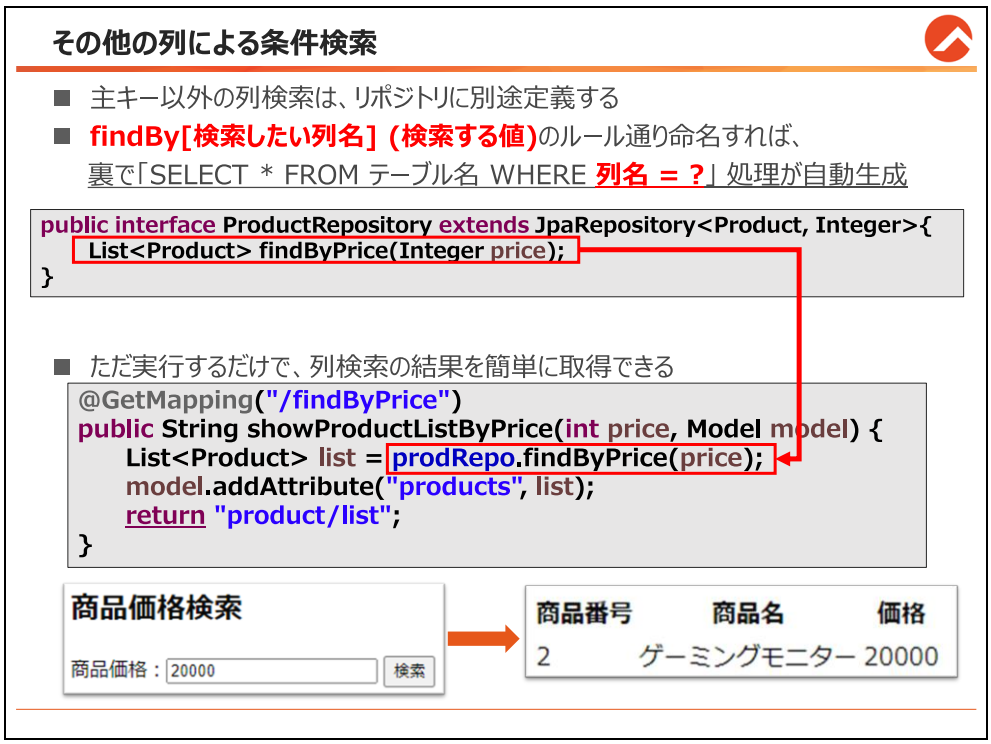
動作確認

http://localhost:8080/product/　←product の後ろに必ず/を付けてください

商品番号に、「1」と入力し、検索ボタンをクリックします。

商品詳細画面へ遷移し、商品番号 1 番のデータが出力されたら、OK です。

3.5.1. 主キーではない列による条件検索



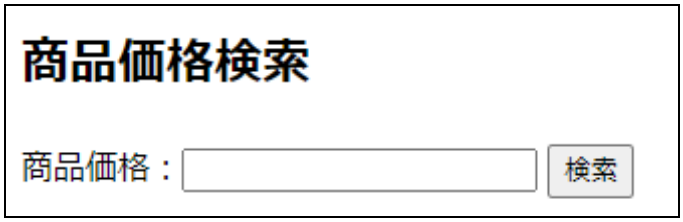
主キー以外の列による検索について

主キーは、汎用的に「id」という名称で使われますので「findById」というメソッド名が既に定義されていますが、主キー以外の普通の列による検索をする時を考えてみましょう。主キー以外の列名の検索は、列名を組み合わせたメソッドを開発者側から定義します。

Spring Data JPA を作った人は、利用者がどんな列名で検索をして来るかまでは予想できません。よって、**利用者自身がメソッド名に、それぞれの列名(正確には、エンティティ内のフィールド名)を付けることで、条件検索ができる**環境をリポジトリに用意しています。

この節では、**任意の列における検索メソッドである findBy〇ooo メソッドをリポジトリに追記**する方法についてご説明します。

【 作業手順①】 価格による検索フォームの追加



…/resources/templates/product/search.html

```
<!--ID 検索の下に追記　-->

<h2>商品価格検索</h2>
        <form th:action="@{/product/findByPrice}">
                商品価格：<input type="text" name="price" />
                <input type="submit" value="検索">
        </form>
```

前の節で作成した、search.html に、今度は商品価格検索フォームを追加しましょう。

リクエストパラメータは、price になり、submit ボタンをクリックすると、「 /product/findByPrice」を URL マッピングしている Controller 側のメソッドが実行されます。

【 作業手順②】 price 列で条件検索をするメソッドをリポジトリに追加

…/repository/ProductRepository.java

```
//Product の import に注意してください。
public interface ProductRepository extends JpaRepository<Product, Integer>{
        List<Product> findByPrice(Integer price);
}
```

次は、リポジトリに抽象メソッドを書き込みます。

一見、インターフェースに抽象メソッドを記載しただけに見えますが、**リポジトリは、決まったルール通りのメソッド名を合わせるだけで、本当にその処理をするメソッドを裏で作ってくれます。**

例えば「**findBy[エンティティのフィールド名](入力値の変数)**」というルールに合わせれば、

```
SELECT * FROM product WHERE price = ○○;
```

という SQL 処理をするメソッドの処理が自動で完成されます。

product テーブルには**同じ価格を持つ商品が複数存在する可能性があるため、このメソッドの戻り値はリスト形式**になっています。もし、検索結果が複数行にならない条件検索をしたいなら、戻り値を「Product」のような、エンティティにすれば良いです。**出力の形を、戻り値を使って選べる**ということです。

findBy 列名 のメソッドを定義する際に注意しないといけないのは、**エンティティに定義されているフィール名と列名に 1 文字でも誤字があったら、アプリケーションが起動しなくなります**ので、特にリポジトリを定義する時は、小文字・大文字や誤字に注意しましょう。

例）findByprice / findbyPrice /FindByPrice / findByPlice→ NG

【 作業手順③】 showProductListByPrice メソッドを作成し、findByPrice のメソッドを実行

…/controller/ProductController.java

```
1://showProductById メソッドの下に追記
2:
3:@GetMapping("/findByPrice")
4:      public String showProductListByPrice(int price, Model model) {
5:              List<Product> list = prodRepo.findByPrice(price);
6:              System.out.println(list);
7:              model.addAttribute("products", list);
8:              return "product/list";
9:      }
```

5 行では、リクエストパラメータ price の値を取得し、prodRepo.findByPrice メソッドを実行しました。

findAll メソッドを実行した時と、出力結果は同じく List になるメソッドですので、6 行以降は showProductList メソッドと全く同じです。

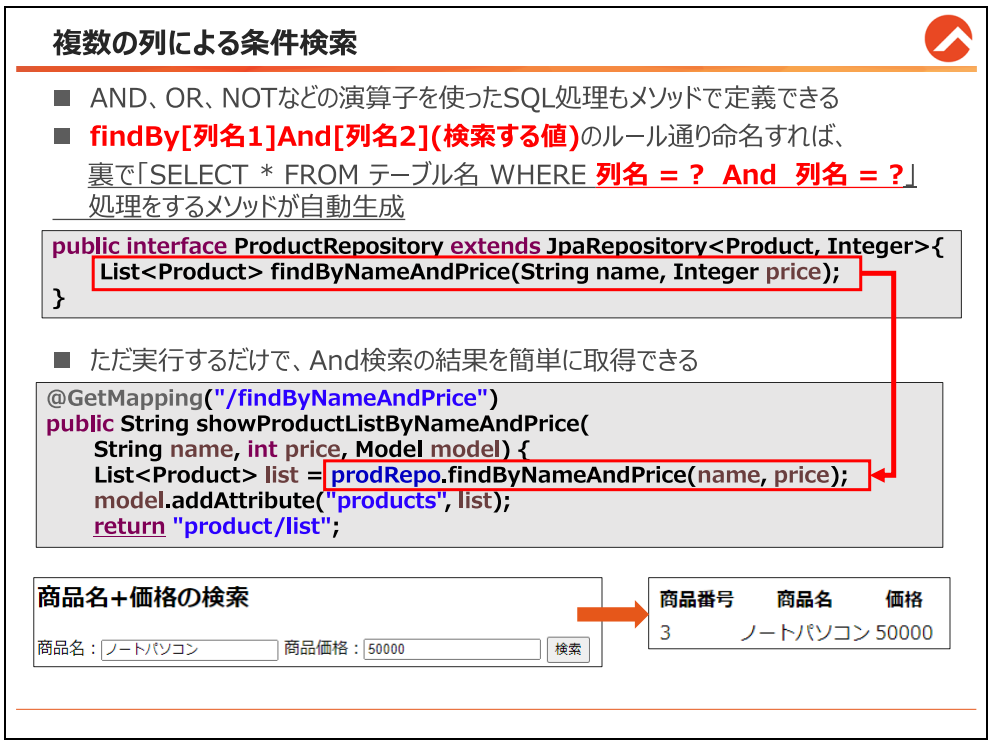
動作確認

http://localhost:8080/product/　←product の後ろに必ず/を付けてください

商品価格フォームに、[20000]と入力し、検索ボタンをクリックします。

商品一覧画面へ遷移し、価格が 20000 のデータが出力されたら、OK です。

3.5.2. 複数の列による条件検索



複数の列による検索について

Spring Data JPA では、「 AND」や「 OR」「 NOT」などの演算子を使った SELECT 文もメソッドとして定義できるようになっています。

では、「 複数の列検索用のメソッド」を定義する方法についてご説明します。

findBy[エンティティのフィールド名]と同様に、リポジトリに追記しましょう。

【 作業手順①】 商品名と価格の検索フォームの追加

…/resources/templates/product/search.html

```
<!--商品価格検索の下に追記 -->

<h2>商品名+価格の検索</h2>
        <form th:action="@{/product/findByNameAndPrice}">
                商品名：<input type="text" name="name" />
                商品価格：<input type="text" name="price" />
                <input type="submit" value="検索">
        </form>
```

前の節で作成した、search.html に、今度は商品名と商品価格両方を使った検索フォームを追加しましょう。

リクエストパラメータは、name と price になり、submit ボタンをクリックすると、「 /product/findByNameAndPrice」を URL マッピングしている Controller 側のメソッドが実行されます。

【 作業手順②】 name と price 列で条件検索をするメソッドをリポジトリに追加

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{
        List<Product> findByPrice(Integer price);
        List<Product> findByNameAndPrice(String name, Integer price);
}
```

次は、リポジトリに追加の抽象メソッドを書き込みます。

今回のルールは、「 findBy[エンティティのフィールド名 1]And [エンティティのフィールド名 2](入力値 1 の変数, 入力値 2 の変数)」です。

```
SELECT * FROM product WHERE name = ○○ AND  price = △△;
```

という SQL 処理をするメソッドが裏で完成されます。

product テーブルには同じ名前で同じ価格を持つ商品が複数存在する可能性があるため、このメソッドの戻り値はリスト形式になっています。もし、検索結果が複数行にならない条件検索をしたいなら、戻り値を「Product」のような、エンティティ型にすれば良いです。

【作業手順③】 showProductListByNameAndPrice メソッドを作成し、findByNameAndPrice メソッドを実行

…/controller/ProductController.java

```
1://showProductListByPrice メソッドの下に追記
2:
3:@GetMapping("/findByNameAndPrice")
4: public String showProductListByNameAndPrice(String name, int price,
                    Model model) {
5:      List<Product> list = prodRepo.findByNameAndPrice(name, price);
6:      System.out.println(list);
7:      model.addAttribute("products", list);
8:      return "product/list";
9: }
```

5 行では、リクエストパラメータ name と price の値を取得し、prodRepo. findByNameAndPrice メソッドを実行しました。

もし、入力値が更に増えるなら、Form クラスを使ってパラメータ取得しても良いでしょう。

6 行以降は showProductList メソッドと全く同じです。

このような AND 検索は、ログイン ID とパスワードのチェックの時、非常に使われる検索の仕方ですので、是非活用してください。

AND 以外にも、OR や NOT も同じ方法でメソッドを定義すれば、その処理のメソッドが裏で自動生成されますので、とても便利です。

参考として、以下のように複数の演算子を組み合わせるのもできます。

例：findByNameOrPriceAndId

動作確認

http://localhost:8080/product/　←product の後ろに必ず/を付けてください

フォームに、[ノートパソコン]と、[50000]と入力し、検索ボタンをクリックします。

商品一覧画面へ遷移し、入力値が商品名と価格に一致しているデータが出力されたら、OK です。

3.5.3. その他の条件検索

その他の条件検索

キーワード	記述形式	SQLでの機能	メソッド定義の例
Or	findBy[フィールド名1]Or[フィールド名2]	論理演算子「OR」	List<Product> findByIdOrPrice(int id, int price);
Like	findBy[フィールド名]Like　又は findBy[フィールド名]Containing	述語「LIKE」	List<Product> findByNameContaining (String name);
IsNull	findBy[フィールド名]IsNull	演算子「IS NULL」	List<Product> findByPriceIsNull();
IsNotNull	findBy[フィールド名]IsNotNull	演算子「IS NOT NULL」	List<Product> findByPriceIsNotNull();
OrderBy	findBy[検索対象のフィールド名]OrderBy[整列の基準となるフィールド名]ASC or DESC	ORDER BY 句	List<Product> findByNameOrderByPriceDesc (String name);
GreaterThan	findBy[フィールド名]GreaterThan	演算子「>」	List<Product> findByPriceGreaterThan(int price);
LessThan	findBy[フィールド名]LessThan	演算子「<」	List<Product> findByPriceLessThan(int price);
Between	findBy[フィールド名]Between	演算子「BETWEEN」	List<Product> findByPriceBetween(int start, int end);
Count	CountBy[フィールド名] [条件]	集合関数COUNT	int countByPriceLessThan(int price);

Spring Data JPA で使用できるメソッドの命名規則を表形式でまとめると、以上のようになります。これらの規則を使うことで、Java のメソッド名を通じて SQL クエリに相当する機能を実現できます。

最後の行の Count については、findBy～ではない定義方法です。該当の条件での検索結果が何件なのか、整数で取得できます。

関数を使うなど、普通の SELECT 文のとは異なる場合は、必ずしも findBy で始まるわけではなく、他にも色んな種類の定義方法がありますので、調べましょう。

※Like の場合、findBy[列名]Like と定義して呼び出しの際は、引数の前後に「%」のようなワイルドカードを付ける必要があります。

例）List<Product> list = prodRepo.findByNameLike("%" + name + "%");

%を前後に付けないと不具合が起きるため、Spring 公式ドキュメントでは、
findBy[列名]Containing を推奨しています。

※全件検索を価格順(降順)で並び替えする時

例）List<Product> list = prodRepo.findAllByOrderByPriceDesc();

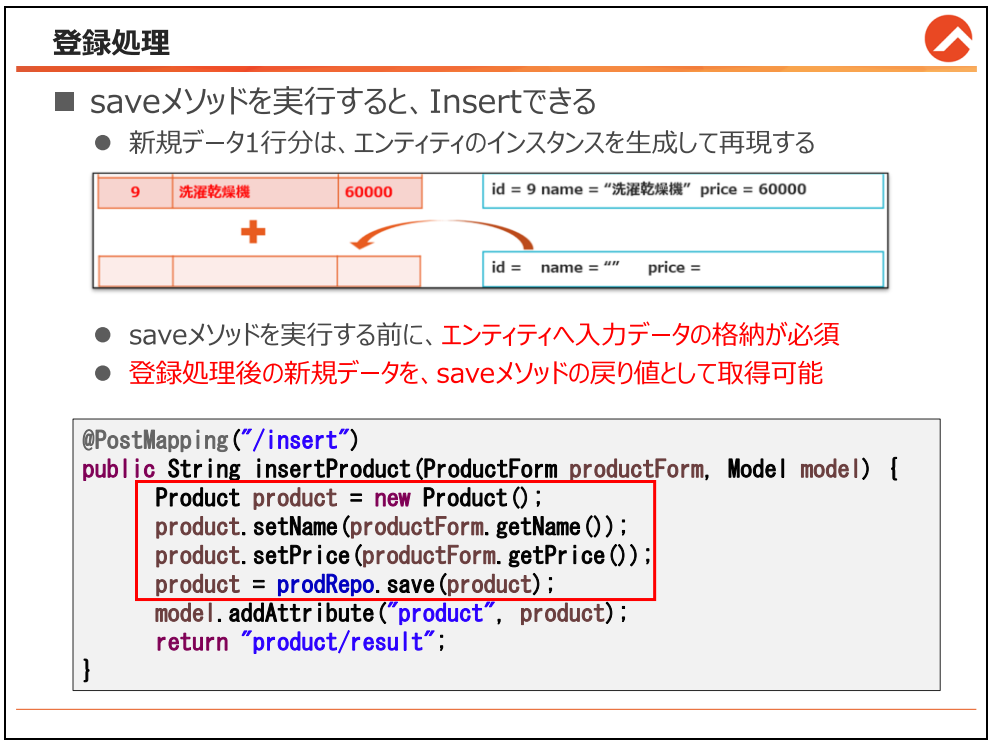
その他のメソッドの例

```
public interface ProductRepository extends JpaRepository<Product,
Integer>{
        List<Product> findByPrice(Integer price);
        List<Product> findByNameAndPrice(String name, Integer price);

        // その他のメソッドの例

        // SQL: SELECT * FROM product WHERE id = ? OR price = ?;
        List<Product> findByIdOrPrice(int id, int price);
        // SQL: SELECT * FROM product WHERE name <> ?;
        List<Product> findByNameNot(String name);
        // SQL: SELECT * FROM product WHERE name LIKE ?;
        List<Product> findByNameLike(String name);
        // SQL: SELECT * FROM product WHERE price IS NULL;
        List<Product> findByPriceIsNull();
        // SQL: SELECT * FROM product WHERE price IS NOT NULL;
        List<Product> findByPriceIsNotNull();
        // SQL: SELECT * FROM product WHERE name = ? ORDER BY price DESC;
        List<Product> findByNameOrderByPriceDesc(String name);
        // SQL: SELECT * FROM product WHERE price > ?;
        List<Product> findByPriceGreaterThan(int price);
        // SQL: SELECT * FROM product WHERE price < ?;
        List<Product> findByPriceLessThan(int price);
        // SQL: SELECT * FROM product WHERE price BETWEEN ? AND ?;
        List<Product> findByPriceBetween(int start, int end);
        // SQL: SELECT COUNT(*) FROM product WHERE price < ?;
        int countByPriceLessThan (int price);

}
```

どんな列を使い、どんな条件を使うかを考えて、自由自在の組み合わせで、異なる SQL 処理を作成できますので、単純なデータベースの操作に使う場合は、高い作業効率を期待できます。

しかし、結合やサブクエリなどを使う複雑な SQL 文については、JPQL や MyBatis のような別のライブラリを利用して、SQL 文を直接実行する方法を採用する場合もありますので、参考にしてください。

3.6. 登録処理



新規データ登録について

今までは、検索関連のデータベース操作を行う方法について学びました。次は、データを新しく登録する方法について学びます。

既存のデータベースプログラミングだと、登録も手間がかかるのですが、Form クラスと、Spring Data JPA の「save」メソッド使うことで、より簡単に登録作業ができます。

データ入力フォーム作成

まずは、ユーザから登録データを入力してもらうための View を作成するところからです。

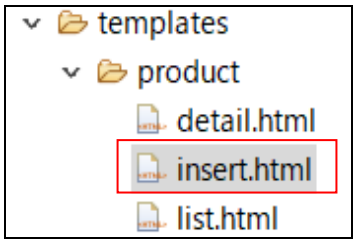
【 作業手順①】 入力 View 遷移用メソッドの定義

…/controller/ProductController.java

```
//findByNameAndPrice メソッドの下に追記

@GetMapping("/insert")
	public String showProductInsertView() {
		return "product/insert";
	}
```

まず、product フォルダの insert.html へ遷移させるだけの、View 遷移用メソッドを作成します。product/insert の URL がリクエストされると、このメソッドにより、入力ページへ遷移するようにします。

【 作業手順②】 入力 View 作成



…/resources/templates/product/insert.html

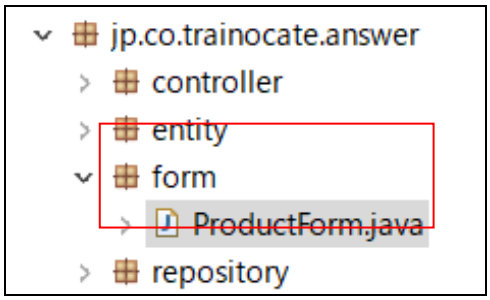
```
<!--<body>前は省略-->
<body>
        <h1>商品登録</h1>
        <form th:action = "@{/product/insert}" method = "POST">
        <p>商品番号：(自動割り当て)</p>
        <p>商品名　：<input type = "text" name = "name"></p>
        <p>商品価格：<input type = "text" name = "price"></p>
        <input type = "submit" value = "登録">
        </form>
</body>
```

th:action 属性値にご注目ください。Viewへ遷移する時も、「 /product/insert」で、submit ボタンをクリックした時も、「 /product/insert」にしていますが、POST 形式でリクエストを送っているため、別々のメソッドを呼び出すことができます。

商品登録(id)は、自動割り当てになっているため、入力しないようにします。

【 作業手順③】 ProductForm の定義

登録データは、通常複数のデータを扱うため、2 章でご紹介しました Form クラスを利用すると便利です。現在、Product エンティティはあるものの、ProductForm が定義されていませんので、定義します。

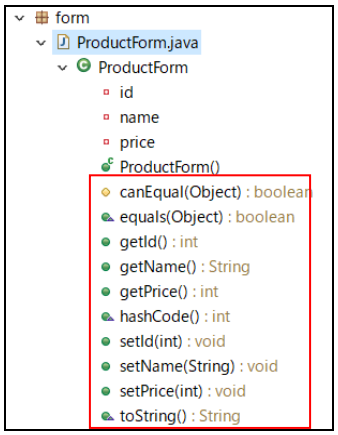


…/form/ProductForm.java

```
1:package jp.co.trainocate.answer.form;
2:
3:import lombok.Data;
4:
5:@Data
6:public class ProductForm {
7:	private Integer id; //商品番号
8:	private String name; //商品名
9:	private Integer price; //商品価格
10:}
```

商品関連のデータを扱うために、[id]と、[name], [price]をフィールドとして持つ Form クラスを作成しました。[id]は登録時には使われませんが、他の処理で使われるので、書くべきです。

5 行の@Data ですが、エンティティの定義の際にご説明しました通り、**@Data アノテーションをつけると、Getter&Setter メソッドは勿論、equals や toString、hashCode メソッドが自動生成されますので、とても便利です。**



【 作業手順④】 登録処理メソッドの定義

…/controller/ProductController.java

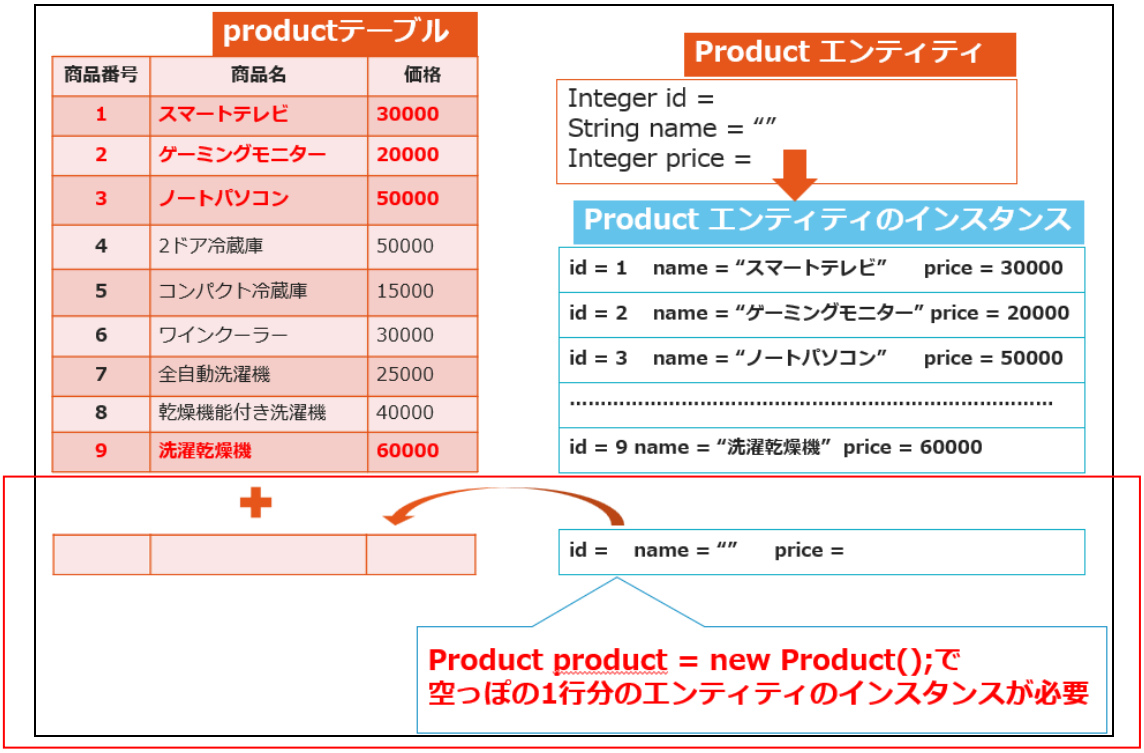
```
1:  //showProductInsertView()の下に追記
2:
3:  @PostMapping("/insert")
4:      public String insertProduct(ProductForm productForm, Model model) {
5:              Product product = new Product();
6:              product.setName(productForm.getName());
7:              product.setPrice(productForm.getPrice());
8:              product = prodRepo.save(product);
9:              model.addAttribute("product", product);
10:             return "/product/result";
11:     }
```

・4 行目: public String insertProduct(ProductForm productForm) {

ユーザからの複数の入力値を受け取るために、productForm クラスをメソッドの引数に入れます。リクエストデータは、Form クラスのフィールドに自動でセットされます。

・5 行目: Product product = new Product();

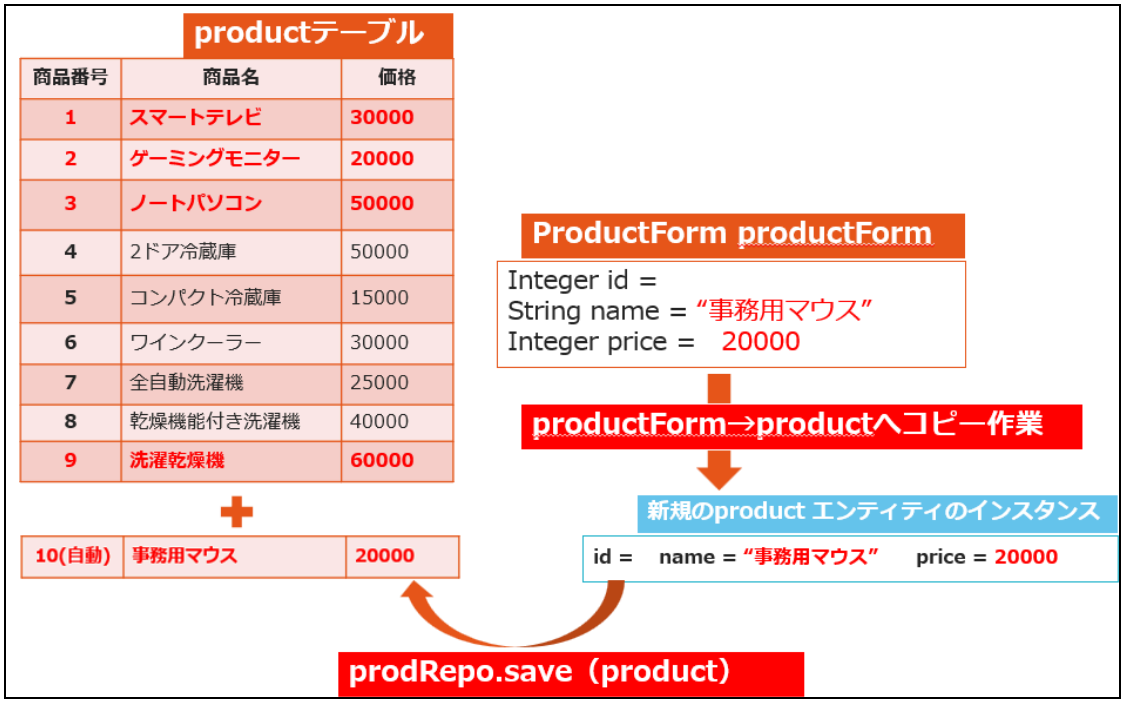
商品の登録をするためには、**予め商品 1 個分をを表す、エンティティのインスタンスを生成する必要がありますので**、new で Product のオブジェクトを生成します。



・8 行目: product = prodRepo.save(product);

先に説明します。Insert の SQL 文を実行させるためには、findAll と同じく、既にリポジトリに定義されている、[save]メソッドを実行すれば、登録処理が完了します。

しかし、エンティティのインスタンスである product は、5 行でオブジェクト生成はしたものの、データを入れないと、空っぽの状態のままになってしまいます。ユーザからのデータを受け取っているのは、product ではなく、4 行の ProductForm の方ですので、データをコピーする必要があります。



・6～7 行目: product.setName(productForm.getName());
product.setPrice(productForm.getPrice());

save メソッドの実行前に、Form インスタンス内のデータを、product へコピーさせます。

この作業により、空っぽだった product にも、ユーザからのデータが格納されました。

・8-9 行目: product = prodRepo.save(product);
model.addAttribute("product", product);

戻り値としては、登録完了した商品情報をエンティティのインスタンスとして取得できます。ここには、自動割り当てされた id も含まれます。結果画面へ表示するため、model に格納します。

・10 行目: return "/product/result";

本当に登録が完了したのか確認するために、商品データを出力する結果画面を表示させます。

…/resources/templates/product/result.html

```
1:<h1>商品・更新結果画面</h1>
 2:     <table>
 3:             <tr>
 4:                     <th>商品番号</th>
 5:                     <th>商品名</th>
 6:                     <th>価格</th>
 7:             </tr>
 8:             <tr>
 9:                     <td th:text="${product.id}"></td>
10:                     <td th:text="${product.name}"></td>
11:                     <td th:text="${product.price}"></td>
12:             </tr>
13:     </table>
14:     <a th:href="@{/product/findAll}">商品一覧へ</a>
15:</body>
```

9 行目の product.id に注目してください。save メソッドの戻り値には、自動採番により割り当てされた ID 番号も含まれているので、表示できます。

動作確認

http://localhost:8080/product/insert

任意の値を入力し、[登録]ボタンをクリックすると、登録後の全件検索結果が表示されます。

「 商品一覧」をクリックすると、一覧画面に遷移します。

一覧でも新規データを確認できます。

【参考】 ModelMapper で「Form→Entity のデータコピー」を簡単にする方法

今回の例では、入力値が 2 つのみでしたので、コピー作業も 2 行で終わりましたが、
例えば、会員登録ページとかなら、10 個以上の値を扱います。
その結果、10 行以上コードを書くことになり、冗長なコードになってしまいます。

その問題を解決するために、「ModelMapper」という外部ライブラリを利用すると便利です。

pom.xml

```
<dependency>
		<groupId>org.modelmapper</groupId>
		<artifactId>modelmapper</artifactId>
		<version>3.0.0</version>
</dependency>
```

まず、Maven を使って、ModelMapper を導入します。

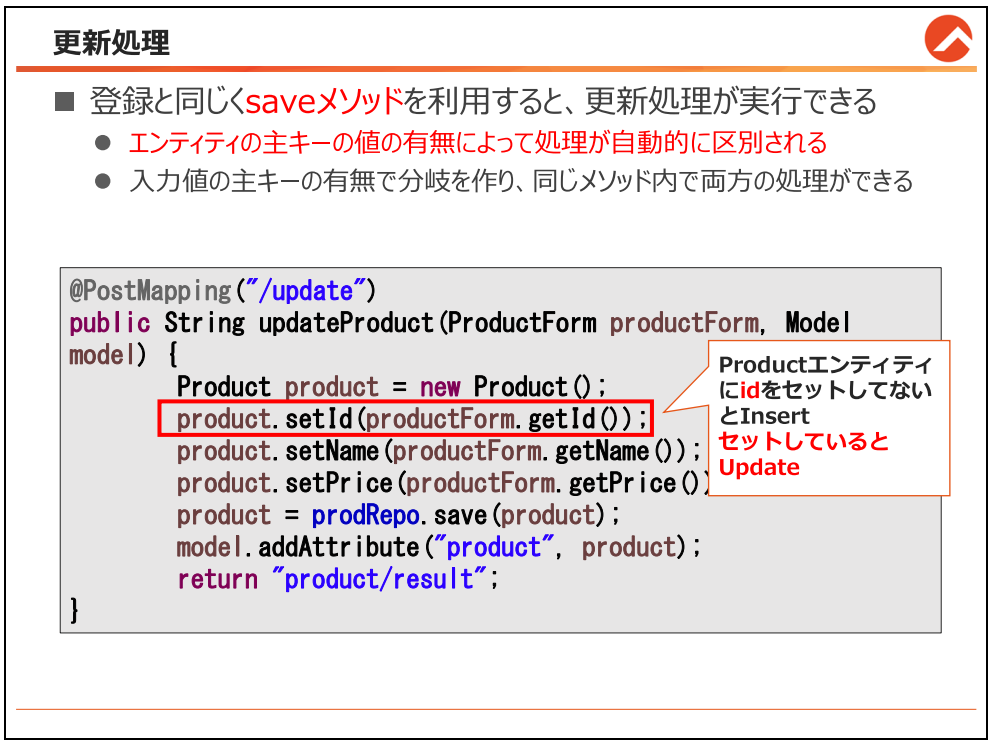
…/controller/ProductController.java

```
@PostMapping("/insert2")
public String insertProduct2(ProductForm productForm, ModelMapper modelMapper,
Model model) {

	Product product = modelMapper.map(productForm, Product.class);
	product = prodRepo.save(product);
	model.addAttribute("product", product);
	return "/product/result";
}
```

ModelMapper を利用するために、メソッドに上記のように ModelMapper のインスタンスを生成した後、
modelMapper.map(Form 名, Entity 名.class) を実行した結果をエンティティの変数に代入すると、**自動で Form クラス内のフィールド名と一致するエンティティ内のフィールドにコピーが行われます。**

扱う入力値が多くなったら、是非こちらの方法を活用してみてください。

3.7. 更新処理



save メソッドは更新処理も可能

登録と更新処理における Spring Data JPA の使い勝手の良さの一つは、両方の操作が同じ save メソッドを使っていることです。

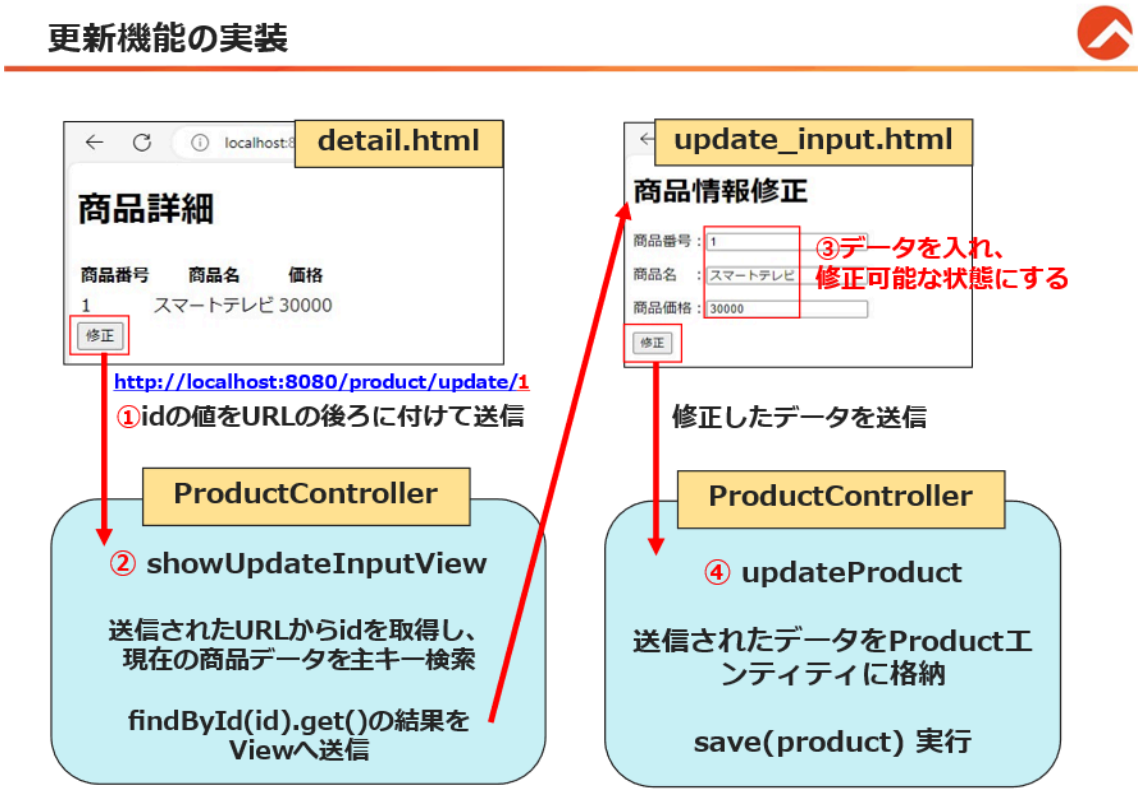
save メソッドを使用することで、新しいレコードの挿入(登録)だけでなく、既存レコードの変更(更新)も簡単に行えます。この柔軟性の秘密は、**エンティティの主キーの値の有無によって処理が自動的に区別される**点にあります。

登録(INSERT)処理:
新しいオブジェクト(エンティティ)を作成し、そのオブジェクトに主キーがまだ割り当てられていない場合(null またはデフォルト値)、save メソッドは新しいレコードをデータベースに挿入する SQL INSERT 文を実行します。

更新(UPDATE)処理:
主キーが割り当てられているエンティティを save メソッドの引数に渡すと、Spring Data JPA は UPDATE 文を実行します。ここで重要なのは、主キーがすでに割り当てられているエンティティは、データベースにすでに存在しているとみなされるため、更新処理が行われます。

このアプローチは、コードの再利用性を高め、データベース処理をより直感的に行えるようにします。特に、**開発者が明示的に INSERT や UPDATE を意識する必要がなく、エンティティの状態に応じて Spring が適切な処理を自動的に行ってくれるため、開発効率が大幅に向上**します。エンティティのライフサイクル管理がこれほどシンプルになると、データモデルとビジネスロジックにより集中することができ、より洗練されたアプリケーションの開発が可能になります。

では、本格的な実装の前に、まずは流れを把握しましょう。



③の修正画面を開いた時、フォームには**現在の商品データが入力されている状態**です。それは、何もしなくても入力されているわけではなく、**事前に手順①と②を行った結果**です。

更新処理は、他の処理とは違い、事前準備が必要だということを、是非理解した上で、進んでください。

【 作業手順①】 詳細画面に、更新ボタンを実装するように修正

更新処理は、まず更新させたい対象を絞る必要があります。**対象を絞る時は、主キーのデータがあれば特定できますので、主キーのデータをもらう必要があります。**その方法の一つとして、本書では、検索から遷移できる「 詳細画面」にボタンを配置させました。

…/resources/templates/product/detail.html

```
<!--</table>の下に追加 -->

<form th:action="@{/product/update/}+${product.id}">
        <input type = "submit" value = "修正">
</form>
```

URL にご注目ください。/product/update/ の後ろに、+ で product.id を付加しています。このように、Controller から取得した**データ(例えば、商品の ID)を URL に組み込む**ことができます。これにより、特定の商品に関連した URL を動的に生成することが可能になります。

その結果、submit ボタンをクリックすると、以下のような **URL が動的に生成**されますので、どの商品を更新するかを Controller のメソッドへ明確に伝えることができます。

http://localhost:8080/product/update/1

では、Controller のメソッドで商品番号 1 を取得する方法についてご説明します。

【 作業手順②】 更新対象データの既存データ取得

…/controller/ProductController.java

```
1://insertProduct メソッドの下に追加
2:
3:@GetMapping("update/{id}")
4:public String showUpdateInputView(@PathVariable int id, Model model) {
5:      Product product = prodRepo..findById(id).orElse(null);
6:      model.addAttribute("product", product);
7:      return "product/update_input";
8:}
```

・3 行目: @GetMapping("update/{id}")

View から動的 URL が送られる場合、同じく動的パラメータを指定する必要があります。
書き方は、{ パラメータ名 } で記載します。

・4 行目: public String showUpdateInputView(@PathVariable int id, Model model) {

3 行で指定した動的 URL の文字列と一致する引数名に**@PathVariable アノテーションをつけると、動的 URL の値を取得**することができます。動的 URL は、{id}なので、int id で合わせます。

5 行からは、更新対象の既存の値を findById メソッドで取得+Model に格納し、
update_input.html へ遷移させています。

【 作業手順③】 修正フォーム(update_input.html)の作成

showUpdateInputView メソッドの return により、既存のデータが表示されている画面へ遷移します。

…/resources/templates/product/update_input.html

```
<h1>商品情報修正</h1>
<form th:action = "@{/product/update}" method = "POST">
<p>商品番号：
<input type = "text" name = "id" th:value = "${product.id}" readonly></input></p>
<p>商品名　：
<input type = "text" name = "name" th:value = "${product.name}"></p>
<p>商品価格：
<input type = "text" name = "price" th:value = "${product.price}"></p>
<input type = "submit" value = "修正">
</form>
```

初めての構文として、「 th:value」が登場しました。

th:value は、Controller の model に格納されているデータを HTML 上で取得して、フォーム内に表示する Thymeleaf 独自の属性です。間違えて、ただの「 value=“”」にすると、値を取得することができませんので、ご注意ください。

商品番号は主キーであり、変更してはいけませんので、readonly の属性を付与して、修正不可にしました。

「 修正」の submit ボタンをクリックすると、フォーム内の入力データ値が全てリクエストパラメータとして/product/update の URL へ送信されます。

【 作業手順④】 修正処理メソッドの定義

…/controller/ProductController.java

```
1:@PostMapping("/update")
2:public String updateProduct(ProductForm productForm, Model model) {
3:      Product product = new Product();
4:      product.setId(productForm.getId());
5:      product.setName(productForm.getName());
6:      product.setPrice(productForm.getPrice());
7:      product = prodRepo.save(product);
8:      model.addAttribute("product", product);
9:      return "/product/result";
10:}
```

・4 行目 : product.setId(productForm.getId());

登録の際は、主キーである「id」をProductエンティティ内にセットしていませんでしたが、更新処理の際は、必ず主キーのフィールドをセットさせます。

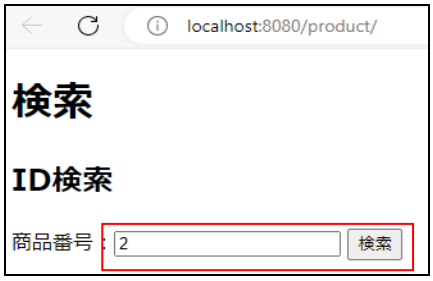
・7 行目 : product = prodRepo.save(product);

saveメソッドの引数に入れたエンティティのフィールドに、主キー(id)の値があったら、新しい値に上書きされる更新処理が実行されます。

その他のコードの解説は、登録と同じですので、割愛します。

動作確認

http://localhost:8080/product/　←product の後ろに必ず/を付けてください



商品番号に、[2]を入力し、[検索]ボタンをクリックします。

商品番号 2 番に対する商品データを確認し、[修正]ボタンをクリックします。

商品名と商品価格を任意の値で修正し、[修正]ボタンをクリックします。

商品一覧ページへ遷移し、2 番の商品が修正されていることを確認します。

【参考】 登録と更新の処理を一つにする方法

…/controller/ProductController.java

```
1:@PostMapping("/insertOrUpdate")
2:public String insertOrUpdateProduct(ProductForm productForm) {
3:     Product product = new Product();
4:     if (productForm.getId() != null) {
5:         // ID がある場合は、既存の商品として扱い、その ID を設定
6:         product.setId(productForm.getId());
7:     }
8:     product.setName(productForm.getName());
9:     product.setPrice(productForm.getPrice());
      // 新しい商品を追加または既存の商品更新
10:    product = prodRepo.save(product);
      // ～～割愛～～
```

上記のコードは、単純の分岐ですが、登録と更新を一つにまとめることができるので、開発効率を高めることができます。

3.8. 削除処理

削除については、更新と同じようなコードを使います。

【 作業手順①】 詳細画面に削除ボタンを追加

…/resources/templates/product/detail.html

```
<!-- detail.html に削除ボタンを追加 -->

<form th:action="@{/product/delete/}+${product.id}">
        <input type = "submit" value = "削除">
</form>
```

Update と同様に、th:action を活用して商品番号を URL に付けて送信します。

【 作業手順②】 削除処理のメソッドを定義

…/controller/ProductController.java

```
//省略

@GetMapping("/delete/{id}")
        public String deleteProduct(@PathVariable int id) {
                prodRepo.deleteById(id);
                return "redirect:/product/findAll";
        }
```

URL から id を取得し、どの商品を削除するかを決めます。

削除用のメソッドは、deleteById(id)を実行すれば、以下の削除 SQL 文が実行されます。

「DELETE FROM product WHERE id = ?」

本当に削除が完了したのか確認するために、全件検索ページを呼び出したいところですが、全件検索は、html ファイルではなく、findAll メソッドを実行していました。

つまり、**return で html ではなく、メソッドを呼び出す URL を指定する必要がある**ということです。

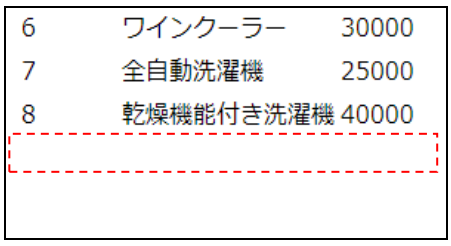
その時は、**return “redirect:/ [呼び出したい URL]”** の書き方にします。

動作確認

http://localhost:8080/product/　**←product の後ろに必ず/を付けてください**

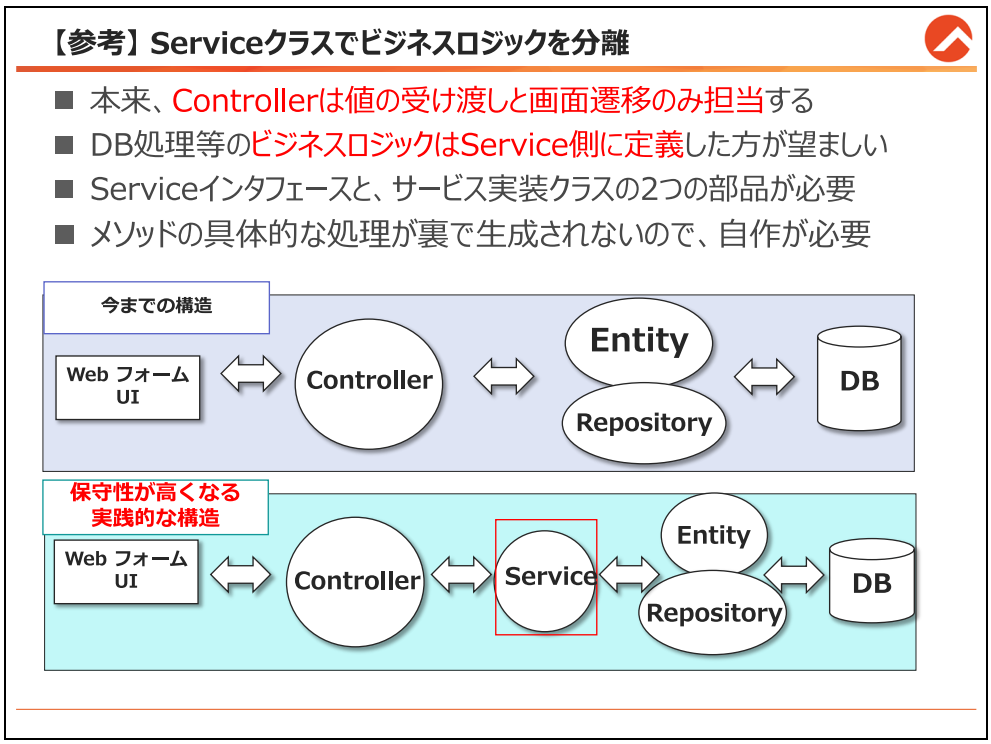
任意の商品番号を検索します。

削除ボタンをクリックします。



商品が削除されたのを確認できれば OK です。

3.9.【 参考】Service クラスでビジネスロジックを分離



これまでの例では、簡単に理解できるように、Controller クラスの中にデータベース処理などのビジネスロジックも書いていました。

しかし、本来 Controller の役割は、データのやり取りと画面の移動にあります。より良い保守性を実現するために、実際の開発現場では**「Service インターフェース」と「Service 実装クラス」と呼ばれる専門の部品を 2 つ作り**、ビジネスロジック(例えば、データベース操作)を集めます。

Service クラスの目的と役割

コントローラーは主に HTTP リクエストの受け取りとレスポンスの送信、画面遷移の管理を行います。一方で、Service クラスはビジネスロジック(例えば、データの検証、計算、データベースの操作など)を担当します。

Service クラスにビジネスロジックを集中させることで、コードの再利用性が高まり、変更や拡張が容易になります。また、コードが整理され、各クラスの役割が明確になるため、保守しやすくなります。

サービスインターフェースとサービス実装クラスを分けるメリット

インターフェースと実装クラスの 2 つに分けて作成する理由は、メリットがあるからです。
まずは、以下の例えでイメージを持ってください。

例え: テレビのリモコンと本体

■ **サービスインターフェース: テレビの「リモコン」**

テレビの電源をつけたり、音量を調整したりする「入口」。

リモコンの操作方法は変わらない（電源ボタンを押せばテレビがつく）。

■ **サービス実装クラス: テレビの「本体」**

実際に映像や音を出す機械。

本体の仕組みは変わるかもしれない（電源の音が変わるとか）が、リモコンの使い方は変わらない。

メリット①：柔軟性が高くなる

■ **分けない場合:**
実装が変わると、呼び出している全ての場所（Controller など）を修正しないといけない。

■ **分けた場合:**
リモコン（インターフェース）を使って呼び出すため、実装（テレビ本体）が変わってもリモコンの使い方は変わらない。

メリット②：実装を切り替えやすい

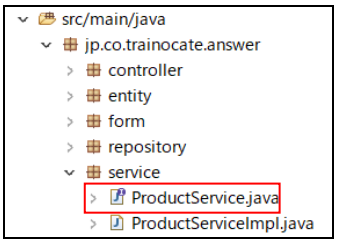
現場では、以下のような場面で実装を切り替えることがよくあります。

■ **データベース→外部 API: データの取得元を変更したい。**

■ **動作確認用のダミー: テスト環境では、ダミーのデータを返すモッククラスを使いたい。**

このような場合、実装クラスを切り替えるだけで、テスト環境や新しい仕様に対応可能になりますので、インターフェースと実装クラスを分けるわけです。

【 作業手順①】サービスインターフェースの作成



[service]パッケージを作成し、その中に[ProductService]**インターフェース**を作成します。

…/service/ProductService.java

```
package jp.co.trainocate.answer.service;

import jp.co.trainocate.answer.entity.Product;

public interface ProductService {
        public Product findProductById(int id);
}
```

学習のため、主キー検索の機能をする抽象メソッドのみ実装します。

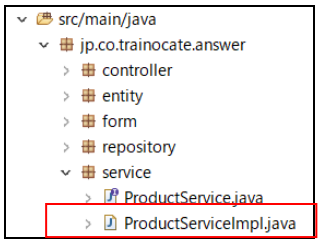
インターフェースは**リモコンだと思ってください。リモコンにはボタンが必要です。リモコンのボタンは抽象メソッドとして表現します。**

ですので、メソッドの書き方は、JPA のリポジトリに検索用のメソッドを定義していた時と似ています。この処理をするためには、どんな引数が必要そうか、どんな戻り値を戻せばいいのかを考えます。

リポジトリのような、メソッド名の規則は特にありません。どんなメソッドなのか意味が通じるメソッド名で大丈夫です。

ただのボタンですので、具体的な処理は書かずに、引数と戻り値だけ決めた抽象メソッドとして作成しておきます。

【 作業手順②】サービス実装クラスの作成



サービスインターフェースで、リモコンのボタン(抽象メソッド)の定義が完了したら、**リモコンのボタンを押した時に実際に処理をする側の、「サービス実装クラス」をセットで作成**する必要があります。

JPA のリポジトリは、リモコンのボタンさえ作っておけば、実際の処理をするメソッドは裏で自動生成されましたが、**サービスの場合は、自作で抽象メソッド一つ一つの具体的な実装が必要**です。

サービス実装クラスは、サービスインターフェース名に「Impl」を追加した名前に合わせるのが一般的です。

…/service/ProductServiceImpl.java

```
 1:import jp.co.trainocate.answer.entity.Product;
 2:import jp.co.trainocate.answer.repository.ProductRepository;
 3:import lombok.RequiredArgsConstructor;
 4:
 5:@Service
 6:@RequiredArgsConstructor
 7:public class ProductServiceImpl implements ProductService{
 8:
 9:    //@Autowired は不要
10:    private final ProductRepository prodRepo;
11:
12:    @Override
13:    public Product findProductById(int id) {
14:            return prodRepo..findById(id).orElse(null);
15:    }
16:}
```

・5 行目： @Service

サービス実装クラスには、@Service アノテーションを付けます。忘れがちですので、注意してください。

・6 行目： @RequiredArgsConstructor

3.3.1「 依存性の注入」の【 依存性の注入方法③】でご紹介した書き方です。
9-10 行目のコードと連携します。
単体テストのしやすさのために、サービス実装クラスではこの方法が推奨されます。

・7 行目： implements ProductService

前の手順で作成したサービスインターフェースと繋げるために、implements します。

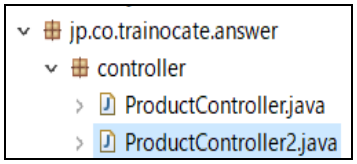
・9 行-10 行目： private final ProductRepository prodRepo;

DB 処理のため ProductRepository を利用したいので、依存性の注入を行います。
6 行目と合わせて記載することで、コンストラクタを自動生成できます。

・13 行-14 行目： return prodRepo..findById(id).orElse(null);

Controller に記載していた DB 処理部分だけ分離させ、DB から取得した商品データをそのまま return しています。ProductService インターフェースで定義した抽象メソッドと同じ呼び方をするように、オーバーライドする必要があります。
(リモコンのボタンを押したら、このメソッドの処理が行われるイメージ)

【 作業手順③】Controller から Service クラスを経由して DB 操作を行う



Service クラスを経由する Controller は、混線を避けるため、
[ProductController2]で新規作成しました。

…/controller/ProductController2.java

```
 1:@Controller
 2:@RequestMapping("product2")
 3:
 4:public class ProductController2 {
 6:    @Autowired
 7:    private ProductService prodService;
 8:
 9://～省略～
10:
11:    @GetMapping("/findById")
12:    public String showProductById(int id, Model model) {
13:           Product product = prodService.findProductById(id);
14:           model.addAttribute("product", product);
15:           return "product/detail";
16:    }
```

・2 行目: @RequestMapping("product2")

Controller が変わったので、異なる基本 URL を設定します。

・7 行目： private ProductService prodService;

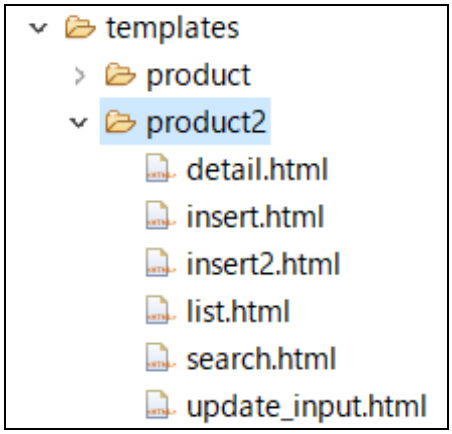
DB 処理は、ProductService の方で担当するので、このクラスで利用できるように、依存性の注入を行います。既存の ProductRepository の依存性の注入は不要になります。

・13 行目： Product product = prodService.findProductById(id);

ここが一番大事です。今までは、Controller のメソッドの中で直接「prodRepo.メソッド」を実行する書き方でしたが、今回は Service クラスに定義されているメソッドを実行させて、その戻り値を利用する書き方になりました。

「ProductService」インターフェース型のインスタンスからメソッドを呼び出していますが、「ProductServiceImpl」クラスに具体的な処理がオーバーライドされているので、実際は「ProductServiceImpl」のメソッドがポリモフィズムで実行されるようになります。

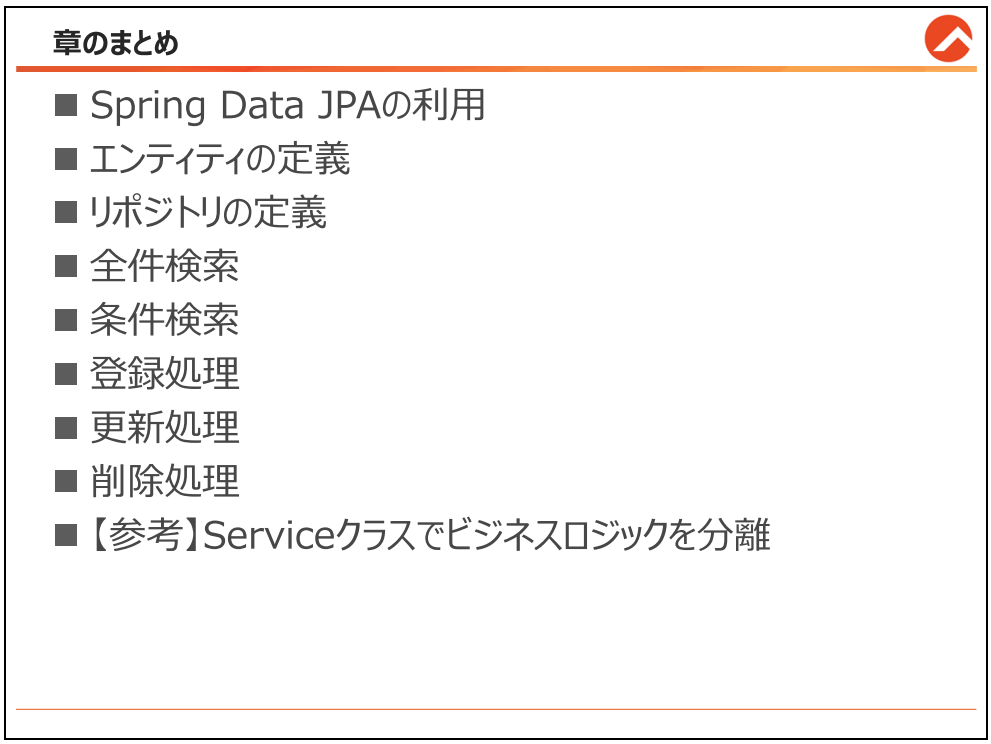
このような書き方をすることで、**Controller はあくまでデータの受け渡しの役割に集中させ、ビジネスロジック(DB 処理)は Service クラスで行わせます。**



※基礎 URL が product2 になるので、product2 フォルダに同じ html を格納する必要があります。

※その他の例は、[ProductController2.java]と、[ProductService.java]、[ProductServiceImpl.java]を参照してください。

章のまとめ



この章では、Spring Data JPA を使った、データ操作方法についてご説明しました。

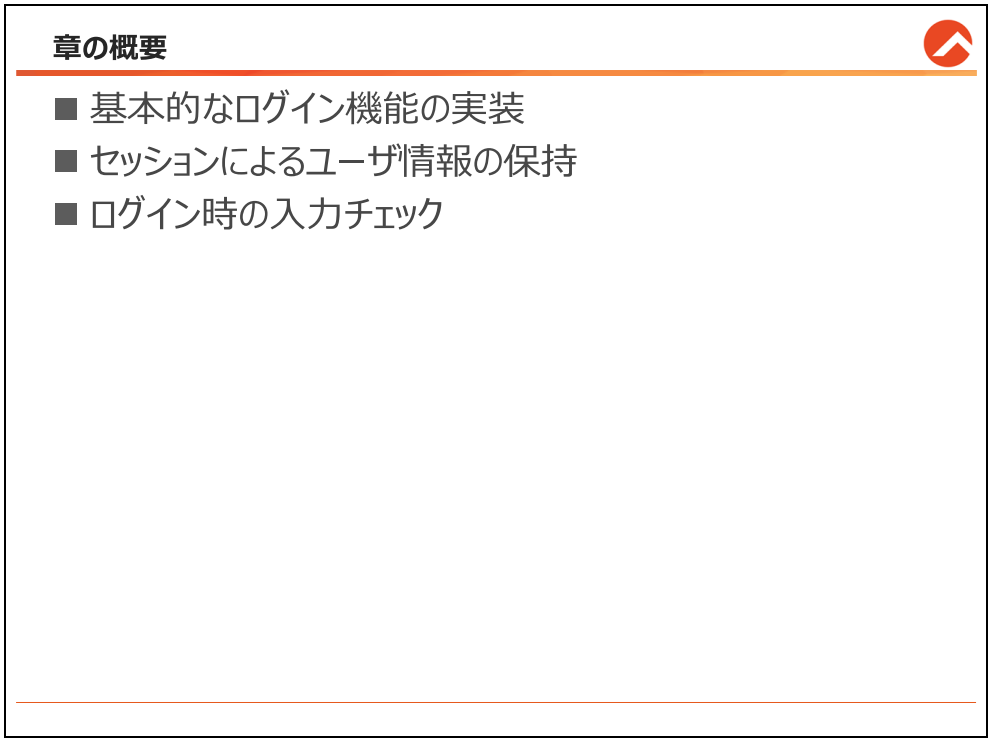
4

第4章 セッションと入力チェック

章の目標

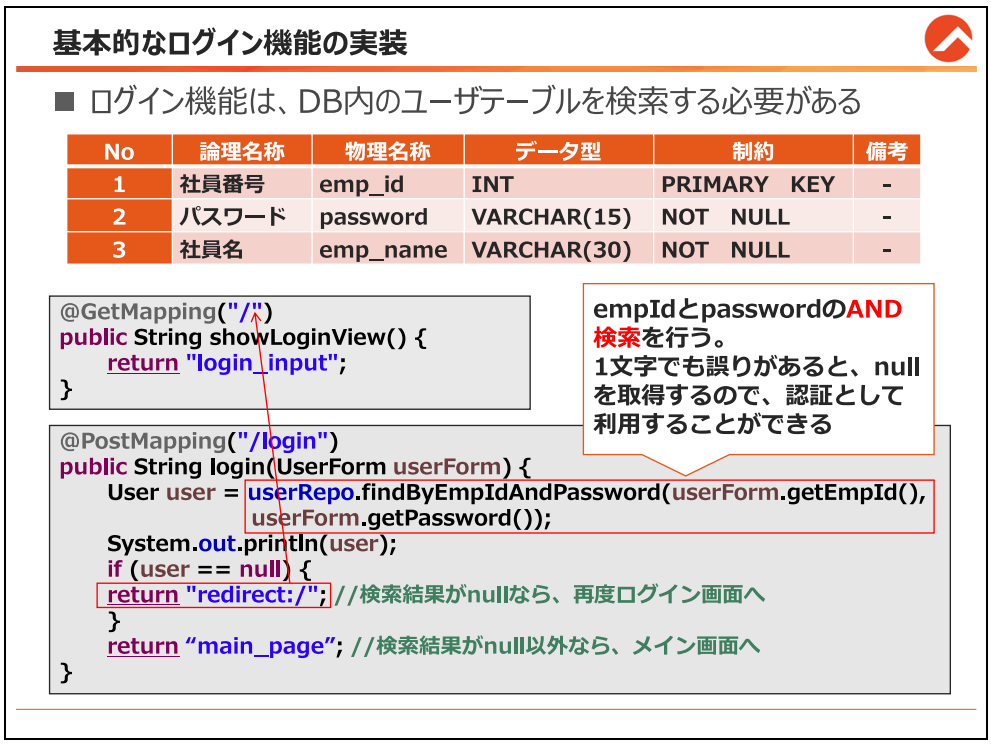
- ■ IDとパスワードによる認証処理を実装できる
- ■ セッションへのデータ保持と、活用方法について理解できる
- ■ 入力チェックとエラーハンドリングを理解し、実装できる

章の概要



この章では、ログイン機能を実装する中で、セッションを活用する方法と、様々な入力チェックを簡単にできる方法をご説明します。そして、エラーメッセージの出力を動的に変える方法についてもご紹介します。

4.1. 基礎的なログイン機能の実装



今まで学習した内容を踏まえて、ログイン機能を実装してみましょう。ログインには、id とパスワードの値を持っているテーブルが必要となりますので、新たに「ユーザ」テーブルを利用します。テーブルとデータは既に spring_sample データベースに格納済みです。

【参考】＜User テーブル＞

No	論理名称	物理名称	データ型	制約	備考
1	社員番号	emp_id	INT	PRIMARY KEY	-
2	パスワード	password	VARCHAR(15)	NOT NULL	-
3	社員名	emp_name	VARCHAR(30)	NOT NULL	-

【参考】＜初期データ＞

emp_id	password	emp_name
123456	pass1234	山田太郎
234567	pass2345	佐藤花子
345678	pass3456	鈴木一郎

【事前準備①】エンティティの定義

…/enitity/User.java

```
 1:@Data
 2:@Entity
 3:@Table(name = "user")
 4:public class User {
 5:
 6:    @Id
 7:    private Integer empId;
 8:
 9:    @Column(length = 15, nullable = false)
10:    private String password;
11:
12:    @Column(name = "emp_name", length = 30, nullable = false)
13:    private String empName;
14:}
```

User テーブルを利用しますので、User エンティティも新しく定義する必要があります。

エンティティの定義については、前の章の Product とほぼ同じですが、12 行の name 属性を見てください。テーブル内の列名は[emp_name]ですが、Java では通常、変数名の単語ごとの区切りに「アンダースコア(_)」を入れるよりは、大文字と小文字の組み合わせにする、「キャーメルケース」を使います。

よって、テーブル内の列名と変数名に違いが発生したら、明示的に name 属性をつける必要があります。

【事前準備②】リポジトリの定義

…/repository/UserRepository.java

```
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.User;

public interface UserRepository extends JpaRepository<User, Integer>{

}
```

エンティティの次は、リポジトリです。3 章の定義方法と同様ですので、説明は割愛いたします。

【事前準備③】Form クラス定義

…/form/UserForm.java

```
package jp.co.trainocate.answer.form;

import lombok.Data;

@Data
public class UserForm {

    private Integer empId;
    private String password;
    private String empName;
}
```

複数の値をユーザから入力してもらうため、Form クラスを用意する必要があります。
Getter&Setter メソッドや、toString メソッドなどの定義は、lombok ライブラリの@Data をまた利用します。2 章の定義方法と同様ですので、説明は割愛いたします。

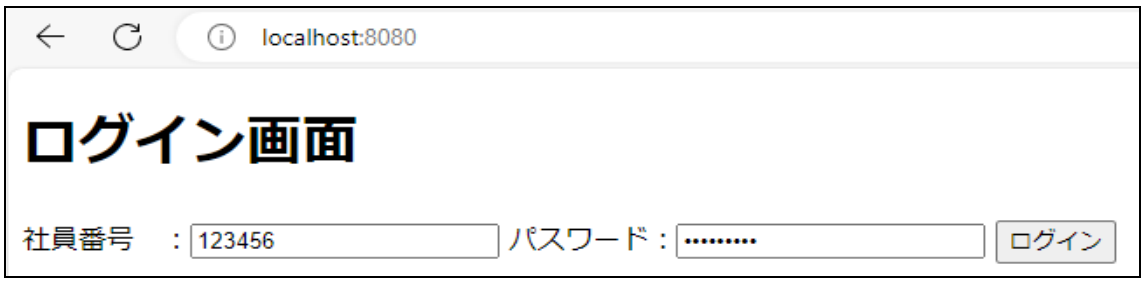
【作業手順①】 LoginController の作成と、ログイン画面遷移用のメソッド作成

…/controller/LoginController.java

```
@Controller
public class LoginController {
        @Autowired
        private UserRepository userRepo;

        @GetMapping("/")
        public String showLoginView() {
                return "login_input";
        }
```

これから、データベースとやり取りをする予定ですので、user テーブルを操作できるように、UserRepository の依存性の注入をしました。
localhost:8080/ をブラウザに入力すると、ログイン画面へ遷移します。

【作業手順②】ログイン画面の作成



…/resources/templates/login_input.html

```
<body>
<h1>ログイン画面</h1>

<form th:action = "@{/login}" method = "post">
社員番号　：<input type = "text" name = "empId"/>
パスワード：<input type = "password" name = "password"/>
<input type = "submit" value = "ログイン">
</form>
</body>
```

2 章で作成したログイン画面をそのまま使います。社員番号は、UserForm のフィールド名と合わせるために、[empId]へ修正しました。

【作業手順③】リポジトリに ID とパスワードの And 検索メソッドの定義

…/repository/UserRepository.java

```
public interface UserRepository extends JpaRepository<User, Integer>{
    public User findByEmpIdAndPassword(Integer empId, String password);
}
```

login_input.html で送信した**社員 ID とパスワードが正しいかどうか判定するため**に、empId と password の And 検索メソッドを定義します。
主キー検索を含むため、1 行しか取得できないメソッドです。よって、戻り値はエンティティ名の User にします。
引数として、empId と password を入れるように定義します。

【 作業手順④】ログイン認証メソッドの定義

…/controller/LoginController.java

```
1:@PostMapping("/login")
2:public String login(UserForm userForm) {
3:      User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
                    userForm.getPassword());
4:      System.out.println(user);
5:      if (user == null) { //結果が null なら、再度ログイン画面へ
6:              return "redirect:/";
7:      }
8:      return "main_page";
9:}
```

・3 行目：

UserRepository に予め定義しておいたメソッドを実装しました。login_input.html からのリクエストパラメータは、userForm 内に自動格納されているため、get メソッドで取得し、引数に入れました。**検索結果は、最大 1 件ですので、User エンティティで受け取ります。**

・5 行の if 文:

社員 ID とパスワードが一致したら、社員データを取得できますが、入力に誤りがあると、何も取得できないので、user 変数は、null のままになります。

この if 文は、**取得できなかった際は、もう一回ログイン画面へ遷移させることで、再入力を促す**コードです。

5 行の if 文内で return されると、即遷移するため、7 行以降が実行されることはありません。

・8 行目 return “main_page”:

データを取得できたなら、ログイン成功ですので、main_page.html へ遷移させます。

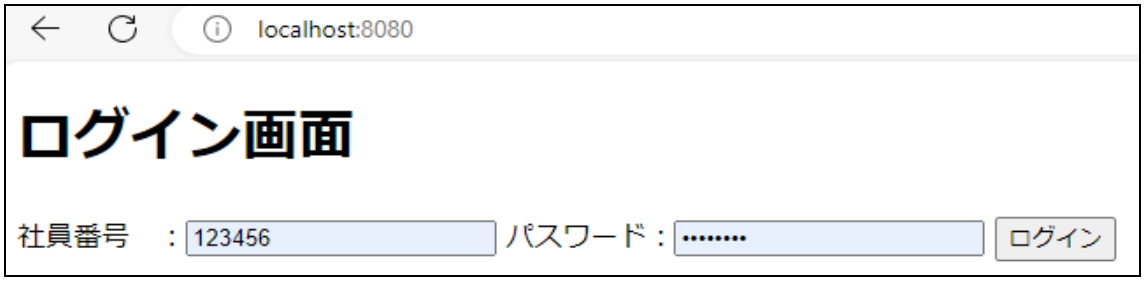
main_page.html は、下記のように、簡単な html ファイルです。

…/resources/templates/main_page.html

```
<!--省略-->
<body>
<h1>メイン画面</h1>
ログインに成功しました。
</body>
```

動作確認

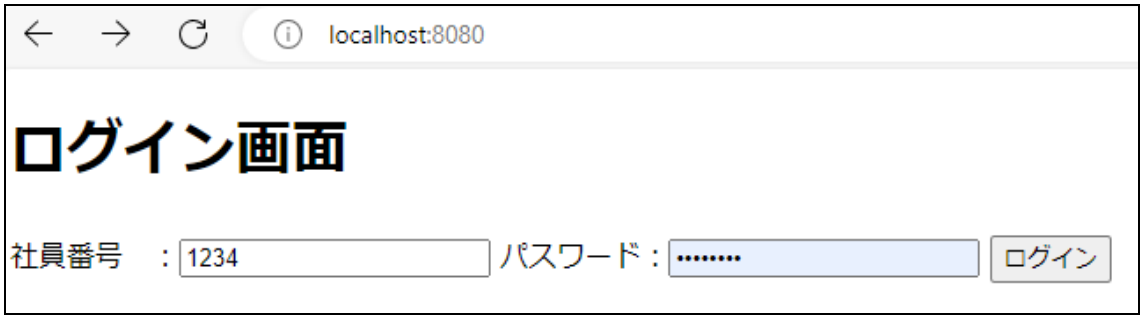
http://localhost:8080/



① ログイン画面から、以下のように入力し、[ログイン]ボタンをクリックします。

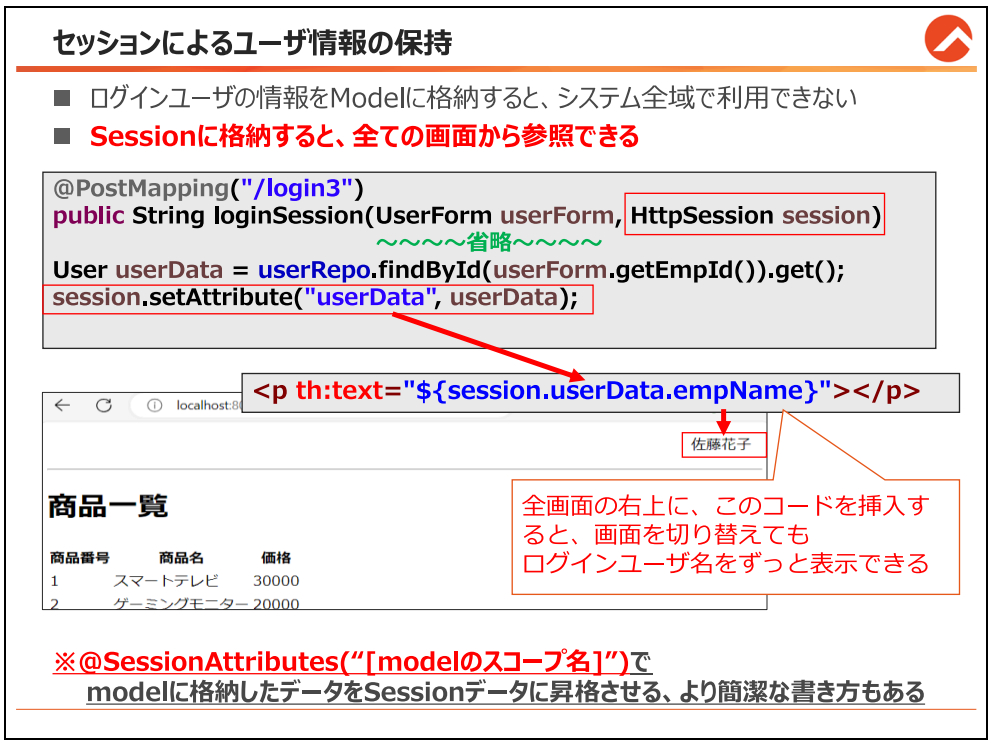
社員番号:123456
パスワード:pass1234

② 正しく入力した場合、メイン画面へ遷移できます。



③ User テーブルに登録されてない Id とパスワードの組み合わせで誤ったログインすると、ログイン画面が再度呼ばれることを確認します。

4.2. セッションによるユーザ情報の保持



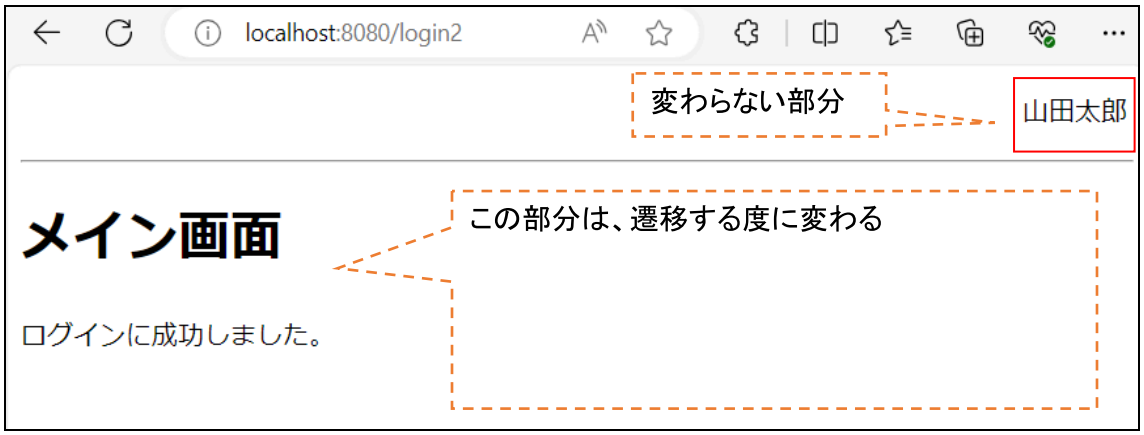
前の節では、一番基本となるログイン機能についてご説明しました。

正しい ID とパスワードを入力したら、ログインができるわけですが、ログイン成功後は、システム内の様々な箇所でユーザの情報を必要とします。

そのため、**ログインが成功したタイミングで、そのユーザの情報をセッションに保持させ、システム全体で参照できるようにすることが大事です。**

この節では、セッションにユーザ情報を保持させ、別の画面でそのユーザ情報を参照するところについてご説明します。

Model に格納した情報は、画面が変わると消える



Web システムでは、多くの異なる画面があり、ユーザはリンクやボタンをクリックすることで、画面遷移を行います。この時、すべての画面で一貫して表示させたいデータもあります。
例えば、上の画像のように、ログインしたユーザの名前が各画面上に常に表示される例は、Web システムにおいて一般的なパターンです。

3 章では、コントローラーからビューへデータを送る方法を学びました。ログイン機能を実装する際には、次のようなコードが使用されます。

```
@PostMapping("/login2")
public String loginNormal(UserForm userForm, Model model) {
        User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
userForm.getPassword());
        System.out.println(user);
        if (user == null) {
                return "redirect:/";
        }
        model.addAttribute("userData", user);
        return "main_page";
}
```

このコードは、ユーザが入力した従業員 ID(empId)とパスワードが正しい場合、ユーザ情報を Model オブジェクトに格納しているコードです。**Model はビューへデータを送るために使用されますが、さらに画面が変わるとそのデータは保持されません。つまり、ユーザが別の画面に移動すると、先ほどのユーザデータは失われてしまいます。**

Model については、3 章の「 findAll メソッドでの全件検索」を参照してください。

【作業手順①】HttpSession 直接利用する方法

Spring でセッションを扱うためには、**HttpSession** のインスタンスを利用します。
この方法は、既存のサーブレット&JSP でセッションを利用する方法でもあります。

…/controller/LoginController.java

```
 1:@GetMapping("/2")
 2:public String showLoginView2() {
 3:      return "login_input2";
 4:}
 5:
 6:@PostMapping("/login2")
 7:public String loginSession(UserForm userForm, HttpSession session) {
 8:      User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
          userForm.getPassword());
 9:      System.out.println(user);
10:      if (user == null) {
11:              return "redirect:/2";
12:      }
13:      session.setAttribute("userData", user);
14:      return "main_page_session";
15:}
```

・7 行目 (HttpSession session):

セッションは、HttpSession のインスタンスを利用することで、データをセッションスコープに格納できたり、セッション情報を消したりできます。 **Spring の Controller に URL がマッピングされたメソッドのことを、改めて「ハンドラメソッド」といいます。このハンドラメソッドの引数に HttpSession を指定するだけで、裏でフレームワークが HttpSession のインスタンス生成と、セッション情報の取得を自動で行ってくれます。**

・13 行目 session.setAttribute("userData", user):

session.setAttribute メソッドは、model.addAttribute と同じような書き方をします。第一引数には、キーワードを記載して、第二引数には、データを指定します。このコードが実行されると、セッションスコープの中に、"userData"という名前でログインユーザの全情報を保持できるようになります。このデータは、**画面が変わっても、システム全域で呼び出すことができます。**

【 作業手順②】 セッションスコープ内のデータを参照する方法

セッションスコープ内のデータを参照する方法について、より詳細に解説します。

まず、session.setAttribute メソッドを用いてデータをセッションに格納できます。

格納したセッションデータは、Thymeleaf テンプレートエンジンを介して簡単にアクセスできます。以下はその例です。

…/resources/templates/main_page_session.html

```
<body>
<p th:text="${session.userData.empName}" style="margin-left:90%"></p>
<hr>
<h1>メイン画面</h1>
<p>ログインに成功しました。</p>

</body>
```

赤枠で示された部分のように、session.[キーワード].[変数名]という形式で記述するだけで、セッションスコープのデータを利用できます。この便利さの背景には、Thymeleaf において session が暗黙のオブジェクトとして既に生成されていることがあります。この方法により、HttpSession オブジェクトをどの画面でも直接利用することが可能です。

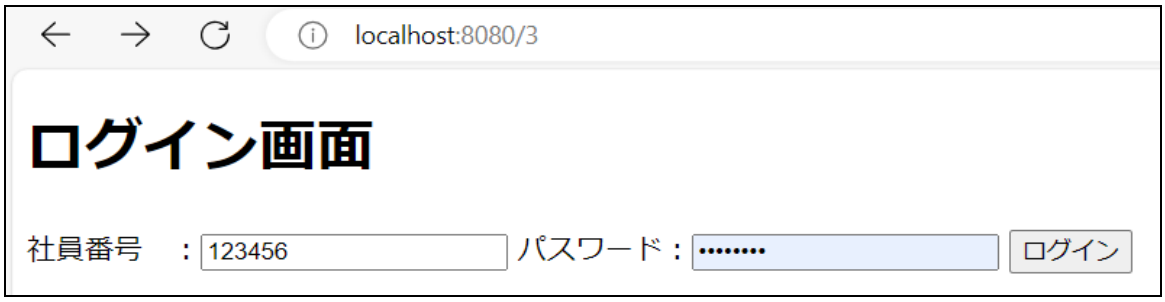
…/resources/templates/product/list.html

```
<body>
 <p th:text="${session.userData.empName}" style="margin-left:90%"></p>
   <hr>
   <h1>商品一覧</h1>
   <table>
        <tr>
        <th>商品番号</th>
        <th>商品名</th>
                    <!--省略-->
```

3 章で記述済みの検索リストを表示するという、ログインとは一切関係ないページ(/product/list.html)にも、同じコードを記述しておきます。

動作確認

http://localhost:8080/2 **<--こちらの URL へアクセスします。**

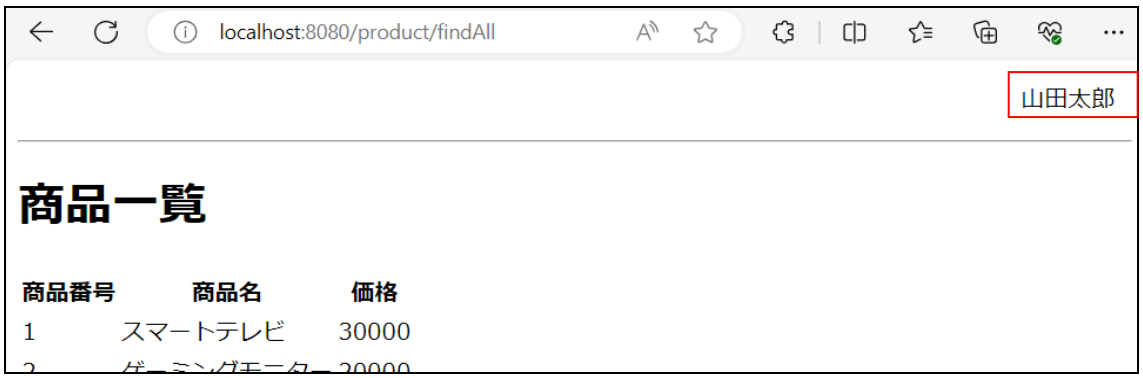


① ログイン画面から、以下のように入力し、[ログイン]ボタンをクリックします。

社員番号:123456
パスワード:pass1234

② 画面の右上に、ログインしたユーザ名が表示されていることを確認します。

③ http://localhost:8080/product/findAll **<--こちらの URL へアクセスします。**



④ 同じコードを書いた、list.html でも、ログインしたユーザ名が確認できます。

【参考】@ SessionAttributes を利用する方法

model に格納したデータと、全く同じデータをセッションにも格納しておきたい場面もあります。その時に活用できる、より簡潔で保守性が高い@SessionAttributes を使う方法についてご紹介します。

…/controller/LoginController.java

```
@Controller
@RequiredArgsConstructor
@SessionAttributes("userData")
public class LoginController {
    private final UserRepository userRepo;
            //省略
    @PostMapping("/login3")
    public String loginNormal(UserForm userForm, Model model) {
          User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
userForm.getPassword());
           System.out.println(user);
           if (user == null) {
                   return "redirect:/3";
           }
           model.addAttribute("userData", user);
           return "main_page_session";
  }
```

上記のソースコードは、model を使ったデータの送信のコードとほぼ同じです。

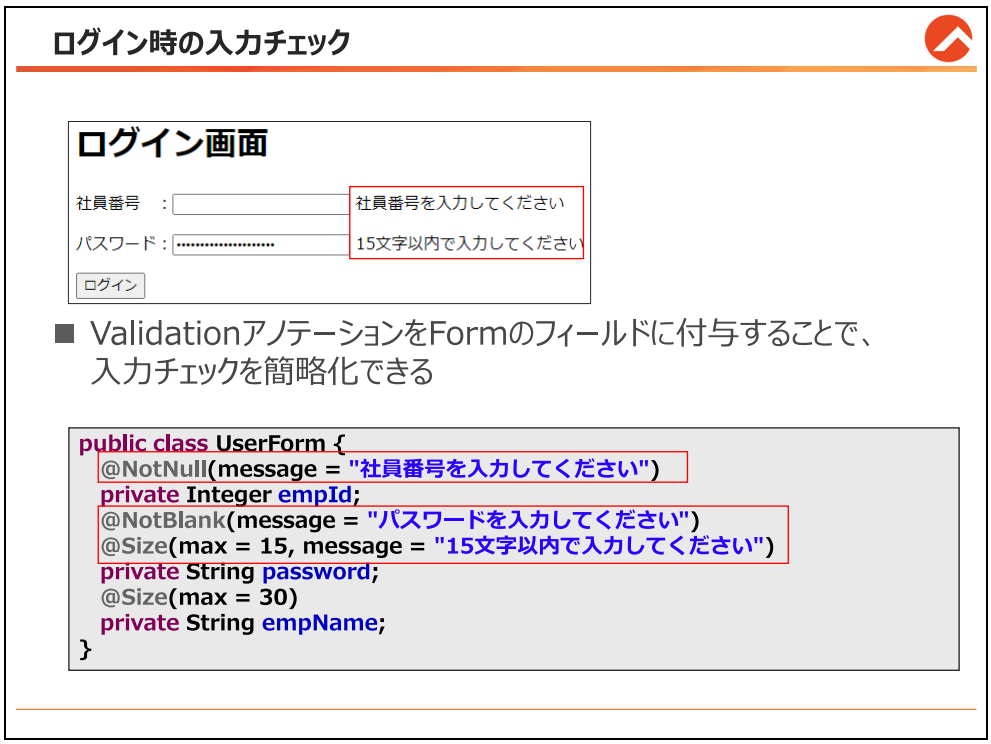
クラス名の上に付与している、**@SessionAttributes("userData")**の部分にご注目ください。

“userData”は、model に userData を格納する際に使ったキーワードです。@SessionAttributes は、model スコープに格納しているデータを、セッションデータとしても利用できるようにします。

HttpSession を直接使うより、コードが簡潔になり、キー名も統一できますので、こちらの方法を推奨します。

※「Spring Security API」を一緒に使うことで、セッションハイジャックなどの脆弱性対策もできます。

4.3. ログイン時の入力チェック



Spring では、フォームに入力した値が想定された形式や文字列であるかをチェックする機能を提供しています。その機能のことを「入力チェック(Validation)」と呼びます。この節では Spring の Validation API を活用した、入力チェックの仕方についてご説明いたします。

【 参考】既存の入力チェックのコード

```
public String validate() {
    if (empId == null) {
        return "社員番号を入力してください";
    }

    if (password == null || password.trim().isEmpty()) {
        return "パスワードを入力してください";
    }

    if (password.length() > 15) {
        return " 15 文字以下で入力してください";
    }

    // 他のフィールドに対するバリデーションも同様に追加

    return null; // 問題がなければ null を返す
}
```

Validation API を使わない、サーブレットなどの場合、入力される値に対して、個別に条件文を作成しておかないといけませんでした。この方法は、記述に時間もかかりますし、保守もしづらい問題がありました。

【 事前準備】 Validation API を利用する

pom.xml

```
<dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Spring の Validation API をインストールすると、**Form クラスのフィールド名に特定のアノテーションをつけるだけで、入力チェック用のコードを省略できるようになります。**

【 作業手順①】 Form クラスに Validation のアノテーションを付与

…/form/UserForm.java

```
 1:package jp.co.trainocate.answer.form;
 2:
 3:import jakarta.validation.constraints.NotBlank;
 4:import jakarta.validation.constraints.NotNull;
 5:import jakarta.validation.constraints.Size;
 6:import lombok.Data;
 7:
 8:@Data
 9:public class UserForm {
10:
11:    @NotNull(message = "社員番号を入力してください")
12:    private Integer empId;
13:
14:    @NotBlank(message = "パスワードを入力してください")
15:    @Size(max = 15, message = "15 文字以内で入力してください")
16:    private String password;
17:
18:     private String empName;
19:}
```

・3-5 行目：

Validation のバージョンによりますが、最新の Validation API は、jakarta.validation.constraints パッケージの方を利用することが推奨されています。

・10 行目　@NotNull：

このアノテーションを付与したフィールドに、**入力値がない場合は、入力エラーだと検知**してくれます。**Integer のフィールドは、@NotNull** を付与します。**message 属性は、出力するメッセージをカスタマイズ**することができます。

・14-15 行　@NotBlank、@Size(max = 15):

@NotNull と似ていますが、こちらは**文字列の入力有無チェックに特化**されています。未入力はもちろん、**空文字(“”)まで検知**することができます。

@Size は、**文字列の文字数制限**を設定できます。Max で最大文字数、min で最小文字数の設定ができます。

【 作業手順②】 ログイン入力ページ遷移用のメソッド定義

…/controller/LoginController.java

```
@GetMapping("/4")
        public String showLoginView4(UserForm userForm) {
                return "validation_login_input";
        }
```

入力チェック機能をつける場合は、**必ず入力画面遷移用メソッドの引数にも、Form クラスを入れておく必要**があります。**Form のデータがないと、エラーが出るからです。**
よく忘れることが多いので、ご注意ください。

【 作業手順③】 入力チェック用メソッドの定義

…/controller/LoginController.java

```
1://省略
2:@PostMapping("/login4")
3:public String loginValidation(@Valid UserForm userForm, BindingResult
bdResult) {

4:        if (bdResult.hasErrors()) {
5:            return "validation_login_input";
6:         }
7:       return "main_page";
8:}
```

・3 行目 @Valid:

@Valid アノテーションは、対象のオブジェクト(この場合は UserForm)が持つ Validation アノテーション(例えば @Size、@NotNull など)に基づいて**入力チェックを実行するために使用**されます。

・3 行目 BindingResult:

BindingResult は、@Valid アノテーションによる入力チェックの結果を保持するオブジェクトです。入力チェックエラーの詳細を取得するために使用されます。通常、メソッドの引数として渡されます。**必ず、@Valid をつけた直後(ここでは、第 2 引数)に記載しないといけないルールがありますので、ご注意ください。**

・4 行目　bdResult.hasErrors()：

BindingResult の **hasErrors メソッドは、入力チェック中にエラーが発生したかどうかを確認するために使用されます。エラーが存在する場合は true を、そうでない場合は false を返します。**このメソッドを利用して条件分岐を行い、エラーがある場合は入力フォームに戻り、エラーメッセージをユーザに表示します。

【 作業手順④】 ログイン入力ページの修正

…/resources/templates/validation_login_input.html

```
 1:<body>
 2:<h1>ログイン画面</h1>
 3:
 4:<form th:action="@{/login4}" method="post" th:object="${userForm}">
 5:
 6:<p>社員番号　：<input type = "text" th:field="*{empId}"/>
 7:                    <span th:errors="*{empId}"></span></p>
 8:
 9:<p>パスワード：<input type="password" th:field="*{password}" />
10:                    <span th:errors="*{password}"></span></p>
11:
12:<input type="submit" value="ログイン">
13:
14:</form>
15:</body>
```

・4 行目　th:object="${userForm}":

th:object 属性は、**「 選択対象オブジェクト」**と呼ばれ、フォームのデータを Controller のオブジェクト(この場合は UserForm オブジェクト) にバインド(結びつけ) するために使用されます。

form タグに記述することで、フォームの入力フィールドの値が、UserForm オブジェクトの対応するプロパティに自動的に格納されます。
つまり、Controller のメソッド内で Model にデータを格納しなくても、Form タグの中で、Thymeleaf 上でデータを受け取れるということです。

フォーム内では、***{...}**という選択式を使用して、userForm オブジェクトのプロパティに簡潔にアクセスできます。**userForm.empId ではなく、*{empId}だけでアクセスできるように**なります。

・6 行目 th:field="*{empId}":

th:field 属性は、対応するフィールドの **name, id, および value 属性を自動的に生成**します。ですので、name、id、value を省略できます。

自動生成される value 属性により、フォーム送信後に入力チェックエラーが発生した場合でも、**ユーザが入力した値がフィールドに保持されます**。これにより、ユーザは入力値を再度入力する手間を省くことができます。

なお、*{empId}は、userForm オブジェクトの empId プロパティを参照しています。

・7 行目 :

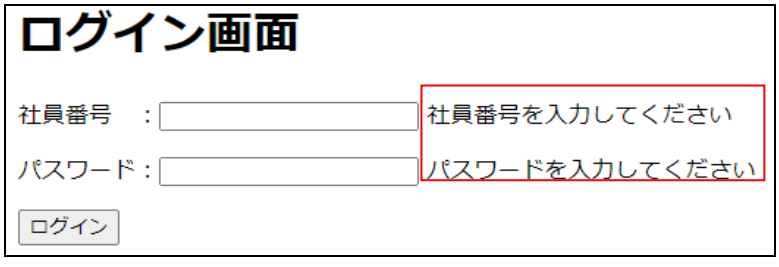
このコードは、empId フィールドに関連する入力チェックエラーがある場合に**エラーメッセージを表示**するために使われます。エラーがない場合は、何も表示されません。

フォームを送信したとき、もし empId フィールドの値が入力チェックのルールに違反していれば(例えば、必須フィールドが空である、など)、〈span〉タグの中に入力チェックごとのエラーメッセージが表示されます。

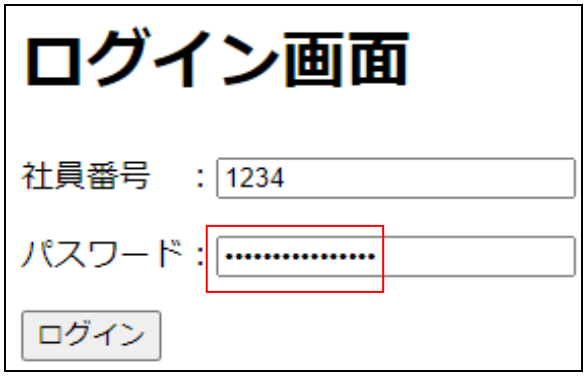
動作確認

http://localhost:8080/4

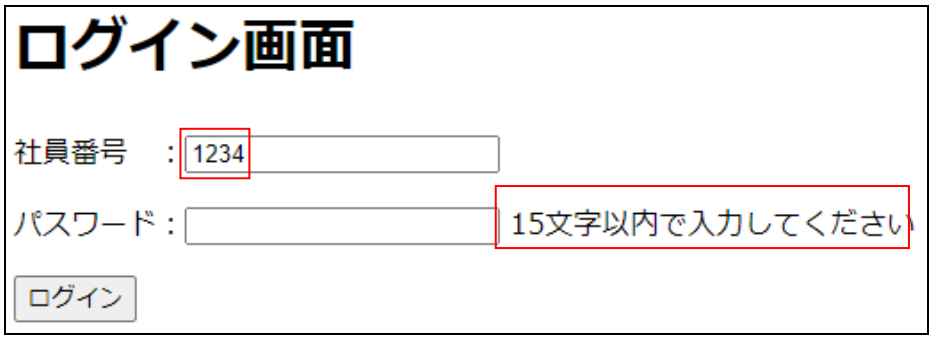
① 何も入力せずに、[ログイン]ボタンをクリックします。



② 入力エラーメッセージが表示されます。



③ パスワードに 16 文字以上の文字列を入力し、[ログイン]ボタンをクリックします。



④ 社員番号はそのまま値が入っている状態で、文字数制限エラーが表示されるか確認します。

【参考】ログイン機能の最終版のソースコード

この章では、ログインについて、以下の実装方法についてご説明しました。

① 入力した ID とパスワードを User テーブル内で And 検索し、正しいかどうかチェック

② ログインユーザのユーザ情報をセッションに格納し、システム全域で活用

③ Validation アノテーションを使った、入力チェック

では、①②③を全て使った最終版のコードを以下に記述します。

…/resources/templates/login_input_final.html

```
<body>
<h1>ログイン画面</h1>

<form th:action="@{/login5}" method="post" th:object="${userForm}">

<p th:if="${loginError}" th:text="${loginError}"></p> <!--5.1.2 参考-->

<p>社員番号　：<input type = "text" th:field="*{empId}"/>
                <span th:errors="*{empId}"></span></p>

<p>パスワード：<input type="password" th:field="*{password}" />
                <span th:errors="*{password}"></span></p>

<input type="submit" value="ログイン">

</form>
</body>
```

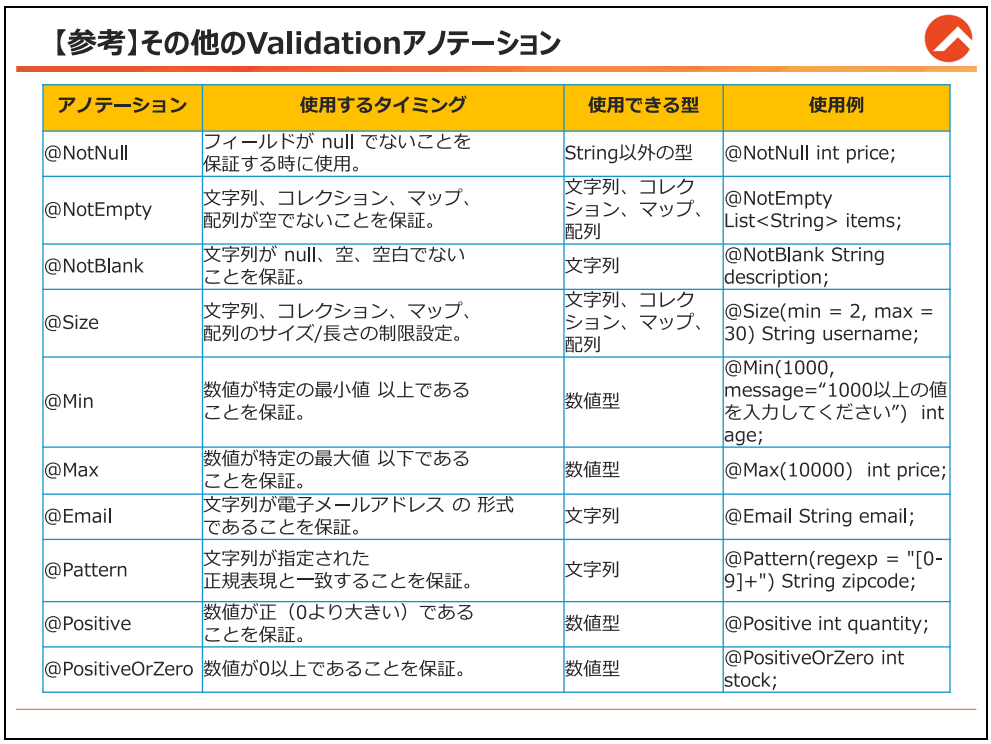
…/controller/LoginController.java

```
@GetMapping("/5")
public String showLoginView5(UserForm userForm) {
        return "login_input_final";
}

@PostMapping("/login5")
public String loginFinal(@Valid UserForm userForm, BindingResult bdResult,
HttpSession session, Model model) {
        //入力エラーチェック
        if (bdResult.hasErrors()) {
                return "login_input_final";
        }

        User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
userForm.getPassword());
        //DB 照会後の ID&パスワードのチェック
        if (user == null) {
        model.addAttribute("loginError", "社員番号またはパスワードが誤
っています。"); // エラーメッセージを追加
        return "login_input_final";
    }

        //正しい入力の場合、ユーザ情報をセッションに格納
        session.setAttribute("userData", user);

        return "main_page_session";
}
```

…/resources/templates/main_page_session.html

```
<body>
<p th:text="${session.userData.empName}" style="margin-left:90%"></p>
<hr>
<h1>メイン画面</h1>
<p>ログインに成功しました。</p>
</body>
```

4.3.1.【 参考】その他の Validation アノテーション



【参考】その他のValidationアノテーション

アノテーション	使用するタイミング	使用できる型	使用例
@NotNull	フィールドが null でないことを保証する時に使用。	String以外の型	@NotNull int price;
@NotEmpty	文字列、コレクション、マップ、配列が空でないことを保証。	文字列、コレクション、マップ、配列	@NotEmpty List<String> items;
@NotBlank	文字列が null、空、空白でないことを保証。	文字列	@NotBlank String description;
@Size	文字列、コレクション、マップ、配列のサイズ/長さの制限設定。	文字列、コレクション、マップ、配列	@Size(min = 2, max = 30) String username;
@Min	数値が特定の最小値 以上であることを保証。	数値型	@Min(1000, message="1000以上の値を入力してください") int age;
@Max	数値が特定の最大値 以下であることを保証。	数値型	@Max(10000) int price;
@Email	文字列が電子メールアドレス の 形式であることを保証。	文字列	@Email String email;
@Pattern	文字列が指定された正規表現と一致することを保証。	文字列	@Pattern(regexp = "[0-9]+") String zipcode;
@Positive	数値が正（0より大きい）であることを保証。	数値型	@Positive int quantity;
@PositiveOrZero	数値が0以上であることを保証。	数値型	@PositiveOrZero int stock;

今回の例文で紹介された@NotNull や、@NotBlank、@Size 以外にも、便利な Validation アノテーションが用意されています。この表に書かれているアノテーションだけでも、一般的な Web システムで使われる入力チェックをケアすることができます。

注意しないといけないのは、上記の表のように、**アノテーションごと「使用できる型」が異なるということです。例えば、数値のデータに@NotBlank を指定するとエラーが出ます。**

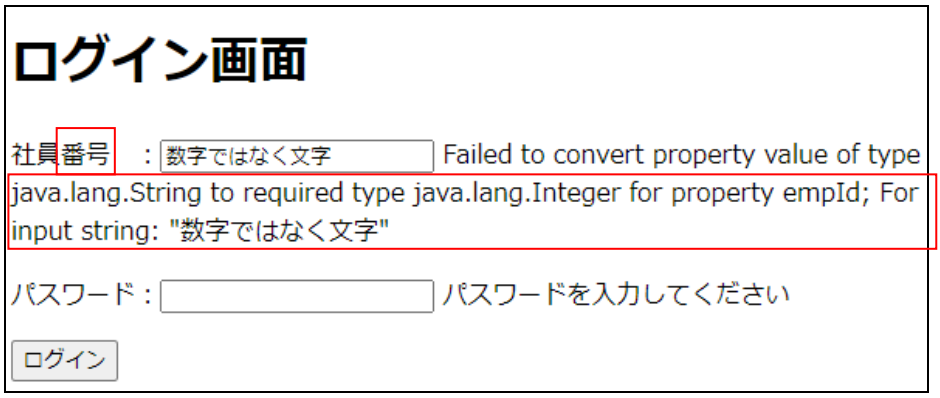
@NotNull については、String 型に適用できないわけではありませんが、@NotBlank の方が空文字や空白のケアができるのでより適切です。

複雑な入力条件は、@Pattern アノテーションで「正規表現」を使うことで、細かく設定できますので、参考にしてください。

4.3.2.【 参考】メッセージのカスタマイズ

前の節で、色んな入力チェック用のアノテーションをご説明しましたが、Integer の型に、文字列を入れてしまった時のような、「型の不一致」や、「日付の整合性チェック」についてのアノテーションがないことに気づいた方もいらっしゃるでしょう。

基本的に、型の不一致による入力チェックエラーと、日付関連のエラーは、th:error が自動的にチェックしてくれます。

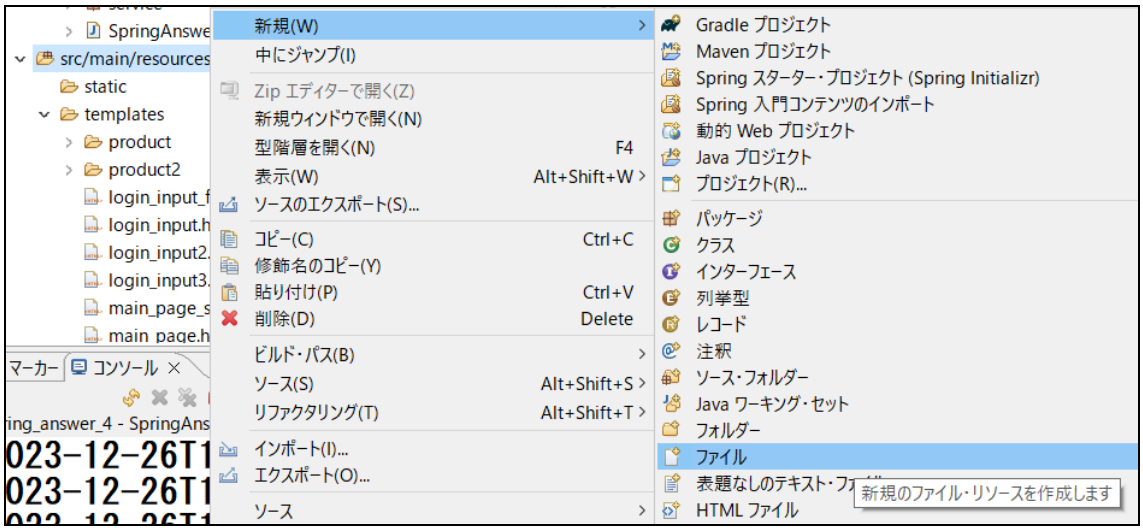


しかし、**自動チェックですので、他のアノテーションのように、message 属性を使った、エラーメッセージの設定ができません。**

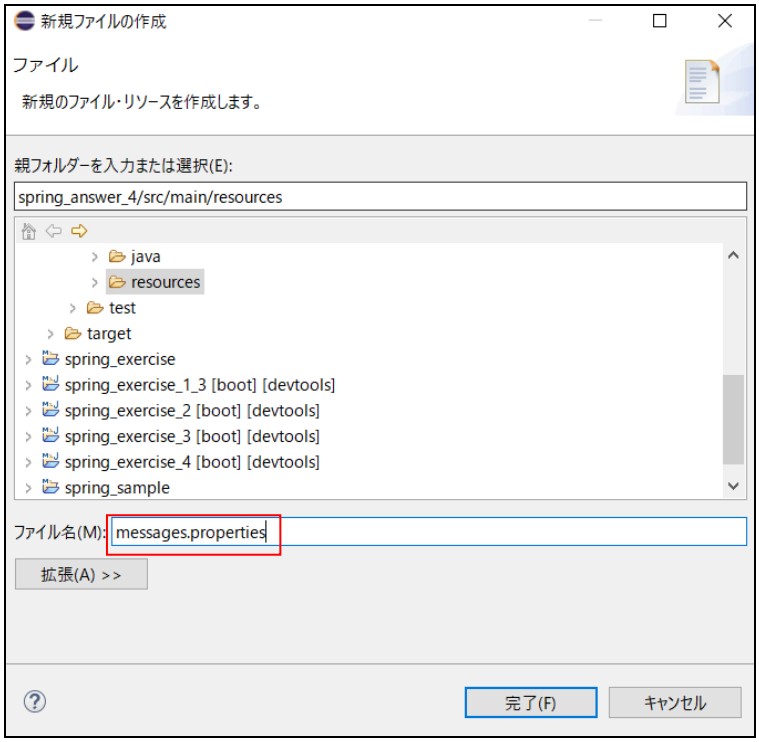
Spring では、メッセージをカスタマイズできるファイルを別途作成できます。

messages.properties の作成

メッセージプロパティのファイルを活用することで、メッセージのカスタマイズができます。



/src/main/resources に、[ファイル]を新規作成します。



ファイル名は、[messages.properties]にします。

messages.properties にカスタマイズしたいメッセージを記入

/resources/messages.properties

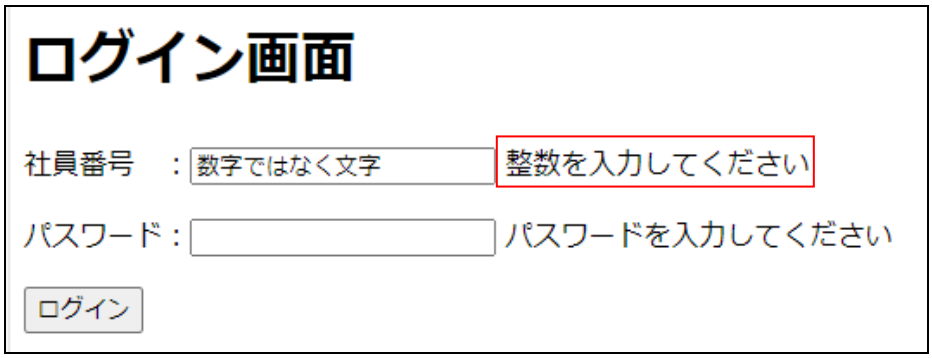
```
typeMismatch.java.lang.Integer=整数を入力してください
typeMismatch.java.lang.Double=小数点を入力してください
typeMismatch.java.util.Date=正しい日付を入力してください
```

messages.properties の中に、エラーメッセージについての表示を変更することができます。

上の例のように、整数と小数点はもちろん、日付についての入力チェックについても、表示されるメッセージを再定義することができます。

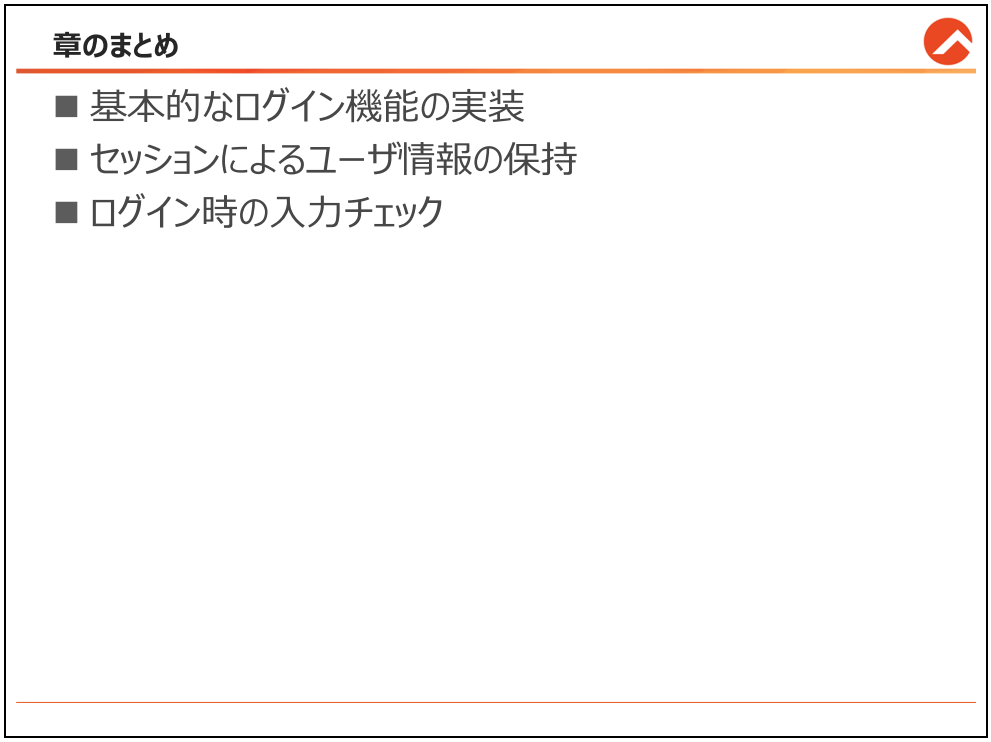
保存後、**[maven install]をすると、修正事項が反映されるようになります。**

外部アプリケーションサーバーで運用するシステムの場合は、必ずサーバーの再起動が必要です。



エラーメッセージが修正されたことを確認できます。

章のまとめ



この章では、ログイン機能に必要なセッションや、Validation API の使い方についてご説明しました。

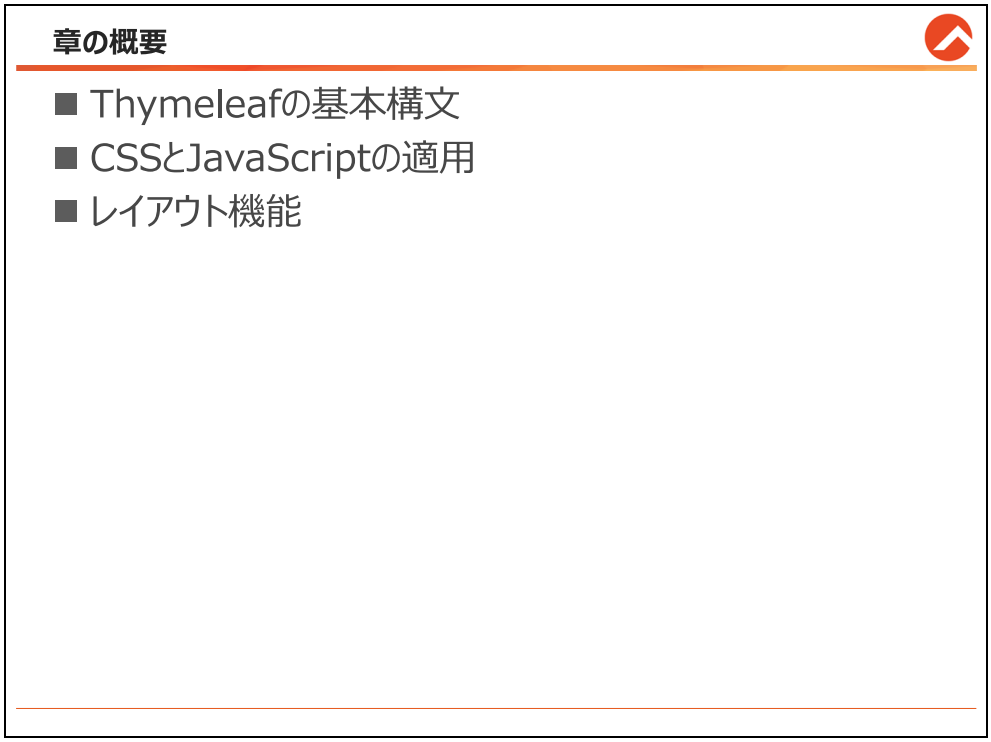
5

第5章 Thymeleaf と Web ページデザイン

章の目標

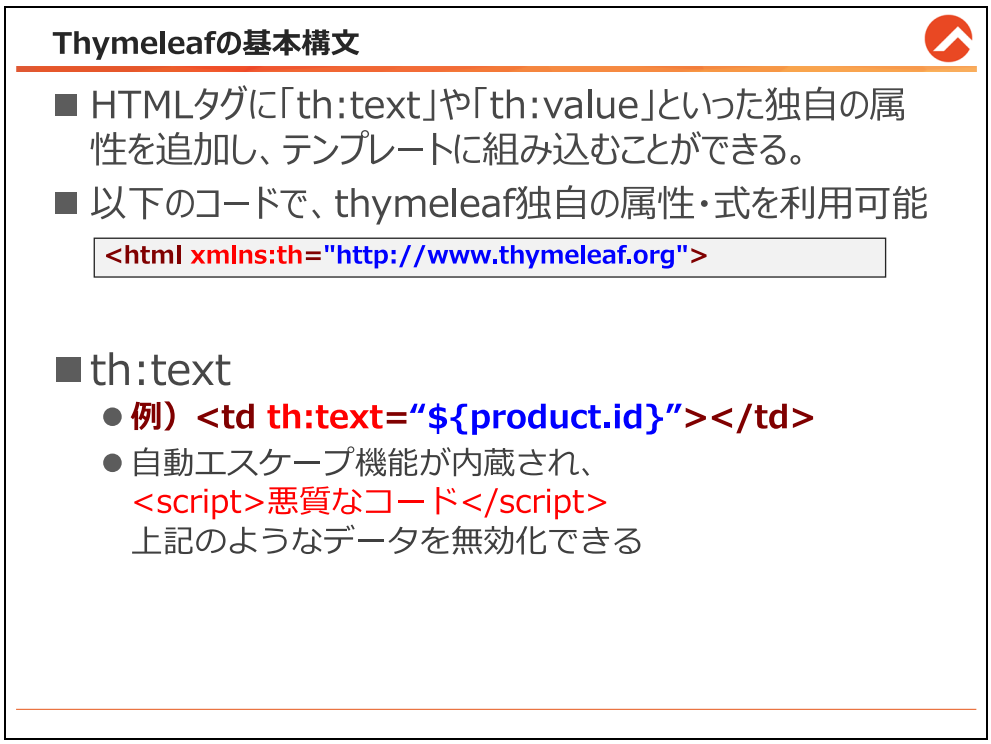
- Thymeleaf の基本構文について理解できる
- Thymeleaf から CSS や JavaScript を活用できる
- レイアウト機能について理解できる

章の概要



今まで、thymeleaf のタグについて、いくつか例文の中でご紹介しました。この章では、改めて thymeleaf の文法をメインの内容にし、CSS と JavaScript を連携する方法についてご説明します。そして、保守性と効率を高めるレイアウト機能についてもご説明します。

5.1. Thymeleaf の基本構文



Thymeleaf について

「Thymeleaf」は、JSP と同様に、Java の「テンプレートエンジン」の一種で、HTML5 形式のテンプレートを変換し、Web ブラウザでの表示を可能にするライブラリです。Thymeleaf は、コントローラーからのデータを受け取ることができますので、コンテンツの動的な変更が実現できます。Thymeleaf の主要な記述方式は以下の通りです。

・属性記述：HTML タグに「th:text」や「th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。

・式記述：属性の値を「${}」や「@{}」の形式で表現し、動的なデータの挿入ができる。

Thymeleaf は Spring Framework との統合が容易であり、Spring アプリケーションでの使用に特に適しています。HTML の中に自然にデータを組み込むことができるため、可読性も高いです。さらに、Thymeleaf はセキュリティ面での配慮もなされており、XSS（クロスサイトスクリプティング）対策などのセキュリティ機能を内蔵しています。

Thymeleaf を使うための事前準備

HTML ファイル内で、Thymeleaf の属性や式を使うためには、以下のようなコードを記載しておく必要があります。

```
1:<!DOCTYPE html>
2:<html xmlns:th="http://www.thymeleaf.org">
3:
4:<head>
5:<meta charset="UTF-8" />
6:<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

・2 行 <html xmlns:th="http://www.thymeleaf.org">:

Thymeleaf テンプレートで使われる HTML の宣言です。この宣言により、Thymeleaf のルールが HTML ページに追加されます。それによって、Thymeleaf の特有の属性や構文が HTML タグ内で認識され、正しく処理されます。

・6 行 <meta name="viewport" content="width=device-width, initial-scale=1.0" />:

この宣言は、Thymeleaf の構文ではありませんが、Web ページがモバイルデバイスで表示される際のビューポート(表示領域)の設定に使われます。デバイスがどのような画面サイズを持っていても、Web ページはそれに適応するように表示されます。

th:text について

```
<td th:text="${product.id}">[データがあったら、置き換えられる]</td>
<td th:text="${product.name}">[データがあったら、置き換えられる]</td>
<td th:text="${product.price}">[データがあったら、置き換えられる]</td>
```

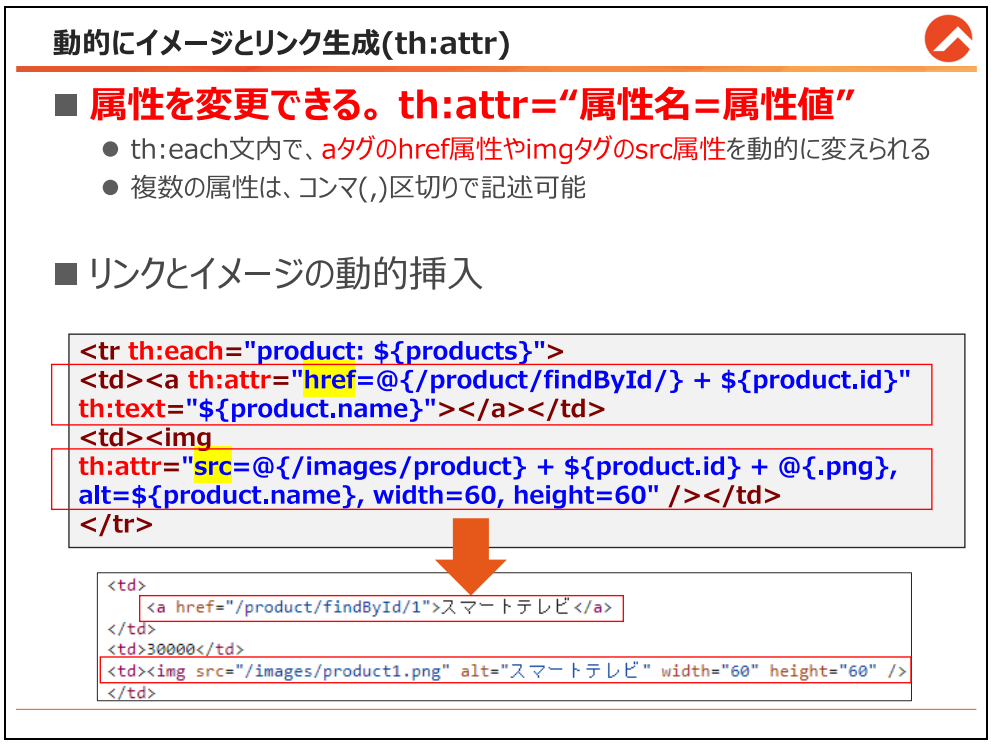
一番基本となる、th:text は、HTML の要素に動的なテキスト内容を挿入するために使用されます。主に、サーバー側で処理されるデータ(例えば、データベースから取得した情報やユーザ入力)を Web ページに表示する際に用います。

<td></td> タグ内のテキストは、th:text 属性に指定された式の結果によって置き換えられます。

th:text の特徴の一つは、クロスサイトスクリプティング(XSS)の対策です。例えば、ユーザが「<script>悪質な JavaScript コード</script>」というデータを入力した場合、XSS 対策が施されていないと、この悪意のある JavaScript コードが HTML 内に組み込まれ、不特定多数のユーザに被害を及ぼす可能性があります。

その対策として、th:text を使用すると、入力データは自動的にエスケープ処理されます。これにより、‘<’や‘>’のような文字は HTML タグとしてではなく、ただの文字列として扱われます。結果として、悪意のある JavaScript コードが実行される心配はなくなります。

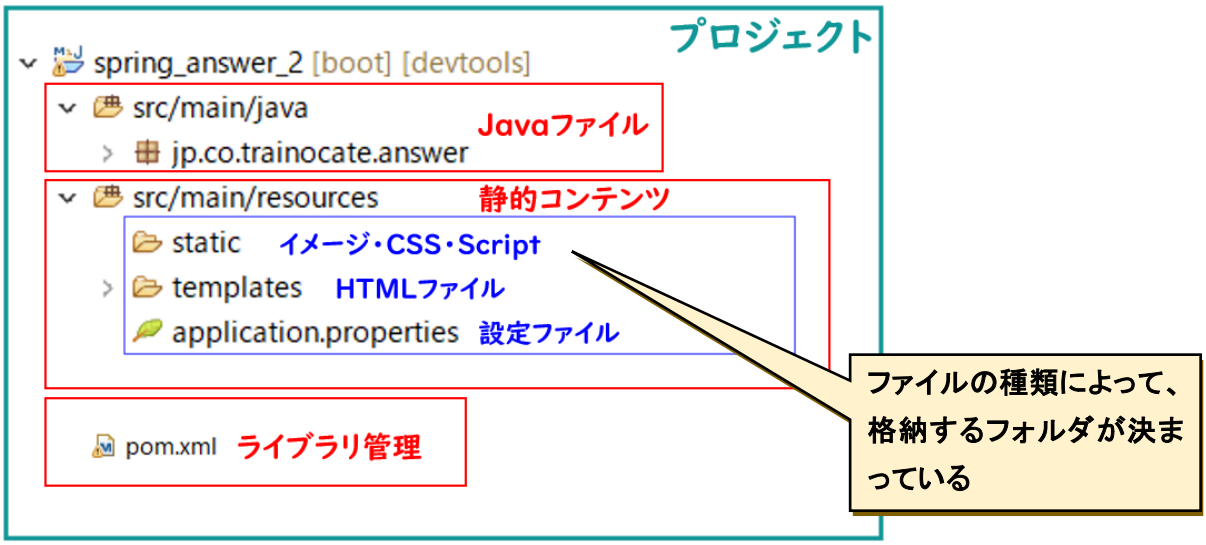
5.1.1. 動的にイメージとリンク生成(th:attr)

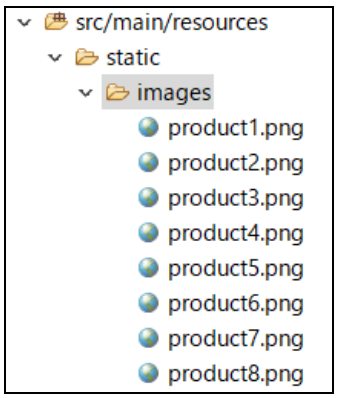


Thymeleaf の th:attr 属性を使用することで、HTML 要素の属性を動的に変更することが可能です。データベースから取得したデータに基づいて異なる属性値を設定することができますので、**Form タグ内の id や class 属性を動的で割り当てることができますし、画像の src 属性やリンクの href 属性などを動的に変えることもできます。**

イメージの動的挿入

【 振り返り】＜Spring Boot プロジェクトの構成＞





Spring Boot 環境では、イメージのような静的ファイルを src/main/resources/static フォルダに格納する必要があります。今回は、予め images フォルダに、商品ごとのイメージを格納しておきました。product の後ろの番号は、商品番号に合わせています。

…/resources/templates/product/list.html

```
<table>
<tr>
        <th>商品番号</th>
        <th>商品名</th>
        <th>価格</th>
        <th>イメージ</th>
```

```
</tr>
<tr th:each="product: ${products}">
        <td th:text="${product.id}"></td>
        <td th:text="${product.name}"></td>
        <td th:text="${product.price}"></td>
        <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
                          alt=${product.name}, width=60, height=60" /></td>
</tr>
```

既存の findByAll のサンプルの list.html に、th:attr のコードを追記しました。

th:attr の構文は **th:attr="属性名=属性値"** という形で、属性値を動的に設定できます。通常、イメージを挿入する際は〈img src="イメージの URL"〉として src 属性に URL を指定しますが、**th:attr を使用すると〈img th:attr="src=@{イメージの URL}"〉のように書き換えることができます**。

この例では、product.id を用いて商品ごとに異なるイメージ URL を生成しています。th:attr を使用する際に複数の属性を指定したい場合は、コンマ(,)で区切ることで設定できます。alt 属性はイメージの表示が失敗した時の代替テキストとして機能し、ここでは商品名を出力しています。また、width と height 属性を指定することでイメージの大きさを統一しています。

出力結果をみると、以下のように変換されているのがわかります。

```
<tr>

    <td>1</td>
    <td>スマートテレビ</td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" /></td>
</tr>
<tr>

    <td>2</td>
    <td>ゲーミングモニター</td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" /></td>
</tr>
<tr>

    <td>3</td>
    <td>ノートパソコン</td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" /></td>
</tr>
```

動的 URL のリンク挿入

Web システムには、以下のようにデータごとに動的なリンクがかかっていることもよくあります。

この場合、**a タグに同じく th:attr 属性値として href を指定すると、動的リンクを生成**することができます。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
   <td th:text="${product.id}"></td>
   <td>
      <a th:attr="href=@{/product/findById/} + ${product.id}"
      th:text="${product.name}"></a>
   </td>
   <td th:text="${product.price}"></td>
   <td><img th:attr="src=@{/images/product} + ${product.id} + @{.png},
         alt=${product.name}, width=60, height=60" />
  </td>
</tr>
```

…/controller/ProductController.java

```
@GetMapping("/findById/{id}")
public String showProductById(@PathVariable int id, Model model) {
        Product product = prodRepo..findById(id).orElse(null);
        model.addAttribute("product", product);
        return "product/detail";
}
```

動的な URL リンクによる呼び出しの場合は、URL から id を受け取る処理が必要となりますので、2 章の update と delete の場合と同様に、@PathVariable アノテーションを付与する必要があります。(その場合、フォームからの id 検索は無効になります)

動作確認

http://localhost:8080/product/findAll

正しく指定した場合、以下のようなコードが生成されることを確認できます。

```
<tr>
    <td>1</td>
    <td>
        <a href="/product/findById/1">スマートテレビ</a>
    </td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>2</td>
    <td>
        <a href="/product/findById/2">ゲーミングモニター</a>
    </td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>3</td>
    <td>
        <a href="/product/findById/3">ノートパソコン</a>
    </td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" />
    </td>
</tr>
```

5.1.2. 条件式による表示制御(th:if)

①th:if と th:unless

Thymeleaf では、th:if や th:unless を活用して、条件に基づいた HTML の表示を制御することができます。これにより、データベースから取得した**データに基づいて異なる表示を行う**ことが可能です。

活用例は多岐にわたりますが、以下にその一例をご紹介します。

…/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
<!--省略-->

<!-- 価格が 30000 より大きい場合、赤色で価格を表示 -->
<td th:if="${product.price > 30000}" th:text="${product.price}"
style="color: red;"></td>

<!-- 価格が 30000 以下の場合、通常のスタイルで価格を表示 -->
<td th:unless="${product.price > 30000}"
th:text="${product.price}"></td>
```

この例では、product.price の値が 30000 を超えるかどうかで、スタイルが適用された価格と、通常スタイルの価格を切り替えて表示しています。**HTML コードは 2 行ですが、条件に合うコードだけが表示されるようになっています。**

②その他の条件式の例

条件式による制御は、活用場面が無限に近いのですが、下記によく使われる例をご紹介します。

(1) 削除フラグに基づくデータの表示:

削除フラグが 0 の場合はデータを表示し、1 の場合は表示しない。
(テーブルに deleteFlag の列を追加する必要があります。)

```
<div th:each="product : ${products}" th:if="${product.deleteFlag == 0}"
th:text="${product.name}"></div>
```

(2) 管理者権限に基づく機能の表示:

特定のユーザー(例えば、管理者) にのみ特定の機能を表示する。

(テーブルに role の列を追加する必要があります。)

```
<div th:if="${session.user.role == 'ADMIN'}">
    <!-- 管理者専用の機能やリンクをここに表示 -->
</div>
```

(3) リストの要素が空かどうかのチェック:

リストが空の場合に、「 情報がありません」というメッセージを表示する。

(#lists.isEmpty(products)は、Thymeleaf で提供する変数式です。
簡単にリストの検索 OK/NG を判定できる構文です。)

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```

⑷ 日付や時間に基づくコンテンツの表示：

現在の日付や時間に基づいて、特定のメッセージやコンテンツを表示する。

（**#dates.～～**は、Thymeleaf で提供する変数式です。
簡単に現在の日付関連のデータを取得できる構文です。）

```
<div th:if="${#dates.dayOfWeek(...) == 2 }">
    今日は月曜特別セールです！
</div>
```

⑸ Radio ボタンのチェックの切り替え：

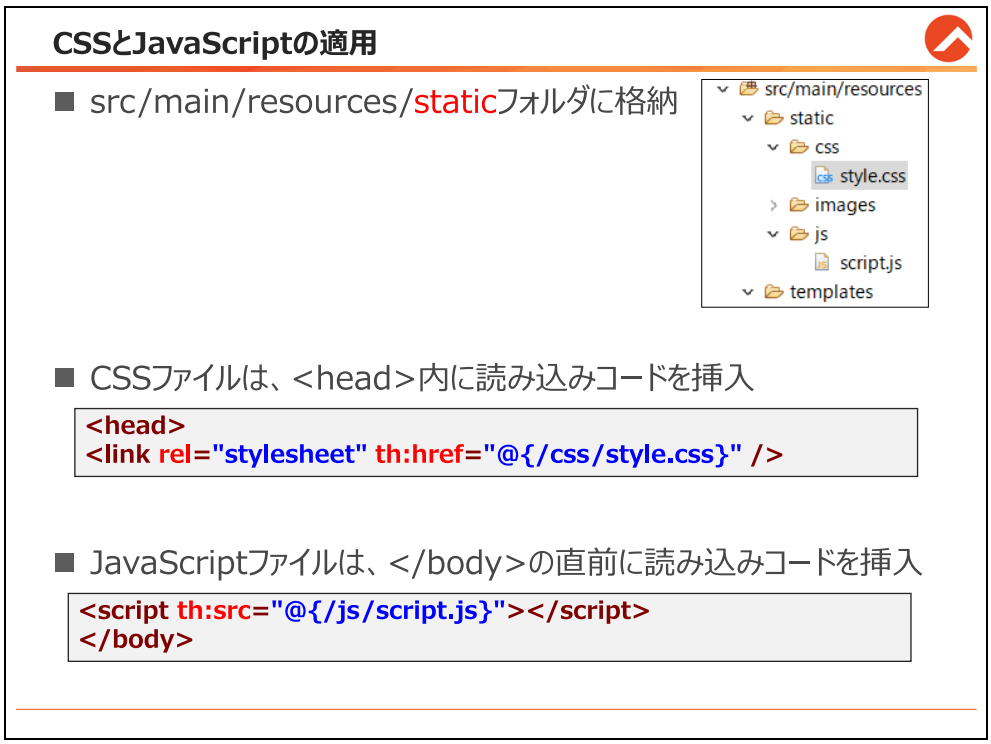
値が 1 か 2 かによって、ラジオボタンの checked 属性を動的に切り替える。

Radio ボタン以外にも、checkbox でも活用できます。

※if-unless でも書けますが、th:checked 属性を使うと、より簡略に記述できます。

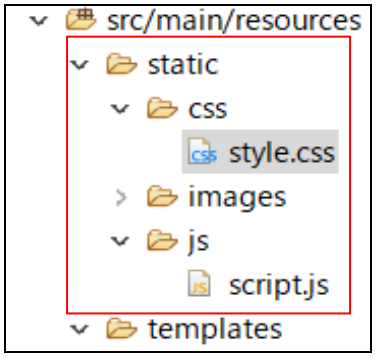
```
<input type="radio" name="gender" th:value="1" th:checked="${gender == 1}" />
<input type="radio" name="gender" th:value="2" th:checked="${gender == 2}" />
```

5.2. CSS と JavaScript の利用



Thymeleaf 内で CSS ファイルと、JavaScript(JS)ファイルを適用するためには、通常の方法では上手くいかないため、この節でご説明いたします。

CSS ファイルと JS ファイルの格納場所



css ファイルと js ファイルは、images と同様に **src/main/resources/static の中に格納しないと Spring Boot プロジェクト内で認識されませんので、必ずこの場所に格納してください。**

CSS ファイルの読み込み

…/resources/templates/product2/list.html

```
<head>
    <link rel="stylesheet" th:href="@{/css/style.css}" />
```

head タグの中に、th:href でファイルパスを記述すれば読み込みできます。/static までのパスは省略されます。

JS ファイルの読み込み

…/resources/templates/product2/list.html

```
<script th:src="@{/js/script.js}"></script>
</body>
```

body の終了タグの直前の場所に、th:src で js ファイルの場所を記述すると、読み込みできます。

サンプルの CSS・JS ファイルのコード

…/resources/static/css/style.css

```
@charset "UTF-8";
body{background-color: black;
     color: white;}
h1{color: yellow;}
```

単純に背景と文字色を反転させるスタイルが記載されています。

…/resources/static/js/script.js

```
document.write("<p>このテキストは JavaScript が生成しました</p>");
```

JavaScript で HTML を生成することもできます。

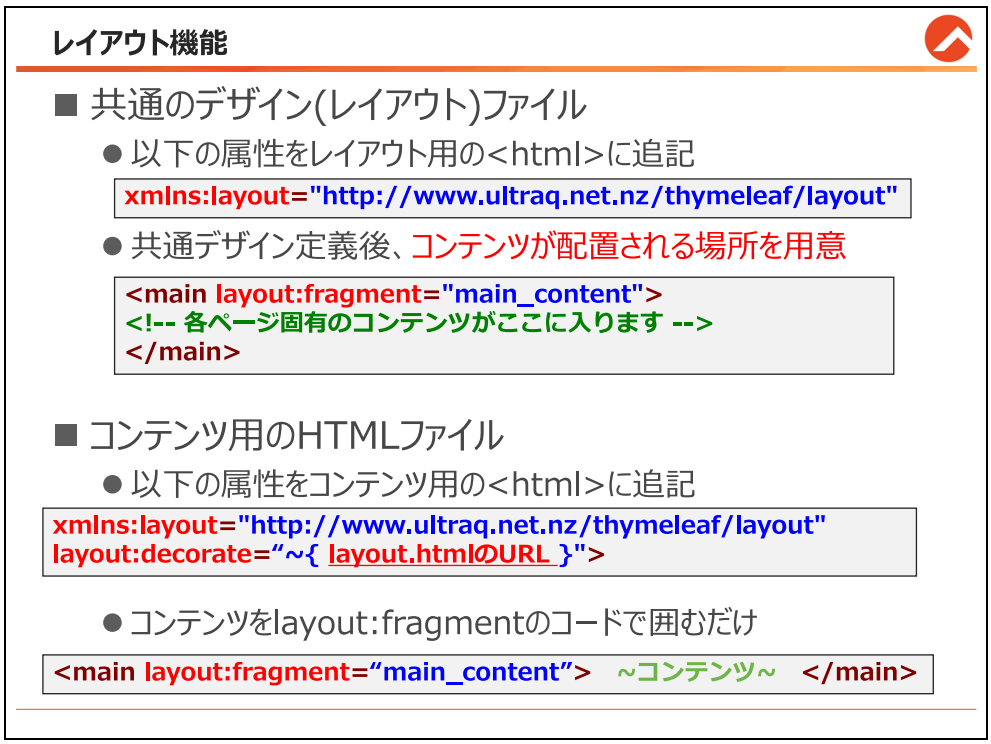
動作確認

http://localhost:8080/product2/findAll

CSS と JavaScript を連携することで、より綺麗で使いやすい画面を作ることができます。

この研修で詳細は割愛しますが、Spring Boot は REST API も簡単に作成することができますので、Ajax などで非同期処理を混ぜることで、アプリ感覚の SPA(Single Page Application)を作るのも容易です。

5.3. レイアウト機能



共通の要素を適用する課題について

一般的に、Web サイトは画面ごとに共通のデザインテーマを持ち、統一されたヘッダー、サイドバー、フッターを採用しています。しかし、Web サイトは数多くの html ファイルで構成されるものです。全ての html ファイルに、ヘッダー・サイドバー・フッターの要素をコピーすることを考えると、作業量も多いですし、保守性も落ちてしまいます。

Thymeleaf はこの問題を解決するための「レイアウト機能」を提供しています。**一度共通のレイアウトファイルを作成すれば、それを各ページに容易に適用することができる**ため、開発プロセスが大いに効率化されます。

それでは、Thymeleaf でのレイアウトの作成方法について詳しくご説明します。

目標画面

今回の研修では、メイン画面は画面ごと置き換えできるようにし、赤枠以外の部分を layout.html に記述します。

【 事前準備】ライブラリを導入

pom.xml

```
<dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

レイアウト機能を使うためには、関連のライブラリが必要です。追加して、maven install をしておきましょう。

【 作業手順①】レイアウトファイルの作成

…/resources/templates/layout.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org"
 3: xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
 4:
 5:<head>
 6:<meta charset="UTF-8" />
 7:<title>Traino 商品</title>
 8:<link rel="stylesheet" th:href="@{/css/layout.css}" />
 9:</head>
10:<body>
11:<!-- ヘッダー -->
12:<header>
13:<h1>Traino 家電商品</h1>
14:</header>
15:<div class="flex-container">
16:<!-- サイドバー -->
17:<nav>
18:     <ul>
19:          <li><a th:href="@{/product/findAll}">全商品一覧</a></li>
20:          <li><a th:href="@{/product/}">商品検索</a></li>
21:          <li><a th:href="@{/product/insert}">商品登録</a></li>
22:          <li><a th:href="@{/5}">ログアウト</a></li>
23:     </ul>
24:</nav>
25:
26:<!-- メインコンテンツ -->
27:<main layout:fragment="main_content">
28:     <!-- 各ページ固有のコンテンツがここに入ります -->
29:</main>
30:</div>
31:<!-- フッター -->
32:<footer>
33:<p>Copyright © 2023 Trainocate Spring. All rights reserved.</p>
34:</footer>
35:</body>
36:</html>
```

この layout.html には、Web サイトの共通要素であるヘッダー、ナビゲーションバー、フッターが定義されています。注目すべき 3 つの箇所をご説明します。

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

レイアウトファイルを定義するためには、既存の html 属性に、xmlns:layout 属性を追記する必要があります。html 要素を 2 回書いたわけではなく、html 要素に 2 つの属性を追加していることに気をつけてください。

・8 行目　<link rel="stylesheet" th:href="@{/css/layout.css}" />

レイアウト専用の css ファイルを別途作成し、読み込ませました。css ファイルの詳細は、デザインの領域ですので、章末の【 参考】の部分をご覧ください。

・27-29 行　<main layout:fragment="main_content">　</main>

この部分のコードは、中身がありません。その理由は、ここは**メインコンテンツが配置される場所**だからです。
「 layout:fragment="識別名"」の属性をつけると、この部分に別の html ファイル(コンテンツ)の fragment 識別名が一致しているコードがコピーされ、**置き換えられます**。

【 作業手順②】コンテンツ用の HTML の修正

…/resources/templates/product/insert.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th=" http://www.thymeleaf.org"
 3: xmlns:layout=" http://www.ultraq.net.nz/thymeleaf/layout"
 4:       layout:decorate=" ~{/layout}" >
 5:
 6:       <!-省略-->
 7:
 8:<body>
 9:<main layout:fragment=" main_content" >
10:
11:<!-既存のコンテンツのソースコード-->
12:
13:</main>
14:</body>
```

修正する HTML ファイルは、商品関連のどの HTML ファイルでも構いません。

・3 行目　xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"

レイアウトにコピーするコンテンツ側の HTML にも、同じくレイアウト関連のルールを宣言しておきます。

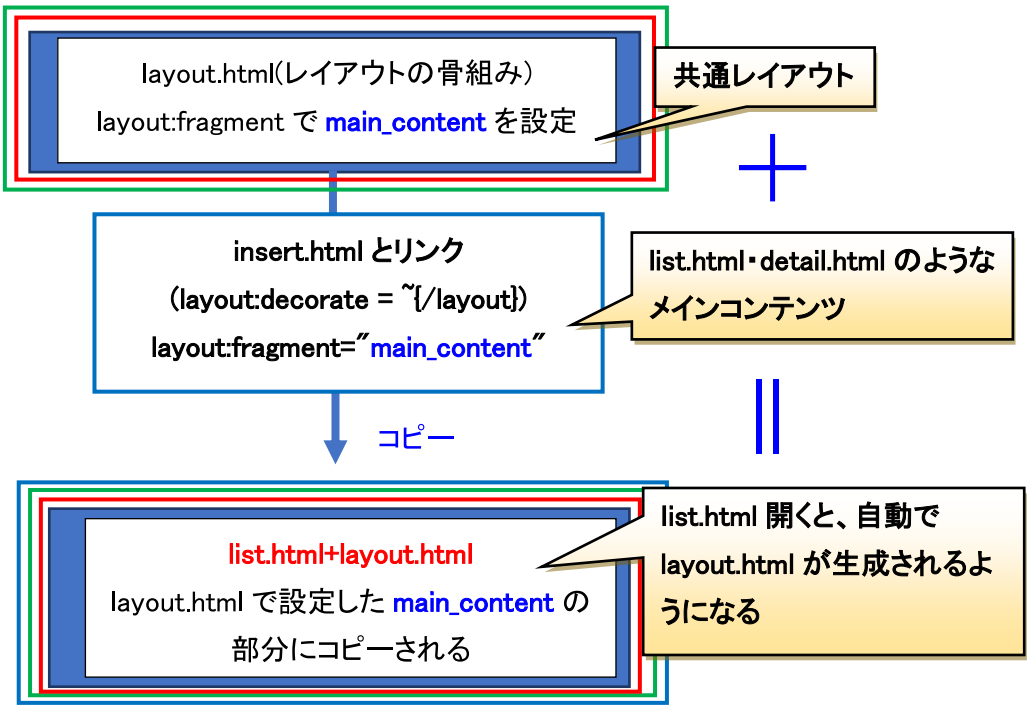
・4 行目　layout:decorate="~{/layout}">

<html>の属性の中に layout:decorate 属性も追記します。この部分の属性値は、**layout.html の場所を URL で指定**すれば OK です。

例えば、layout が product フォルダ内にあるなら、"~{/product/layout}" のように記述すれば OK です。URL を間違えると、レイアウト機能が使えませんので、ご注意ください。

・9-13 行目　<main layout:fragment="main_content">

既存の<body>内に書いていたソースコードを、全てこの要素で囲みます。先ほどの layout.html の fragment 識別名が"main_content"でしたので、同じ値を入れます。そうすることで、**9-13 行内の全ての HTML コードが、layout.html の 28 行にコピーされ、レイアウト+コンテンツの新しい HTML が生成されます。insert.html へ return して一覧画面を開くと、自動で layout.html の共通部分まで生成された画面が表示されます。**



今回は、insert.html のみ修正しましたが、レイアウトを適用させたい全ての html ファイルを、同じように修正すれば、以下のようにシステム全体に同じデザインが適用されます。

localhost:8080/product/findById/2
Traino家電商品
全商品一覧
商品検索
商品登録
ログアウト
商品詳細
商品番号 商品名 価格
2 ゲーミングモニター 20000
修正
削除
Copyright © 2023 Trainocate Spring. All rights reserved.

localhost:8080/product/insert
Traino家電商品
全商品一覧
商品検索
商品登録
ログアウト
商品登録
商品番号：(自動割り当て)
商品名 ：
商品価格：
登録
Copyright © 2023 Trainocate Spring. All rights reserved.

【参考】layout.css のソースコード

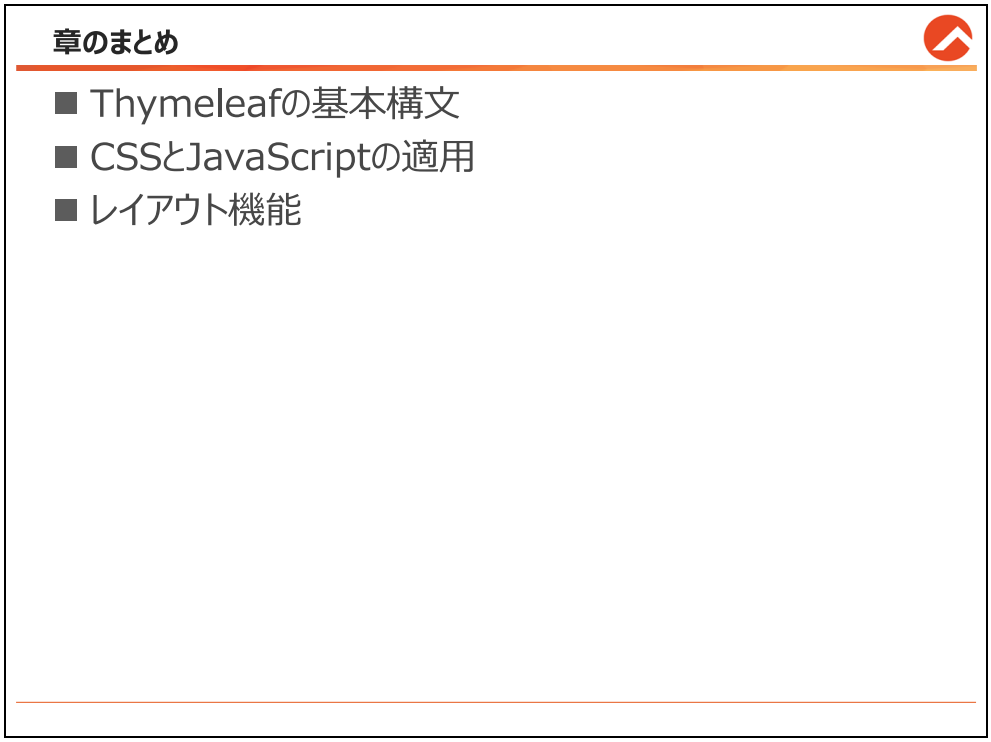
…/resources/static/css/layout.css

```
/* 全体のベーススタイリング */
html, body {
    height: 100%;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    font-family: UD デジタル 教科書体 NP-B, sans-serif;
}

/* ヘッダーのスタイリング */
header {
        background-color: #E44D26;
    color: white;
    padding: 10px 0;
    text-align: center;
    border-bottom: 3px solid #C1351D;
}

/* サイドバーのスタイリング */
nav {
    background-color: #333; /* 黒 */
    color: white;
    padding: 15px;
    width: 200px;
    flex-shrink: 0;
}

nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

nav ul li a {
    color: white;
```

```
        text-decoration: none;
        padding: 10px 0;
        display: block;
        font-size: 1.2em; /* テキストサイズを大きくする */
}

nav ul li a:hover {
        background-color: #555;
}

/* メインコンテンツのスタイリング */
main {
        background-color: #fff; /* 白色の背景 */
        padding: 30px;
        overflow-y: auto; /* 必要に応じてスクロールバーを表示 */
        flex-grow: 1;
}

/* フッターのスタイリング */
footer {
        background-color: #333; /* 黒 */
        color: white;
        text-align: center;
        padding: 10px 0;
        width: 100%;
}

/* フレックスボックスのレイアウト */
.flex-container {
        display: flex;
        flex: 1;
        flex-direction: row;
}

@media (max-width: 768px) {
        .flex-container {
                flex-direction: column;
        }
```

```
        nav {
            width: 100%;
        }
}

/* フッターが常に画面の最下部に表示されるようにする */
.flex-container {
        min-height: 0; /* フレックスコンテナの min-height をリセット */
}

body > footer {
        flex-shrink: 0;
}
```

章のまとめ



この章では、Spring Boot でシステムを実装する際に、追加で知っておくべき機能についてご説明しました。本書で全て紹介できなかった Spring の便利な機能もまだまだありますが、一番基本となる内容は網羅しましたので、これから皆様の Spring 開発知識の土台になると確信します。是非マスターしてください。

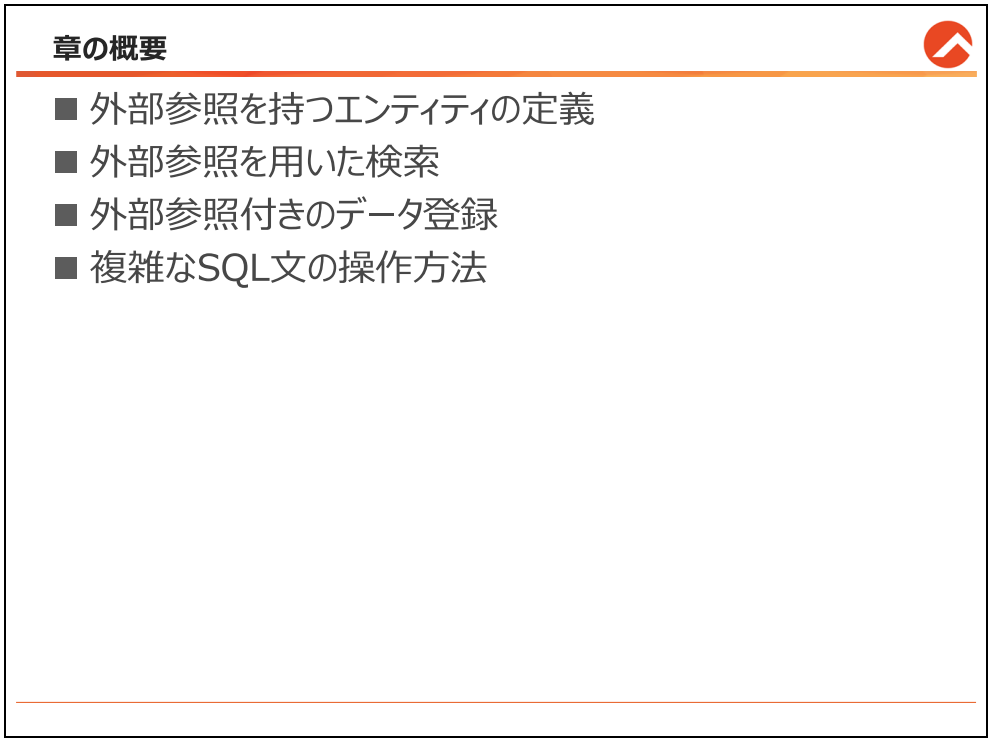
6

第6章　外部参照と複雑な SQL 操作

章の目標

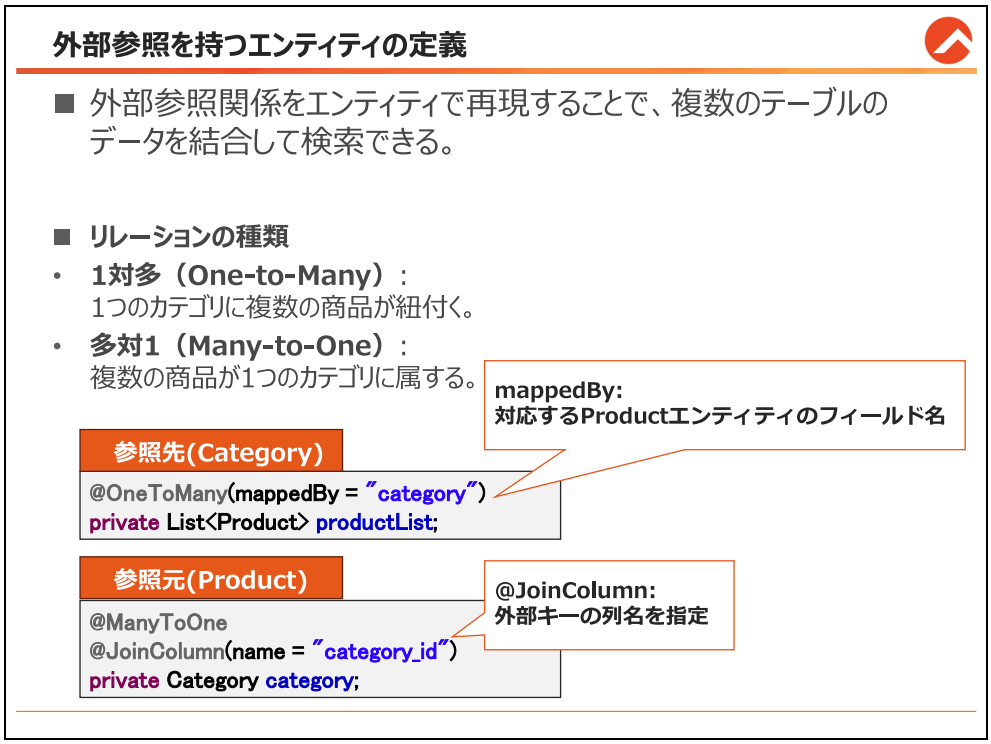
- 外部参照を持つエンティティを利用し、データベース間のリレーションシップを設定できる
- Spring Data JPA を用いた外部参照付きの検索や登録を実装できる
- 複雑な SQL クエリを実行し、実践的なデータ操作ができる

章の概要



本章では、外部参照を持つエンティティの操作方法について学びます。複数のテーブル間にリレーションシップがある場合のエンティティ定義や検索、登録、更新の方法を具体的に解説し、さらに複雑な SQL クエリをネイティブクエリで実装する方法を紹介します。Spring Data JPA の標準的な機能を基盤とし、実務に役立つデータ操作を学ぶことを目指します。

6.1. 外部参照を持つエンティティの定義



3 章では、1 つのテーブルに対応するエンティティを定義し、Spring Data JPA を用いた基本的な操作方法を学びました。本節では、**複数のテーブル間に外部参照の関係がある場合に、どのようにエンティティを設計し、リレーションシップを定義するのかを学びます。**具体的には、「 商品(Product) 」テーブルと「 カテゴリ(Category) 」テーブルのリレーションシップをエンティティとしてどのように表現するかを解説します。

【 作業手順①】新たなテーブルと、データの準備

以下のように、「 カテゴリ(Category) 」テーブルを新規作成し、既存の「 商品(Product) 」テーブルと外部参照で結合します。

※本書では、学習のためにデータベースが事前に構築されています。実務では、必要に応じてテーブルや外部キーの設定を行ってください。

＜Category テーブル＞

No	論理名称	物理名称	データ型	制約	備考
1	カテゴリ ID	id	INT	PRIMARY KEY	−
2	カテゴリ名	name	VARCHAR(30)	NOT NULL	−

<Category テーブルに格納されるデータ>

カテゴリ ID	カテゴリ名
1	テレビ・モニター
2	コンピュータ
3	冷蔵庫
4	洗濯機

このデータを基に、「 Product」テーブルにも新たに category_id 列を追加し、外部参照の関係を持たせる必要があります。

<既存の Product テーブルへの変更>

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	30	–	–
3	価格	price	INT	–	–	–
4	カテゴリ ID	category_id	INT		FOREIGN KEY	(Category テーブルの id 列を参照)

この修正により、Product テーブルと Category テーブルが外部キーで結びつきます。

商品番号	商品名	価格	カテゴリ ID
1	スマートテレビ	30000	1
2	ゲーミングモニター	20000	1
3	ノートパソコン	50000	2
4	2 ドア冷蔵庫	50000	3
5	コンパクト冷蔵庫	15000	3
6	ワインクーラー	30000	3
7	全自動洗濯機	25000	4
8	乾燥機能付き洗濯機	40000	4
9	洗濯乾燥機	60000	4

カテゴリを管理する「Category」テーブルを追加し、外部参照を利用する形に拡張する準備が整いました。

外部参照とは

外部参照とは、1 つのテーブルが別のテーブルのデータを参照する関係を指します。データベースでは外部キー(Foreign Key)を使用してこの関係を表現します。

テーブル同士のリレーショナル(関係)が結ばれると、その関係性を表すために、以下の用語で区別します。

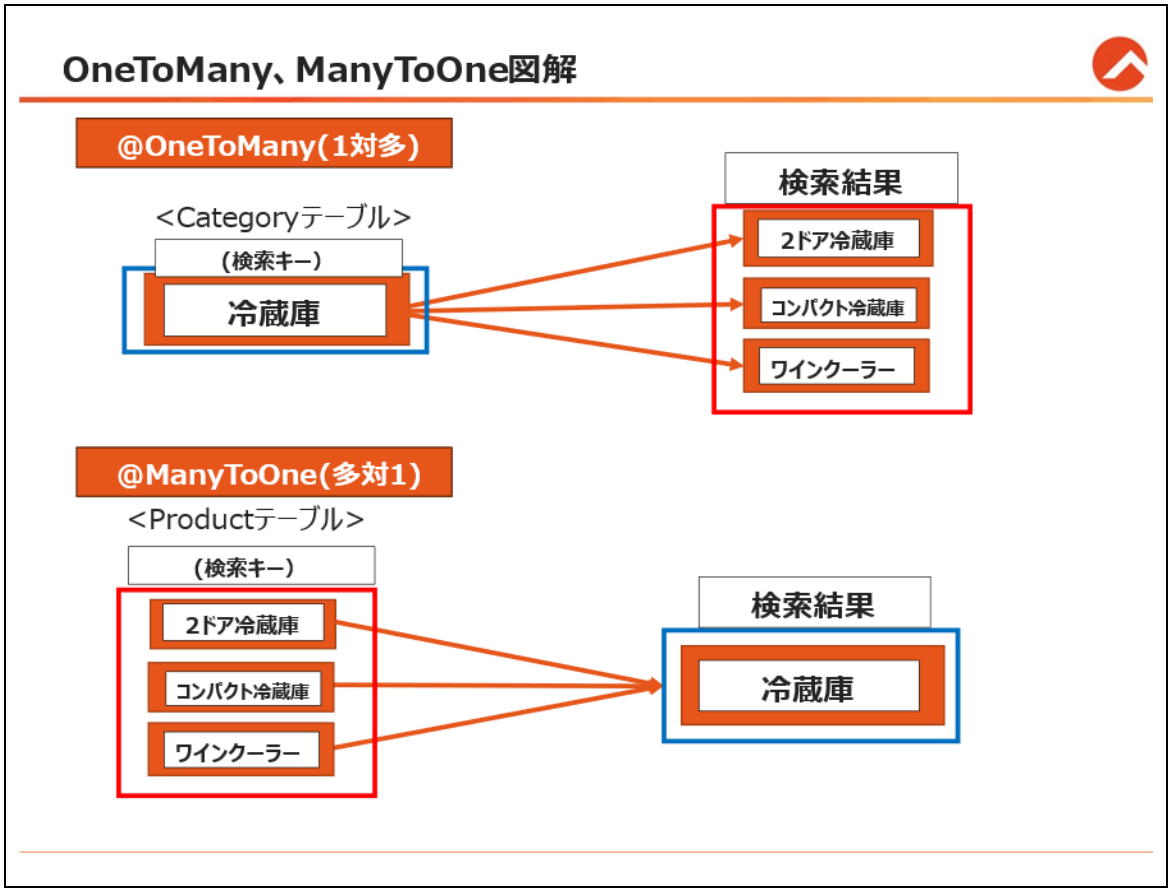
リレーションの種類

- 1 対多(One-to-Many)

  1 つのカテゴリ(例: 冷蔵庫)に対して複数の商品(例: 2 ドア冷蔵庫、ワインクーラー)が紐付く関係です。

- 多対 1(Many-to-One)

  複数の商品が 1 つのカテゴリに紐付く関係です。全ての商品は必ずいずれかのカテゴリに属します。



Category テーブルからの視点では、Product テーブルとの関係は、1 対多と表現できます。

Product テーブルからの視点では、Category テーブルとの関係は、多対 1 と表現できます。

【 作業手順②】参照先エンティティの作成

では、テーブルとの関係性を、Java の世界にエンティティとして、忠実に再現しましょう。
まずは、category テーブルに対応するエンティティ Category を作成します。

…/entity/Category.java

```
package jp.co.trainocate.answer.entity;

import jakarta.persistence.OneToMany;

@Entity
@Data
@Table(name = "category")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 30)
    private String name;

    @OneToMany(mappedBy = "category")
    private List<Product> productList;
}
```

上記の Category エンティティは、category テーブルに対応しています。各フィールドはテーブル内の列に対応しており、特に id フィールドは主キーとして定義されています。

また、**productList フィールド**は 後述する Product エンティティを格納するリストで、**一対多の関係を作るために複数のデータを格納する必要がある**ため、必須です。

@OneToMany アノテーションは、1 つのカテゴリが複数の商品を持つ「1 対多」の関係を表します。たとえば、「冷蔵庫」というカテゴリに、「2 ドア冷蔵庫、コンパクト冷蔵庫、ワインクーラー」が該当するパターンです。

@OneToMany アノテーションには **mappedBy 属性**があります。この属性は、後述する Product エンティティに定義されるフィール名を記載します。Product エンティティの説明の後に、再度ご確認ください。

【 作業手順③】参照元エンティティの修正

次に、Product テーブルに対応するエンティティ Product を修正します。Product テーブルは category テーブルを外部参照しているため、Product エンティティには Category エンティティをフィールドとして含めます。

…/entity/Product.java

```
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;

@Entity
@Data
@Table(name = "product")

public class Product {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Integer id;

        @Column(length = 30, nullable = false)
        private String name;

        @Column(nullable = false)
        private Integer price;

        @ManyToOne
        @JoinColumn(name = "category_id")
        private Category category;
}
```

最後の category フィールドにご注目ください。今まではテーブル上の列名がエンティティのフィールド名に合わせる必要がありました。よって、以下のようなテーブル構成なら、「 Integer categoryId」と定義するべきでしょう。

4	カテゴリ ID	category_id	INT		FOREIGN KEY	(Category テーブルの id 列を参照)

しかし、category_id ではなく、「 Category category」 のフィールドを定義しています。

JPA では、外部キー(例: category_id 列)を直接操作せず、リレーション先のエンティティ(例: Category)をフィールドとして扱います。これにより、オブジェクト指向の設計とデータベース操作を統一できます。

```
@ManyToOne
@JoinColumn(name = "category_id", nullable = false)
private Category category;
```

このように、**外部参照の関係を持つテーブルのエンティティを作成する際には、列名ではなく、参照先のエンティティをフィールドとして定義します。**Product テーブルは Category テーブルと外部参照の関係にあるため、Product エンティティ内に Category 型のフィールドを追加しています。

そして、**@ManyToOne アノテーションは、複数のアイテムが 1 つのカテゴリに属する「 多対 1」の関係を表します**。Product エンティティの category フィールドに付与されています。

たとえば、「 2 ドア冷蔵庫、コンパクト冷蔵庫、ワインクーラー」のようなデータは、「 冷蔵庫」のカテゴリに属しているということです。

@JoinColumn アノテーションは、参照元テーブルのどの列と参照先テーブルのどの列が外部参照制約の関係にあるかを定義します。 **name 属性には、外部キーとして使用するテーブル列名(例: category_id)を指定します。**

このアノテーションを記述すると、**検索時に参照関係にあるテーブルが内部結合され、自動的にデータを取得**できます。つまり、**Product テーブルを対象に検索したら、Category テーブルのデータも取得できるようになる**ということです。なお、参照関係にあるテーブルのレコードはエンティティのフィールドに代入されます。

これにより、テーブル間のリレーションシップが Spring 上で明確に定義され、データベース操作が容易になります。

【 作業手順④】 CategoryRepository の作成

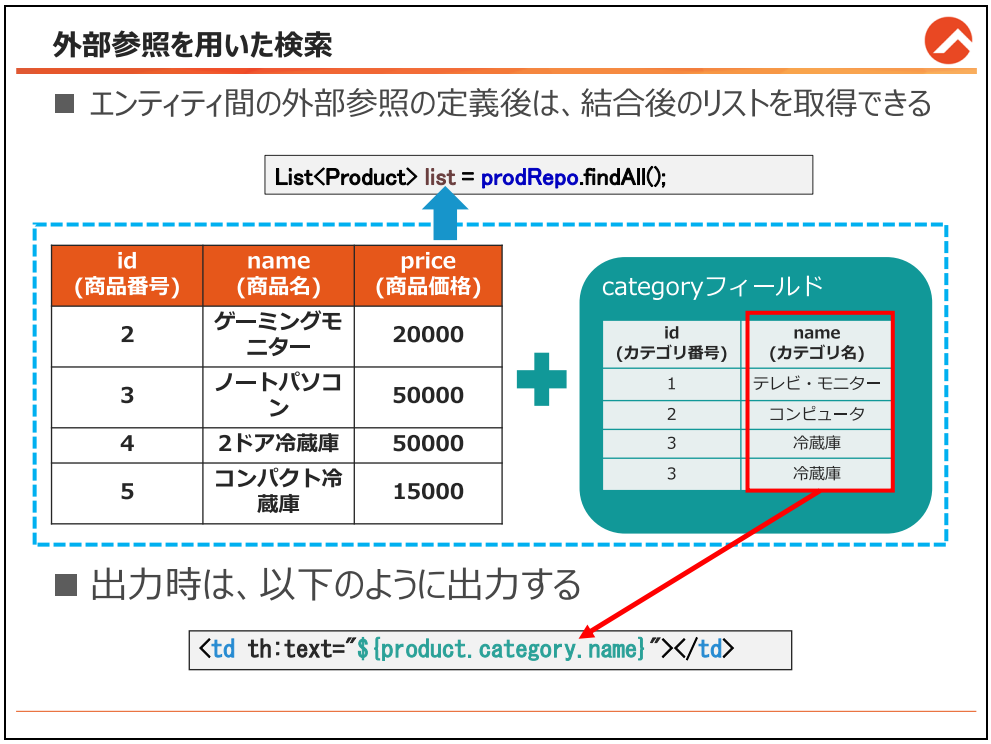
…/repository/CategoryRepository.java

```
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.Category;

public interface CategoryRepository extends JpaRepository<Category,
Integer> {

}
```

Category の検索も後ほど必要となりますので、リポジトリを作成します。

「Category」は、別 API と名前が被ることがありますので、今後、Import の際は、注意してください。

6.2. 外部参照を用いた検索



本節では、Spring Data JPA を用いて、外部参照を持つテーブル間の基本的な検索を実装する方法を学びます。外部参照を定義したエンティティを活用し、関連データを簡単に取得する基礎的な操作に重点を置きます。

【 作業手順①】商品コントローラーの再確認

以下は、3 章で作成した ProductController とほぼ同じ内容です。ただし、外部参照をエンティティに定義したため、挙動に変化があります。

…/controller/ProductController.java

```
@Controller
@RequestMapping("/product")
public class ProductController {
	@Autowired
	private ProductRepository prodRepo;

	@GetMapping("/findAll")
	public String showProductList(Model model) {
		List<Product> list = prodRepo.findAll();
	  //System.out.println(list); //外部参照の定義後は、エラーになる
		model.addAttribute("products", list);
		return "product/list";
	}
```

■ findAll メソッドの結果

外部参照を定義した後、findAll メソッドの実行結果には、**Product のデータに加えて、関連する Category のデータも含まれる**ようになります。

一覧取得時に、関連するカテゴリ情報も自動的に取得されます。

■ 標準出力の注意点

外部参照を持つエンティティは構造が複雑になります。特に、エンティティ間の双方向参照が設定されている場合、エンティティのインスタンスを System.out.println で出力すると、スタックオーバーフローエラーが発生する可能性があります。

Product が Category を参照し、Category が Product を再参照するためです。

【 作業手順②】View にカテゴリ名を出力するよう修正

商品データをリスト表示する際に、カテゴリ名も合わせて表示するように list.html を修正します。以下のコードは、商品リストを HTML テーブルとして表示するサンプルです。
変更点として、**カテゴリ名を表示する列を追加し、product.category.name でカテゴリ名を取得**しました。

…/resources/templates/product/list.html

```
<h1>商品一覧</h1>
        <table>
                <tr>
                        <th>商品番号</th>
                        <th>商品名</th>
                        <th>価格</th>
                        <th>カテゴリ</th>
                </tr>
                <tr th:each="product: ${products}">
                        <td th:text="${product.id}"></td>
                        <td th:text="${product.name}"></td>
                        <td th:text="${product.price}"></td>
                        <td th:text="${product.category.name}"></td>
                </tr>
        </table>
```

カテゴリ名の取得

<td th:text="${product.category.name}"></td>の部分で、Product エンティティに紐付く Category エンティティの name フィールドを表示できます。

一覧取得時に、関連するカテゴリ情報も自動的に取得されます。product.category は Product エンティティの@ManyToOne で定義されたリレーションを利用して、Category エンティティを取得できます。

このように、外部参照が正しく定義されていると、対象のエンティティだけではなく、紐づいている別エンティティのデータ群も取得可能です。

【参考】null データの参照時のエラー対策

外部参照が定義されたエンティティでは、関連するデータ(ここではカテゴリ)が設定されていない場合に注意が必要です。具体的には、**product.category が null の場合に product.category.name を参照しようとすると、エラーが発生します。**

このような場合、カテゴリが未設定(null)でもエラーを回避し、適切に表示できるように処理を追加します。

① th:if 文を利用

5 章で学習したとおり、Thymeleaf では、th:if 属性を利用して条件を設定することができます。

```
<td th:if="${product.category != null}" th:text="${product.category.name}"></td>
<td th:unless="${product.category != null}">未設定</td>
```

■ **th:if="${product.category != null}"**
この部分は、「カテゴリが設定されている場合」に実行されます。カテゴリ名を表示します。

■ **th:unless="${product.category != null}"**
この部分は、「カテゴリが未設定の場合」に実行されます。「未設定」という文字列を表示します。

② 条件式を 1 行で記述する方法(三項演算子)

上記のように複数行で条件を記述する代わりに、条件式を使って 1 行で記述する方法もあります。

```
<td th:text="${product.category != null ? product.category.name : '未設定'}"></td>
```

■ **product.category != null:**

「カテゴリが設定されているか」をチェックします。

■ **?(三項演算子):**

条件が true の場合に左側の値(product.category.name)を表示します。

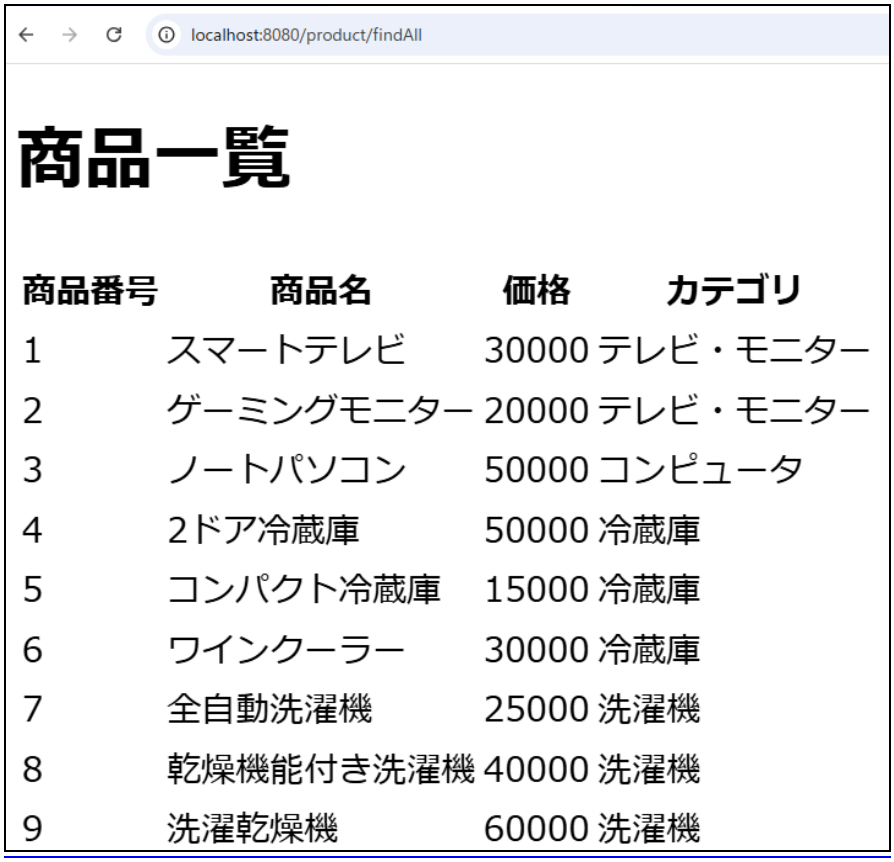
■ **:(else 部分):**

条件が false の場合に右側の値('未設定')を表示します。

動作確認

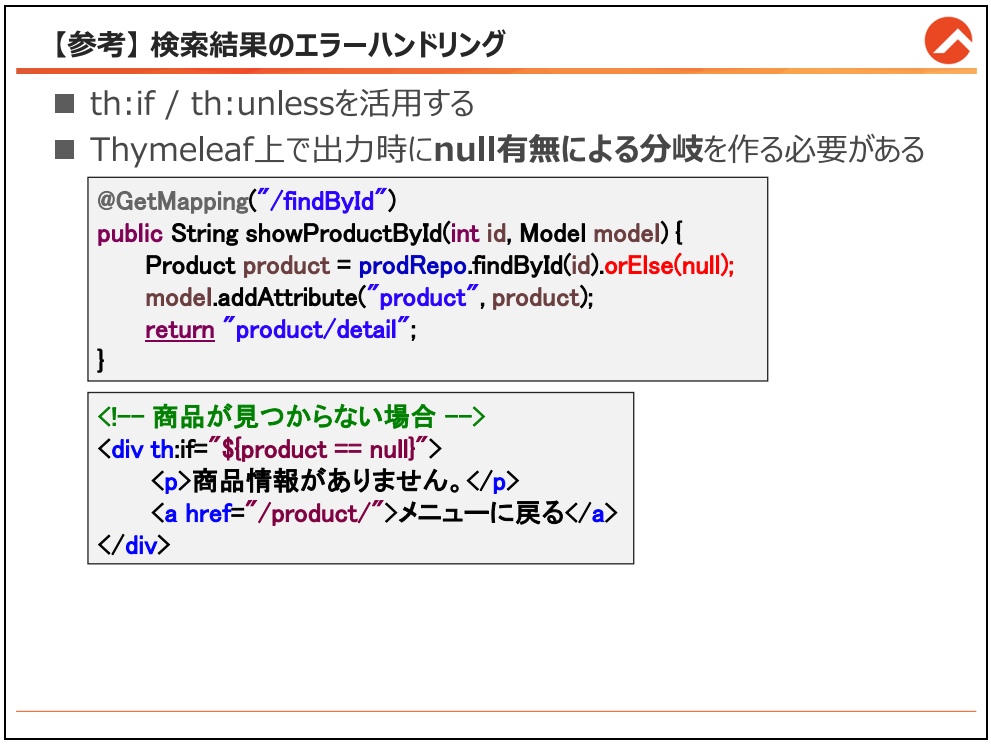
http://localhost:8080/product/findAll

3 章と同様に、Controller の URL マッピングが、[/product]
メソッドの URL マッピングが[/findAll] になっているため、このような URL を指定します。



カテゴリ列に商品ごとのカテゴリ名が結合され表示されることを確認できます。

6.2.1.【 参考】検索結果のエラーハンドリング



本節では、検索時のエラーハンドリングについて学びます。存在しない ID で検索した場合に発生するエラーを回避し、より安全にデータを取得する方法について学びます。また、カテゴリ名を表示することで、リレーションの復習も行います。

【 作業手順①】 findById(id).orElse(null)の確認

…/controller/ProductController.java

```
//省略

@GetMapping("/findById")
	public String showProductById(int id, Model model) {
		Product product = prodRepo.findById(id).orElse(null);
		model.addAttribute("product", product);
		return "product/detail";
	}
```

findById(id).orElse(null)を使用すると、データが存在しない場合に null が返され、安全に処理を進めることができます。

もちろん、エンティティ間の定義が完了しているので、**findById でも結合した結果を取得**できます。

【 作業手順②】商品が見つからなかった際の表示の切り替え

…/resources/templates/product/detail.html

```
<h1>商品詳細</h1>

    <!-- 商品が見つからない場合 -->
    <div th:if="${product == null}">
        <p>商品情報がありません。</p>
        <a href="/product/">検索メニューに戻る</a>
    </div>

    <!-- 商品が見つかった場合 -->
    <div th:unless="${product == null}">
        <table>
            <tr>
                <th>商品番号</th>
                <th>商品名</th>
                <th>価格</th>
                <th>カテゴリ</th>
            </tr>
            <tr>
                <td th:text="${product.id}"></td>
                <td th:text="${product.name}"></td>
                <td th:text="${product.price}"></td>
                <td th:text="${product.category.name}"></td>
            </tr>
        </table>

    <!--...省略-->

    </div>
```

で囲まれた部分は、Controller で Model に格納されたデータの有無に応じて、表示内容を切り替える処理を行っています。

このように、**複数行の Thymeleaf 条件式は、視認性や保守性を高めるために を使用することを推奨します。**

※単体のデータの有無ではなく、リスト内の判定については、5.1.2 の②を再度ご確認ください。

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```

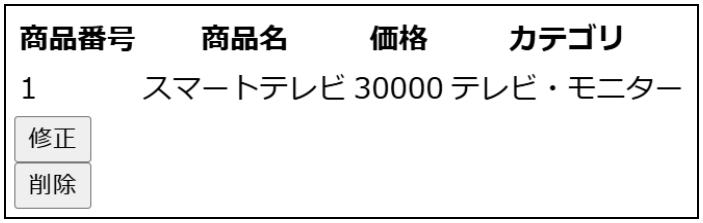
動作確認

http://localhost:8080/product/

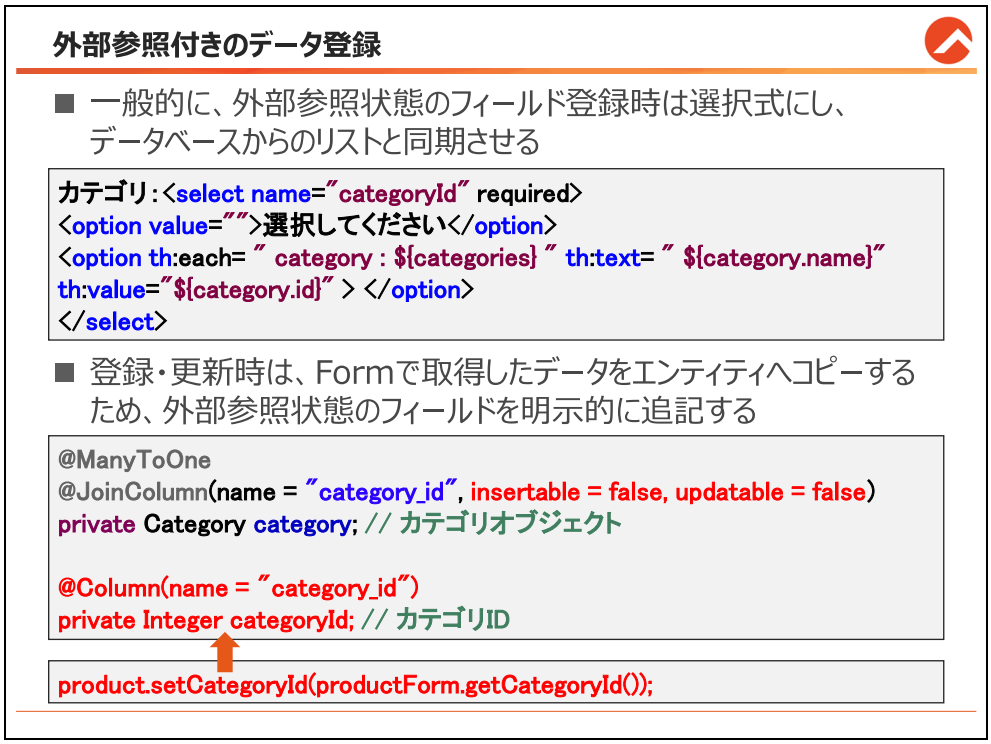
① ID 検索フォームに、存在しない商品番号を入れ、検索します。

② 例外のエラーページの代わりに、商品情報なしのメッセージが表示されることを確認してください。

③ 検索ページに戻り、今度は「 1」で商品番号検索を行い、カテゴリ名を含めた商品データが表示されるかを確認してください。



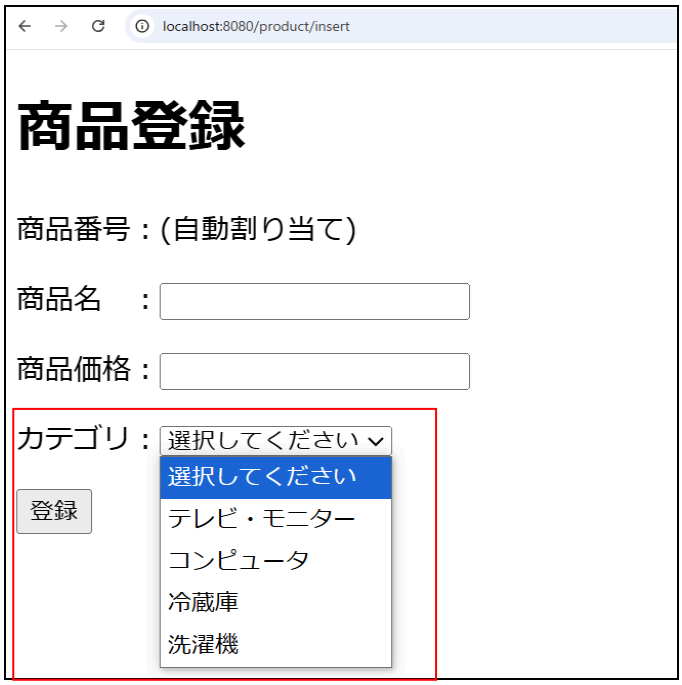
6.3. 外部参照付きのデータ登録



本節では、商品データとカテゴリデータのリレーションを考慮したデータ登録を実装します。特に、商品情報を登録する際にカテゴリを選択する方法を学びます。

また、フォームから送信されたカテゴリ ID を商品エンティティに関連付けて登録する際に直面する課題を説明し、その解決方法を実装します。

商品登録に選択式のカテゴリを追加する時



今回は、商品データにカテゴリを登録する機能を追加します。
カテゴリのように、多対 1 の外部参照関係を持つデータは、Web 上では選択式(セレクトメニュー) で登録するのが一般的です。

以下は、固定値を用いたセレクトメニューの例です。

…/resources/templates/product/insert.html

```
<h1>商品登録</h1>
        <form th:action = "@{/product/insert}" method = "POST">
        <p>商品番号：(自動割り当て)</p>
        <p>商品名　：<input type = "text" name = "name"></p>
        <p>商品価格：<input type = "number" name = "price"></p>
        <p>カテゴリ：
                <select name="categoryId" required>
                        <option value = "">選択してください</option>
                        <option value = "1">テレビ・モニター</option>
                        <option value = "2">コンピュータ</option>
                        <option value = "3">冷蔵庫</option>
                        <option value = "4">洗濯機</option>
                </select>
        </p>
```

このように固定値を記載する方法は簡単ですが、以下のような問題点があります。

- **新しいカテゴリの追加や既存カテゴリの修正が手間**
  HTML コードを手動で変更する必要があるため、管理が煩雑になります。
- **データベースとの不一致**
  UI とデータベースが同期しなくなる可能性があり、意図しない不具合が発生します。

これらの問題を解決するために、データベースからカテゴリデータを取得し、動的にセレクトメニューを生成する方法を採用します。

【 作業手順①】 登録処理の準備 – ProductForm の修正

登録画面のフォームでは以下のデータが送信されます。

- 商品名(name)
- 商品価格(price)
- カテゴリ ID(categoryId)

新たに追加された categoryId をコントローラで受け取るため、ProductForm クラスを修正します。

…/form/ProductForm.java

```
@Data
public class ProductForm {
        private Integer id; //商品番号
        private String name; //商品名
        private Integer price; //商品価格

        private Integer categoryId; //カテゴリ ID
}
```

送信される際に、select 要素の name 属性値が、categoryId ですので、Integer categoryId と定義します。

【 作業手順②】 登録画面遷移用のコントローラーの修正

…/controller/ProductController.java

```
import jp.co.trainocate.answer.entity.Category;
import jp.co.trainocate.answer.repository.CategoryRepository;

@Controller
@RequestMapping("/product")
public class ProductController {
	@Autowired
	private ProductRepository prodRepo;
	@Autowired
	private CategoryRepository cateRepo;

//検索系メソッドは省略

@GetMapping("/insert")
	public String showProductInsertView(Model model) {
		// カテゴリリストを取得
		 List<Category> list = cateRepo.findAll();
		 model.addAttribute("categories", list);
		 return "product/insert";  // 登録入力画面へ
	}
```

次は、Controller のハンドラメソッドを修正します。元々、@GetMapping(“/insert”)のメソッドは、今まで単純に登録をする入力画面へ遷移するだけの単純なコードでした。

ただ、<option>にカテゴリデータを動的に出力するには、コントローラで事前にモデルにカテゴリリストを格納して送信しておく必要があります。

コードの解説:

- **CategoryRepository の依存性注入:**

  カテゴリデータをデータベースから取得するため、CategoryRepository を @Autowired で注入します。

- **findAll()による全件取得:**

  カテゴリリストを findAll()メソッドで取得します。

- **model.addAttribute:**

  カテゴリリストを **categories** という名前でモデルに格納し、ビューで利用可能にします。

- **入力画面テンプレートへの遷移:**

  return "product/insert";で商品登録画面へ遷移します。

これで、登録画面へカテゴリのリストの送信ができましたので、登録画面の View を実装しましょう。

【 作業手順③】 カテゴリのリストの動的表示

```
<p>カテゴリ：<select name="categoryId" required>
        <option value="">選択してください</option>
        <option th:each="category : ${categories}"
                th:text="${category.name}"
                th:value="${category.id}" >
        </option>
    </select>
    </p>
```

修正点の説明:

- th:each:
  データベースから取得したカテゴリリスト(categories)をループ処理で展開します。
- th:text:
  メニューに表示するカテゴリ名(category.name)を動的に設定します。
- th:value:
  フォーム送信時にカテゴリ ID(category.id)をコントローラに渡します。登録処理ではこの ID を使用します。

ポイント:

- データベースのカテゴリデータに変更があっても、フォームは自動的に更新されます。
- セレクトメニューの<option>タグがデータベースの内容と常に同期します。

【作業手順④】登録処理の実装 － 不具合の確認

ProductForm に categoryId フィールドを追加したことで、登録処理用のメソッドでカテゴリ ID を受け取ることができるようになりました。次に、ProductForm のデータを Product エンティティにコピーして保存する処理を実装します。

…/controller/ProductController.java

```
@PostMapping("/insert")
public String insertProduct(ProductForm productForm) {
	Product product = new Product();
	product.setName(productForm.getName());
	product.setPrice(productForm.getPrice());

	// カテゴリ ID を設定する処理を追加しようとする
	product.setCategoryId(productForm.getCategoryId()); // エラー発生

	prodRepo.save(product);
	return "redirect:/product/findAll";
}
```

3 章で学習した通り、prodRepo.save(product)をする前には、ProductForm のデータを Product エンティティにコピーしておく必要がありました。

しかし、いざ product.setCategoryId( ) メソッドを呼び出そうとしたら、そもそも**Product エンティティには、categoryId ではなく、Category category でフィールドを定義していたため、setCategoryId メソッドが生成されていません。**よって、**エラー**になります。

```
@Entity
@Data
@Table(name = "product")

public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 30, nullable = false)
    private String name;

    @Column(nullable = false)
    private Integer price;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category; //categoryIdではない

}
```

【 作業手順⑤】Product エンティティに categoryId フィールドの追加

作業手順④の不具合を解決するため、**Product エンティティに categoryId フィールドを追加**します。
ただし、categoryId と category の 2 つのフィールドが同じ列を参照するため、重複マッピングに注意が必要です。

…/entity/Product.java

```
// 上記省略
  @Column(nullable = false)
  private Integer price;

  @ManyToOne
  @JoinColumn(name = "category_id", insertable = false, updatable = false)
  private Category category; // カテゴリオブジェクト

  @Column(name = "category_id")
  private Integer categoryId; // カテゴリ ID
}
```

フィールド追加のポイント

- **@Column(name = "category_id")を明示的に指定する理由:**

  データベースの category_id 列を categoryId フィールドにマッピングするために必要です。
  指定しないと、フィールド名を基に自動推論され、列重複によるエラーが出ます。

- **insertable = false, updatable = false を設定する理由:**

  @ManyToOne の category フィールドと@Column の categoryId が同じカラムを参照していると、**登録や更新時にどちらを優先するか Hibernate が判断できず**、以下のようなエラーが発生します

  > Repeated column in mapping for entity: Product column: category_id (should be mapped with insert="false" update="false")

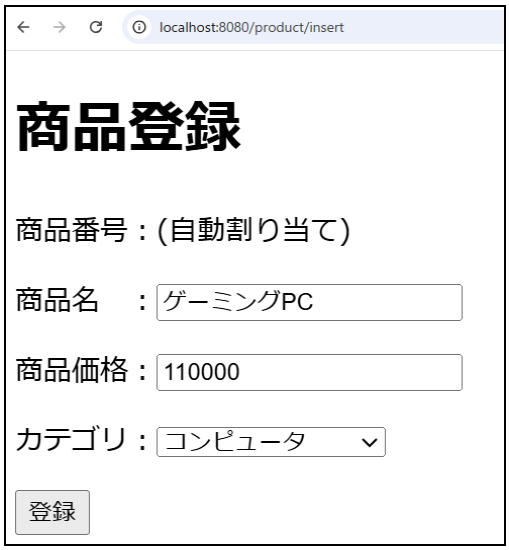
  insertable = false, updatable = false を設定することで、**category フィールドが登録や更新に関与しないようにし、categoryId だけを使用するように制御**できます。

Product エンティティの修正後、保存すると、登録処理用メソッドのエラーが消え、正常にカテゴリ ID を登録できるようになります。

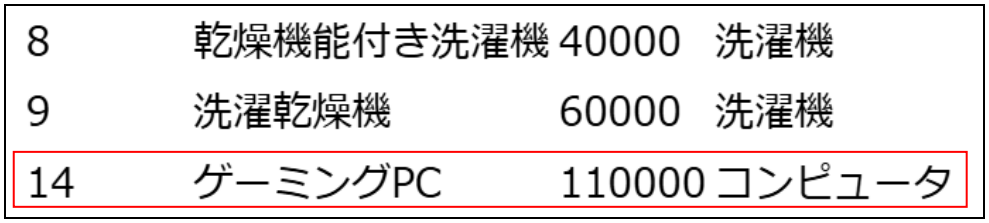
動作確認

http://localhost:8080/product/insert

① 任意のデータを入力・選択し、「登録」ボタンをクリックします。

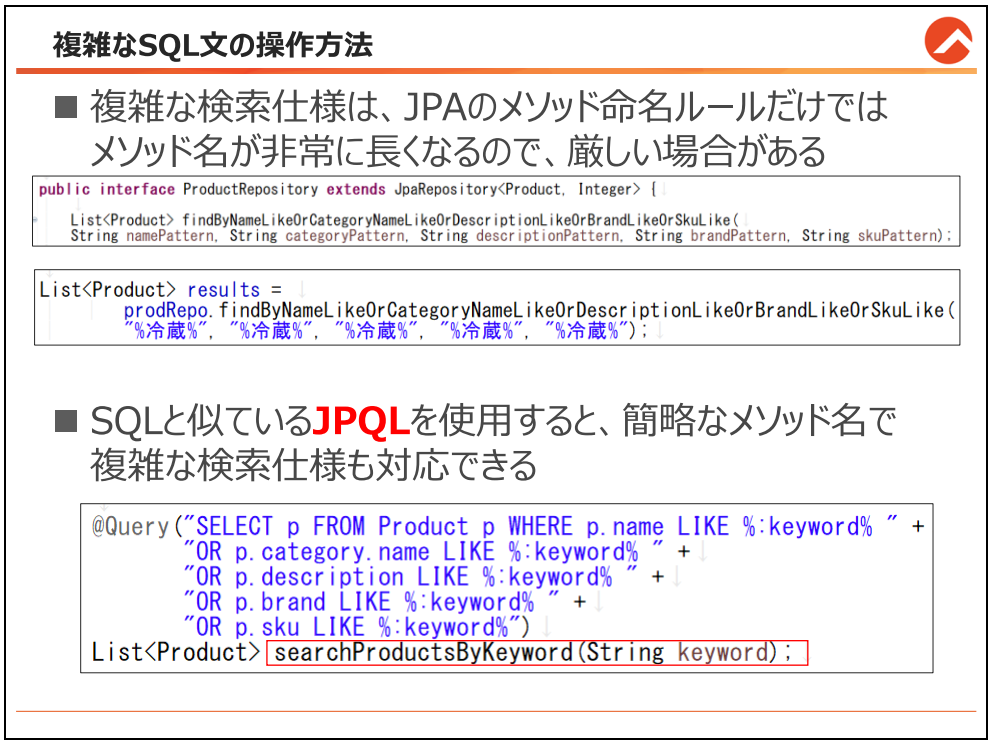


② 商品一覧に、カテゴリを含む新規データが追加されたら OK です。



※この節では、登録処理のみ説明しましたが、更新処理も同様に categoryId を受け取り、エンティティにコピーする 1 行さえ追記すれば、実装できますので参考にしてください。

6.4. 複雑な SQL 文の操作方法



これまでの章では、Spring Data JPA のリポジトリメソッド名を利用してクエリを実行する方法を学びました。しかし、複雑な SQL 文を作成する場合、リポジトリメソッド名が非常に長くなり、可読性や保守性が低下するという問題があります。

本節では、JPA の JPQL(Java Persistence Query Language)を使用して、クエリを記述する方法を学びます。JPQL は SQL に似た構文を持ちつつ、Java のエンティティを操作するために最適化されています。

JPQL の基本構文を学び、簡単な例を通じて理解を深めます。最後に【 参考】として、複雑な仕様に基づいた JPQL の実装例を紹介します。

6.4.1. 複雑な検索をメソッド名で定義する際のデメリット

実務では、テーブル内の列が増えたり、複雑な仕様の検索をしないといけない場面がよくあります。そのような状況で、Spring Data JPA のメソッド命名規則を使用して複雑な検索を実現しようとすると、いくつかのデメリットが発生します。

以下の例を基に説明します。

＜例: 拡張された Product テーブル＞

以下のように、Product テーブルが拡張されている状態を仮定します。

列の物理名	列の論理名
id	商品番号
name	商品名
price	価格
description	商品説明
brand	ブランド名
sku	SKU コード
category_id	カテゴリ ID
price	価格

そして、以下のような検索機能を実装する必要があったと仮定します。

- **例題: 商品とカテゴリを検索する仕様**

  「商品名」「カテゴリ名」「商品説明」「ブランド名」「SKU コード」の**いずれかに部分一致する商品を検索**し、該当する商品一覧を取得します。

- **出力形式:**

  商品番号、商品名、カテゴリ名、商品説明、ブランド名、SKU コード、価格

Spring Data JPA のメソッド命名規則で再現する場合

3 章で学習した通り、Spring Data JPA では findBy メソッド名に条件を列挙してクエリを作成できます。しかし、この仕様を満たすために適用すると、以下のような非常に長いメソッド名になってしまいます。

<ProductRepository に定義>

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

List<Product> findByNameLikeOrCategoryNameLikeOrDescriptionLikeOrBrandLikeOrSkuLike(
  String namePattern, String categoryPattern, String descriptionPattern, String
brandPattern, String skuPattern);
```

以下のように、全ての列に対して同じ検索キーワードを適用する場合でも、それぞれ個別のパラメータを渡す必要があります。

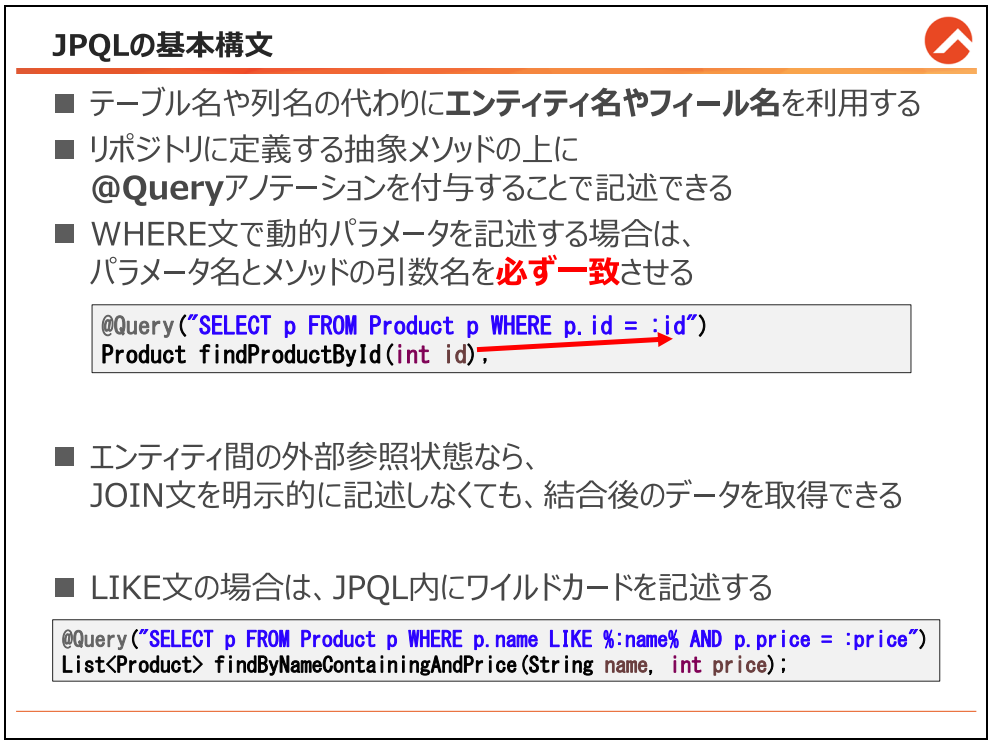
＜ProductController から、メソッドを呼び出す時＞

```
List<Product> results =
productRepository.findByNameLikeOrCategoryNameLikeOrDescriptionLikeOrBrandLikeOrSkuLike(
    "%冷蔵%", "%冷蔵%", "%冷蔵%", "%冷蔵%", "%冷蔵%");
```

非常に長いメソッド名と、冗長なパラメータを渡す必要があるため、JPA のメソッド命名規則だと、厳しいです。このままだと、可読性はもちろん、保守性も低い実装になってしまいます。

このような**複雑な検索条件を効率的に実装するには、SQL 文を直接記述する方法が適しています。**次節では、Spring Data JPA の JPQL と、@Query アノテーションを使用して SQL 文を直接記述する方法について説明します。

6.4.2. JPQL の基本構文



JPQL の特徴

JPQL(Java Persistence Query Language)は、JPA エンティティを基に操作を行うクエリ言語です。

SQLと似た構文ですが、**テーブル名や列名ではなく、エンティティ名やフィールド名を使用**します。

JPQL の主な特徴:

- **エンティティモデルに基づいてクエリを記述**
  データベースのテーブル構造を意識することなく、Java のオブジェクト指向に沿ったクエリが記述できます。
- **リポジトリに直接記述可能**
  クエリはリポジトリインターフェースに記述しますので、以前の章で学んだ感覚のまま実装できます。
- **メソッド名の命名規則が不要**
  JPQL を使用する場合、自由にメソッド名を設定できます。

全件検索の実装

JPQL の基本を学ぶため、すべての商品データを取得するクエリを JPQL で実装します。以下のようにリポジトリインターフェースにクエリを記述します。

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

        @Query("SELECT p FROM Product p")
         List<Product> findAllProducts();

        //他の抽象メソッドは省略
}
```

- **findAllProducts**
  メソッド名は自由に決められます。今回は分かりやすい名前にしています。引数と戻り値を開発者が決めることは、前の章と同じです。全件検索ですので、引数無しで戻り値は List<Product>で定義しています。
- **@Query** アノテーションの付与
  JPQL を直接文字列として、@Query アノテーションの引数に記述できます。
- **SELECT p FROM Product p**
  エンティティ Product をすべて取得するクエリです。p は表別名で、エンティティを簡潔に参照するために使用します。
  SELECT * FROM Product の代わりに、このような記述方法を使います。

JPQL を記述する際、以下のルールを守る必要があります。

1. **エンティティ名を使用**
   Product(エンティティ名)を使用します。テーブル名(product)ではありません。
2. **フィールド名を使用**
   エンティティのフィールド名を指定します。列名(name や price)ではなく、Java のフィールド名をそのまま使います。
3. **SQL と似ているが異なる点もある**
   SQL とほぼ同じ構文ですが、SELECT 句や WHERE 句でテーブルではなくエンティティを扱うことを意識しましょう。

次は、コントローラーの全件検索用メソッドで、呼び出すメソッドを「 findAll」から、
「 findAllProducts」に変更します。

…/controller/ProductController.java

```
@GetMapping("/findAll")
        public String showProductList(Model model) {
                List<Product> list = prodRepo.findAllProducts();
                model.addAttribute("products", list);
                return "product/list";
        }
```

http://localhost:8080/product/findAll で動作確認を行うと、従来と同じように全件検索の結果が表示されることを確認できます。

JPQL の大きな利点は、エンティティ同士に外部参照(リレーション) が設定されている場合、JOIN 文を明示的に記述しなくても、関連するエンティティ(例えば Category) のデータも自動的に取得できる点です。

【 参考】大量データを扱う場合の注意点

大量のデータを扱う場合、関連する結合データが自動的に取得されることで、パフォーマンスに影響を与えることがあります。
このような場合、関連データのロード方法を変更することでパフォーマンスを改善できます。

以下のように@ManyToOne アノテーションのフェッチタイプを LAZY に変更します。

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "category_id")
private Category category;
```

結合データ(Category) は、明示的にアクセスされるまでロードされません。結果的に、無駄なデータのロードを防ぎ、クエリの実行速度が向上します。

WHERE 文の基本構文

これまでの章では、findById のようなリポジトリメソッドを使用し、検索条件をメソッドの引数で渡すことでデータを取得していました。

JPQL を使用した場合も同じように、検索条件を引数で渡してデータを検索できます。違いとしては、**@Query を使用して WHERE 文を直接記述する**点です。

以下は、指定された主キー(id)に一致する商品データを検索する JPQL の例です。

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

    @Query("SELECT p FROM Product p")
     List<Product> findAllProducts();

    @Query("SELECT p FROM Product p WHERE p.id = :id")
     Product findProductById(int id);
```

- **WHERE p.id = :id**

  Product エンティティの **id フィールドが指定された値(:id)と一致**するデータを検索します。

- **:id**

  プレースホルダとして使用されるパラメータ。実行時に引数 id の値が挿入されます。

Java データベースプログラミングとの比較

理解のため、既存の知識である JDBC で主キー検索を行う場合と比較します。

```
String sql = "SELECT * FROM product WHERE id = ?";
PreparedStatement pstm = connection.prepareStatement(sql);
pstm.setInt(1, id);
```

- **JDBC でのプレースホルダ**

  SQL 文中の「 ?」がプレースホルダとして動的に値を受け取ります。
  パラメータの位置は、1 番目、2 番目と番号で指定します。

- **JPQL でのプレースホルダ**

  JPQL では、:id のような名前付きパラメータを使用します。番号ではなく、**パラメータ名と引数名を一致させることで紐付け**されるため、直感的で扱いやすいのが特徴です。

…/controller/ProductController.java

```
@GetMapping("/findById")
        public String showProductById(int id, Model model) {
                //Product product = prodRepo.findById(id).orElse(null);
                Product product = prodRepo.findProductById(id);
                model.addAttribute("product", product);
                return "product/detail";
        }
```

findById の代わりに、JPQL で定義したメソッドを呼び出します。

http://localhost:8080/product/ で ID 検索の動作確認をすると、同じ主キー検索ができるのが確認できます。

名前の LIKE 検索と価格の AND 検索の実装

商品名に特定のキーワードが含まれるデータを検索し、かつ、価格が指定された値と一致する商品のみを取得するという条件は、既存の JDBC 式では、以下のような記述が必要でした。

```
String sql = "SELECT * FROM product WHERE name LIKE ? AND price = ?";
PreparedStatement pstmt = conn.prepareStatement(sql));
pstmt.setString(1, "%" + name + "%"); // 名前の部分一致
pstmt.setInt(2, price); // 価格の条件
```

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{
        //他メソッドは省略

  @Query("SELECT p FROM Product p WHERE p.name LIKE %:name% AND p.price = :price")
    List<Product> findByNameContainingAndPrice(String name, int price);
}
```

JPQL を使う場合、name を囲む**ワイルドカード(%など)も一緒に記述**します。

…/controller/ProductController.java

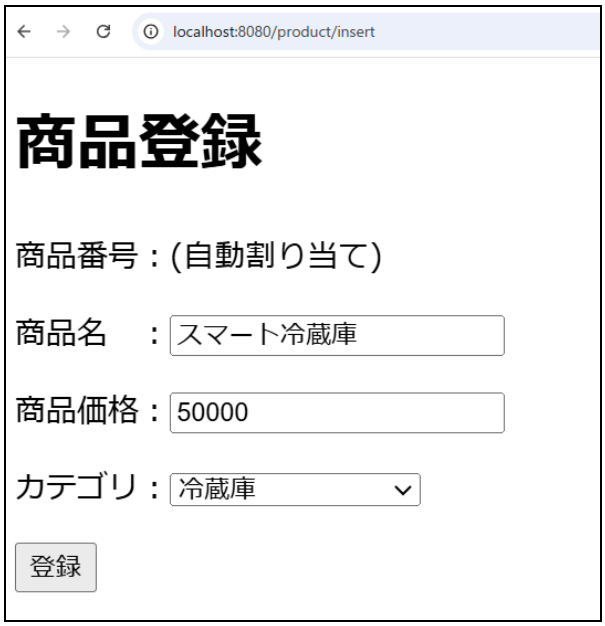
```
@GetMapping("/findByNameAndPrice")
public String showProductListByNameAndPrice(String name, int price, Model model)
{
    //List<Product> list = prodRepo.findByNameAndPrice(name, price);
    List<Product> list = prodRepo.findByNameContainingAndPrice(name, price);
        model.addAttribute("products", list);
        return "product/list";
}
```

コントローラーでは、既存の findByNameAndPrice メソッドの代わりに、JPQL で新たに定義した findByNameContainingAndPrice メソッドを呼び出すように修正します。

動作確認

http://localhost:8080/product/insert

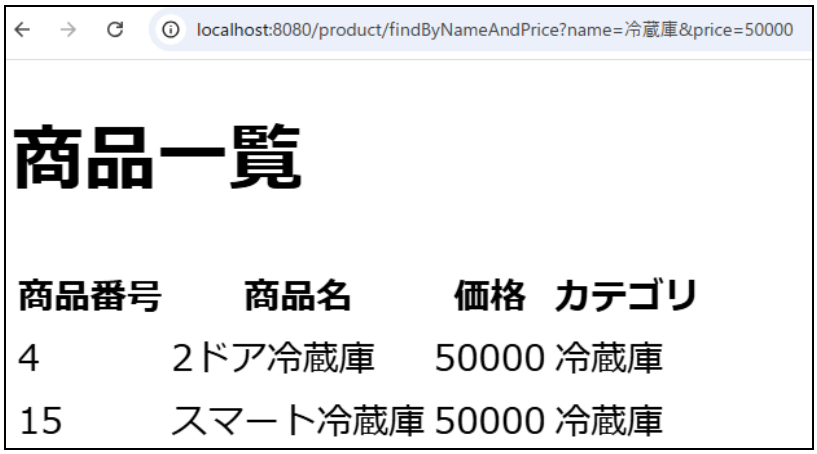
① スマート冷蔵庫と、50000、冷蔵庫を入力し、新たな商品を登録します。



② 以下の URL から、検索画面に遷移します。

http://localhost:8080/product/

③ 商品名+価格の検索欄に、「 冷蔵庫」と、「 50000」と検索します。



JPQL で定義した通り、「 冷蔵庫」というキーワード検索と、価格の AND 検索が行われていると正常です。

【参考】6.4.1 の例題を JPQL で作成する例

…/repository/ProductRepository.java

```
//【 参考】複雑な仕様の例題のサンプル①
        @Query("SELECT p FROM Product p LEFT JOIN p.category c " +
                        "WHERE p.name LIKE %:keyword% " +
                        "OR c.name LIKE %:keyword% " +
                        "OR p.description LIKE %:keyword% " +
                        "OR p.brand LIKE %:keyword% " +
                        "OR p.sku LIKE %:keyword%")
        List<Product> searchProductsByKeyword(String keyword);

//【 参考】複雑な仕様の例題のサンプル②
        @Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword% " +
            "OR p.category.name LIKE %:keyword% " +
            "OR p.description LIKE %:keyword% " +
            "OR p.brand LIKE %:keyword% " +
            "OR p.sku LIKE %:keyword%")
        List<Product> searchProductsByKeyword(String keyword);
```

サンプル①は、より純粋な SQL 文に近い書き方で、**JOIN を利用した結合を明示的に記述**しています。

サンプル②は、**JOIN を利用せずに、p.category.name という書き方でエンティティの外部参照関係の定義を利用**する書き方です。

どちらも同じ結果になりますが、②の方がオブジェクト指向ですので、推奨します。

【参考】集計関数と GROUP BY の場合

Product と Category を対象に、JPQL を使った 集計関数(SUM, COUNT)と GROUP BY のリポジトリを作成する方法も参考として紹介します。

■ 実装するリポジトリメソッド

1. カテゴリごとの商品の合計価格を取得する (SUM(price))
2. カテゴリごとの商品数を取得する (COUNT(*))
3. 指定した価格以上の商品があるカテゴリ一覧を取得する (HAVING を使用)

【作業手順①】 DTO(集計結果を受け取るためのクラス)の作成

JPQL では DTO(データ転送オブジェクト) を作成して、結果をオブジェクトとして受け取るのが一般的です。

■カテゴリごとの合計価格を受け取る DTO

```
public class CategorySalesDTO {
    private String categoryName;
    private Long totalSales;

    public CategorySalesDTO(String categoryName, Long totalSales) {
        this.categoryName = categoryName;
        this.totalSales = totalSales;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public Long getTotalSales() {
        return totalSales;
    }
}
```

■カテゴリごとの商品数を受け取る DTO

```
public class CategoryProductCountDTO {
    private String categoryName;
    private Long productCount;

    public CategoryProductCountDTO(String categoryName, Long
productCount) {
        this.categoryName = categoryName;
        this.productCount = productCount;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public Long getProductCount() {
        return productCount;
    }
}
```

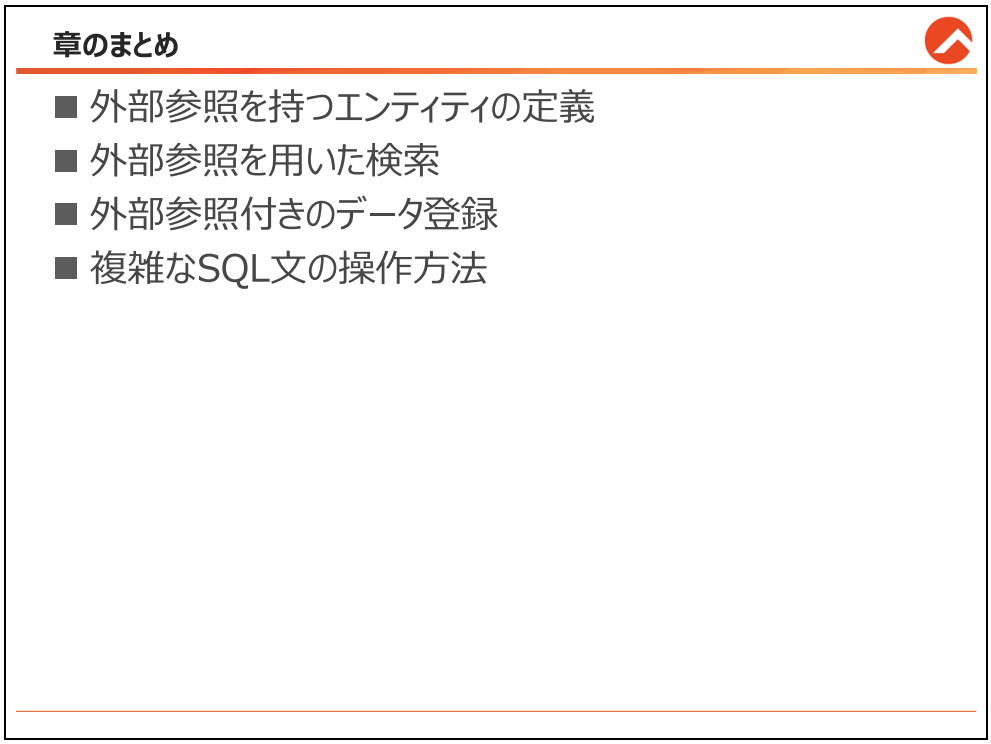
【 作業手順②】リポジトリの作成

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Integer> {

    /* 1. カテゴリごとの商品の合計価格を取得 */
    @Query("SELECT new jp.co.example.dto.CategorySalesDTO(p.category.name,
SUM(p.price)) FROM Product p GROUP BY p.category.name")
    List<CategorySalesDTO> findTotalSalesByCategory();

    /*  2. カテゴリごとの商品数を取得 */
    @Query("SELECT new
jp.co.example.dto.CategoryProductCountDTO(p.category.name, COUNT(p)) " +
           "FROM Product p GROUP BY p.category.name")
    List<CategoryProductCountDTO> findProductCountByCategory();

    /*  3. 指定した価格以上の商品があるカテゴリを取得( HAVING を使用)  */
    @Query("SELECT new jp.co.example.dto.CategorySalesDTO(p.category.name,
SUM(p.price)) " +
     "FROM Product p GROUP BY p.category.name HAVING SUM(p.price) >= :minSales")
    List<CategorySalesDTO> findCategoriesByMinSales(@Param("minSales") Long
minSales);
}
```

■まとめ

メソッド	目的	JPQL クエリ
findTotalSalesByCategory()	カテゴリごとの売上合計を取得	SUM(price) GROUP BY category.name
findProductCountByCategory()	カテゴリごとの商品数を取得	COUNT(*) GROUP BY category.name
findCategoriesByMinSales(Long minSales)	価格の合計が指定値以上のカテゴリを取得	HAVING SUM(price) >= minSales

章のまとめ



この章では、結合のための外部参照や JPQL について学習しました。

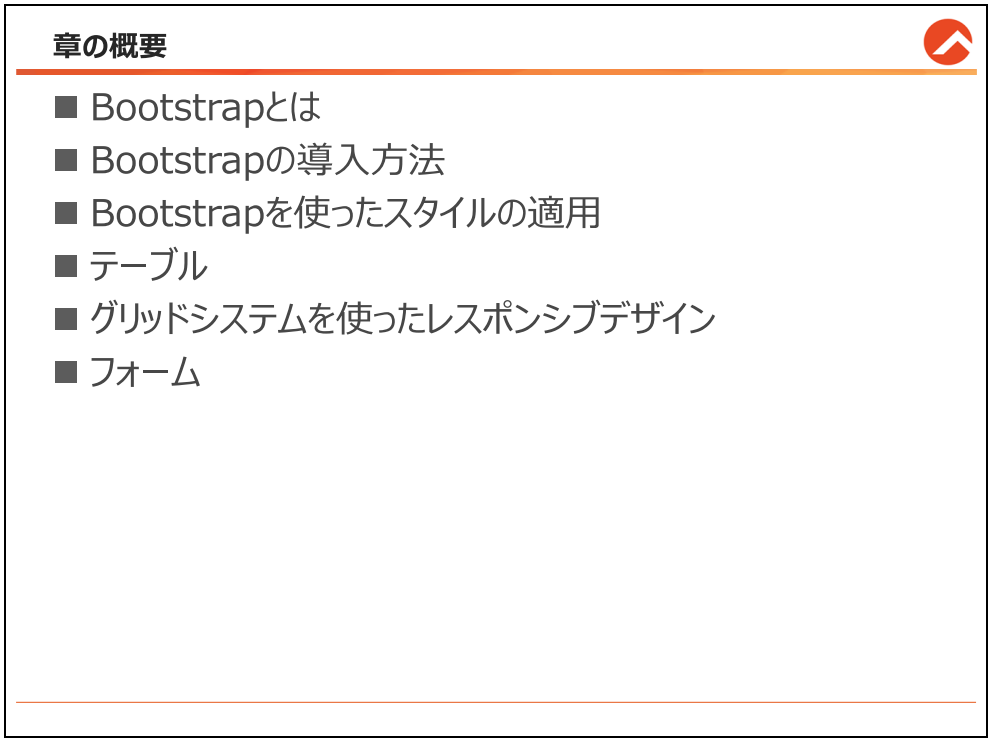
7

第7章 Bootstrap

章の目標

- Bootstrap の特徴と導入方法を習得する
- Bootstrap でよく使うスタイルについて実装できる
- Bootstrap でテーブルのカスタマイズができる
- グリッドシステムとレスポンシブデザインについて理解できる
- Bootstrap でフォームを実装できる

章の概要

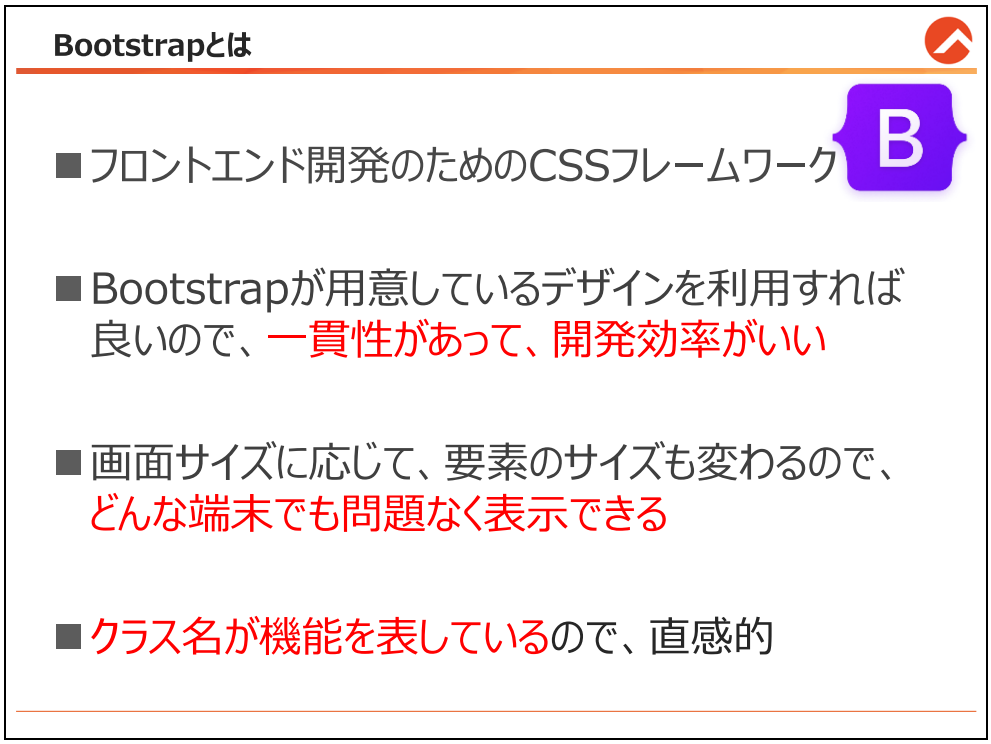


7 章では、Bootstrap の基本的な概要から導入方法、具体的な使い方までを学びます。

- **Bootstrap の概要**
  Bootstrap とは、統一されたデザインとレスポンシブな UI を簡単に作成するためのフロントエンドフレームワークです。
- **導入方法**
  CDN を利用した簡単な導入方法と、ローカルから読み込む方法を説明します。
- **具体的な使い方**
  グリッドシステム、ボタン、フォーム、テーブルなどの基本コンポーネントを用いた実践的な例を学びます。

この章を通じて、Bootstrap を効率よく活用するスキルを身に付けましょう。

7.1. Bootstrap とは



Bootstrap とは

Bootstrap は、Web サイトや Web アプリケーションのフロントエンド開発を簡単にするためのオープンソースのフレームワークです。もともと(旧)Twitter 社の社内プロジェクトとして始まり、現在は多くの開発者に利用されています。

特徴:

- モバイルファースト: 最初からスマートフォンでの表示を意識して設計されています。
- 統一されたデザイン: 一貫性のある UI を素早く構築可能です。
- レスポンシブデザイン: さまざまな画面サイズに適応します。

本書では、(2024 年 7 月基準)最新バージョンである 5.3 バージョンを利用します。

システム開発における外見の課題

システム開発では、機能の実装だけでなく、ユーザが使いやすい見た目やデザインも重要です。しかし、既存の方法で外見を整えるのは、次のような課題があります。

既存の方法: 手動で CSS 記述

たとえば、以下のように「青いテキスト」を表示したい場合、CSS を手動で記述する必要があります。

```
<!-- CSS を直接記述 -->
<style>
    .blue-text {
        color: blue;
    }
</style>
<p class="blue-text">青いテキスト</p>
```

もしくは、インラインスタイルを使う方法もあります。

```
<!-- インラインスタイル -->
<p style="color: blue;">青いテキスト</p>
```

この方法の課題

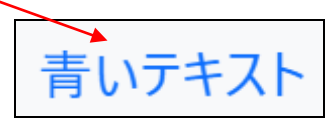
1. **時間がかかる:**
   テキストの色やサイズを変更するたびに CSS を記述する必要があります。
2. **保守性が低い:**
   クラス名やインラインスタイルが増えると、どこで定義されているか追跡するのが大変です。
3. **知識が必要:**
   CSS の構文を理解していないと適切な記述ができません。
4. **マルチデバイス対応が大変:**
   スマホやタブレット、PC で異なるレイアウトを実装する必要があり、工数が増えます。

Bootstrap での解決

Bootstrap を使えば、CSS を記述する手間が省けます。
HTML の**要素に特定のクラスを付与するだけ**で、スタイルが適用されます。

＜同じ「青いテキスト」を Bootstrap で実装する場合＞

```
<p class="text-primary">青いテキスト</p>
```



他の例（ボタンのデザイン）

Bootstrap を使えば、以下のようなデザインも簡単に実装できます。

従来の方法:

```
<style>
    .custom-button {
        background-color: #007bff;
        color: white;
        padding: 10px 20px;
        border: none;
        border-radius: 5px;
    }
</style>
<button class="custom-button">送信</button>
```

Bootstrap を使う方法:

```
<button class="btn btn-primary">送信</button>
```

Bootstrap を使うメリット

1. **短時間で実装可能:**

   クラスを付与するだけで、すぐにデザインが適用されます。

2. **一貫性のあるデザイン:**

   Bootstrap には統一されたスタイルが用意されているため、プロジェクト全体でデザインが統一されます。

3. **CSS の記述量の減少:**

   CSS ファイルを記述する頻度が激減し、誰でも簡単にデザインを適用できます。

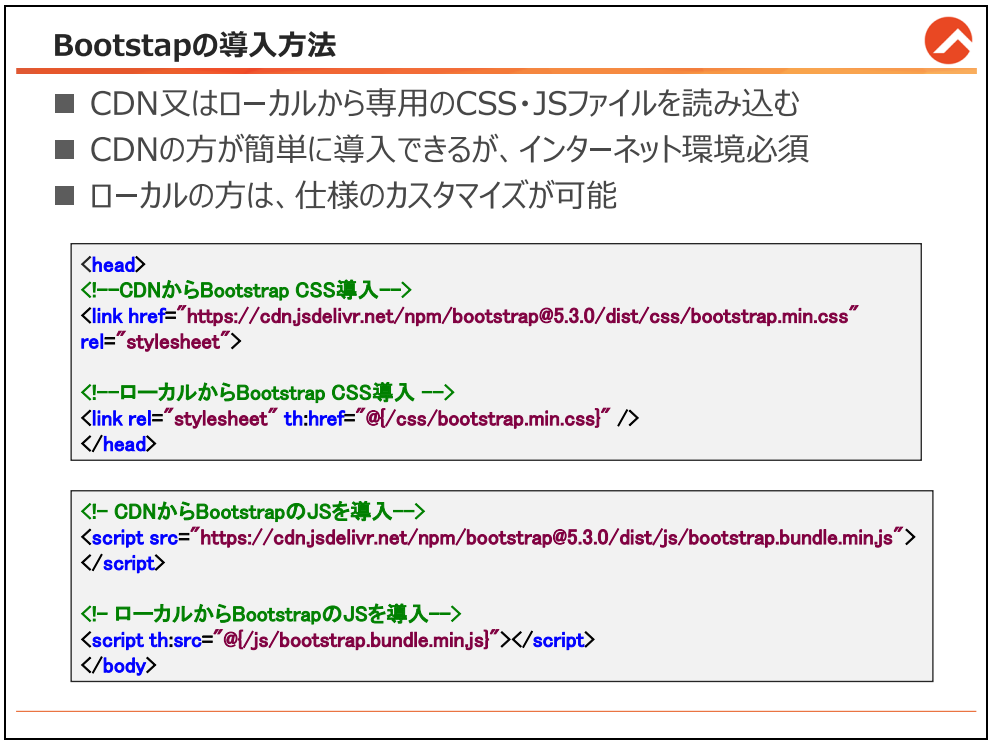
4. **直感的に使える:**

   クラス名で機能を表しているため、初心者でも簡単に理解できます。

5. **レスポンシブデザインが簡単**

   スマホやタブレット、PC、どの端末でも最適な表示ができます。

7.2. Bootstrap の導入方法



Bootstrap をプロジェクトに導入する方法は、次の 2 つがあります：

1. CDN(Content Delivery Network)を利用する方法
2. ローカル(Spring プロジェクト内)に導入する方法

※バージョンによって、詳細が変わる可能性がありますので、以下の公式ホームページのドキュメントを参照してください。

https://getbootstrap.jp/

CDN を利用した導入

CDN(Content Delivery Network)を利用すると、簡単に Bootstrap を導入できます。
CDN とは、世界中に配置されたサーバーのネットワークを利用して、コンテンツを配信する仕組みです。Bootstrap の公式 CDN を使うことで、ローカルにファイルをダウンロードする必要がなく、すぐに使い始めることができます。手順は下記となります。

① <head>要素内に、<meta name="viewport">を記載

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

モバイルデバイスでの適切なレスポンシブ動作のために記載する必要があります。

② Bootstrap の css ファイルの読み込む

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

<head>内に上記の URL を記載します。

③ Bootstrap のjsファイルを読み込む

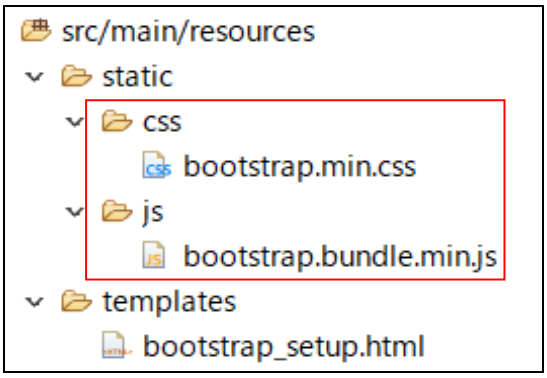
```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
```

<body>の末尾に上記の URL を記載します。

Bootstrap を Web サイト内で利用したいなら、上記のコードは必須です。
URL は、公式ホームページからお勧めの URL を参照することができます。

ローカル(Spring プロジェクト内) からの導入

5 章で学習した通り、Thymeleaf 上で CSS・JS ファイルを読み込むことができます。
事前に Bootstrap の CSS・JS ファイルをダウンロードし、「static」フォルダ内に格納しておくと、格納場所を指定して、読み込むことができます。



① 以下のコードを<head>に記入

```
<link rel="stylesheet" th:href="@{/css/bootstrap.min.css}" />
```

② 以下のコードを<body>の末尾に記入

```
<script th:src="@{/js/bootstrap.bundle.min.js}"></script>
```

下記は、Bootstrap を導入したシンプルな HTML の例です。

…/controller/ProductController.java

```
//省略

@GetMapping("/7-2-1")
	public String showBootStrapSetup() {
		return "bootstrap_setup";
	}
```

html ファイルに遷移させるために、遷移用メソッドを定義します。

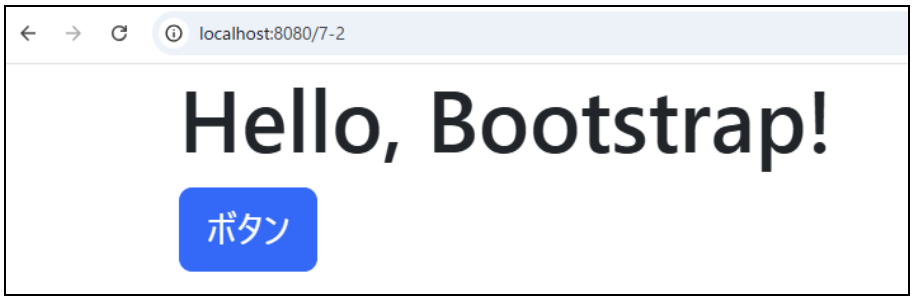
…/resources/templates/bootstrap_setup.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
<meta charset="UTF-8" />
<!-- ①以下のコードを記入することで、モバイル端末での表示を最適化できます。-->
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Bootstrap Example</title>

<!-- ② Bootstrap CSS をローカルから読み込む -->
<link rel="stylesheet" th:href="@{/css/bootstrap.min.css}" />

<!-- ② Bootstrap CSS を CDN から読み込む -->
<link href=
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>
<!-- コンテンツ -->
<div class="container">
    <h1>Hello, Bootstrap!</h1>
    <button class="btn btn-primary">ボタン</button>
</div>

<!-- ③ <body>の末尾に、Bootstrap の JS をローカルから読み込む-->
<script th:src="@{/js/bootstrap.bundle.min.js}"></script>

<!-- ③ <body>の末尾に、Bootstrap の JS を CDN から読み込む -->
<script src=
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>
```

サンプルコードには、両方の方法を記述していますが、実際は **CDN とローカルのうち、どちらかのみ記述**してください。**本研修では、ローカルからの読み込みを採用**します。

動作確認

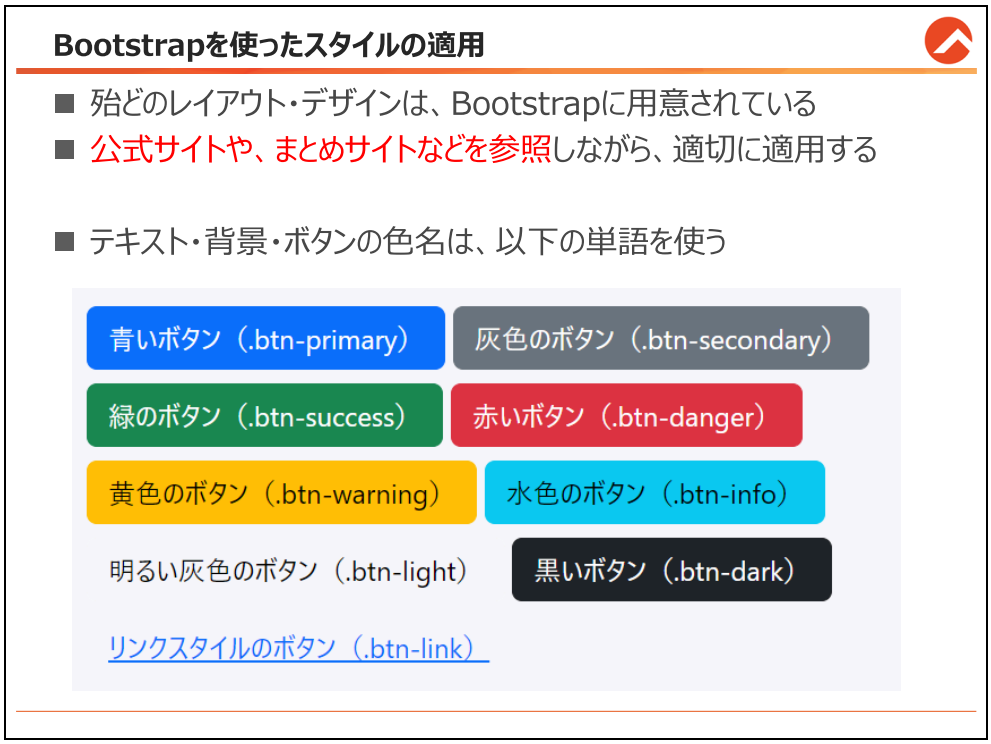
http://localhost:8080/7-2



青いボタンが表示されたら、問題なく導入ができているということです。

最新情報は公式サイト(https://getbootstrap.jp/) で確認してください。

7.3. Bootstrap を使ったスタイルの適用



Bootstrap は、Web サイトでよく使われるデザインや機能があらかじめ用意されています。特に、クラスを HTML に付与するだけで簡単にスタイルを適用できる点が大きな特徴です。

すべてのスタイルを一度に学ぶことは難しいですが、この節では特によく使うスタイルをいくつか紹介します。また、詳細な情報は以下の公式サイトや参考サイトで確認できます。

- Bootstrap 公式サイト(https://getbootstrap.jp/)
- とほほの Bootstrap 5 入門(https://www.tohoho-web.com/bootstrap5/index.html)

7.3.1. テキスト

Bootstrap では、テキストの配置を簡単に変更できます。

- **左揃え**: .text-start
- **中央揃え**: .text-center
- **右揃え**: .text-end

…/resources/templates/style_sample.html

```
<h2 class="text-center">配置( Alignment) </h2>

    <p class="text-start">左揃え( .text-start) </p>
    <p class="text-center">中央揃え( .text-center) </p>
    <p class="text-end">右揃え( .text-end) </p>
```

※CSS の text-align プロパティと同じく、ブロック内のインライン要素(テキスト、イメージ)の位置を調整できます。ブロック要素を動かすことはできません。

7.3.2. 色

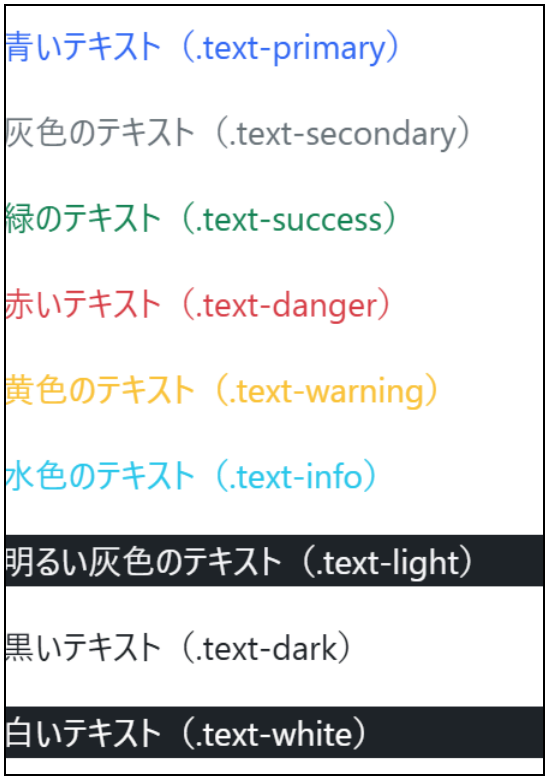
色のスタイルは、テキスト、背景、ボーダーなどに簡単に適用できます。まずは、色を変更する基本を学びましょう。

■ テキストの色

Bootstrap の色クラスは「色名」ではなく、**「その色がよく使われる用途」に基づいています**。

…/resources/templates/style_sample.html

```
<p class="text-primary">青いテキスト( .text-primary) </p>
    <p class="text-secondary">灰色のテキスト( .text-secondary) </p>
    <p class="text-success">緑のテキスト( .text-success) </p>
    <p class="text-danger">赤いテキスト( .text-danger) </p>
    <p class="text-warning">黄色のテキスト( .text-warning) </p>
    <p class="text-info">水色のテキスト( .text-info) </p>
    <p class="text-light bg-dark">明るい灰色のテキスト( .text-light)</p>
    <p class="text-dark">黒いテキスト( .text-dark) </p>
    <p class="text-white bg-dark">白いテキスト( .text-white) </p>
```



■ 背景色

背景色のクラスは、テキストの色クラスと同じ命名ルールです。**text-が bg-に変わるだけ**です。
薄い色は、「 -subtle」を追記することで表現できます。(Bootstrap 5.3 以降の機能)

…/resources/templates/style_sample.html

```
<div class="bg-primary text-white">青い背景( .bg-primary) </div>
<div class="bg-secondary text-white">灰色の背景( .bg-secondary) </div>
<div class="bg-success text-white">緑の背景( .bg-success) </div>
<div class="bg-danger text-white">赤い背景( .bg-danger) </div>
<div class="bg-warning text-dark">黄色の背景( .bg-warning) </div>
<div class="bg-info text-white">水色の背景( .bg-info) </div>
<div class="bg-light text-dark">明るい灰色の背景( .bg-light) </div>
<div class="bg-dark text-white">黒い背景( .bg-dark) </div>
<div class="bg-white text-dark">白い背景( .bg-white) </div>
<div class="bg-primary-subtle">薄い青い背景( .bg-primary-subtle) </div>
<div class="bg-secondary-subtle">薄い灰色( .bg-secondary-subtle) </div>
<div class="bg-success-subtle">薄い緑の背景( .bg-success-subtle) </div>
<div class="bg-danger-subtle">薄い赤い背景( .bg-danger-subtle) </div>
<div class="bg-warning-subtle">薄い黄色の背景 .bg-warning-subtle)</div>
<div class="bg-info-subtle">薄い水色の背景( .bg-info-subtle) </div>
<div class="bg-light-subtle">薄い明るい灰色( .bg-light-subtle) </div>
<div class="bg-dark-subtle">薄い黒い背景( .bg-dark-subtle) </div>
```

青い背景（.bg-primary）
灰色の背景（.bg-secondary）
緑の背景（.bg-success）
赤い背景（.bg-danger）
黄色の背景（.bg-warning）
水色の背景（.bg-info）
明るい灰色の背景（.bg-light）
黒い背景（.bg-dark）
白い背景（.bg-white）
薄い青い背景（.bg-primary-subtle）
薄い灰色の背景（.bg-secondary-subtle）
薄い緑の背景（.bg-success-subtle）
薄い赤い背景（.bg-danger-subtle）
薄い黄色の背景（.bg-warning-subtle）
薄い水色の背景（.bg-info-subtle）
薄い明るい灰色の背景（.bg-light-subtle）
薄い黒い背景（.bg-dark-subtle）

7.3.3. ボタン

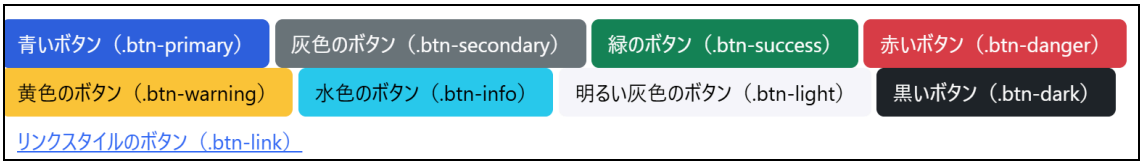
Bootstrap を使えば、HTML のボタン要素にクラスを追加するだけで、簡単にデザインを適用できます。ここでは、基本的なボタンの色やサイズ、状態について解説します。

■ ボタンの色

Bootstrap では、**btn クラスを基礎として、さらに色を指定するクラスを追加**することで、簡単にさまざまな色のボタンを作成できます。

…/resources/templates/style_sample.html

```
<button class="btn btn-primary">青いボタン( .btn-primary) </button>
<button class="btn btn-secondary">灰色のボタン( .btn-secondary)
</button>
<button class="btn btn-success">緑のボタン( .btn-success) </button>
<button class="btn btn-danger">赤いボタン( .btn-danger) </button>
<button class="btn btn-warning">黄色のボタン( .btn-warning) </button>
<button class="btn btn-info">水色のボタン( .btn-info) </button>
<button class="btn btn-light">明るい灰色のボタン( .btn-light)
</button>
<button class="btn btn-dark">黒いボタン( .btn-dark) </button>
<button class="btn btn-link">リンクスタイルのボタン( .btn-link)
</button>
```



※btn-link は見た目がリンクのようになります。
※<button>だけでなく、<input>や<a>タグにも適用可能です。

■ ボタンのサイズ

btn クラスに加えて、サイズを指定するクラスを追加することで、大きなボタンや小さなボタンを作成できます。

…/resources/templates/style_sample.html

```
<button class="btn btn-dark btn-lg">大きなボタン( .btn-lg) </button>
<button class="btn btn-dark">普通サイズボタン</button>
<button class="btn btn-dark btn-sm">小さなボタン( .btn-sm) </button>
```

■ 無効化されたボタン

Bootstrap では、ボタンを無効化(クリックできない状態に)するためのクラスが用意されています。条件によって、無効化・有効化を切り替えする実装の時使われます。

…/resources/templates/style_sample.html

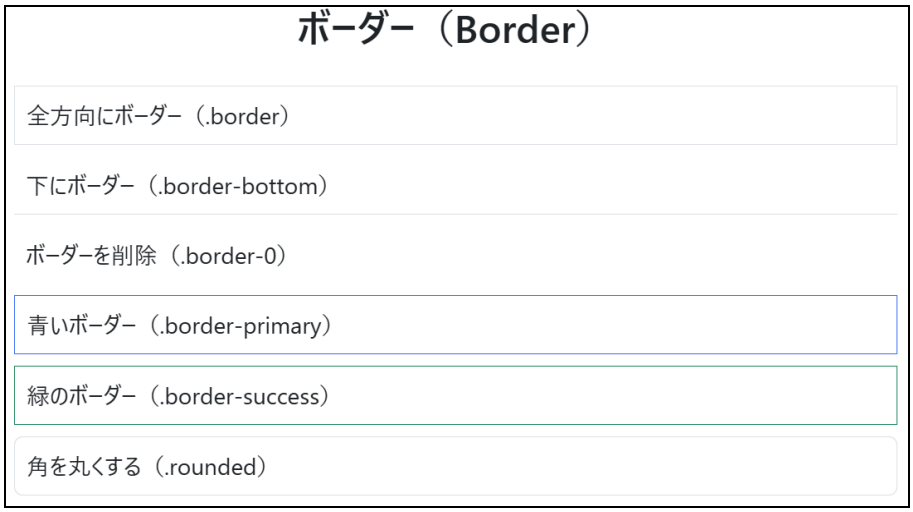
```
<button class="btn btn-secondary disabled">無効化されたボタン
</button>
```

※マウスで当ててみると、クリックアニメーションが発生しないことがわかります。

7.3.4. ボーダー(Border)

ボーダーは、要素に枠線を追加するためのクラスです。Bootstrap では、枠線の追加、削除、色の指定、角の丸みなどを簡単に設定できます。

```
<h2 class="text-center mb-4">ボーダー( Border) </h2>
<div class="border p-2 mb-2">全方向にボーダー( .border) </div>
<div class="border-bottom p-2 mb-2">下にボーダー( .border-bottom) </div>
<div class="border-0 p-2 mb-2">ボーダーを削除( .border-0) </div>
<div class="border border-primary p-2 mb-2">青いボーダー
( .border-primary) </div>
<div class="border border-success p-2 mb-2">緑のボーダー
( .border-success) </div>
<div class="border rounded p-2 mb-2">角を丸くする( .rounded) </div>
```



※p や mb のクラスは、次のページで説明します

※border-0 は条件によって、枠線の有無を決める実装の時使われます。

7.3.5. 余白

■ マージン(Margin)

マージンは、要素の**外側の余白**を調整するために使用します。

クラスの命名ルール

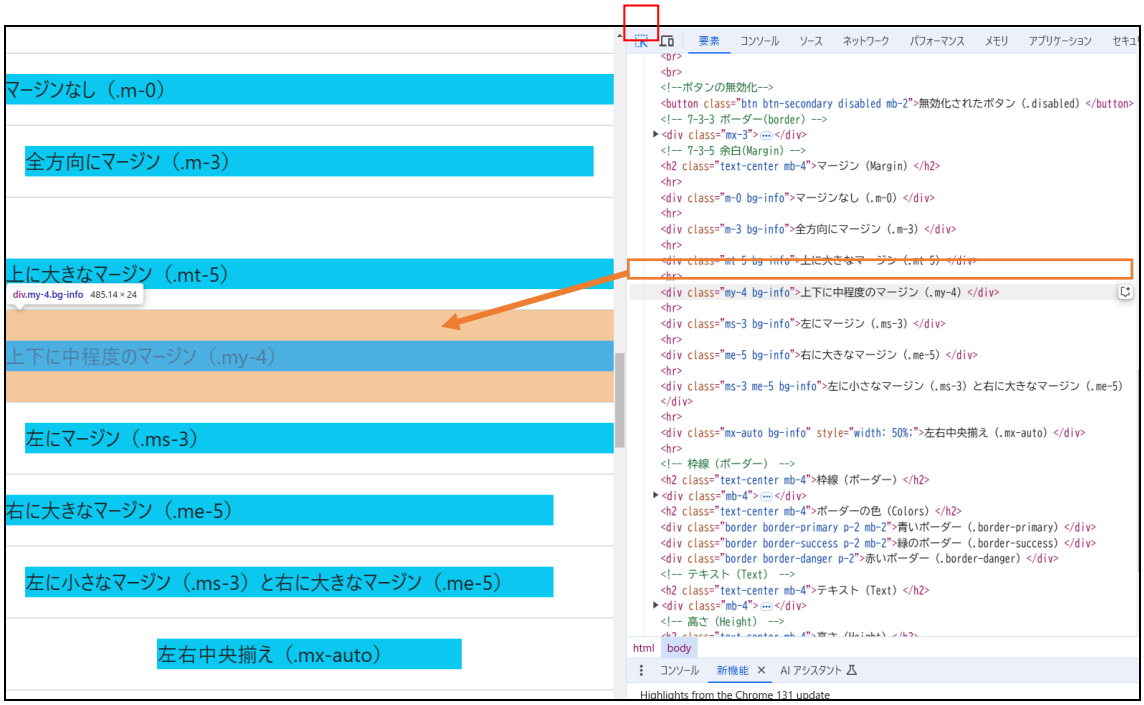
- m-0 ～ m-5: 全方向のマージンを設定
- mt-(上)、mb-(下)、ms-(左/スタート)、me-(右/エンド)で特定の方向に設定
- mx-(左右)、my-(上下)でまとめて設定

```
<hr>
<div class="m-0 bg-info">マージンなし( .m-0) </div>
<hr>
<div class="m-3 bg-info">全方向にマージン( .m-3) </div>
<hr>
<div class="mt-5 bg-info">上に大きなマージン( .mt-5) </div>
<hr>
<div class="my-4 bg-info">上下に中程度のマージン( .my-4) </div>
<hr>
<div class="ms-3 bg-info">左にマージン( .ms-3) </div>
<hr>
<div class="me-5 bg-info">右に大きなマージン( .me-5) </div>
<hr>
<div class="ms-3 me-5 bg-info">左に小さなマージン( .ms-3) と右に大きな
マージン( .me-5) </div>
<hr>
<div class="mx-auto bg-info" style="width: 50%;">左右中央揃え( .mx-auto)
</div>
<hr>
```

※<hr>は、水平線を引く要素です。余白の範囲の参考のため記述されています。
(デフォルトで少しの上下の余白が生成されます)

要素の余白を簡単に調べられる方法についてご紹介します。

①「 F12」で「 開発者ツール」を開きます
② 以下の赤枠のボタンをクリックします。(又は Ctrl+Shift+C)
③「 要素」タブの調べたい HTML コードにマウスを当てます。
※ マウスを<div class = “my-4 bg-info”>上下に中程度のマージン(.my-4) に当てたので、余白の範囲が表示されました。

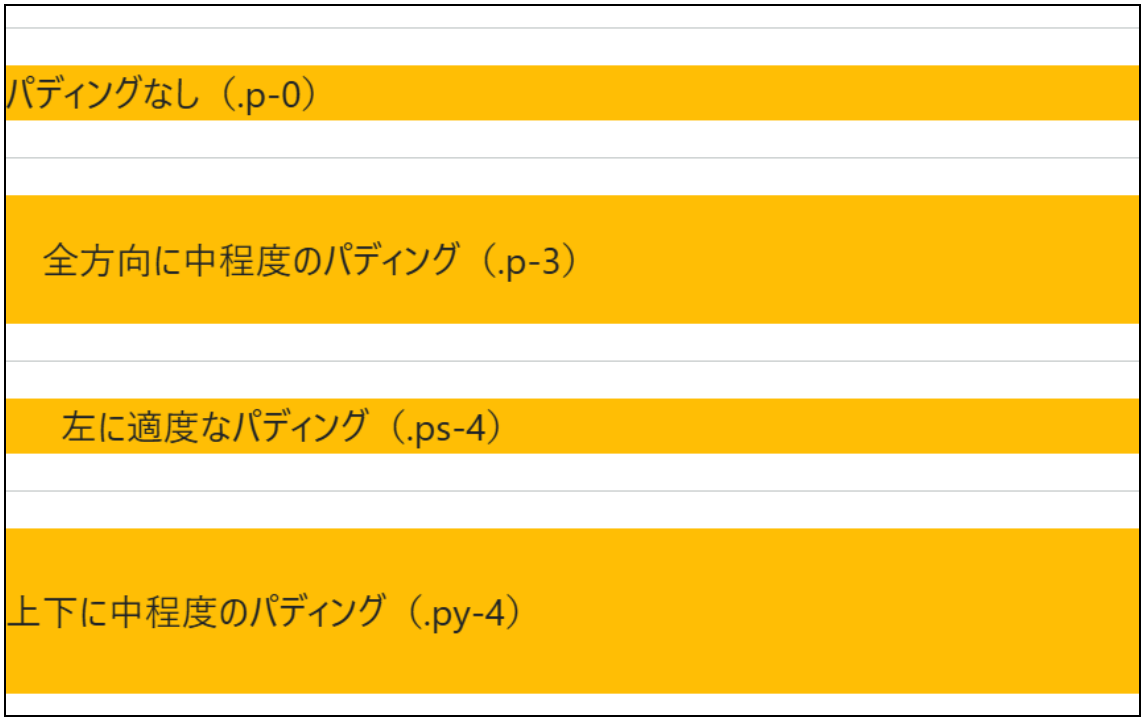


■ パディング(Padding)

パディングは、**要素の内側の余白**を調整するために使います。
Bootstrap では、p-を使ったクラス名を指定するだけで簡単に設定できます。

- p-0 ～ p-5　　全方向のパディングを設定
- pt-0 ～ pt-5　　上のパディングを設定
- pb-0 ～ pb-5　　下のパディングを設定
- ps-0 ～ ps-5　　左のパディングを設定(start)
- pe-0 ～ pe-5　　右のパディングを設定(end)
- px-0 ～ px-5　　左右のパディングを設定
- py-0 ～ py-5　　上下のパディングを設定

```
<hr>
<div class="bg-warning p-0">パディングなし( .p-0) </div>
<hr>
<div class="bg-warning p-3">全方向に中程度のパディング( .p-3) </div>
<hr>
<div class="bg-warning ps-4">左に適度なパディング( .ps-4) </div>
<hr>
<div class="bg-warning py-4">上下に中程度のパディング( .py-4) </div>
<hr>
```



7.3.6. ブロック要素の幅の調整(高さと横幅)

Web ページのデザインでは、ブロック要素の高さや横幅を適切に調整することが重要です。Bootstrap では、簡単にこれらを設定するクラスが用意されています。この節では、「高さ」と「横幅」の調整方法を説明します。

■ 高さ(Height)の調整

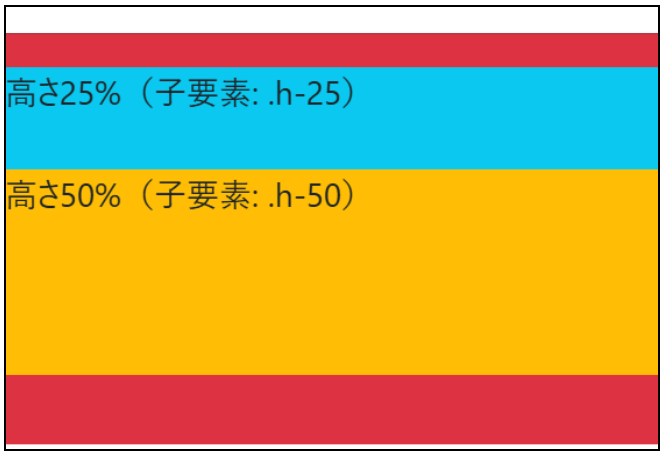
- h-25　親要素の高さの 25%を設定
- h-50　親要素の高さの 50%を設定
- h-75　親要素の高さの 75%を設定
- h-100　親要素の高さの 100%を設定
- h-auto　コンテンツに応じた自動調整

※h-35 のような微調整は別途カスタマイズが必要です。

ブロック要素の高さを設定する場合、**親要素の高さ**が重要です。子要素の高さは、親要素の高さに依存して計算されます。

```
<!-- 親要素 -->
<div class="bg-danger" style="height: 200px;">
        <hr>
        <!-- 子要素 -->
        <div class="bg-info h-25">高さ 25%( 子要素: .h-25) </div>
        <div class="bg-warning h-50">高さ 50%( 子要素: .h-50) </div>

</div>
```

高さ 200px 固定の親要素(赤いブロック)がある場合、
子要素に h-25 と h-50 と指定すると、200px の 25%と 50%の高さが適用されます。

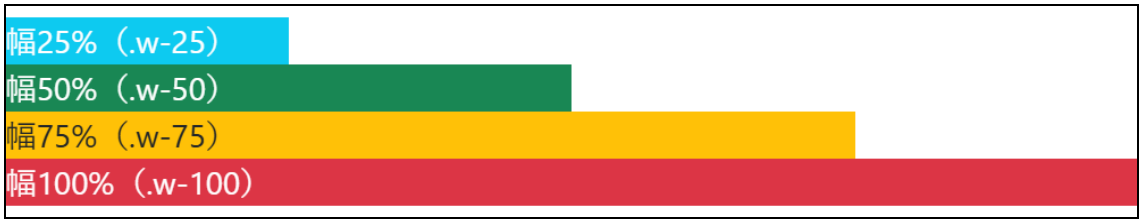


■ 横幅(Width)の調整

高さと同様に、ブロック要素の横幅も Bootstrap 上で調整できます。

- w-25　　親要素の幅の 25%
- w-50　　親要素の幅の 50%
- w-75　　親要素の幅の 75%
- w-100　　親要素の幅の 100%(全幅)
- w-auto　　コンテンツに応じた自動幅

```
<div class="bg-info text-white w-25">幅 25%( .w-25) </div>
<div class="bg-success text-white w-50">幅 50%( .w-50) </div>
<div class="bg-warning text-dark w-75">幅 75%( .w-75) </div>
<div class="bg-danger text-white w-100">幅 100%( .w-100) </div>
```



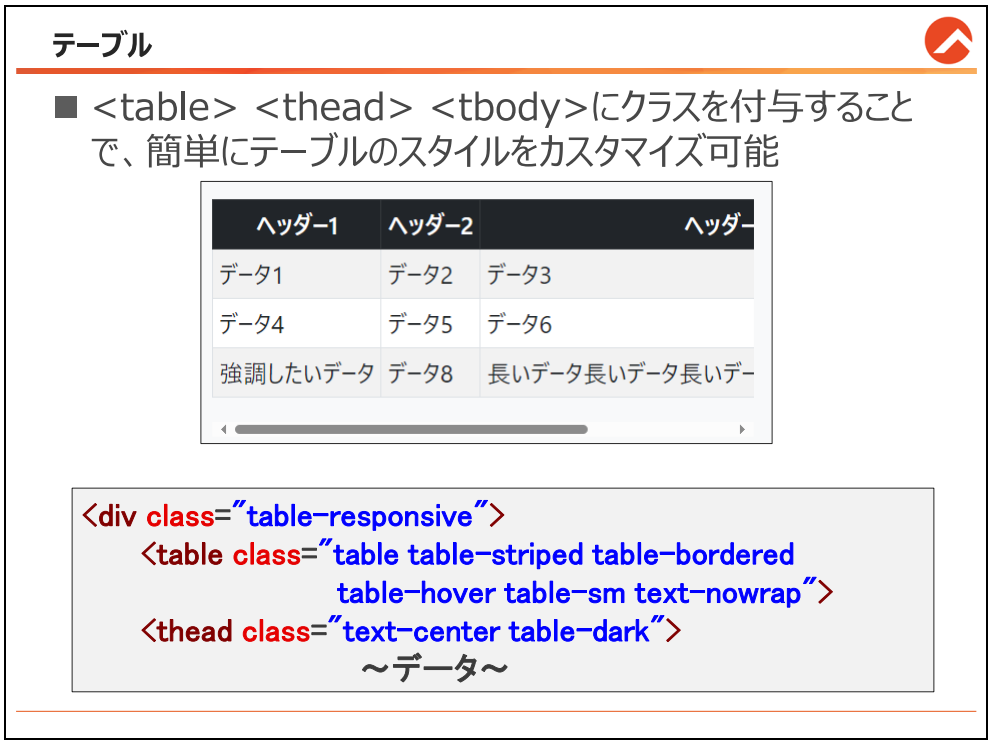
動作確認

http://localhost:8080/7-3

是非実行して、実際の動きを確認してください。

今回のサンプルは、あくまでよく使うクラスの一例ですので、紹介できていないクラスがたくさん残っています。他の節で、少しずつ新しいクラスを紹介していきますが、それでも全てを網羅することが難しいので、公式サイトや、まとめサイトで必要なクラスを調査して適用させるのが、Bootstrap を極める第一歩です。

7.4. テーブル

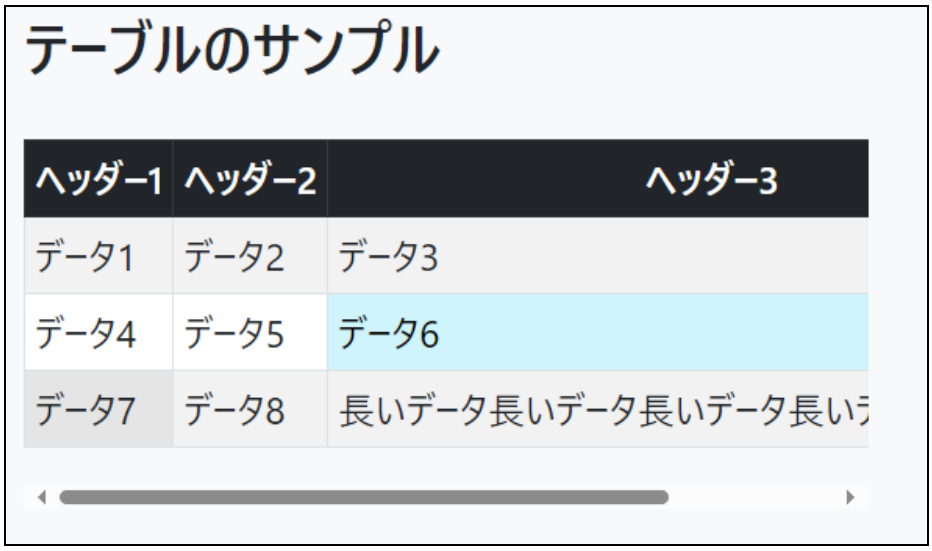


Bootstrap のテーブルクラス

Bootstrap を使用することで、テーブルのスタイルを簡単にカスタマイズすることができます。基本的なクラスは以下の通りです。

- **table**: 基本的なテーブルスタイルを適用します。
- **table-striped**: 各行を交互に色付けして、視認性を向上させます。
- **table-bordered**: テーブルに枠線を追加します。
- **table-hover**: 行にカーソルを合わせたときに強調表示します。
- **table-dark**: テーブル全体に暗いテーマを適用します。
- **table-sm**: コンパクトなテーブルスタイルを適用します。
- **table-responsive**: テーブルを狭い画面でもスクロール可能にします。table 要素を別途の div で囲う必要があります。
- **table-active**: アクティブな行やセルを強調するためのクラス。
- **table-primary(色名)**:状態に応じた行やセルの背景色を設定するためのクラス。
  text-nowrap: テキストの折り返しを防ぎ、一行で表示します。このクラスをテーブルのヘッダーやセルに適用することで、テキストが縦に表示されるのを防ぎます。

※その他に、テーブルの**ヘッダーとデータ行のスタイルを個別**で行うことができるように、通常は、<table>の中に**<thead>と<tbody>要素**を分けて実装します。



…/resources/templates/table_sample.html

```
<h2 class="mb-4">テーブルのサンプル</h2>

<div class="table-responsive">
    <table class="table table-striped table-bordered table-hover table-sm
                  text-nowrap">
        <thead class="text-center table-dark">
            <tr>
                <th>ヘッダー1</th>
                <th>ヘッダー2</th>
                <th>ヘッダー3</th>
            </tr>
        </thead>
        <tbody>
            <tr class="table-info">
                <td>データ 1</td>
                <td>データ 2</td>
                <td>データ 3</td>
            </tr>
            <tr >
                <td>データ 4</td>
                <td>データ 5</td>
                <td>データ 6</td>
            </tr>
```

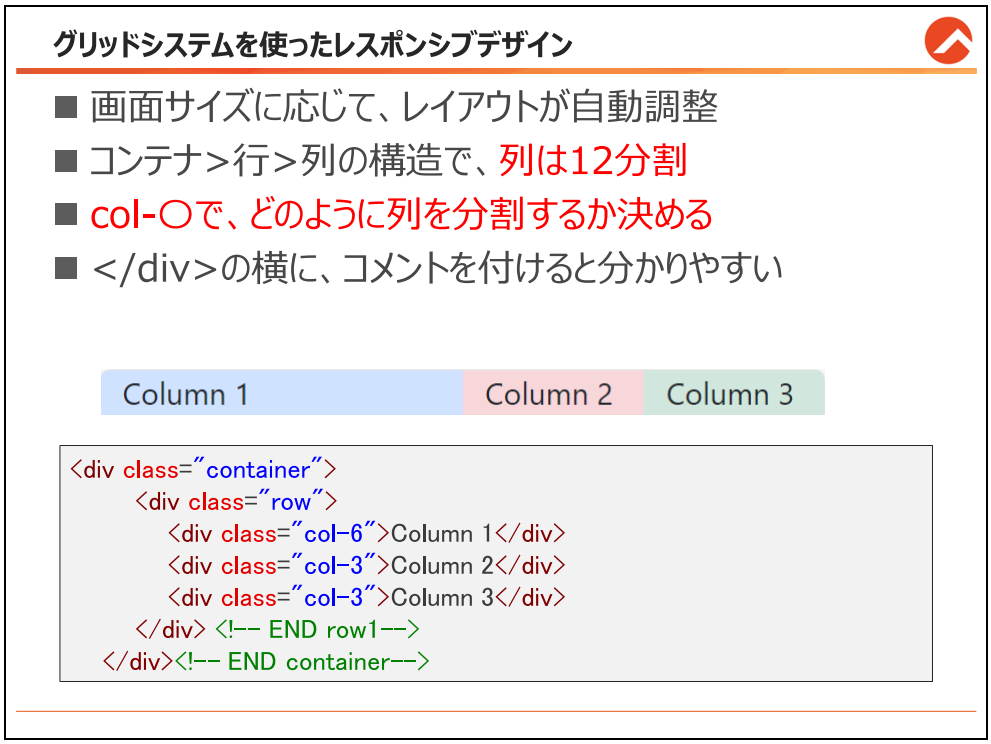
```
            <tr>
                <td class = "table-active">強調したいデータ</td>
                <td>データ 8</td>
                <td>長いデータ長いデータ長いデータ長いデータ長いデータ</td>
            </tr>
        </tbody> <!-- END table body -->
    </table> <!-- END table -->
</div> <!-- END responsive table wrapper -->
</div> <!-- END container -->
```

■動作確認

http://localhost:8080/7-4

class 属性を一つ消す度に、どう変わっていくのかを確認すると、よりわかりやすいです。

7.5. グリッドシステムを使ったレスポンシブデザイン



グリッドシステムの基本概念

Bootstrap のグリッドシステムを使うと、画面サイズに応じて自動的にレイアウトが調整される Web ページを簡単に作成できます。従来の CSS レイアウトで苦労していた、**ブロック要素の縦積みや横方向への配置が非常に簡単になります。**

Bootstrap の**グリッドシステムは、12 列**のシステムを採用しています。
1 行(row)が最大で 12 の列(column)に分割できるということです。このシステムを使って、柔軟でレスポンシブなレイアウトを作成できます。

グリッドシステムでは、以下の 3 つの基本要素を使います。

① コンテナ(Container): **ページの中央に配置**され、内容を適切に配置するための基本的なレイアウト要素です。全体の幅を制御します。

② 行(Row):カラム(列)を含む水平のレイアウト要素です。コンテナ要素の中に記述します。

③ カラム(Column):列のことを「 カラム」と言います。カラムは、コンテンツを配置するための垂直のレイアウト要素です。最大 12 カラムまで使用できます。

これらの要素は入れ子構造で、HTML 要素の class 属性を使って指定します。

入れ子構造は、<table><tr><td>の関係と似ています。

基本的なグリッドレイアウト

① コンテナ(Container)

** **は、ページの中央にコンテンツを配置し、左右に適切な余白を追加するための基本的なレイアウト要素です。container を使用すると、幅が固定されており、**デバイスの画面サイズに応じて最大幅が変わります。コンテンツが常に中央に配置されます**ので、読みやすさが向上します。

一方、** **は、画面全体に広がるコンテナを作成します。余白なしで、親要素の**幅いっぱい**にコンテンツを広げたい場合に使用します。例えば、全幅のヘッダーやフッターを作成する場合に便利です。

② 行(Row)

** **は、水平に配置される行です。row の中にカラム(列) が入ります。row を使用することで、カラムが適切に並び、縦方向のスペースを均等に分けることができます。

③ カラム(Column)

** **のように、カラム(列) は、コンテンツを配置するための垂直のレイアウト要素です。カラムは、グリッドシステムの基礎となり、1 行を最大 12 のカラムに分割することができます。以下に具体的な例を示します。

…/resources/templates/grid_sample.html

```
1:<div class="container">
2:    <div class="row">
3:        <div class="col-6 bg-primary-subtle">Column 1</div>
4:        <div class="col-3 bg-danger-subtle">Column 2</div>
5:        <div class="col-3 bg-success-subtle">Column 3</div>
6:    </div> <!-- END row1-->
7:</div><!-- END container-->
```

**・1 行目 **

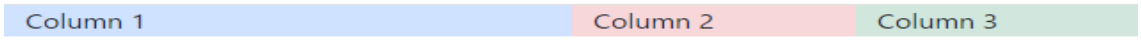
今回のサンプルでは、外側を container でコンテナを作っています。画面の中央にコンテンツが配置されます。もし、container-fluid にすると、横幅いっぱいに変わります。

**・2 行目 **

コンテナの中に、row を class 属性として持つ div 要素を入れます。後述のカラムを配置するためにも、必ず row が必要です。まるで table 要素に、<tr>がないと、<td>も書けないのと同じです。

・3 行目～5 行目 <div class="col-6～

1 つの row の中に、複数のカラムを配置できます。このサンプル通りだと、以下のように表示されます。



** : このカラムは、12 分の 6、つまり全幅の 50%を占めます。**

** : このカラムは、12 分の 3、つまり全幅の 25%を占めます。**

Bootstrap のグリッドは 12 列構成で、合計 12 未満なら空白、12 超なら折り返し、均等配分には番号なしの col を使います。

今回のサンプルの場合は、6 + 3 + 3 = 12 でした。

背景色はカラムの視覚的な区別を付けるために使用しています。

・6 行目～7 行目 <!--コメント-->

グリッドシステムでは、 要素を使うことが多いので、閉じタグだけでは、どの要素の閉じタグなのか、区別がつきません。そのため、**どの要素の閉じタグなのかを明確にするため、コメントを記載することが多いです。**特に、row が複数ある場合は「 END row1」や、「 END row2」とかでコメントを残すことで実際の範囲も読みやすくなるので、重要です。

【 参考】 知っておくと便利になるカラムのコツ

・12 未満なら、残りのスペースは自動的に分配されます。

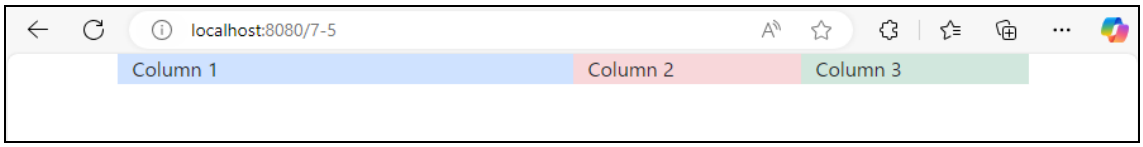
・col-auto と設定すると、コンテンツの長さによって、横幅が自動的に計算されます。

・カラムを画面の中央寄せしたい時は、row クラスに加えて、「 justify-content-center」クラスも適用します。

・g-1 (g-5 まである) を row クラスに加えると、カラム同士の間隔を調整できます。
微調整することが多いので、覚えておいてください。

■動作確認

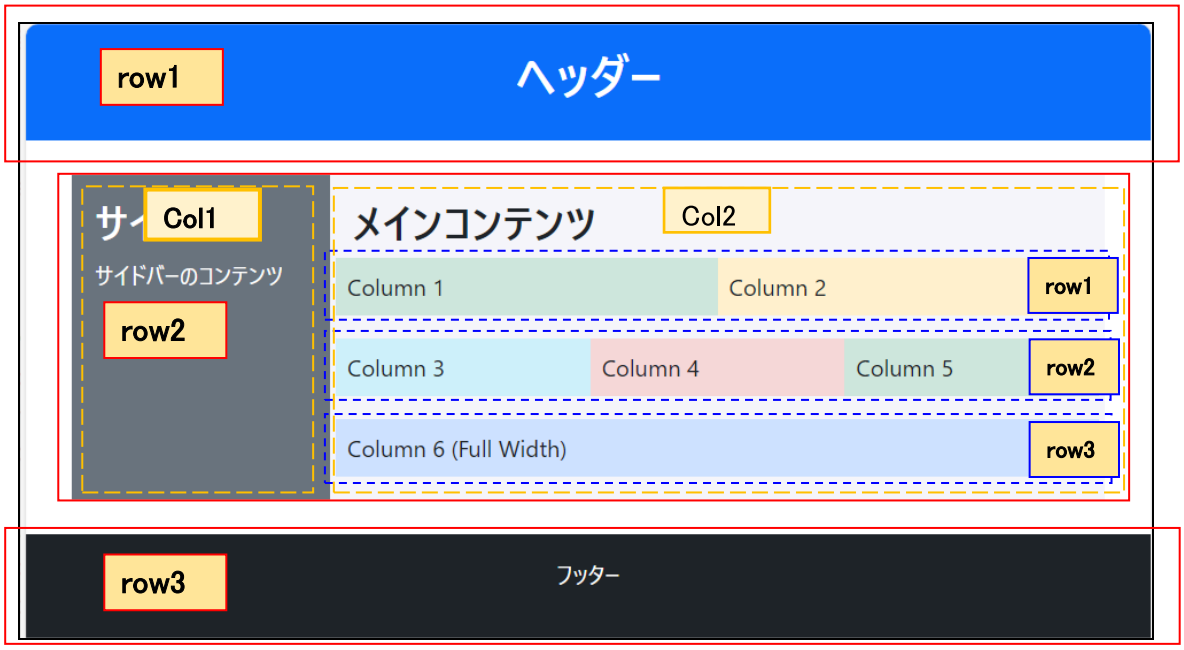
http://localhost:8080/7-5



※画面の横幅を調整しても、同じ割合で要素のサイズが調整されることを確認してください。

※より詳しい構文は、https://getbootstrap.jp/docs/5.3/layout/grid/ を参照

【参考】グリッドシステムの応用



今まで学習した内容を応用すると、上のような画面を作成できます。

ポイントは、3 つです。

1. ヘッダーとフッターは、全幅に広がるようにするためﾆ container-fluid」を使用し、サイドバーとメインコンテンツは中央に配置するためﾆ container」を使用しています。
2. 各要素にはマージンとパディングを指定して、内部と外部に余白を入れています。
3. サイドバーとメインコンテンツは列(Column)を 2 つで分けて 1 つの行(row)に配置し、メインコンテンツの中にはさらに 3 つの行(row)を入れ子にして配置しています。
4. col-md-3 と col-md-9 の md の意味について説明します。

   横幅が長い PC 画面の場合は 3:9 の割合が維持されますが、**タブレット端末以下の端末になった際は 3:9 のレイアウトではなく、より最適なレイアウトに調整される**という意味です。
   sm にすると、スマホ以下の端末で調整されます。

…/resources/templates/grid_sample2.html

```
<!-- ヘッダー -->
<div class="container-fluid bg-primary text-white text-center py-3">
    <h1>ヘッダー</h1>
</div> <!-- END container1-->

<!-- メインコンテンツとサイドバー -->
<div class="container my-4">
    <div class="row">
        <div class="col-md-3 bg-secondary text-white p-3 mb-3 mb-md-0">
            <h2>サイドバー</h2>
            <p>サイドバーのコンテンツ</p>
        </div> <!-- END sidebar col-->
        <div class="col-md-9 bg-light p-3">
            <h2>メインコンテンツ</h2>
            <div class="row">
                <div class="col-6 p-2 bg-success-subtle">Column 1</div>
                <div class="col-6 p-2 bg-warning-subtle">Column 2</div>
            </div> <!-- END main col row1-->
            <div class="row mt-3">
                <div class="col-4 p-2 bg-info-subtle">Column 3</div>
                <div class="col-4 p-2 bg-danger-subtle">Column 4</div>
                <div class="col-4 p-2 bg-success-subtle">Column 5</div>
            </div> <!-- END main col row2-->
            <div class="row mt-3">
                <div class="col-12 p-2 bg-primary-subtle">Column 6 (Full Width)</div>
            </div> <!-- END main col row3-->
        </div> <!-- END main col-->
    </div> <!-- END row1-->
</div> <!-- END container2-->

<!-- フッター -->
<div class="container-fluid bg-dark text-white text-center py-3">
    <p>フッター</p>
</div> <!-- END container3-->
```

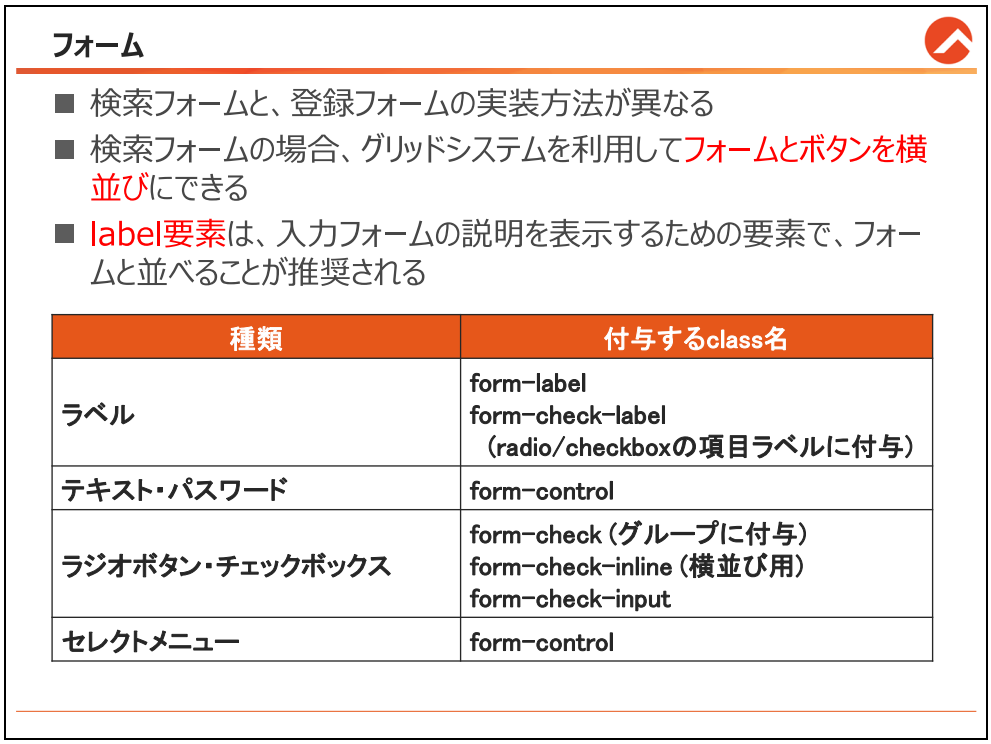
■動作確認

http://localhost:8080/7-5-2

※画面の横幅を調整しても、同じ割合で要素のサイズが調整されることを確認してください。

※より詳しい構文は、 https://getbootstrap.jp/docs/5.3/layout/columns/ を参照

7.6. フォーム



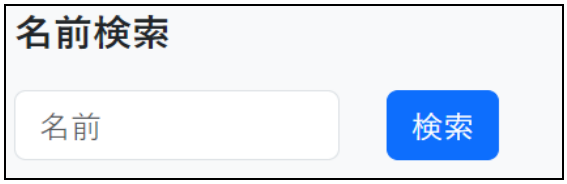
Bootstrap は、フォームのスタイルを簡単に整えるための豊富なクラスを提供しています。このクラスを使うことで、デザインが統一され、モダンな Web ページを手軽に作成できます。

フォームの種類について

フォームには、検索バーのようなシンプルなフォームと、登録ページのように複数の入力フィールドを連続して配置する垂直フォームの 2 種類のスタイルがあります。

検索フォーム

検索フォームは、ユーザが少量の情報を入力できるように設計されています。このフォームは、ページの一部として目立たせる必要があるため、シンプルかつ機能的なデザインが求められます。



…/resources/templates/form_sample.html

```
 1:<body class="bg-light p-4">
 2:<div class="container"> <!-- START container -->
 3:
 4:    <div class="row"> <!-- START row1 -->
 5:        <div class="col-md-3"> <!-- START sidebar -->
 6:            <div class="fs-5 fw-bold mb-3">名前検索</div>
 7:
 8:            <form class="row">
 9:                <div class="col-8">
10:                    <input type="text" class="form-control"
11:                        placeholder="名前">
12:                </div>
13:                <div class="col-4">
14:                    <button type="submit"
15:                        class="btn btn-primary">検索</button>
16:                </div>
17:            </form>
18:
19:        </div> <!-- END sidebar -->
20:    </div> <!-- END row1 -->
21:</div> <!-- END container -->
```

検索フォームは、前の節で学習した、グリッドシステムを利用します。

**・5 行目 **

フォームとは直接関係がないコードですが、横幅の 3 割のみ占めるように、限定をさせることで、左サイドバーのようなレイアウトを作ることができます。検索バーは、サイドバーに配置させることが多いので、予めカラムを生成しました。

・8 行目 <form class="row">

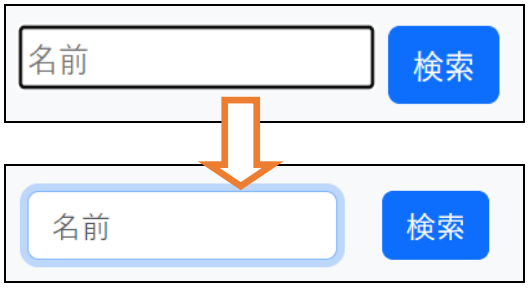
ではなく、<form>要素を row として生成すると、フォームを別途の行として扱えることができます。

**・9 行目・13 行目 **

入力フォームに 8、ボタンに 4 の割合でカラムを生成しています。

・10 行目 <input type="text" class="form-control"～

Bootstrap のフォーム関連クラスの中で、一番大事なクラスです。テキストフォームにこのクラスを付与することで、**Bootstrap 式のテキストフォームのデザインを適用**することができます。



form-control 無しでフォームをクリック

form-control 付与後、フォームをクリック

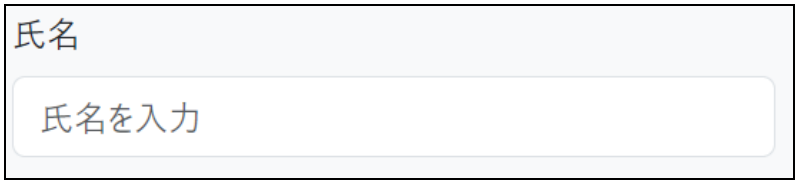
他にも、form-control を付与することで、できることが色々ありますので、公式サイトを参照してください。

https://getbootstrap.jp/docs/5.3/forms/form-control/

登録ページのフォーム

登録ページでは、複数の入力フィールドを連続して配置する垂直フォームを使用します。以下に、よく使われるパターンの入力フィールドを 1 つずつ紹介し、構文の説明とサンプルコードをご説明します。

1. テキストフォーム(input type = text、password)




```
<label for="inputName" class="form-label">氏名</label>
<input type="text" class="form-control" id="inputName"
  placeholder="氏名を入力">
```

フォームには、**「ラベル」**という要素があります。ラベルとは、入力フォームの説明を表示するための要素です。ラベルを使わず、普通にテキストを配置してもいいですが、通常は、ラベルを使って説明する実装が推奨されます。

for 属性を使って、**関連付けたい入力フィールドの id を指定**すると、**ラベルをクリックした時、関連する入力フィールドが自動的にフォーカス**されます。

- **form-label クラス:**
  label 要素に適用されるクラスで、ラベルを適切にスタイルし、フォーム全体の見た目を整えます。

- **form-control クラス:**
  入力フィールドに適用されるクラスで、テキスト入力フィールドが Bootstrap のスタイルで適用されます。

2. ラジオボタン、チェックボックス(input type = radio、checkbox)

```
<label class="form-label">性別</label><br>
<div class="form-check form-check-inline">
      <input type="radio" class="form-check-input" id="genderMale"
      name="gender" value="male">
      <label for="genderMale" class="form-check-label">男性</label>
</div>

<div class="form-check form-check-inline">
      <input type="radio" class="form-check-input" id="genderFemale"
      name="gender" value="female">
      <label for="genderFemale" class="form-check-label">女性</label>
</div>
```

ラジオボタンをBootstrap 実装する際は、2 つの種類のラベルが必要です。「 性別」というラベルと、「 男性・女性」という個別のラベルを実装する必要があります。

ラジオボタンと、チェックボックスは必要に応じて、以下のクラスを適用します。

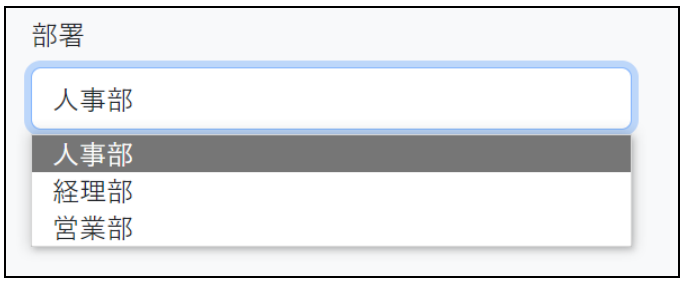
- form-check **クラス**:
  チェックボックスやラジオボタンのグループに適用されるクラスで、適切な間隔とスタイルを設定します。

- **form-check-inline クラス**:
  form-check クラスに加えて form-check-inline クラスを使用することで、ラジオボタンをインラインブロック要素にし、**横並びに配置**します。削除したら、改行されます。

- form-check-input **クラス**:
  チェックボックスやラジオボタンに共通のデザインを適用できます。

- form-check-label **クラス**:
  チェックボックスやラジオボタンのラベルに適用します。for 属性と連携するので、**レベルをクリックするだけでも、ラジオボタンがチェックされる**便利な UI を作ります。

3. セレクトメニュー(select-option):




```
<label for="department" class="form-label">部署</label>
<select class="form-select" id="department">
        <option value="hr">人事部</option>
        <option value="finance">経理部</option>
        <option value="sales">営業部</option>
</select>
```

セレクトメニューは、<select>要素に、「form-select」クラスを適用するくらいです。

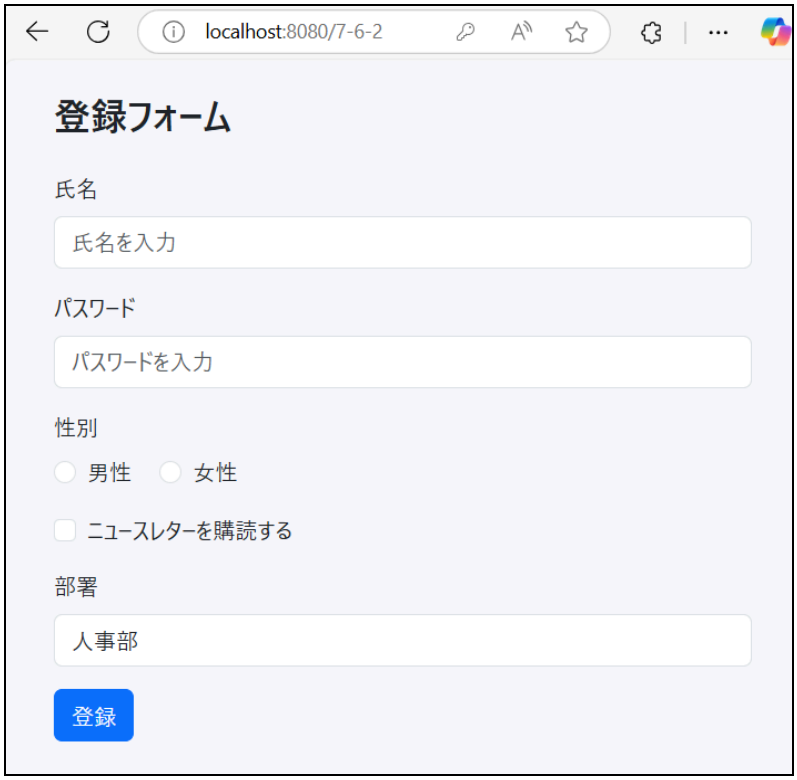
登録フォームのサンプルコード

…/resources/templates/form_sample2.html

```
<body class="bg-light p-4">

<div class="container"> <!-- START container -->
    <form class="row justify-content-center"> <!-- START form row -->
        <div class="col-md-4"> <!-- START form column -->
            <h2 class="mb-4">登録フォーム</h2>

            <div class="mb-3"> <!--テキストフォーム-->
                <label for="inputName" class="form-label">氏名</label>
                <input type="text" class="form-control" id="inputName"
                    placeholder="氏名を入力">
            </div>
            <div class="mb-3"> <!--パスワードフォーム-->
                <label for="inputPassword"
                    class="form-label">パスワード</label>
                <input type="password" class="form-control"
                    id="inputPassword" placeholder="パスワードを入力">
```

```
            </div>
            <div class="mb-3"> <!--ラジオボタン-->
                <label class="form-label">性別</label><br>
                <div class="form-check form-check-inline">
                    <input type="radio" class="form-check-input"
                        id="genderMale" name="gender" value="male">
                    <label for="genderMale"
                        class="form-check-label">男性</label>
                </div>
                <div class="form-check form-check-inline">
                    <input type="radio" class="form-check-input"
                        id="genderFemale" name="gender" value="female">
                    <label for="genderFemale"
                        class="form-check-label">女性</label>
                </div>
            </div>
            <div class="mb-3"> <!--チェックボックス-->
                <div class="form-check">
                    <input type="checkbox" class="form-check-input"
                        id="subscribeNewsletter">
                    <label for="subscribeNewsletter"
                       class="form-check-label">ニュースレターを購読する</label>
                </div>
            </div>
            <div class="mb-3"> <!--セレクトメニュー-->
                <label for="department" class="form-label">部署</label>
                <select class="form-select" id="department">
                    <option value="hr">人事部</option>
                    <option value="finance">経理部</option>
                    <option value="sales">営業部</option>
                </select>
            </div>
            <button type="submit" class="btn btn-primary">登録</button>
        </div> <!-- END form column -->
    </form> <!-- END form row -->
</div> <!-- END container -->
```

■動作確認

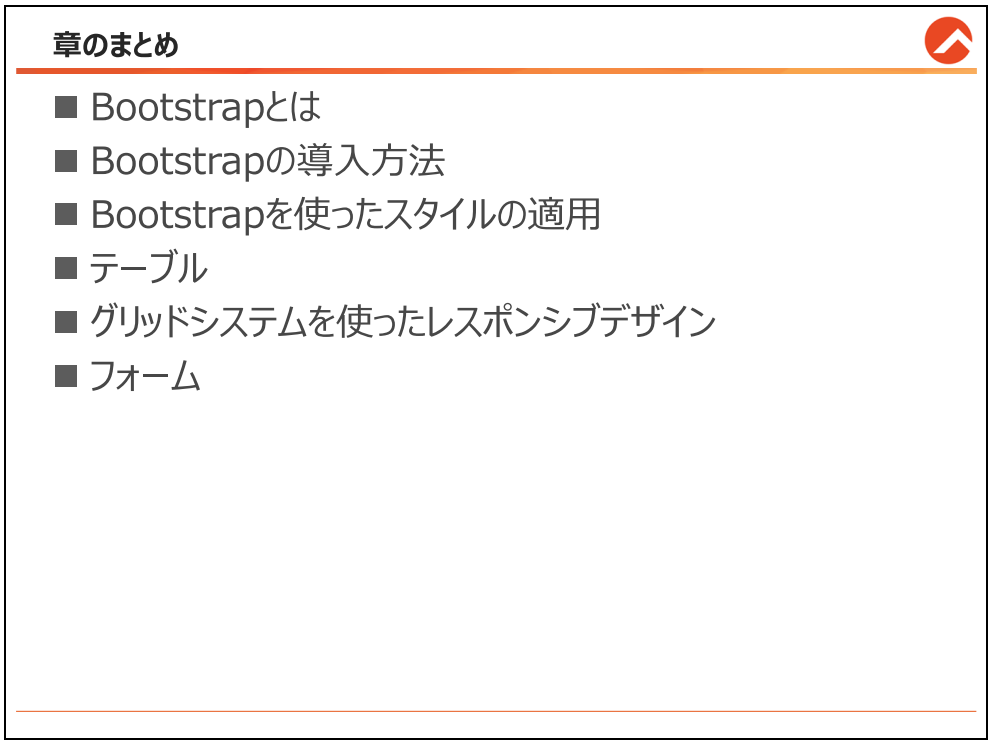


・画面のサイズを変えても、配置が維持されることを確認してください。

・各フォームが統一されたデザインになっていることを確認してください。

・ラジオボタンと、セレクトメニューが正常動作することを確認してください。

章のまとめ



この章では、Bootstrap についてご説明しました。本書で説明した内容は、全てを網羅しているわけではありませんので、今回学習した知識を基に、必要に応じて公式サイトなどを調べながら実装する習慣をつけてください。

A

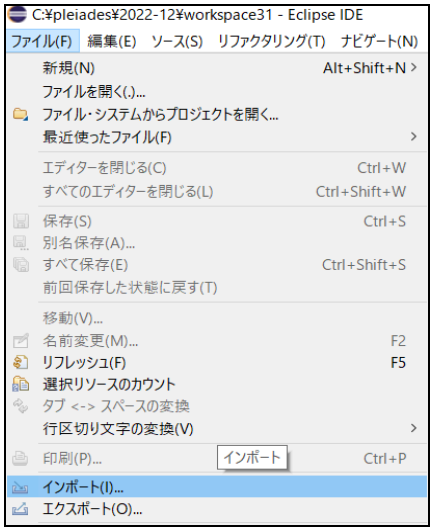
付録 A 環境構築手順

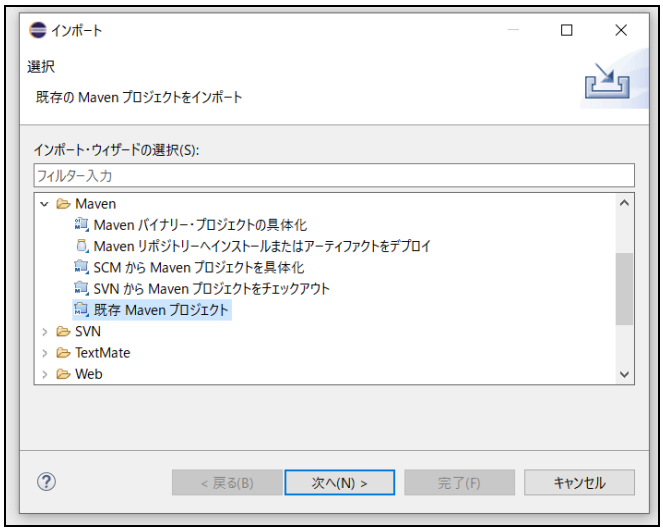
Import 手順

1. エクスポート済みの Spring プロジェクトの zip ファイルを用意します。

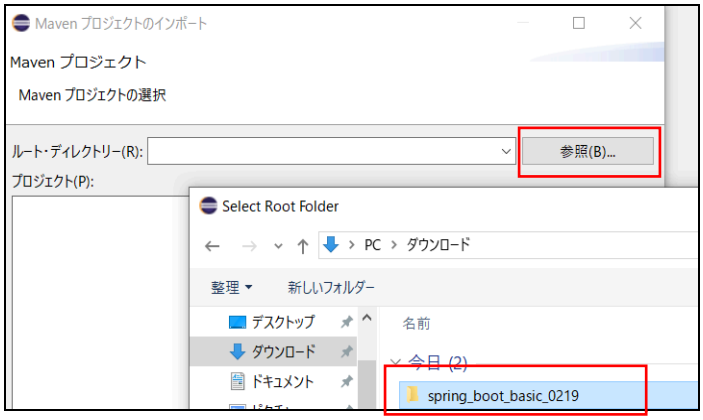
2. Zip ファイルの圧縮を解凍してください。

3. Eclipse の[ファイル]→[インポート]→[Maven]→[既存 Maven プロジェクト]をクリックします。

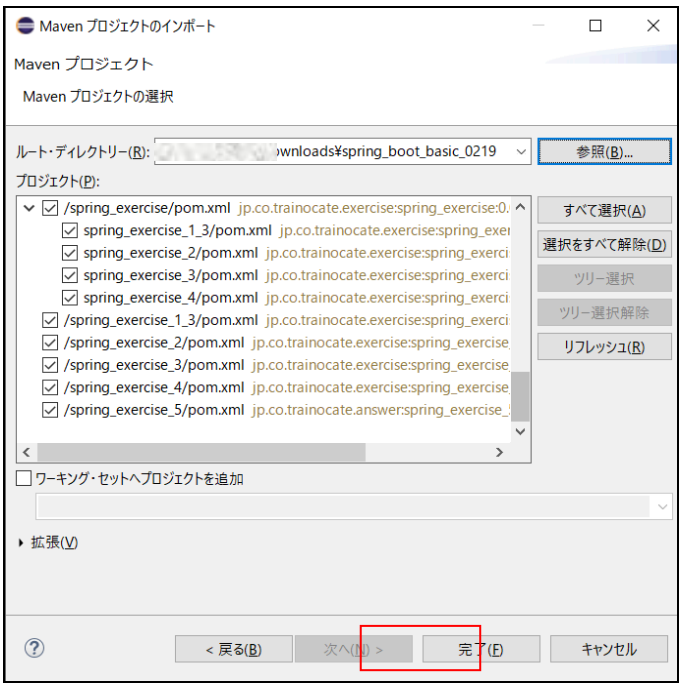




4. [参照]→「圧縮解凍後のフォルダ」を選択し、[フォルダの選択]をクリックします。



5. インポートの準備が終わったことを確認できれば、[完了]をクリックします。



6. プロジェクトエラーがなくなるまで待ちます。

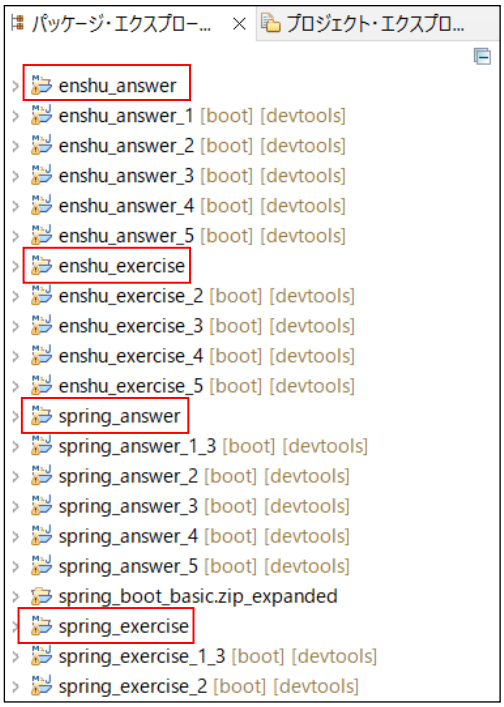
7. 本書で扱う Spring のプロジェクトは、親プロジェクトと子プロジェクトに分かれています。
赤枠で表示している、**後ろに番号がないプロジェクトが親プロジェクトです。**

enshu_answer： 演習ガイド解答

enshu_exercise： 演習ガイド問題

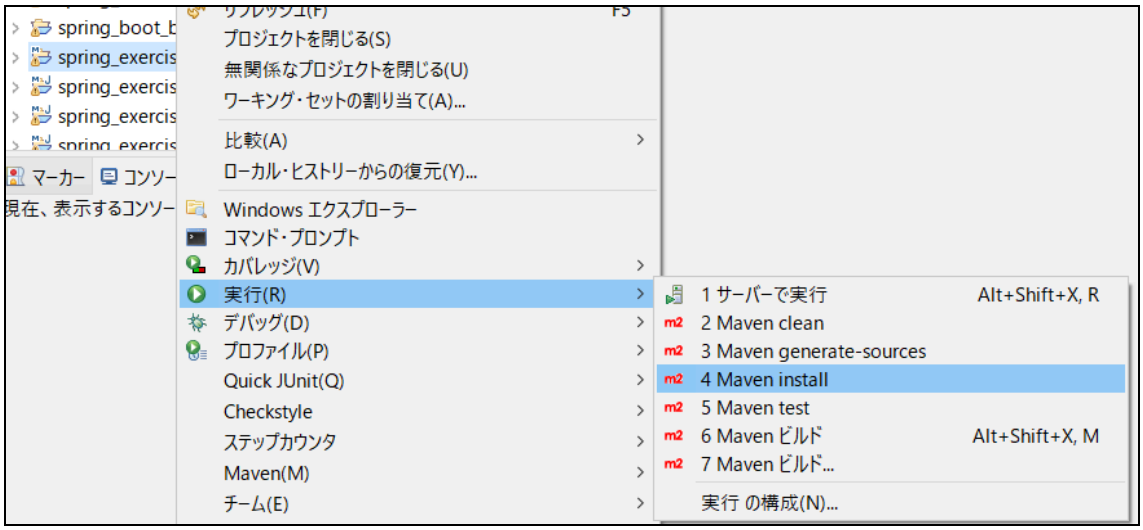
spring_answer： テキスト内のサンプル

spring_exercise： テキスト内の穴埋め問題



8. **実行したいプロジェクトは、実行する前に必ず先に親プロジェクトを右クリック→[実行]→[maven install]をしてください。**

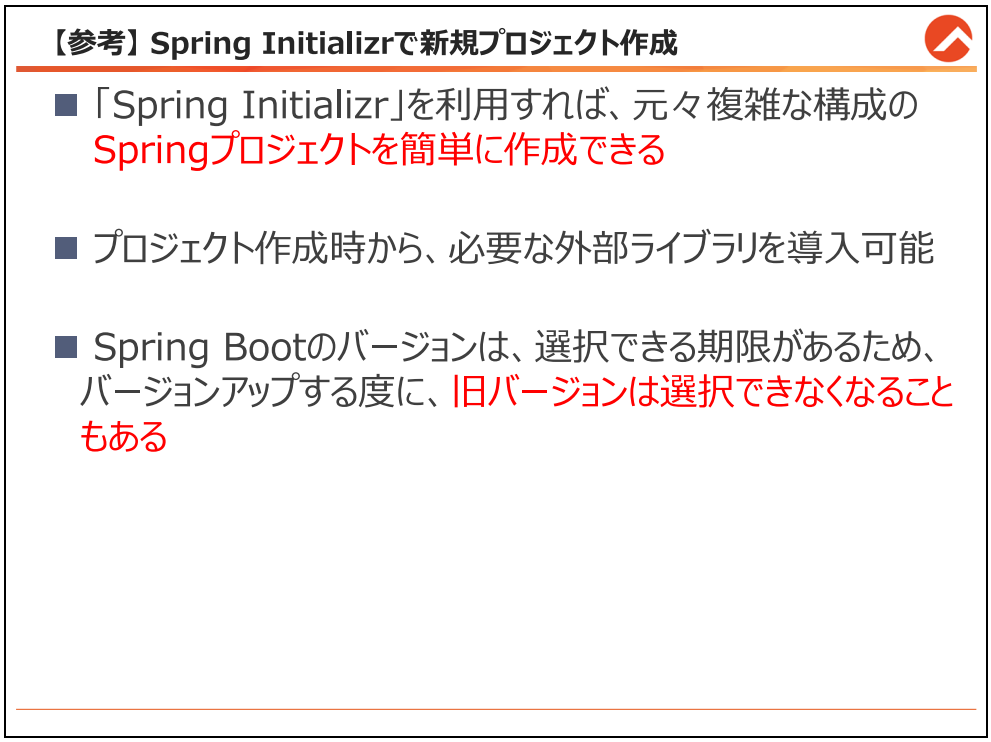
例）[spring_exercise_2]プロジェクトを実行したいなら、まず親プロジェクトである、[spring_exercise]プロジェクトを右クリック→[Maven install]をクリック



9. 以下のようにコンソール画面に「BUILD SUCCESS」という結果が表示されたら、子プロジェクトも実行できるようになります。

```
[INFO] Reactor Summary for spring_exercise 0.0.1-SNAPSHOT:
[INFO]
[INFO] spring_exercise_1_3 ................................ SUCCESS
[INFO] spring_exercise_2 .................................. SUCCESS
[INFO] spring_exercise_3 .................................. SUCCESS
[INFO] spring_exercise_4 .................................. SUCCESS
[INFO] spring_exercise .................................... SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  28.842 s
[INFO] Finished at: 2024-01-23T21:27:55+09:00
```

新規プロジェクト作成方法



Spring Initializr で Spring Boot プロジェクト作成

元々Spring プロジェクトの初期環境構築は複雑ですが、Spring Boot の「 Spring Initializr」を利用すると、初期環境構築を自動化できます。その特徴をご説明します。

① 簡単なセットアップ

Spring Initializr を使用すると、必要なライブラリや設定ファイルが最初から揃っているプロジェクトを簡単に作成できます。おかげで、初心者でもプロジェクトの基本的な構造を理解しやすくなります。

② カスタマイズ可能

プロジェクトを作成する時に、必要なライブラリ(例えば、Spring Web, JPA など)を自分で選択できます。必要な機能だけを含む、スリムで効率的なプロジェクトを作成することができます。

③ 時間の節約

一から設定を行う必要がないため、開発を始めるまでの時間を大幅に短縮できます。

④ 常に最新のバージョン

Spring Initializr は、Java や Eclipse とは関係なく、「 https://start.spring.io」で管理していますので、常に最新の Spring Boot のバージョンと互換性があります。

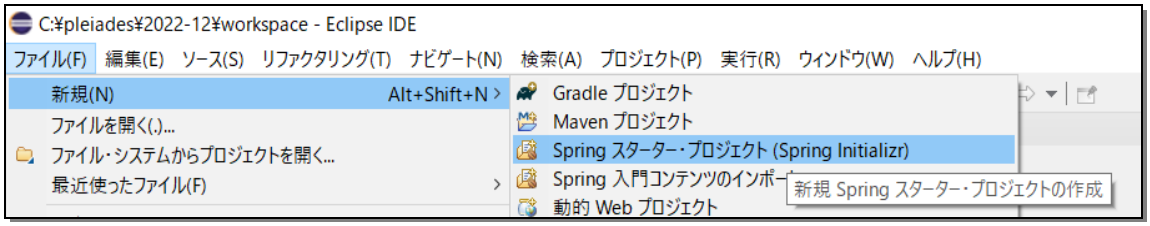
その分、**新規プロジェクト作成時は、過去のバージョンは選択できなくなりますので、注意が必要です。**

Spring Boot の「Spring Initializr」で新規 Spring プロジェクトを作成する方法をご説明します。

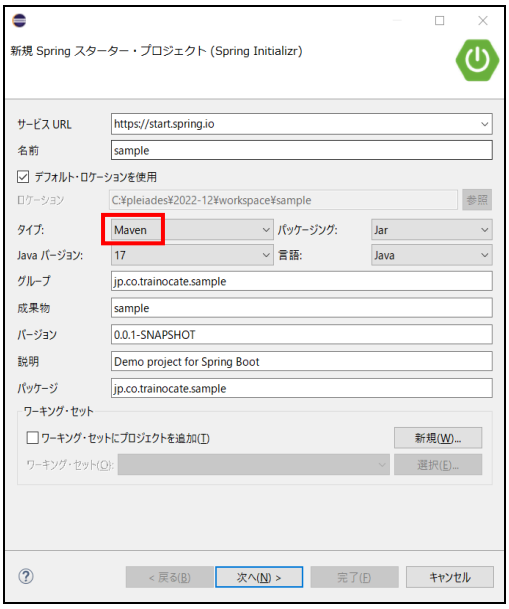
1. 2020 年以降の Eclipse(pleiades) 又は、Spring Tool Suite を起動します。

   ※本書では、Eclipse 2022(pleiades)を使用します。

2. [ファイル]→[新規]→[Spring スターター・プロジェクト(Spring Initializr)]をクリックします。



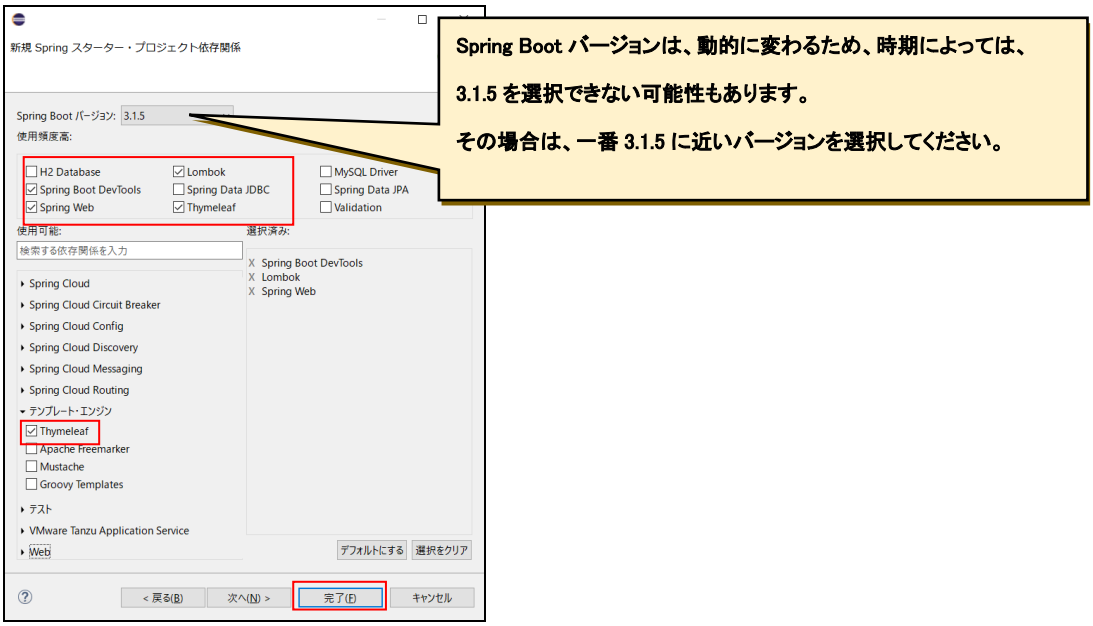
3. [タイプ]は、[Maven]に変更し、[名前]・[グループ]・[成果物]・[パッケージ]は適切な文字列を入れ、[次へ]をクリックします。



4. プロジェクト立上げの際に、最初からインストールしたい外部ライブラリを選択できます。

   初期のインストールライブラリとして一般的なライブラリは、以下の 4 つですので、チェックを入れて[完了]をクリックしてください。

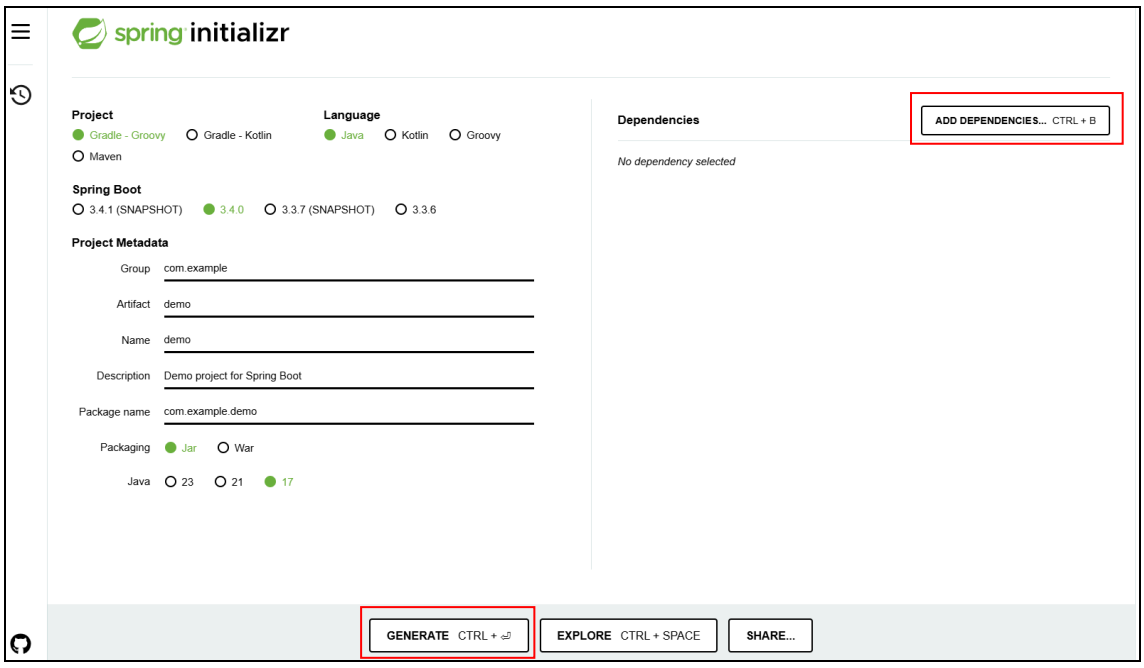
   ・Spring Web
   ・Spring Boot DevTools
   ・Thymeleaf
   ・Lombok



5. プロジェクト作成には、時間がかかりますので、Eclipse の右下の状態バーが 100%になるまでお待ちください。

セキュリティ問題で、Eclipse 上で新規プロジェクト作成ができない場合

※もし、セキュリティの問題で、Eclipe から Spring Initializr を開くことができない場合は、ブラウザから、https://start.spring.io にアクセスしてください。



「 ADD DEPENDENCIES」をクリックし、

・Spring Web
・Spring Boot DevTools
・Thymeleaf
・Lombok

などのライブラリを選択してください。

記載完了後、「 GENERATE」ボタンをクリックすると、zip ファイルとして Export されます。

前の節の、Import 手順に従って、Import してください。

B

付録 B　よく起きるエラーまとめ

404 エラーが出る場合：

1. @Controller を書き忘れていませんか？
2. @GetMapping や@PostMapping の URL は正しいですか？
3. Controller に@RequestMapping をしているか確認しましたか？
4. Get 方式で送信して、@PostMapping で受け取っていませんか？またはその逆ではありませんか？
5. Controller のクラスファイルを一回以上実行していますか？Eclipse で実行をしないと、そもそもクラスファイルがコンパイルされませんので、Java が認識できず、404 エラーになります。
6. テンプレートファイルの配置は正しいですか？
   Thymeleaf テンプレートファイルは、通常 src/main/resources/templates ディレクトリに配置されるべきです。誤ったディレクトリにファイルがあると、Spring Boot はテンプレートを見つけることができず、404 エラーを引き起こします。
7. テンプレートファイルの拡張子は正しいですか？
   Thymeleaf テンプレートのファイル名は.html で終わる必要があります。間違った拡張子を使用していると、Spring Boot がテンプレートを認識できずに 404 エラーが発生します。

500 Thymeleaf エラー

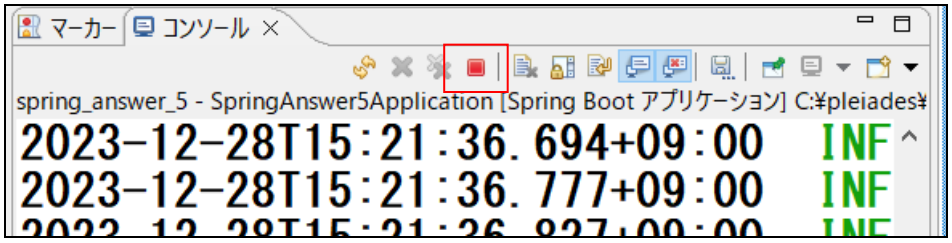
500 エラーは、基本的にエラーメッセージの中に、どの行で問題発生したかの情報が表示されますので、慌てずエラーメッセージを確認しましょう。

1. Thymeleaf の閉じタグ忘れはありませんか？
2. メソッドの引数を書き忘れていませんか？
3. 属性値の誤りはありませんか？
   Thymeleaf では、特定の属性を使ってデータを表示したり操作したりします。例えば、th:text や th:each などです。
4. モデル属性は一致していますか？
   Controller からビューに渡されるモデル属性(例えば、model.addAttribute(″data″, dataObject))と、Thymeleaf テンプレート内で参照している属性名が一致しているかを確認します。
5. Thymeleaf の式に誤りはありませんか？
   Thymeleaf では、${…}や*{…}などの式を使ってオブジェクトのプロパティにアクセスしま

す。これらの式に誤った構文や存在しないプロパティを参照していると、エラーが発生します。

8080 ポート衝突エラー

Eclipse で Spring Boot アプリケーションを実行する際は、必ず動いているサーバーを止めてから実行しないといけません。



止めずに何回も実行を続けていると、8080 ポートの衝突が発生する可能性があります。

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Web server failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure this application to listen on another port.
```

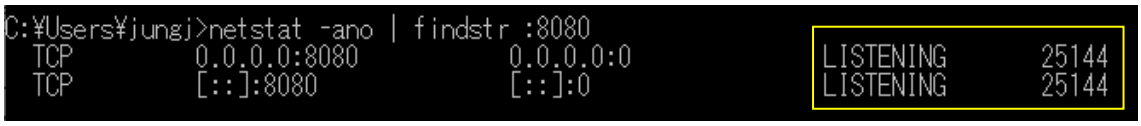
上記のようなエラーが出た際は、以下の手順を試してください。

1. Window キーを入力し、[cod]と入力してコマンドプロンプトを起動します。

2. 以下のコマンドを入力してください。8080 以外のポート番号で実行している場合は、その番号に変えてください。

```
netstat -ano | findstr :8080
```

3. LISTENING の後ろに、番号が表示されます。



4. 以下のコマンドを入力してください。**/PID の後ろには、各自の LISTENING の番号を入れてください。(25144 ではありません)**

```
taskkill /PID 25144 /F
```

5. プロセスを強制終了できましたので、Eclipse から再実行できるようになりました。

Eclipse 上で赤線が発生したり、呼び出したいメソッドが候補から出ない

1. Import 先が間違っている可能性が高いです。テキストを改めて確認し、import 先が現在の Import 先と合っているのかを確認してください。

2. 大文字・小文字は正しいのか、確認してください。

設定ファイル・イメージ・CSS・JavaScript などを修正しても反映されない

1. 修正したファイルは、全て保存しましたか？
2. Maven install と、Spring Boot アプリケーションの再起動はしましたか？
3. html ファイルを修正して保存しても、html コードが反映されない場合は、pom.xml の以下のコードを消して maven install 後、再度貼り付けて maven install してください。

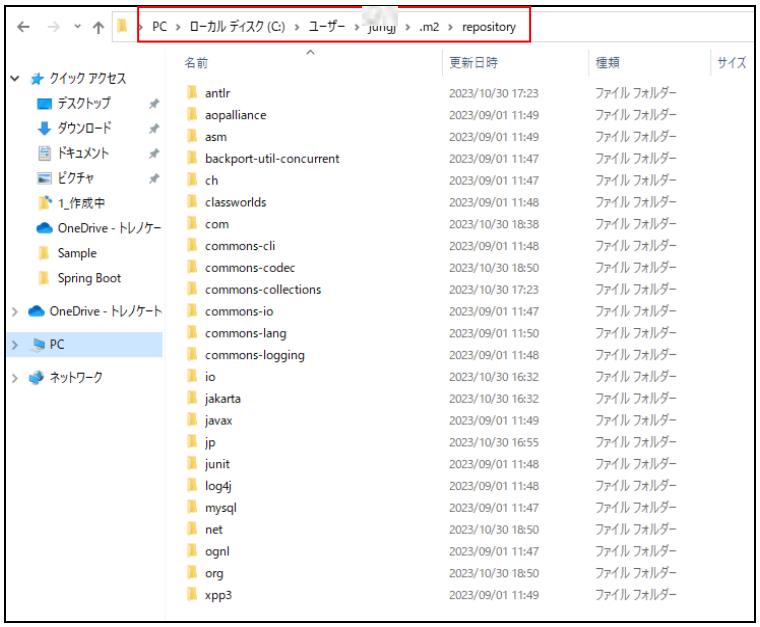
```
4.      <dependency>
5.          <groupId>org.springframework.boot</groupId>
6.          <artifactId>spring-boot-devtools</artifactId>
7.          <scope>runtime</scope>
8.          <optional>true</optional>
9.      </dependency>
```

10. キャッシュが残っていたりしませんか？
    この場合、キャッシュの削除、または異なるブラウザで実行してください。
11. JavaScript にエラーはないのか、開発者モードのコンソール上で確認しましたか？

インポート時にエラーが発生する時

pom.xml によって、ダウンロードされたライブラリのファイルは、以下のパスに保存されます。

PC¥ローカル ディスク(C:)¥ユーザ¥[ユーザ名]¥.m2¥repository



インポートエラーが出る場合は、ここにあるファイルがエラーの原因になる可能性がありますので、repository フォルダを削除してください。その後、インポートし直してください。

※セキュリティ上、Maven Repository へのインターネット接続ができない PC 環境では、インポートする前に、講師の方から研修中に使用するライブラリが入っている repository フォルダをまとめて配布しますので、こちらの repository フォルダにコピーを行ってからインポートを行ってください。

無断複写・複製の禁止
本書は著作権法上の保護を受けています。
本書の全部または一部を無断で複写複製することは
禁じられています。

Spring Boot 入門
～サーブレット&JSP からのステップアップ～

2024 年 02 月06 日　第 1 版　発行
2024 年 03 月15 日　第 2 版　改訂　著　者
2024 年 12 月 20 日　第 3 版　改訂　発行者
2026 年 03 月 13 日　第 4 版　改訂

トレノケート株式会社

〒 163-6027
東京都新宿区西新宿 6 丁目 8 番 1 号
住友不動産新宿オークタワー 27 階
電話 03-3347-9686

http://www.trainocate.co.jp/

落丁本、乱丁本はお取り替えいたします。
© 2024 Trainocate Japan, Ltd.

JAC0177G-T1-04